

Федеральное государственное автономное образовательное учреждение
высшего образования
«Томский государственный университет систем управления и радиоэлектроники»
(ТУСУР)

На правах рукописи



Шабля Юрий Васильевич

**МЕТОДОЛОГИЯ ПОСТРОЕНИЯ АЛГОРИТМОВ КОДИРОВАНИЯ
ИНФОРМАЦИОННЫХ ОБЪЕКТОВ НА ОСНОВЕ РЕКУРСИВНЫХ
КОМПОЗИЦИЙ СТРУКТУР ДЕРЕВЬЕВ И/ИЛИ**

Специальность 1.2.3 — «Теоретическая информатика, кибернетика»

Диссертация на соискание учёной степени доктора физико-математических наук

Научный консультант:
доктор технических наук, доцент
Кручинин Дмитрий Владимирович

Томск — 2025

Оглавление

Введение	5
Глава 1. Анализ современного состояния исследований в области построения алгоритмов комбинаторной генерации для решения задач кодирования информации	14
1.1 Представление информации в форме комбинаторных множеств .	14
1.2 Основные задачи в области комбинаторной генерации и их связь с процессом кодирования информации	18
1.3 Методы построения алгоритмов комбинаторной генерации	25
1.4 Связь производящих функций с комбинаторными множествами .	35
1.5 Выводы по главе	42
Глава 2. Методология построения алгоритмов кодирования информационных объектов	44
2.1 Структуры деревьев И/ИЛИ и алгоритмы генерации их вариантов	44
2.2 Расширение алгебры функций мощности комбинаторных множеств	53
2.2.1 Композиция функций с известными структурами деревьев И/ИЛИ	54
2.2.2 Операторы суммирования и произведения	60
2.2.3 Условный оператор	62
2.2.4 Нулевое значение	66
2.2.5 Композиция функций с известными алгоритмами ранжирования и генерации по рангу	68
2.2.6 Рекурсивные функции	69
2.2.7 Расширенная алгебра функций мощности комбинаторных множеств	71
2.3 Рекурсивные композиции структур деревьев И/ИЛИ для дискретных структур, связанных с числами Фубини	73
2.3.1 Рекурсивная композиция структур деревьев И/ИЛИ на основе биномиальных коэффициентов	76
2.3.2 Рекурсивная композиция структур деревьев И/ИЛИ на основе чисел Стирлинга второго рода	81

2.3.3	Рекурсивная композиция структур деревьев И/ИЛИ на основе чисел Эйлера первого рода	89
2.4	Выводы по главе	98

Глава 3. Кодирование информационных объектов на основе

	операций над производящими функциями	99
3.1	Рекурсивные композиции структур деревьев И/ИЛИ для комбинаторных множеств, сформированных в результате выполнения операций над производящими функциями	99
3.1.1	Сложение производящих функций	100
3.1.2	Умножение производящих функций	101
3.1.3	Возведение производящей функции в степень	103
3.1.4	Композиция производящих функций	105
3.2	Кодирование информационных объектов, мощность множества которых определяется алгебраической производящей функцией .	108
3.3	Алгоритмы кодирования для частных случаев алгебраических производящих функций	122
3.4	Выводы по главе	133

Глава 4. Кодирование информационных объектов,

	представленных решеточными путями	135
4.1	Комбинаторные множества решеточных путей	135
4.2	Алгоритмы кодирования для частных случаев решеточных путей	138
4.2.1	Северо-восточные решеточные пути	138
4.2.2	Решеточные пути Дика	145
4.2.3	Решеточные пути Деланнуа	150
4.2.4	Решеточные пути Шредера	156
4.2.5	Решеточные пути Моцкина	163
4.2.6	Вычислительные эксперименты	169
4.3	Алгоритмы кодирования для направленных решеточных путей .	173
4.3.1	Простые направленные решеточные пути	174
4.3.2	Простые направленные решеточные пути с ограничениями	185
4.3.3	Направленные решеточные пути	189
4.3.4	Направленные решеточные пути с ограничениями	193
4.4	Выводы по главе	197

Глава 5. Кодирование информационных объектов,	
представленных словами контекстно-свободных грамматик	199
5.1 Комбинаторные множества слов контекстно-свободных грамматик	199
5.2 Метод кодирования слов контекстно-свободных грамматик	203
5.3 Алгоритмы кодирования для частных случаев	
контекстно-свободных грамматик	206
5.4 Метод получения явных и рекуррентных формул для подсчета	
количества слов заданной длины	216
5.5 Выводы по главе	232
Глава 6. Блочное кодирование цифровых данных	234
6.1 Общая схема блочного кодирования на основе алгоритмов	
комбинаторной генерации	234
6.2 Структуры деревьев И/ИЛИ для блочного кодирования	238
6.3 Кодирование информационных объектов, связанных с прикладным	
программным обеспечением	250
6.3.1 Кодирование текстовых файлов	250
6.3.2 Кодирование растровых изображений	254
6.3.3 Кодирование журналов событий информационных систем	259
6.3.4 Кодирование реляционных баз данных	274
6.4 Выводы по главе	274
Заключение	276
Список литературы	279
Приложение А. Акт внедрения в учебный процесс	306
Приложение Б. Свидетельства о государственной регистрации	
программ для ЭВМ	308

Введение

Актуальность темы. Информация играет ключевую роль в современном мире. Развитие информационных технологий и цифровизация всех сфер жизни человека приводит к экспоненциальному росту производимой, обрабатываемой и хранимой информации. В свою очередь, это приводит к потребности в развитии существующего и разработке нового математического, алгоритмического и программного обеспечения для организации соответствующих информационных процессов.

Теория информации обеспечивает математическую основу процессов передачи, обработки и хранения информации. В этом контексте важную роль играет кодирование информации, которое в широком смысле реализует переход от одного способа представления информации к другому. При этом применение методов кодирования информации может быть вызвано следующими целями:

- преобразование информации в форму, удобную для последующей ее передачи или обработки;
- сжатие данных для организации рационального хранения;
- обеспечение конфиденциальности информации (шифрование);
- обеспечение целостности информации (помехоустойчивое кодирование).

В зависимости от типа решаемой задачи, а также учитывая специфику обрабатываемой информации, существует огромное количество различных методов кодирования информации. Каждый метод кодирования информации обладает собственной спецификой с точки зрения используемого математического аппарата, а также характеризуется ограничениями на применимость и различными оценками эффективности. Разработке теоретических основ в области методов кодирования информации посвятили свои исследования С.Е. Shannon, R.M. Fano, D.A. Huffman, J. von Neumann, R.W. Hamming, A.V. Aho, A. Lempel, В.А. Котельников, А.Н. Колмогоров, В.И. Левенштейн, А.А. Марков и другие.

Работа с дискретными структурами при решении задач кодирования информации является типичной ситуацией в связи с переходом общества к цифровым технологиям: структуры, характеризующиеся непрерывностью, подвергаются дискретизации, и в результате все сводится, например, к двоичной логике. Если обрабатываемый информационный объект (сущность, содержащая информацию) представляет собой описание сложной дискретной структуры, то путем фиксиро-

вания значений ее параметров формируется конечное множество всех возможных вариантов аналогичных структур. Таким образом, получаем конечное множество дискретных структур, для которых известны правила их построения, то есть имеем дело с комбинаторным множеством. Простым примером такого информационного объекта является двоичная последовательность длины n : образуется путем упорядочивания n элементов, каждый из которых имеет одно из двух возможных значений (0 или 1), при этом количество всех возможных вариантов двоичных последовательностей равно 2^n . В качестве более сложных структур можно привести текстовые файлы, цифровые изображения, базы данных и многое другое.

Одним из возможных вариантов математической основы для методов кодирования информации является применение методов комбинаторной генерации. Комбинаторная генерация представляет собой научное направление на стыке теоретической информатики и дискретной математики, в рамках которого рассматривается процесс получения алгоритмов генерации элементов комбинаторных множеств. Разработке теоретических основ в области методов комбинаторной генерации посвятили свои исследования E.M. Reingold, D.L. Kreher, A.V. Goldberg, D.E. Knuth, F. Ruskey, J. Sawada, A. Williams, E. Barcucci, S. Bacchelli, V. Vajnovszki, P. Flajolet, C. Martinez, X. Molinero, Б.Я. Рябко, В.В. Кручинин и другие. Если рассмотреть такие классы алгоритмов комбинаторной генерации, как ранжирование (ranking) и генерация по рангу (unranking), то на их основе можно задать биективное отображение между комбинаторным множеством и ограниченным подмножеством множества целых чисел. Следовательно, применение указанных алгоритмов позволяет организовать процесс кодирования информационных объектов, формирующих комбинаторное множество. В таком случае алгоритм ранжирования кодирует заданный информационный объект вычисленным числовым значением (ранг), а алгоритм генерации по рангу выполняет обратное преобразование (декодирование), то есть восстанавливает исходный информационный объект по известному значению его ранга.

Для разработки алгоритмов комбинаторной генерации могут применяться различные универсальные подходы, например, базирующиеся на поиске с возвратом, ЕСО-правилах, символьном методе, префиксах строк, перестановках, деревьях И/ИЛИ. В рамках данного диссертационного исследования рассматривается задача кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ. Примерами таких информационных объектов служат как общие математические

структуры (перестановки и сочетания элементов множества, подмножества, деревья, решеточные пути, выражения формальных языков), так и структуры, связанные с прикладным программным обеспечением (например, текстовые сообщения, журналы событий информационных систем, растровые изображения, реляционные базы данных). Достоинством применения метода на основе деревьев И/ИЛИ является его гибкость в представлении дискретных структур, так как деревья И/ИЛИ обеспечивают компактное представление и оптимизируют вычислительные ресурсы за счет иерархической организации. Кроме того, метод на основе деревьев И/ИЛИ позволяет учитывать сразу несколько параметров, описывающих дискретную структуру, а также реализовывать алгоритмы ранжирования и генерации по рангу. Однако, несмотря на наличие исследований по разработке алгоритмов комбинаторной генерации на основе деревьев И/ИЛИ для простых дискретных структур (таких как сочетания, перестановки, разбиения и композиции чисел), остается **актуальным решение проблемы** кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ.

Цели и задачи исследования. Целью диссертационной работы является создание теоретических и методологических основ для разработки новых эффективных методов решения задачи кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ.

Для достижения поставленной цели были решены следующие задачи:

1. Провести аналитический обзор современного состояния исследований в области построения алгоритмов комбинаторной генерации для решения задач кодирования информации;
2. Развить и формализовать методологию построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ;
3. Исследовать связь рекурсивных композиций структур деревьев И/ИЛИ с производящими функциями и операциями над ними;
4. Апробировать предложенную методологию построения алгоритмов кодирования информационных объектов на примере комбинаторных множеств решеточных путей;
5. Исследовать связь рекурсивных композиций структур деревьев И/ИЛИ с деревьями вывода контекстно-свободных грамматик;

6. Разработать методы кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ (текстовые сообщения, растровые изображения, журналы событий информационных систем, реляционные базы данных);

7. Создать комплекс программ, реализующих разработанные алгоритмы кодирования, для проверки их работоспособности и оценки эффективности.

Объект исследования. Объектом исследования является кодирование информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ.

Предмет исследования. Предметом исследования являются методы построения алгоритмов комбинаторной генерации на основе рекурсивных композиций структур деревьев И/ИЛИ.

Методы исследования. В диссертационной работе применялись методы построения алгоритмов комбинаторной генерации, получения явных выражений коэффициентов производящих функций, анализа вычислительной сложности алгоритмов, объектно-ориентированного программирования, статистического анализа и сжатия данных.

Научная новизна полученных результатов:

1. Развита методология построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ. Отличительной особенностью является расширение используемой алгебры функций мощности комбинаторных множеств: доступность операторов суммирования, произведения, условного оператора и нулевых значений, а также операции композиции функций мощности других комбинаторных множеств;

2. Разработан метод кодирования информационных объектов, мощность множества которых определяется алгебраической производящей функцией. Отличительной особенностью является использование рекуррентной формулы на основе коэффициентов k -й степени производящей функции для построения рекурсивной композиции структур деревьев И/ИЛИ;

3. Разработан метод кодирования направленных решеточных путей, отличающийся биективным отображением множества допустимых шагов решеточного пути на множество потомков ИЛИ-узла дерева И/ИЛИ. Доказаны теоремы о рекуррентных формулах для подсчета количества направленных решеточных путей на плоскости;

4. Разработан метод кодирования информационных объектов, строящихся по правилам вывода контекстно-свободных грамматик, отличительной особенностью которого является использование биективного отображения деревьев вывода на структуры вариантов деревьев И/ИЛИ. Доказаны теоремы о явных формулах для подсчета количества слов длины n , выводимых из заданного нетерминального символа однозначной контекстно-свободной грамматики;

5. Разработан метод кодирования журналов событий информационных систем, отличающийся предварительной обработкой записей журнала событий путем декомпозиции составных атрибутов, а также путем применения алгоритма ранжирования вариантов структур деревьев И/ИЛИ.

Теоретическая значимость работы. Теоретическая значимость результатов диссертационной работы заключается в развитии методологии построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ за счет расширения используемой алгебры функций мощности комбинаторных множеств. Разработанные методы кодирования информационных объектов, представленных различными классами решеточных путей либо выражениями контекстно-свободных языков, могут применяться при решении задач сжатия данных и моделирования соответствующих сложных дискретных структур (например, в области биоинформатики и хемоинформатики). Предложенные методы кодирования информационных объектов, связанных с прикладным программным обеспечением (таких как текстовые сообщения, журналы событий информационных систем, растровые изображения, реляционные базы данных) послужат теоретическим фундаментом для развития новых технологий проектирования файловых архиваторов.

Практическая значимость работы. Практическая значимость результатов диссертационной работы заключается в создании теоретических основ и программного обеспечения, расширяющих перечень инструментов проведения исследований в области разработки алгоритмов кодирования информации. Полученные в рамках апробации методологии алгоритмы кодирования подтверждают эффективность и универсальность ее применения для широкого разнообразия информационных объектов. Разработанные методы кодирования информационных объектов, представляющих собой журналы событий информационных систем, демонстрируют дополнительную эффективность по сравнению с результатами, полученными с помощью известных алгоритмов сжатия данных.

Основные этапы диссертационного исследования выполнены в рамках проведения научно-исследовательских работ, поддержанных грантами Российского фонда фундаментальных исследований (РФФИ) и Российского научного фонда (РНФ), а также программой «Приоритет 2030».

Положения, выносимые на защиту:

1. Расширение используемой алгебры функций мощности комбинаторных множеств обеспечивает возможность построения алгоритмов комбинаторной генерации на основе рекурсивных композиций структур деревьев И/ИЛИ для множеств сложных дискретных структур. В том числе, это позволяет комбинировать существующие алгоритмы для различных комбинаторных множеств;
2. Разработанный метод позволяет строить алгоритмы кодирования информационных объектов размера n , мощность множества которых определяется алгебраической производящей функцией $A(x) = \sum_{n \geq 0} a(n) x^n$, удовлетворяющей уравнению $A(x) = \sum_{i=0}^m P_i(x) \cdot A(x)^i$, где $P_i(x) = \sum_{j=0}^{n_i} p_i(j) x^j$, $p_i(j) \in \mathbb{Z}_{\geq 0}$ и $p_1(0) = 0$, с полиномиальной временной сложностью $O((n_1 + \dots + n_m + m) \cdot n^3)$;
3. Разработанный метод позволяет строить алгоритмы кодирования информационных объектов, представленных направленными решеточными путями с произвольным набором допустимых шагов вида $(a_i; b_i)$, где $a_i > 0$;
4. Полученная связь рекурсивных композиций деревьев И/ИЛИ с контекстно-свободными грамматиками позволяет строить алгоритмы кодирования для информационных объектов, которые определяются более чем одним параметром и для построения которых применимы правила вывода контекстно-свободных грамматик. Разработанные алгоритмы кодирования позволяют сжать вторичную структуру РНК более чем в 2 раза по сравнению с использованием для хранения троичной системы счисления;
5. Использование разработанного метода кодирования журналов событий позволило увеличить коэффициент сжатия для исследуемых наборов журналов событий системы управления обучением Moodle и журналов событий безопасности операционной системы Windows.

Соответствие паспорту специальности. Диссертация выполнена в соответствии с паспортом научной специальности 1.2.3 «Теоретическая информатика, кибернетика». В соответствии с п. 1 «Теория информации» в работе решается проблема кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ; представлена общая методология построения алгоритмов кодирования и разрабо-

таны методы кодирования отдельных классов дискретных структур (исследованы как общие математические структуры, так и структуры, связанные с прикладным программным обеспечением). В соответствии с п. 3 «Теория сложности алгоритмов и вычислений» проведены исследования оценки временной сложности разработанных алгоритмов кодирования и подсчета количества элементов комбинаторных множеств. В соответствии с п. 4 «Математическая теория языков и грамматик» в работе предложено биективное отображение деревьев вывода слов контекстно-свободных грамматик на структуры вариантов деревьев И/ИЛИ, что расширяет возможности применения методологии построения алгоритмов кодирования на информационные объекты, описываемые словами формальных языков.

Достоверность результатов. Достоверность результатов диссертационной работы обеспечивается строгостью применения математических методов, доказательством сформулированных утверждений, проверкой теоретических положений вычислительными экспериментами, сравнением с существующими решениями.

Внедрение результатов работы. Результаты диссертационной работы использованы в ходе выполнения следующих научно-исследовательских работ:

- «Разработка алгоритмов и программного обеспечения индексирования больших объемов данных на основе новых методов комбинаторной генерации» (грант РНФ на 2018–2020 гг., проект № 18-71-00059, исполнитель);
- «Методы комбинаторной генерации на основе деревьев И/ИЛИ с применением теории производящих функций» (грант РФФИ на 2018–2019 гг., проект № 18-31-00201, руководитель);
- «Защита электронных документов за счет разработки методов и алгоритмов встраивания цифровых водяных знаков повышенной робастности» (грант РФФИ на 2019–2021 гг., проект № 19-47-703003, исполнитель);
- «Исследование коэффициентов степеней производящих функций многих переменных» (грант РФФИ на 2019–2021 гг., проект № 20-31-70037, исполнитель);
- «Разработка методов и алгоритмов комбинаторной генерации на основе применения схем рекурсивных композиций деревьев И/ИЛИ» (стипендия Президента РФ для молодых ученых и аспирантов, осуществляющих перспективные научные исследования и разработки по приоритетным направлениям модернизации российской экономики на 2021–2023 гг.);
- «Разработка математического, алгоритмического и программного обеспечения комбинаторной генерации для решения задач хранения и обработки

больших объёмов данных» (грант РНФ на 2022–2025 гг., проект № 22-71-10052, исполнитель).

Результаты диссертационной работы внедрены в учебный процесс ФГАОУ ВО «ТУСУР» на факультете вычислительных систем и используются при обучении студентов по направлению подготовки «Информатика и вычислительная техника».

Личный вклад автора. Автору принадлежит ключевая роль в получении основных результатов диссертационной работы. Все результаты, составляющие научную основу, и выносимые на защиту положения получены автором лично или при непосредственном его участии. Авторский вклад заключается в развитии методологии построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ. Автором самостоятельно разработаны методы кодирования информационных объектов, представленных различными классами решеточных путей и выражениями контекстно-свободных языков, а также предложены методы кодирования информационных объектов, связанных с прикладным программным обеспечением. Соавторы, принимавшие участие в отдельных направлениях исследований и в разработке программного обеспечения, указаны в списке основных публикаций по теме диссертационной работы.

Апробация работы. Основные результаты диссертационной работы докладывались и обсуждались на международных и всероссийских научных конференциях:

- Международная научно-техническая конференция студентов, аспирантов и молодых ученых «Научная сессия ТУСУР» (2015–2019, 2021, 2023–2025 гг., г. Томск, ТУСУР);
- Международная научно-практическая конференция «Электронные средства и системы управления» (2015–2021, 2023–2024 гг., г. Томск, ТУСУР);
- Международная конференция студентов, аспирантов и молодых ученых «Перспективы развития фундаментальных наук» (2021–2024 гг., г. Томск, ТПУ);
- Всероссийская научная конференция молодых ученых «Наука. Технологии. Инновации» (2015–2016, 2018–2019, 2022–2023 гг., г. Новосибирск, НГТУ);
- Международная научно-практическая конференция молодых ученых «Прикладная математика и информатика: современные исследования в области естественных и технических наук» (2018–2019, 2023 гг., г. Тольятти, ТГУ);
- VIII Международная научная конференция «Конвергентные когнитивно-информационные технологии» (30 ноября – 2 декабря 2023 г., г. Москва, МГУ);

- The 3rd Algorithmic and Enumerative Combinatorics Summer School (1–5 августа 2016 г., Австрия, г. Хагенберг, Университет им. И. Кеплера);
- The Mediterranean International Conference of Pure & Applied Mathematics and Related Areas (2018, 2021–2022, 2024 гг., Турция, г. Анталья, Университет Акдениз);
- The Mediterranean International Conference of Pure & Applied Mathematics and Related Areas (2019, 2023 гг., Франция г. Париж, Университет Эври);
- The 32th International Conference of the Jangjeon Mathematical Society (17–19 июля 2019 г., г. Владивосток, ДВФУ);
- The VIII International Conference «Engineering and Telecommunication» (24–25 ноября 2021 г., г. Москва, МФТИ);
- The 13th Symposium on Generating Functions of Special Numbers and Polynomials and their Applications (11–13 марта 2023 г., Турция, г. Анталья, Университет Акдениз);
- The 30th International Conference on Difference Equations and Applications (15–19 июля 2025 г., Китай, г. Гуанчжоу, Гуанчжоуский университет).

Публикации по теме диссертации. Основные результаты диссертационного исследования опубликованы в 25 статьях в рецензируемых научных изданиях перечня ВАК (18 статей относятся к категории К1, 6 статей относятся к категории К2, 1 статья относится к категории К3). Из них 16 статей (Article) опубликованы в зарубежных научных изданиях, индексируемых Web of Science или Scopus (из них 4 статьи в журналах, входящих в первый квартиль рейтинга JCR от Web of Science). Дополнительно имеется 8 публикаций (Conference paper) в научных изданиях, индексируемых Web of Science или Scopus, а также 43 публикации в иных научных изданиях. Получены 7 свидетельств о государственной регистрации программ для ЭВМ.

Объем и структура работы. Диссертация состоит из введения, шести глав основной части, заключения, списка литературы и двух приложений. Полный объем диссертации составляет 314 страниц, включая 90 рисунков, 23 таблицы и 86 алгоритмов. Список литературы содержит 274 наименования.

Глава 1. Анализ современного состояния исследований в области построения алгоритмов комбинаторной генерации для решения задач кодирования информации

В данной главе представлен анализ современного состояния исследований в области построения алгоритмов комбинаторной генерации для решения задач кодирования информации. Вводятся основные понятия диссертационного исследования и демонстрируются возможности применения результатов научного направления комбинаторной генерации в области разработки алгоритмов кодирования информационных объектов. Рассматривается связь производящих функций с комбинаторными множествами и функциями их мощности, а также влияние операций над производящими функциями на формирование новых дискретных структур.

1.1 Представление информации в форме комбинаторных множеств

Основополагающей характеристикой современного информационного общества является доминирование информации как ключевого ресурса. В данном случае под термином «информация» понимаются любые сведения независимо от формы их представления [1]. Интенсивное развитие информационных технологий способствовало информатизации всех сфер жизнедеятельности человека и общества. Кроме того, появление глобальной сети Интернет значительно упростило доступ к огромному количеству данных, а популярность цифровых технологий способствует генерации новой информации. Статистический отчет «Digital 2024: Global overview report» [2] подтверждает тот факт, что число пользователей различных цифровых устройств неуклонно растет с каждым годом по всему миру. Соответственно, растет и объем данных, производимых такими пользователями.

Согласно аналитическим отчетам международной исследовательской компании «International Data Corporation», к 2028 году прогнозируется значение общего объема созданных по всему миру цифровых данных, равное 394 зеттабайт ($394 \cdot 10^{21}$ байт). На рисунке 1.1 представлены результаты оценки общего объема созданных данных с 2010 по 2023 годы и прогнозные значения до 2028 года [3], что также подтверждает экспоненциальный рост объема генерируемых данных.

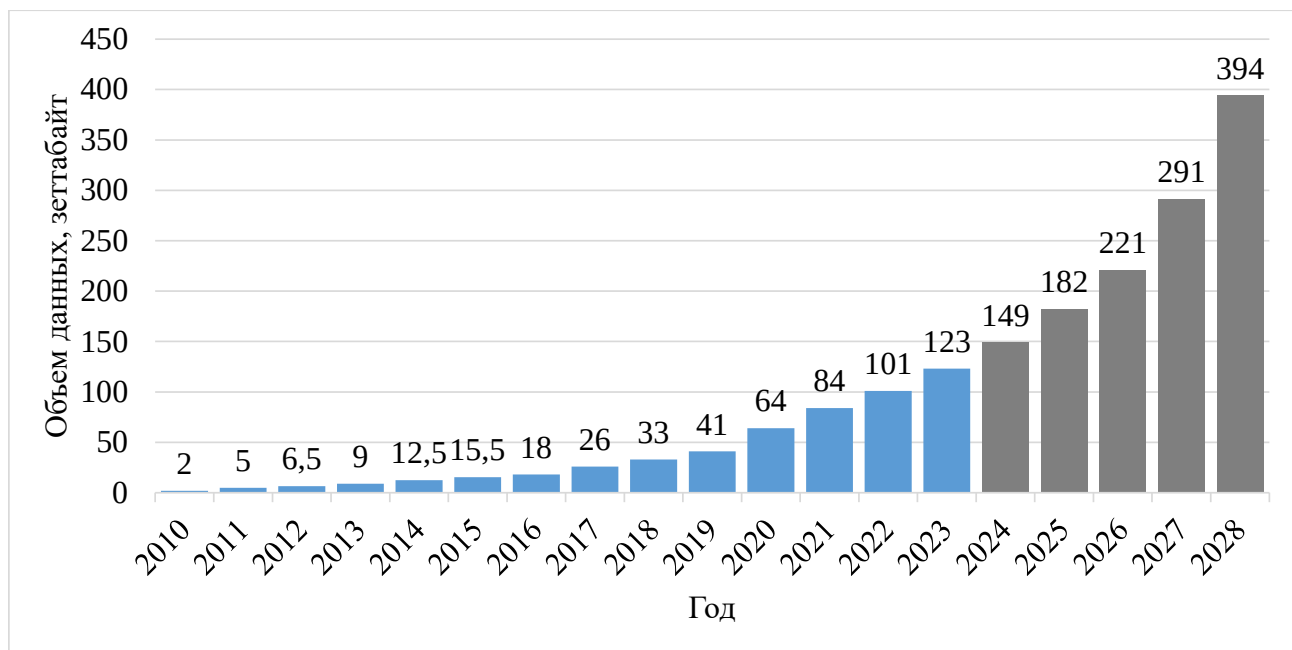


Рисунок 1.1 — Объем созданных цифровых данных по годам

Следовательно, возникает потребность в развитии теоретической базы и разработке соответствующих инструментов для работы с такими большими объемами данных.

Исследования в области теории информации сформировали теоретические основы организации современных информационных технологий. При этом особое место в теории информации занимает раздел, связанный с разработкой методов кодирования информации. В широком смысле, кодирование информации реализует переход от одного способа представления информации к другому [4]. Обратное преобразование, в результате которого из закодированной формы представления информации восстанавливается ее исходная форма, называется декодированием. За счет использования различных методов кодирования информации реализуется решение следующих практических задач:

1. Преобразование информации в форму, удобную для последующей ее передачи или обработки. Например, устную речь можно представить письменно в виде текста, а аналоговый сигнал можно оцифровать, что позволит воспользоваться совершенно другими инструментами для обработки исходной информации;

2. Сжатие данных [5; 6]. В данном случае решается задача минимизации ресурсов, требуемых для хранения информации, что особенно важно в условиях ограниченных ресурсов и способствует быстрой передаче данных. При этом возможно как сжатие данных без потерь (полученная в результате декодирования информация полностью совпадает с информацией до ее кодирования), так и

сжатие данных с потерями (полученная в результате декодирования информация незначительно, с точки зрения дальнейшего использования, отличается от информации до ее кодирования);

3. Обеспечение свойства конфиденциальности информации, реализуется путем применения методов шифрования [7; 8]. В ходе процесса шифрования информация преобразуется в форму, полностью скрывающую суть ее содержимого, а восстановление исходной формы возможно только при наличии определенной секретной информации (ключ шифрования);

4. Обеспечение свойства целостности информации, реализуется путем применения методов помехоустойчивого кодирования [9; 10]. В данном случае решается задача обнаружения ошибок, появившихся в ходе передачи данных (например, за счет применения контрольных сумм или хеширования). Кроме того, существуют методы кодирования информации, способные самостоятельно исправить часть обнаруженных ошибок.

Таким образом, имеющаяся теоретическая база в области теории информации и кодирования создает фундамент для развития и поддержания устойчивых и безопасных информационных систем. При этом разработка новых и эффективных методов кодирования информации позволит улучшить качество работы соответствующих информационных процессов.

Применение цифровых технологий накладывает ограничения на формы представления обрабатываемого информационного объекта (объект, содержащий в себе некоторую информацию, то есть несет определенную смысловую нагрузку [11]). Рассматриваемый информационный объект должен обладать конкретным описанием процедуры построения его структуры, так как компьютер способен выполнять только последовательность конкретных инструкций, составленных с использованием набора допустимых вычислительных операций. Кроме того, параметры такой структуры должны определяться значениями конечного множества и не быть бесконечными, так как компьютер обладает ограниченным количеством вычислительных ресурсов. Для цифровых устройств простейшим примером такой структуры является бит, который задается одним из двух возможных значений: 0 — отсутствие сигнала, 1 — наличие сигнала. Примером посложнее является двоичная последовательность длины n , которая образуется путем упорядочивания n значений бит, а количество всех возможных вариантов двоичных последовательностей равно 2^n . Частным случаем такой структуры является байт, то есть двоичная последовательность длины 8.

В качестве информационного объекта с более сложной структурой можно привести текст. По своей сути текст представляет собой последовательность n символов, взятых из множества m допустимых символов (алфавит) с возможностью повтора. Тогда количество всех возможных вариантов текстов равно m^n . Например, для заданного алфавита $\{a,b,c\}$, состоящего из $m = 3$ символов, можно составить следующие $3^2 = 9$ разных текстов, состоящих из $n = 2$ символов:

$$\{aa,ab,ac,ba,bb,bc,ca,cb,cc\}.$$

Еще одним примером информационного объекта, активно используемого в информационных системах, являются растровые изображения. По своей сути растровое изображение представляет собой последовательность $n \cdot m$ пикселей, формирующих прямоугольник размера n на m (высота — n пикселей, ширина — m пикселей). При этом каждый пиксель имеет цвет, взятый из множества k допустимых цветов (цветовая палитра) с возможностью повтора. Тогда количество всех возможных вариантов растровых изображений равно $k^{n \cdot m}$. Например, для заданной цветовой палитры $\{\blacksquare, \color{red}\blacksquare\}$, состоящей из $k = 2$ цветов, можно составить следующие $2^{2 \cdot 2} = 16$ разных растровых изображений размера $n = 2$ на $m = 2$:

$$\{\begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \blacksquare \\ \blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \blacksquare \\ \color{red}\blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \blacksquare \\ \color{red}\blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \color{red}\blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \color{red}\blacksquare \\ \blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \color{red}\blacksquare \\ \color{red}\blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \blacksquare & \color{red}\blacksquare \\ \color{red}\blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \blacksquare \\ \blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \blacksquare \\ \color{red}\blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \color{red}\blacksquare \\ \blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \color{red}\blacksquare \\ \blacksquare & \color{red}\blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \color{red}\blacksquare \\ \color{red}\blacksquare & \blacksquare \end{smallmatrix}, \begin{smallmatrix} \color{red}\blacksquare & \color{red}\blacksquare \\ \color{red}\blacksquare & \color{red}\blacksquare \end{smallmatrix}\}.$$

Аналогично можно описать процедуру построения практически для любого обрабатываемого информационного объекта в информационной системе. Например, видеозапись можно представить как последовательность кадров (изображений), аудиозапись — как последовательность значений амплитуд оцифрованного аудиосигнала, реляционная база данных — как множество иерархически связанных таблиц, где каждая таблица может быть представлена последовательностью текстовых данных. Следовательно, каждый такой информационный объект можно охарактеризовать как некоторую дискретную структуру. Кроме того, путем фиксирования значений параметров рассматриваемой дискретной структуры формируется конечное множество всех возможных вариантов аналогичных структур. Конечное множество дискретных структур, для которых известны правила их построения, называется комбинаторным множеством, а его элементы — комбинаторными объектами [12].

Таким образом, множества обрабатываемых информационных объектов могут быть представлены как комбинаторные множества, что, в свою очередь, позволяет использовать различные классы комбинаторных алгоритмов.

1.2 Основные задачи в области комбинаторной генерации и их связь с процессом кодирования информации

Комбинаторные алгоритмы играют важную роль в области теоретической информатики, так как являются эффективным инструментом при работе с дискретными структурами [12; 13]. Одной из общих задач, связанных с обработкой комбинаторных множеств, является построение алгоритмов комбинаторной генерации [14]. Алгоритмы комбинаторной генерации позволяют перенумеровать элементы комбинаторного множества (за счет применения алгоритма ранжирования), а также в обратную сторону реализовать их генерацию как в единичном варианте (алгоритм генерации по рангу или случайная генерация), так и исчерпывающим перебором (последовательная генерация).

При исследовании алгоритмов комбинаторной генерации применяется следующая классификация [15]:

1. Алгоритм последовательной генерации (Listing);
2. Алгоритм случайной генерации (Random selection);
3. Алгоритм ранжирования (Ranking);
4. Алгоритм генерации по рангу (Unranking).

Определение 1.1. *Алгоритм последовательной генерации (Listing) — алгоритм, который для заданного комбинаторного множества A последовательно генерирует все элементы $a \in A$.*

При этом порядок, согласно которому генерируются комбинаторные объекты, формируется заложенными в алгоритме теоретическими основами, то есть правилами обхода элементов множества. На рисунке 1.2 представлен пример, демонстрирующий работу алгоритма последовательной генерации для множества текстов, состоящих из двух символов с использованием алфавита $\{a, b, c\}$. В данном случае результатом применения алгоритма Listing является сгенерированная последовательность всех элементов комбинаторного множества с использованием лексикографического упорядочивания.

Определение 1.2. *Алгоритм случайной генерации (Random selection) — алгоритм, который для заданного комбинаторного множества A случайным образом генерирует один элемент $a \in A$.*

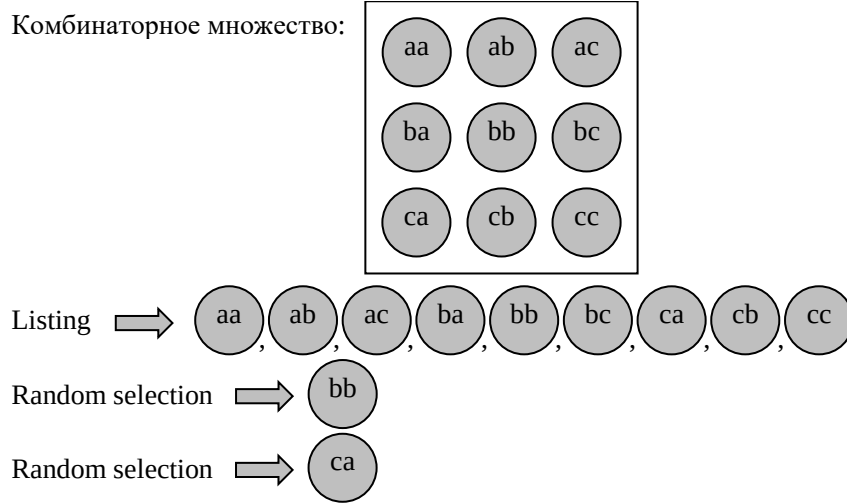


Рисунок 1.2 — Пример выполнения алгоритмов Listing и Random selection

При этом распределение вероятностей генерации комбинаторных объектов может быть как равномерным (каждый объект обладает одинаковыми шансами на то, что именно он будет сгенерирован при очередном запуске алгоритма случайной генерации), так и неравномерным. На рисунке 1.2 также представлен пример, демонстрирующий работу алгоритма случайной генерации. В данном случае в результате первого применения алгоритма случайной генерации сформирован объект с текстом «bb», а в результате второго применения — объект с текстом «ca».

Определение 1.3. *Алгоритм ранжирования (Ranking) — алгоритм, который для заданного элемента a комбинаторного множества A вычисляет его уникальный порядковый номер r , также называемый рангом.*

Следовательно, реализуется отображение из комбинаторного множества A в множество неотрицательных целых чисел $\mathbb{Z}_{\geq 0}$:

$$\text{Rank} : A \rightarrow \mathbb{Z}_{\geq 0}$$

или

$$r = \text{Rank}(a),$$

где $a \in A$ и $r \in \mathbb{Z}_{\geq 0}$. В данном случае в качестве начала отсчета использовано значение 0, то есть первый объект в упорядоченном ряду имеет ранг $r = 0$. При этом упорядочивание комбинаторных объектов формируется заложенными в алгоритме ранжирования теоретическими основами. Также, ввиду уникальности значений рангов, если $r_i = \text{Rank}(a_i)$ и $r_j = \text{Rank}(a_j)$, то для $a_i, a_j \in A$ и $a_i \neq a_j$ следует $r_i \neq r_j$. На рисунке 1.3 представлен пример, демонстрирующий работу

алгоритма ранжирования для множества текстов, состоящих из двух символов с использованием алфавита $\{a,b,c\}$. В данном случае в результате первого применения алгоритма ранжирования для объекта с текстом «bb», согласно лексикографического упорядочивания элементов комбинаторного множества, вычислен ранг 4. В результате второго применения алгоритма ранжирования для объекта с текстом «ca» вычислен ранг 6, в результате третьего применения — для объекта с текстом «ab» вычислен ранг 1.

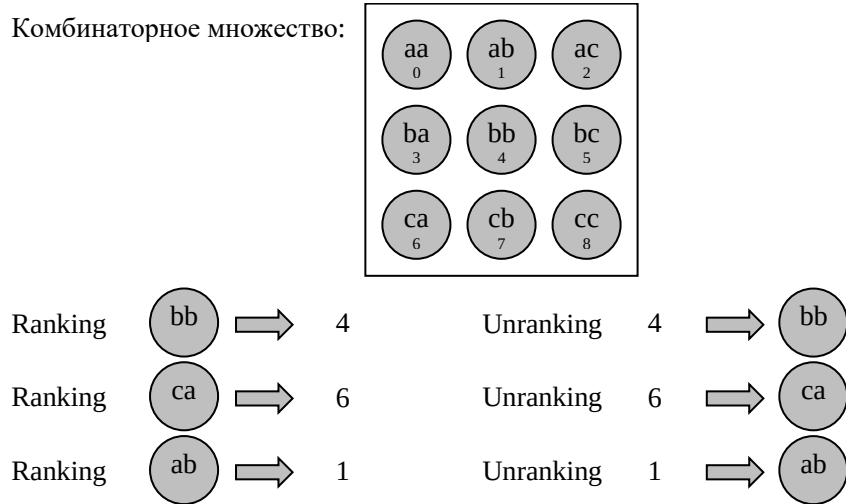


Рисунок 1.3 — Пример выполнения алгоритмов Ranking и Unranking

Определение 1.4. *Алгоритм генерации по рангу (Unranking) — алгоритм, который для заданного ранга r генерирует соответствующий ему элемент a комбинаторного множества A .*

Следовательно, реализуется отображение из множества неотрицательных целых чисел $\mathbb{Z}_{\geq 0}$ в комбинаторное множество A :

$$\text{Unrank} : \mathbb{Z}_{\geq 0} \rightarrow A$$

или

$$a = \text{Unrank}(r),$$

где $a \in A$ и $r \in \mathbb{Z}_{\geq 0}$. На рисунке 1.3 также представлен пример, демонстрирующий работу алгоритма генерации по рангу. В данном случае в результате первого применения алгоритма генерации для ранга 4, согласно лексикографического упорядочивания элементов комбинаторного множества, сгенерирован объект с текстом «bb». В результате второго применения алгоритма генерации для ранга 6 сгенерирован объект с текстом «ca», в результате третьего применения — для ранга 1 сгенерирован объект с текстом «ab».

Алгоритмы ранжирования и генерации по рангу реализуют обратные друг для друга преобразования и формируют биективное отображение между комбинаторным множеством и ограниченным подмножеством неотрицательных целых чисел, поэтому справедливы следующие соотношения:

$$a = \text{Unrank}(\text{Rank}(a)),$$

$$r = \text{Rank}(\text{Unrank}(r)).$$

Кроме того, на основе алгоритма генерации по рангу можно реализовать работу как алгоритма последовательной генерации, так и случайной генерации. Случайная генерация происходит за счет применения алгоритма генерации по рангу для выбранного случайным образом одного из допустимых значений рангов рассматриваемого комбинаторного множества. Если же в цикле последовательно перебрать все допустимые значения рангов рассматриваемого комбинаторного множества и к каждому из них применить алгоритм генерации по рангу, то результат выполнения будет аналогичен применению алгоритма последовательной генерации. Однако такому варианту реализации алгоритма последовательной генерации характерны высокие оценки вычислительной сложности. Таким образом, алгоритм генерации по рангу выступает универсальным алгоритмом комбинаторной генерации.

Каждый тип алгоритмов комбинаторной генерации специализируется на решении определенного рода практических задач. В частности, алгоритмы последовательной генерации позволяют решить задачу полного перебора всех возможных структур при заданных значениях их параметров. Следовательно, алгоритмы последовательной генерации можно применить при решении задач оптимизации дискретных структур, то есть когда происходит их перебор и по итогу отбирается структура с наилучшими характеристиками. Также алгоритмы последовательной генерации можно применить при решении задач моделирования сложных дискретных структур. В таком случае за счет применения алгоритма комбинаторной генерации сразу создаются структуры, удовлетворяющие заданным ограничениям, то есть отпадает потребность проверки соблюдения требуемых условий у произвольно созданной структуры. Например, если случайным образом без участия алгоритма комбинаторной генерации создать последовательность одинакового количества открывающих и закрывающих скобок, то совсем необязательно будет сформирована последовательность корректно вложенных скобок.

Обзор большого количества известных алгоритмов последовательной генерации для основных классических комбинаторных структур (последовательности, перестановки, сочетания, разбиения, деревья) представлен в научных монографиях D.E. Knuth [12; 13] и F. Ruskey [15]. Одним из наиболее известных математических приемов, применяемых при разработке алгоритмов последовательной генерации, является метод поиска с возвратом (Backtracking), который проявляется в реализации обхода внутренних конструкций генерируемых структур и их частичном изменении при переходе от одной структуры к другой. Однако известны подходы с использованием отличающегося математического аппарата, такие как ЕСО-метод [16] или метод на основе перестановок [17].

Алгоритмы случайной генерации позволяют решить задачу формирования выборочной совокупности объектов из множества всех возможных структур при заданных значениях их параметров. Полученная выборка может быть использована для тестирования других алгоритмов, работающих с исследуемыми дискретными структурами. В основе работы алгоритмов случайной генерации лежит выбор случайного пути обхода или случайного правила построения внутренних конструкций генерируемых структур, реализуемый за счет использования генераторов случайных чисел. Применение алгоритмов случайной генерации для создания тестовых выборок становится особенно актуальным в следующих случаях, в которых полный перебор требует большого количества ресурсов:

- исследуется сложная дискретная структура, генерация которой требует большого количества ресурсов;
- исследуется дискретная структура, характеризующаяся значительным разнообразием при заданных значениях ее параметров, то есть существует большое количество вариантов таких структур.

Алгоритмы генерации по рангу могут быть использованы в качестве основы алгоритмов последовательной или случайной генерации, что делает их применимыми при решении описанных выше задач. Такой вариант реализации алгоритмов комбинаторной генерации предоставляет следующие преимущества:

- в случае реализации последовательной генерации алгоритм генерации по рангу позволяет задать стартовую позицию, начиная с которой продолжится последовательный перебор структур;
- в случае реализации случайной генерации алгоритм генерации по рангу позволяет задать вероятностное распределение генерируемых структур за счет настройки вероятностей выбора значений рангов.

В то же время комбинирование алгоритмов ранжирования и генерации по рангу позволяет задать биективное отображение между комбинаторным множеством и ограниченным подмножеством неотрицательных целых чисел. Алгоритм ранжирования позволяет представить заданную дискретную структуру в форме одного целого числа (ранг), то есть выполняется кодирование исходного информационного объекта. Алгоритм генерации по рангу позволяет восстановить исходную дискретную структуру из значения соответствующего ей ранга, то есть выполняется декодирование информационного объекта. Следовательно, на основе алгоритмов комбинаторной генерации можно организовать процесс кодирования информационных объектов, представляющих собой сложные дискретные структуры.

При этом за счет кодирования информационных объектов с использованием алгоритмов комбинаторной генерации могут достигаться разные цели:

- если значение ранга требует меньше памяти для хранения, чем исходное представление дискретной структуры, то достигается эффект сжатия данных;
- если часть информации, требуемой для выполнения обратного преобразования с помощью алгоритма генерации по рангу, сохранена в секрете, то обеспечивается свойство конфиденциальности информации;
- значение ранга также может быть использовано в качестве контрольной суммы, вычисленной для исследуемой дискретной структуры.

В общем виде идея кодирования информации на основе алгоритмов комбинаторной генерации с целью сжатия данных была предложена в [18]. В частности, в указанной работе исследовалась задача кодирования данных, которые представляют собой слова заданной длины из некоторого формального языка, за счет применения алгоритма ранжирования. На рисунке 1.4 схематично изображен процесс кодирования информационного объекта a , принадлежащего комбинаторному множеству A , значением ранга r , полученным с помощью алгоритма ранжирования. Также указано обратное восстановление объекта a из значения ранга r за счет применения алгоритма генерации по рангу.

В рамках работы алгоритма ранжирования задается некоторое упорядочивание элементов рассматриваемого комбинаторного множества. Таким образом, значение ранга r представляет собой порядковый номер информационного объекта в упорядоченном ряду всех элементов комбинаторного множества. Обозначим через $|A|$ количество всех объектов, содержащихся в комбинаторном множестве A . Тогда значение ранга r для произвольного объекта a комбинаторного множе-

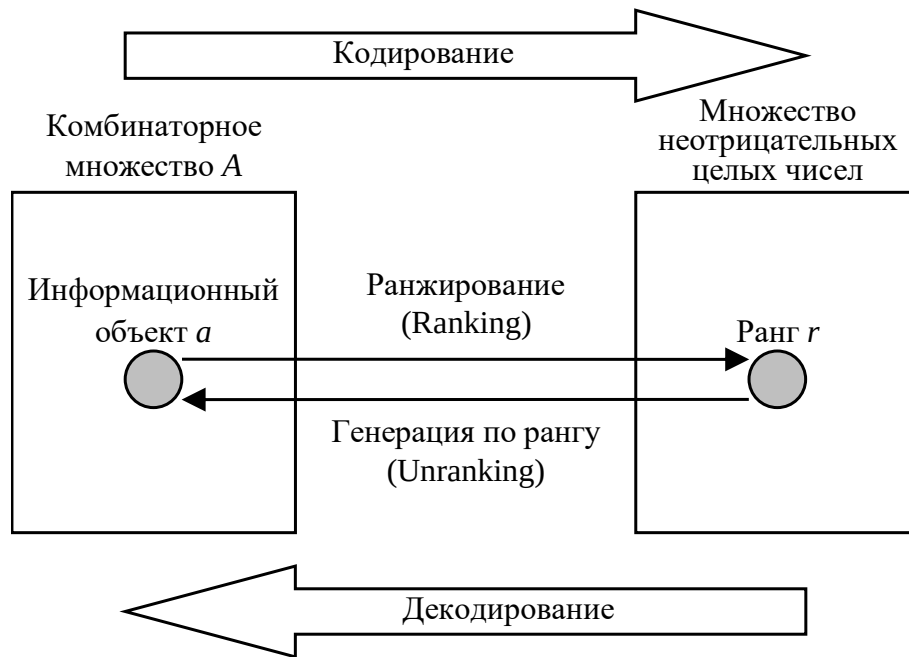


Рисунок 1.4 — Кодирование информации с помощью алгоритмов комбинаторной генерации

ства A принадлежит множеству $\{0, 1, \dots, |A| - 1\}$. Следовательно, чтобы хранить значение одного произвольного ранга r , достаточно

$$S_1 = \lceil \log_2 |A| \rceil \text{ бит.}$$

Дополнительно обозначим через S_2 количество бит, требуемых для хранения информационного объекта a в исходной форме представления. Следовательно, если $S_2 > S_1$, то в результате кодирования происходит сжатие данных, то есть получаем значение коэффициента сжатия

$$K = \frac{S_2}{S_1}$$

больше единицы.

При этом имеются следующие закономерности касемо качества сжатия:

- чем сложнее описание структуры информационного объекта, тем больше требуется количества бит памяти для его хранения (то есть увеличивается значение S_2), что приводит к увеличению коэффициента сжатия;
- чем более разнообразна структура информационных объектов, тем больше элементов содержится в комбинаторном множестве A (то есть увеличивается значение $|A|$ и, как следствие, увеличивается значение S_1), что приводит к уменьшению коэффициента сжатия.

Заметим, что данный вариант сжатия данных за счет выполнения алгоритмов комбинаторной генерации относится к классу методов сжатия без потерь, так как алгоритм генерации по рангу восстанавливает исходный информационный объект без каких-либо изменений.

Пример использования подхода к сжатию данных на основе алгоритмов комбинаторной генерации представлен в [19; 20]. В данной работе рассмотрена конкретная задача сжатия измерительной информации, обрабатываемой системой геомониторинга подвижных объектов. При этом предложенный алгоритм сжатия базируется на представлении измерительной информации в виде сочетания символов фиксированного алфавита.

Кроме того, имеются предпосылки к созданию универсального метода сжатия данных на основе алгоритмов комбинаторной генерации [21]. В данной работе исследуется возможность построения формальной грамматики, правила вывода которой позволят сформировать последовательность бит рассматриваемого файла. Далее с помощью алгоритма ранжирования для последовательности примененных правил вывода реализуется кодирование файла.

Таким образом, наличие подобных исследований подтверждает актуальность совершенствования существующих и разработки новых методов кодирования информации, в том числе на основе методов комбинаторной генерации.

1.3 Методы построения алгоритмов комбинаторной генерации

Определив для кодируемого информационного объекта соответствующее комбинаторное множество, следующим шагом необходимо разработать алгоритмы ранжирования и генерации по рангу. Область разработки алгоритмов комбинаторной генерации активно развивается, о чем свидетельствуют регулярные публикации в высокорейтинговых научных изданиях. Далее представлены примеры недавних публикаций по научным исследованиям в области разработки новых алгоритмов комбинаторной генерации. Например, одним из актуальных направлений является изучение различных древовидных структур и методов их генерации. Так, в работе [22] исследована задача разработки алгоритмов комбинаторной генерации для множества структур (k, m) -арных деревьев порядка n с использованием B -упорядочивания. Авторами разработаны модифицированные

версии алгоритмов ранжирования и генерации по рангу, которые обладают лучшей оценкой вычислительной сложности по сравнению с аналогами и не требуют хранения результатов предварительных вычислений. В работе [23] предложены алгоритмы ранжирования и генерации по рангу для бинарных деревьев с линейной временной сложностью за счет выполнения предварительных вычислительных шагов. При этом для представления бинарных деревьев, состоящих из n узлов, используется вектор $v = [v_0, v_1, \dots, v_{n-1}]$, называемый «Catalan cipher vector» [24].

Кроме того, достаточно популярной является тематика генерации дискретных структур, представляющих собой последовательности элементов или слова (то есть последовательности символов, взятых из заданного конечного множества, называемого алфавитом). Например, в работе [25] предложены алгоритмы ранжирования и генерации по рангу с использованием лексикографического упорядочивания для комбинаторного множества беспорядков, где перестановка n элементов представлена в нотации циклов. В работе [26] решена аналогичная задача комбинаторной генерации, но с использованием представления перестановки n элементов в виде слова длины n . В работе [27] разработаны алгоритмы ранжирования и генерации по рангу с использованием лексикографического упорядочивания для комбинаторного множества слов длины n , составленных из алфавита k символов и имеющих одинаковые префикс и суффикс. Данные алгоритмы комбинаторной генерации были модифицированы в работе [28], что позволило улучшить показатели вычислительной сложности. В работе [29] исследуются алгоритмы ранжирования и генерации по рангу для комбинаторного множества подслов длины k , полученных из слов длины n с использованием алфавита σ символов.

Также стоит отметить распространенность научных исследований, связанных с изучением различных вариантов упорядочивания элементов комбинаторных множеств. Например, наиболее известным и простым для понимания является лексикографическое упорядочивание: в работе [30] исследуются перечислительные задачи, образующие «Двенадцатеричный путь» [31], и для каждой из них рассмотрен алгоритм генерации по рангу с использованием лексикографического упорядочивания. Однако использование лексикографического порядка зачастую не является оптимальным решением с точки зрения затрачиваемых ресурсов. Например, при реализации последовательной генерации в лексикографическом порядке могут наблюдаться значительные изменения между соседними элементами комбинаторного множества. Эту проблему можно решить за счет использования упорядочивания кода Грея, которое сводит к минимуму изменения элементов

комбинаторного множества относительно их предшественников. Так, в работе [32] разработаны алгоритмы ранжирования и генерации по рангу с использованием упорядочивания кода Грея для комбинаторного множества остовных деревьев одного класса графовых структур данных. Предложенные алгоритмы комбинаторной генерации имеют линейную временную сложность и характеризуются изменением двух ребер, связанных с одной из n вершин графа, при переходе от одного остовного дерева графа к другому. В работе [33] разработаны алгоритмы ранжирования и генерации по рангу с использованием упорядочивания кода Грея для комбинаторного множества последовательностей вида $(a_1, \dots, a_n)$, где каждый элемент a_i принадлежит множеству $\{0, 1, \dots, m - 1\}$.

Помимо решения общих теоретических задач существуют научные исследования, направленные на решение прикладных задач с использованием идей комбинаторной генерации. Например, в работе [34] идеи комбинаторного анализа нашли свое применение в оптимизации процессов генерации органоидов, полученных из стволовых клеток человека. В работе [35] методы комбинаторной генерации легли в основу алгоритма и программной реализации, позволяющей совершать атаки на парольные системы с использованием словарей скомпрометированных паролей. В работе [36] показан метод генерации фрагментов цифровой гиперплоскости, основываясь на полученных результатах применения обобщенного алгоритма Евклида.

Также особого внимания заслуживает тот факт, что разработка алгоритмов комбинаторной генерации для общих математических структур является предметом многочисленных исследований на протяжении последних десятилетий. Простым примером такой классической математической структуры с продолжительной историей в области разработки алгоритмов комбинаторной генерации являются перестановки. Перестановка n элементов ($n \geq 0$) — это последовательность (упорядоченный набор) всех элементов, принадлежащих множеству n элементов. Таким образом, перестановка задает некоторое упорядочивание элементов исходного множества и имеется ровно $n!$ различных вариантов перестановки n элементов. Появление языков программирования высокого уровня стимулировало разработку многочисленных алгоритмов последовательной генерации для комбинаторного множества перестановок [37–39]. В работе [40] представлен подробный анализ эффективности программных реализаций существующих на тот момент алгоритмов. Несмотря на наличие алгоритмов для последовательной генерации перестановок, которые эффективны с точки зрения временной

сложности и по затратам памяти, не теряет своей актуальности разработка новых алгоритмов, отличающихся использованием оригинальных подходов к представлению перестановок или способов их преобразования [41–47]. Также существуют исследования, решающие задачи ранжирования и генерации по рангу для комбинаторного множества перестановок [48–53]. Кроме того, решаются задачи разработки алгоритмов комбинаторной генерации для более сложных типов перестановок элементов, учитывающих дополнительные ограничения, например: перестановки с заданным количеством инверсий элементов [54], перестановки с заданным расположением подъемов [55–57], перестановки с заданным количеством циклов [58], перестановки без неподвижных точек (беспорядки) [25; 26; 59; 60].

Другим примером хорошо изученной дискретной структуры являются сочетания элементов множества. Сочетание из n элементов по m ($0 \leq m \leq n$) — это подмножество m различных элементов выбранных из множества n элементов. Для комбинаторного множества сочетаний существует большое количество алгоритмов комбинаторной генерации, которые отличаются типом реализуемого алгоритма, способом упорядочивания, а также наличием дополнительных ограничений. Так, в работе [61] исследуется быстроедействие целого ряда алгоритмов последовательной генерации для комбинаторного множества сочетаний, которые были разработаны до 1980 года. При этом наилучшие показатели быстрогодействия показал алгоритм, предложенный в [62]. Следующим этапом в данном направлении стала разработка алгоритмов последовательной генерации для комбинаторного множества сочетаний с улучшенными показателями быстрогодействия за счет реализации параллельных вычислений [63–72]. Такие алгоритмы эффективны с точки зрения оценки временной сложности за счет разделения вычислений, а также за счет хранения в памяти промежуточных результатов. При этом эффективный с точки зрения требуемой памяти алгоритм последовательной генерации для комбинаторного множества сочетаний представлен в работе [73]. Дополнительно существуют исследования, направленные на последовательную генерацию сочетаний с использованием различных видов упорядочивания элементов комбинаторного множества (например, алгоритмы из [74–77]). Кроме разработки алгоритмов последовательной генерации, также известно большое количество исследований, направленных на разработку алгоритмов генерации по рангу для комбинаторного множества сочетаний [78–85]. Обзор характеристик данных алгоритмов комбинаторной генерации представлен в работе [86].

Еще одним примером классической математической структуры, широко используемой в области дискретной математики и теоретической информатики, являются древовидные структуры данных. Наиболее изученной древовидной структурой данных с точки зрения разработки алгоритмов комбинаторной генерации являются полные бинарные (или двоичные) деревья. Полное бинарное дерево — это дерево, в котором каждый внутренний узел имеет ровно два узла-потомка. Наличие большого количества исследований в области разработки алгоритмов комбинаторной генерации для множества полных бинарных деревьев обусловлено их связью с числами Каталана [87]: число Каталана C_n определяет количество всех возможных полных бинарных деревьев, состоящих из n внутренних узлов. Так, в работе [88] предложен алгоритм последовательной генерации для комбинаторного множества полных бинарных деревьев, который не содержит циклов и обрабатывает древовидную структуру данных с использованием указателей на узлы-потомки. Также широко используется подход к кодированию структур бинарных деревьев в виде различных числовых последовательностей. Например, в работе [89] представлены результаты разработки алгоритмов последовательной генерации для комбинаторного множества бинарных деревьев, представленных последовательностями значений весов узлов дерева с разными вариантами их обхода (w -последовательности, предложенные в [90]). В работе [91] введен новый способ кодирования бинарных деревьев в виде LC-последовательностей (RC-последовательностей), в основе которых лежит указание метки левого (правого) узла-потомка для каждого узла, а также предложен алгоритм последовательной генерации таких последовательностей. Последующая оптимизация алгоритмов комбинаторной генерации для LC-последовательностей и RC-последовательностей представлена в работе [92]. Существуют и другие варианты представления структуры бинарного дерева, применяемые при разработке алгоритмов комбинаторной генерации, например: последовательность значений уровней листьев бинарного дерева [93], двоичная последовательность (аналогично способу представления правильных скобочных последовательностей) [94; 95], z -последовательность [96], последовательность «codeword» для двоичной последовательности [97]. Кроме того, известно большое количество разработанных алгоритмов ранжирования и генерации по рангу для комбинаторного множества полных бинарных деревьев [88; 90; 92; 93; 96]. Разработки, связанные с исследованием полных бинарных деревьев, легли в основу алгоритмов комбинаторной генерации для множества t -арных деревьев, то есть деревьев с t узлами-потомками для каждого внутреннего

узла [96; 98–103]. Также существуют алгоритмы комбинаторной генерации для других классов древовидных структур, таких как (k, m) -арные деревья [22; 104; 105], АВЛ-деревья [106], B -деревья [107], деревья с заданным количеством внутренних и конечных узлов [108; 109] и другие.

Таким образом, можно сделать вывод о существовании комплексной теоретико-алгоритмической базы по решению проблем комбинаторной генерации для классических математических дискретных структур. В то же время сохраняется актуальность разработки новых алгоритмов, которые совершенствуют или обобщают существующие решения, либо отличаются оригинальными идеями по обработке дискретных структур. Однако все предлагаемые идеи носят частный характер и могут быть использованы только для конкретного вида дискретных структур. Следовательно, результаты таких научных исследований невозможно напрямую использовать при разработке алгоритмов комбинаторной генерации для других классов дискретных структур. Тогда исследователь сталкивается с разработкой принципиально новых алгоритмов, не опирающихся на существующие решения, что является нетривиальной задачей. В целях решения указанной проблемой в области комбинаторной генерации разрабатываются теории, направленные на создание универсальных подходов к построению новых алгоритмов без жесткой привязки к виду обрабатываемой дискретной структуры. Далее представлены существующие на сегодняшний день методы, предназначенные для разработки новых алгоритмов комбинаторной генерации и характеризующиеся некоторой степенью универсальности их применения:

1. Метод поиска с возвратом: описание данного метода представлено в работе [110]. Метод носит общий характер и находит применение при решении широкого круга поисковых задач, так как представляет собой вариацию полного перебора. Основная идея метода поиска с возвратом заключается в переборе возможных путей расширения текущего частичного решения задачи размера i до решения задачи размера $i + 1$. Если расширение текущего частичного решения задачи размера i не является возможным, то проверяется либо другое частичное решение задачи размера i , либо происходит возврат к задаче размера $i - 1$ и продолжается поиск альтернативных ее решений. При этом получение решения задачи искомого размера n является фактом генерации очередного элемента рассматриваемого комбинаторного множества. Как результат, реализация полного перебора описанным образом позволяет последовательно сгенерировать исчерпывающий перечень комбинаторных объектов заданного размера. Также

в работе [14] предложена оптимизация метода поиска с возвратом, заключающаяся во введении ограничивающих функций, которые позволяют заблаговременно отсекалть бесперспективные ветви генерирующего дерева;

2. ЕСО-метод: описание данного метода представлено в цикле работ [16; 111–113]. Подобно методу поиска с возвратом, основная идея ЕСО-метода заключается в организации перехода от комбинаторного объекта размера i к комбинаторному объекту размера $i + 1$. При этом для перечисления исходного комбинаторного множества используется формализм ЕСО-правил [16], на основе которых формируются итоговые алгоритмы комбинаторной генерации. В частности, в работе [111] описана идея применения ЕСО-правил для реализации случайной генерации, а в работе [112] показано применение ЕСО-правил для реализации последовательной генерации. Впоследствии ЕСО-метод нашел применение при решении задач исчерпывающей генерации для различных классов комбинаторных множеств перестановок [114–121];

3. Метод на основе перестановок: описание данного метода представлено в работе [17]. Основная идея метода заключается в представлении рассматриваемых комбинаторных объектов в виде подмножества комбинаторного множества перестановок. Следовательно, требуется промежуточное представление комбинаторного объекта в виде перестановки. При этом последовательная генерация перестановок осуществляется путем реализации переходов от текущей перестановки к последующей. Основным отличием данного метода от ЕСО-метода является реализация упорядочивания с минимальным количеством изменений в последующей перестановке относительно текущей. Впоследствии метод на основе перестановок нашел применение при решении задач исчерпывающей генерации для различных классов комбинаторных множеств с биективным отображением на подмножества комбинаторного множества перестановок: алгебраические структуры решеток [122], разбиения прямоугольника на прямоугольники [123], деревья исключения [124], ациклические ориентации [125], бинарные деревья [126];

4. Символьный метод: описание данного метода представлено в цикле работ [127–130]. Основная идея символьного метода заключается в декомпозиции производящей функции для последовательности значений мощности комбинаторного множества, при этом декомпозиция должна состоять только из разрешенных операторов. Также данный метод работает только с производящими функциями одной переменной, то есть обрабатывает только комбинаторные объекты с одним количественным параметром (например, размер комбинаторного объек-

та). В частности, в работе [127] описана идея применения символьного метода для реализации случайной генерации, а в работе [129] показано применение символьного метода для реализации последовательной генерации. Кроме того, данный метод позволяет получать алгоритмы генерации по рангу [128], что подтверждается разработанными алгоритмами и их программными реализациями, представленными в [131; 132]. Однако решение обратной задачи ранжирования с использованием символьного метода на текущий момент не представлено;

5. Метод на основе префиксов слов: описание данного метода представлено в работе [133]. Основная идея метода заключается в представлении рассматриваемых комбинаторных объектов в виде подмножества комбинаторного множества слов для заданного алфавита символов. При этом полученное слово кодируется в виде числа, представляющего собой порядковый номер в рамках лексикографического упорядочивания всех допустимых слов, то есть реализуется алгоритм ранжирования. Также представлен процесс обратного преобразования (декодирования), то есть реализация алгоритма генерации по рангу. Однако для работы этих алгоритмов комбинаторной генерации требуется решение сопутствующих перечислительных задач, а именно подсчет количества слов с заданным префиксом. Примеры успешной апробации метода на основе префиксов слов для получения алгоритмов комбинаторной генерации представлены в работах [134–136];

6. Метод на основе деревьев И/ИЛИ: описание данного метода представлено в работах [137; 138]. Основная идея метода заключается в представлении рассматриваемых комбинаторных объектов в виде множества вариантов структуры дерева И/ИЛИ. В свою очередь, это позволяет выполнить обход структуры дерева И/ИЛИ с целью последовательного получения всех его вариантов, то есть реализуется алгоритм последовательной генерации, а также пронумеровать каждый вариант по отдельности (алгоритм ранжирования) и построить вариант дерева И/ИЛИ с заданным номером (алгоритм генерации по рангу). Важно отметить, что для построения структуры дерева И/ИЛИ требуется наличие формулы для вычисления мощности комбинаторного множества, которая должна состоять только из разрешенных математических операций. В случае отсутствия такой формулы можно воспользоваться методами получения явных формул для коэффициентов производящих функций [139; 140], в том числе для производящих функций нескольких переменных [141; 142].

В таблице 1.1 представлена совокупность возможностей и требований указанных выше универсальных методов, предназначенных для разработки

новых алгоритмов комбинаторной генерации. Рассмотрим наименования столбцов таблицы 1.1 и использованные в них обозначения:

– Столбцы «Listing», «Random selection», «Ranking» и «Unranking»:

«+»: метод содержит описание процесса построения алгоритма комбинаторной генерации соответствующего типа;

«–»: метод не содержит описание процесса построения алгоритма комбинаторной генерации соответствующего типа;

«±»: метод не содержит описание процесса построения алгоритма комбинаторной генерации соответствующего типа, но он может быть реализован на основе другого типа алгоритма;

– Столбец «Функция мощности»:

«+»: метод требует наличия формулы для вычисления мощности комбинаторного множества;

«–»: метод не требует наличия формулы для вычисления мощности комбинаторного множества;

– Столбец «Требуемая структура»: указана вспомогательная структура, которую необходимо построить для заданного комбинаторного множества или объекта прежде чем приступить к построению алгоритмов комбинаторной генерации.

Таблица 1.1 — Сравнение универсальных методов по разработке алгоритмов комбинаторной генерации

Метод	Listing	Random selection	Ranking	Unranking	Функция мощности	Требуемая структура
Метод поиска с возвратом	+	–	–	–	–	–
ЕСО-метод	+	+	–	–	–	ЕСО-правило
Метод на основе перестановок	+	–	–	–	–	Перестановка
Символьный метод	+	+	–	+	+	Производящая функция
Метод на основе префиксов слов	±	±	+	+	+	Слово
Метод на основе деревьев И/ИЛИ	+	±	+	+	+	Дерево И/ИЛИ

Таким образом, для разработки новых методов кодирования информации первые три метода (метод поиска с возвратом, ЕСО-метод и метод на основе перестановок) не являются применимыми, поскольку они предназначены исключительно для разработки алгоритмов последовательной генерации. Несмотря на возможность реализации алгоритма генерации по рангу, символьный метод также неприменим для кодирования информации из-за отсутствия обратного преобразования — алгоритм ранжирования. Кроме того, данный метод позволяет учитывать только один параметр для комбинаторных объектов (например, размер объекта), что ограничивает рассмотрение более сложных дискретных структур.

Оставшиеся два метода (на основе префиксов слов и на основе деревьев И/ИЛИ) позволяют реализовать процесс кодирования информации, а также оба требуют наличия функции мощности и вспомогательной структуры. Однако в рамках диссертационного исследования предпочтение отдается методу на основе деревьев И/ИЛИ, так как:

- построение структуры дерева И/ИЛИ строго определено для функций мощности, удовлетворяющих требованиям метода;
- решение задачи биективного отображения между комбинаторным множеством и множеством вариантов дерева И/ИЛИ часто является прямым следствием комбинаторного доказательства формулы для вычисления мощности.

Если же использовать метод на основе префиксов слов, то он не содержит каких-либо рекомендаций относительно решения задачи биективного отображения между комбинаторным множеством и подмножеством комбинаторного множества слов. В связи с этим такая задача должна решаться индивидуально для каждого конкретного комбинаторного множества и без опоры на существующие аналоги решения. Кроме того, применение данного метода требует решения проблемы подсчета количества слов с заданным префиксом, что является отдельной задачей перечислительной комбинаторики и предполагает предварительные исследования.

Следовательно, метод на основе деревьев И/ИЛИ представляет собой более подходящее на текущий момент решение, пригодное для задач кодирования информации. Однако основное ограничение данного метода заключается в требовании наличия формулы для вычисления мощности исходного комбинаторного множества, которая, в свою очередь, должна удовлетворять ряду условий. Расширение алгебры функций, используемых для вычисления мощности комбинаторных множеств, позволит разрабатывать алгоритмы комбинаторной генерации для более широких классов дискретных структур.

1.4 Связь производящих функций с комбинаторными множествами

Рассмотрим произвольный класс дискретных структур A . Обозначим через A_n подмножество комбинаторных объектов с заданным значением n некоторого количественного параметра (например, размер объекта), то есть $A_n \subseteq A$. Следовательно, получаем класс дискретных структур A , который объединяет в себе комбинаторные множества A_n для всех $n \geq 0$, то есть

$$\bigcup_{n \geq 0} A_n = A,$$

причем $A_i \cap A_j = \emptyset$ для $i \neq j$. Если рассмотреть возрастающую последовательность значений параметра n , где $n \geq 0$, то получим следующую числовую последовательность для мощностей комбинаторных множеств A_n :

$$|A_0|, \quad |A_1|, \quad |A_2|, \quad |A_3|, \quad \dots$$

Полученной числовой последовательности можно сопоставить производящую функцию вида:

$$|A_0| + |A_1| x + |A_2| x^2 + |A_3| x^3 + \dots = \sum_{n \geq 0} |A_n| x^n = A(x).$$

Таким образом, с помощью производящей функции $A(x)$ задается последовательность значений мощностей для целого класса дискретных структур.

Определение 1.5. Обыкновенной производящей функцией (или производящей функцией) произвольной числовой последовательности $(a_n)_{n \geq 0} = (a_0, a_1, a_2, a_3, \dots)$ называется формальный степенной ряд вида

$$A(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3 + \dots = \sum_{n \geq 0} a_n x^n.$$

Далее прописными латинскими буквами с переменной x внутри круглых скобок $(A(x), B(x), C(x), \dots)$ будем обозначать производящие функции, а их коэффициенты при x^n — строчными вариантами написания букв с параметром n в виде нижнего индекса $(a_n, b_n, c_n, \dots)$. Альтернативно для обозначения коэффициентов производящей функции может использоваться запись с круглыми скобками, то есть $a(n) = a_n$ для производящей функции $A(x)$.

Заметим, что, в отличие от степенных рядов математического анализа, в формальных степенных рядах переменной x не присваиваются числовые значения и, как следствие, не рассматривается вопрос сходимости. В данном случае переменная требуется для определения позиции конкретного значения числовой последовательности. Также широко используется оператор квадратных скобок $[x^n]A(x)$, который возвращает значение коэффициента при x^n для формального степенного ряда $A(x)$, то есть

$$[x^n]A(x) = [x^n] \left(\sum_{k \geq 0} a_k x^k \right) = a_n.$$

Производящие функции являются эффективным инструментом для решения задач в различных разделах математики, таких как комбинаторика, математический анализ и статистика. В частности, методы теории производящих функций активно применяются в комбинаторике, поскольку они позволяют получить компактное представление дискретных структур (используя одну функцию определяется мощность бесконечного числа комбинаторных множеств). Значительный вклад в развитие методов теории производящих функций и их применение для решения комбинаторных задач внесли J. Riordan [143], L. Comtet [144], P. Flajolet и R. Sedgewick [145], H.S. Wilf [146], R. Stanley [147], Г.П. Егорычев [148], О.В. Кузьмин [149], С.К. Ландо [150] и другие ученые.

Кроме того, методы теории производящих функций активно используются при изучении свойств полиномов. В таком случае производящие функции позволяют также компактно представить целые семейства полиномов (коэффициент при степени формальной переменной является частным случаем полинома при заданных значениях его параметров). Фундаментальные исследования в области специальных полиномов и связанных с ними производящих функций представлены в научных монографиях таких авторов, как R.P. Voas и R.C. Buck [151], S. Roman [152], H.M. Srivastava и H.L. Manocha [153].

Таким образом, с помощью производящих функций удобно описывать множества информационных объектов. При этом особое внимание уделяется задаче нахождения формул для вычисления коэффициентов производящих функций, так как они позволяют получить информацию о частном конкретном информационном объекте с заданными параметрами. Одним из возможных путей решения данной задачи является использование математического аппарата композит производящих функций [154–157], который позволяет получать явные и

рекуррентные формулы для коэффициентов производящей функции на основе декомпозиции ее выражения.

Также заслуживает внимания изучение того, как операции над производящими функциями влияют на связанные с ними множества информационных объектов. Если рассмотреть комбинаторные множества A_n и B_n , которые содержат комбинаторные объекты размера n (или любого другого количественного параметра) в количестве $|A_n|$ и $|B_n|$ соответственно, то на основе последовательностей значений $|A_n|$ и $|B_n|$ при разных n можно задать следующие производящие функции с коэффициентами $a(n)$ и $b(n)$:

$$\sum_{n \geq 0} |A_n| x^n = \sum_{n \geq 0} a(n) x^n = A(x),$$

$$\sum_{n \geq 0} |B_n| x^n = \sum_{n \geq 0} b(n) x^n = B(x).$$

Выполнение операций над производящими функциями $A(x)$ и $B(x)$ формирует новые комбинаторные множества, связанные с элементами комбинаторных множеств A_n и B_n :

— если применить операцию сложения, то получаем следующую производящую функцию:

$$A(x) + B(x) = H(x) = \sum_{n \geq 0} h(n) x^n.$$

Из теории производящих функций следует, что коэффициенты производящей функции $H(x)$ определяются по формуле

$$h(n) = a(n) + b(n).$$

Значит, производящая функция $H(x)$ задает последовательность значений мощности комбинаторных множеств H_n ($n \geq 0$), элементы которых формируются путем дизъюнктивного объединения комбинаторных множеств A_n и B_n , то есть

$$H_n = A_n \sqcup B_n.$$

Таким образом, H_n содержит комбинаторные объекты размера n , каждый из которых соответствует комбинаторному объекту, принадлежащему либо A_n , либо B_n . При этом для вычисления мощности комбинаторного множества H_n справедлива формула

$$|H_n| = |A_n| + |B_n|. \quad (1.1)$$

– если применить операцию вычитания, то получаем следующую производящую функцию:

$$A(x) - B(x) = H(x) = \sum_{n \geq 0} h(n) x^n.$$

Из теории производящих функций следует, что коэффициенты производящей функции $H(x)$ определяются по формуле

$$h(n) = a(n) - b(n).$$

Значит, производящая функция $H(x)$ задает последовательность значений мощности комбинаторных множеств H_n ($n \geq 0$), элементы которых формируются путем разности комбинаторных множеств A_n и B_n , то есть

$$H_n = A_n \setminus B_n.$$

Однако это справедливо только при условии, что комбинаторные множества A_n и B_n содержат объекты одного класса дискретных структур и B_n является подмножеством A_n , то есть $B_n \subseteq A_n$. Таким образом, H_n содержит комбинаторные объекты размера n , каждый из которых соответствует комбинаторному объекту, принадлежащему A_n и не принадлежащему B_n . Тогда для вычисления мощности комбинаторного множества H_n справедлива формула

$$|H_n| = |A_n| - |B_n|. \quad (1.2)$$

– если применить операцию умножения, то получаем следующую производящую функцию:

$$A(x) \cdot B(x) = H(x) = \sum_{n \geq 0} h(n) x^n.$$

Из теории производящих функций следует, что коэффициенты производящей функции $H(x)$ определяются по формуле

$$h(n) = \sum_{i=0}^n a(i) \cdot b(n-i).$$

Значит, производящая функция $H(x)$ задает последовательность значений мощности комбинаторных множеств H_n ($n \geq 0$), элементы которых формируются путем декартова произведения комбинаторных множеств A_i и B_{n-i} для всех $i \in \{0, 1, \dots, n\}$, то есть

$$H_n = (A_0 \times B_n) \cup (A_1 \times B_{n-1}) \cup \dots \cup (A_{n-1} \times B_1) \cup (A_n \times B_0) = \bigcup_{i=0}^n A_i \times B_{n-i}.$$

Таким образом, H_n содержит комбинаторные объекты, представляющие собой упорядоченные пары вида (a, b) , где $a \in A_i$ и $b \in B_{n-i}$ ($i \in \{0, 1, \dots, n\}$), так что суммарный размер комбинаторных объектов a и b равен n . При этом для вычисления мощности комбинаторного множества H_n справедлива формула

$$|H_n| = \sum_{i=0}^n |A_i| \cdot |B_{n-i}|. \quad (1.3)$$

— если применить операцию возведения в k -ю степень, то получаем следующую производящую функцию:

$$A(x)^k = H_k(x) = \sum_{n \geq 0} h(n, k) x^n.$$

Из рекуррентной формулы

$$A(x)^k = A(x) \cdot A(x)^{k-1}, \quad A(x)^0 = 1$$

следует, что коэффициенты производящей функции $H_k(x)$ для $n, k \geq 0$ определяются по формуле

$$h(n, k) = \begin{cases} \delta(n, k), & k = 0; \\ \sum_{i=0}^n a(i) \cdot h(n-i, k-1), & 0 < k \leq n, \end{cases}$$

где $\delta(i, j)$ — символ Кронекера, который равен 1 при $i = j$ и равен 0 при $i \neq j$. Значит, производящая функция $H_k(x)$ задает последовательность значений мощности комбинаторных множеств $H_{n,k}$ ($n, k \geq 0$), элементы которых формируются путем декартова произведения k различных комбинаторных множеств A_i ($i \in \{0, 1, \dots, n\}$), так что суммарное значение индексов равно n , то есть

$$H_{n,k} = \bigcup_{i_1 + \dots + i_k = n} A_{i_1} \times \dots \times A_{i_k}.$$

Таким образом, $H_{n,k}$ содержит комбинаторные объекты, представляющие собой кортежи вида $(a_1, \dots, a_k)$, где $a_j \in A_{i_j}$ ($j \in \{1, \dots, k\}$), так что суммарный размер комбинаторных объектов a_j равен n . При этом для вычисления мощности комбинаторного множества $H_{n,k}$ для $n, k \geq 0$ справедлива формула

$$|H_{n,k}| = \begin{cases} \delta(n, k), & k = 0; \\ \sum_{i=0}^n |A_i| \cdot |H_{n-i, k-1}|, & 0 < k \leq n. \end{cases} \quad (1.4)$$

— если применить операцию композиции, то получаем следующую производящую функцию:

$$A(x) \circ B(x) = A(B(x)) = H(x) = \sum_{n \geq 0} h(n) x^n.$$

Данная операция над производящими функциями определена только для случая, когда $b(0) = 0$. Однако комбинаторным множествам свойственно использование пустого объекта для случая объекта размера $n = 0$, то есть $b(0) = 1$. Тогда для получения требуемого вида нужно преобразовать производящую функцию путем вычитания ее свободного коэффициента, то есть $B(x) := B(x) - b(0)$, либо путем домножения на переменную x , то есть $B(x) := x \cdot B(x)$. Выполнив подстановку $B(x)$ вместо переменной x в записи формального степенного ряда для $A(x)$, получаем

$$A(B(x)) = \sum_{i \geq 0} a(i) B(x)^i = \sum_{k \geq 0} a(k) \sum_{n \geq 0} B^\Delta(n, k) x^n = \sum_{n \geq 0} \sum_{k \geq 0} a(k) B^\Delta(n, k) x^n,$$

где $B^\Delta(n, k)$ представляет собой коэффициент при x^n для k -й степени производящей функции $B(x)$, для которой $b(0) = 0$, то есть $B^\Delta(n, k) = [x^n]B(x)^k$. Функция коэффициентов $B^\Delta(n, k)$ называется композитой производящей функции $B(x)$, причем $B^\Delta(n, k) = \delta(n, k)$ при $k = 0$ и $B^\Delta(n, k) = 0$ при $k > n$ [154; 155]. Следовательно, коэффициенты производящей функции $H(x)$ определяются по формуле

$$h(n) = \sum_{k=0}^n a(k) \cdot B^\Delta(n, k).$$

Значит, производящая функция $H(x)$ задает последовательность значений мощности комбинаторных множеств H_n ($n \geq 0$), элементы которых формируются путем декартова произведения комбинаторных множеств A_k и $B_{n,k}$ для всех $k \in \{0, 1, \dots, n\}$, то есть

$$H_n = (A_0 \times B_{n,0}) \cup (A_1 \times B_{n,1}) \cup \dots \cup (A_{n-1} \times B_{n,n-1}) \cup (A_n \times B_{n,n}) = \bigcup_{k=0}^n A_k \times B_{n,k}.$$

Таким образом, H_n содержит комбинаторные объекты, представляющие собой упорядоченные пары вида (a, b) , где $a \in A_k$ и $b \in B_{n,k}$ ($k \in \{0, 1, \dots, n\}$). При этом комбинаторные множества $B_{n,k}$ формируются в результате применения операции возведения производящей функции $B(x)$ в k -ю степень, то есть для случая $k > 0$ множество $B_{n,k}$ содержит комбинаторные объекты в виде кортежей

$(b_1, \dots, b_k)$, где $b_j \in B_j$ ($j \in \{1, \dots, k\}$). Также заметим, что размер объекта $a \in A_k$ соответствует количеству объектов b_j в кортеже $(b_1, \dots, b_k)$, что позволяет комбинировать свойства двух различных дискретных структур. Например, использование операции композиции производящих функций и правил вычисления их композит позволило получить новые результаты в следующих областях: явные формулы для специальных полиномов [158–160], явные формулы для специальных чисел [161; 162], критерии простоты числа [163–167]. В результате получаем, что для вычисления мощности комбинаторного множества H_n справедлива формула

$$|H_n| = \sum_{k=0}^n |A_k| \cdot |B_{n,k}|, \quad (1.5)$$

где на основании формулы (1.4) для $n, k \geq 0$ получаем

$$|B_{n,k}| = \begin{cases} \delta(n,k), & k = 0; \\ \sum_{i=0}^n |B_i| \cdot |B_{n-i,k-1}|, & 0 < k \leq n. \end{cases}$$

Исходя из вышеизложенного, производящие функции представляют собой дополнительный инструмент для изучения комбинаторных множеств с точки зрения их перечисления. Так, производящие функции помогают решать задачи перечислительной комбинаторики, потому что их коэффициенты показывают количество комбинаторных объектов в множестве с заданным значением параметра. Кроме того, декомпозиция производящей функции показывает взаимосвязь исследуемого комбинаторного множества с более простыми дискретными структурами.

Однако существуют более сложные дискретные структуры, которые определяются не только одним количественным параметром. Тогда для описания значений мощности множеств таких структур применимы обобщения обыкновенных производящих функций на случай нескольких формальных переменных [168; 169]. В частности, если комбинаторные объекты определяются двумя количественными параметрами n и m , то есть рассматривается комбинаторное множество $A_{n,m}$, тогда для числовой последовательности значений мощности комбинаторных множеств $A_{n,m}$ можно сопоставить производящую функцию вида:

$$|A_{0,0}| + |A_{1,0}|x + |A_{0,1}|y + |A_{1,1}|xy + \dots = \sum_{n \geq 0} \sum_{m \geq 0} |A_{n,m}| x^n y^m = A(x,y).$$

Определение 1.6. *Производящей функцией многих переменных называется формальный степенной ряд вида*

$$A(x, y, \dots, z) = \sum_{n \geq 0} \sum_{m \geq 0} \dots \sum_{l \geq 0} a_{n, m, \dots, l} x^n y^m \dots z^l.$$

В общем случае, если комбинаторные объекты определяются набором количественных параметров $n, m, \dots, l$, то есть рассматривается комбинаторное множество $A_{n, m, \dots, l}$, тогда для числовой последовательности значений мощности комбинаторных множеств $A_{n, m, \dots, l}$ можно сопоставить производящую функцию многих переменных:

$$\begin{aligned} & \sum_{n \geq 0} \sum_{m \geq 0} \dots \sum_{l \geq 0} |A_{n, m, \dots, l}| x^n y^m \dots z^l = \\ & = \sum_{n \geq 0} \sum_{m \geq 0} \dots \sum_{l \geq 0} a_{n, m, \dots, l} x^n y^m \dots z^l = A(x, y, \dots, z). \end{aligned}$$

Аналогично случаю обыкновенных производящих функций, выполнение операций над производящими функциями многих переменных, связанными с комбинаторными множествами, также формируют новые комбинаторные множества.

1.5 Выводы по главе

В данной главе введены основные понятия диссертационного исследования, касающиеся разработки алгоритмов комбинаторной генерации и их применения для решения задач кодирования информации. В частности, показана возможность представления множеств информационных объектов, обрабатываемых цифровыми устройствами, в форме комбинаторных множеств. В свою очередь, это позволяет использовать классы алгоритмов комбинаторной генерации, такие как ранжирование и генерация по рангу, для организации процесса кодирования информационных объектов, формирующих комбинаторное множество.

Проведенный аналитический обзор современного состояния исследований в области комбинаторной генерации показал применимость соответствующих методов для решения задач кодирования информации. Как результат, в данной главе представлена общая схема кодирования на основе алгоритмов комбинаторной генерации для информационных объектов, представляющих собой сложные

дискретные структуры. Однако существующая теоретико-алгоритмическая база в области комбинаторной генерации носит частный характер, то есть предлагаемые решения направлены только на конкретные дискретные структуры и не могут быть обобщены на другие. Следовательно, это не может быть использовано в качестве основы для единой методологии построения алгоритмов кодирования информационных объектов и требуются универсальные решения.

Сравнительный анализ существующих универсальных методов, предназначенных для разработки алгоритмов комбинаторной генерации, показал преимущество метода на основе деревьев И/ИЛИ для решения задач кодирования информации. Однако основное ограничение данного метода заключается в требовании наличия формулы для вычисления мощности исходного комбинаторного множества, которая также должна удовлетворять ряду условий. Расширение алгебры функций, используемых для вычисления мощности комбинаторных множеств, позволит разрабатывать алгоритмы комбинаторной генерации для более широких классов дискретных структур. В свою очередь, решение данной задачи позволит организовать процесс кодирования для информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ.

Кроме того, в данной главе рассмотрена связь производящих функций с комбинаторными множествами и их функциями мощности, а также исследовано влияние операций над производящими функциями на формирование новых дискретных структур. Дальнейшее изучение этой связи позволит комбинировать существующие алгоритмы комбинаторной генерации для сложных дискретных структур на основе декомпозиции соответствующих производящих функций.

Таким образом, решение перечисленных задач позволит сформировать единую методологию построения алгоритмов кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ. Именно этому посвящены последующие главы диссертации.

Глава 2. Методология построения алгоритмов кодирования информационных объектов

В данной главе представлены основные результаты в области разработки методологии построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ. Представлено описание математического аппарата деревьев И/ИЛИ и их применение в задачах комбинаторной генерации. Предложено расширение используемой алгебры функций мощности комбинаторных множеств для построения рекурсивных композиций структур деревьев И/ИЛИ. В качестве демонстрации возможностей методологии построения алгоритмов комбинаторной генерации для разных рекурсивных композиций структур деревьев И/ИЛИ рассмотрены дискретные структуры, связанные со специальными числами Фубини.

2.1 Структуры деревьев И/ИЛИ и алгоритмы генерации их вариантов

Древовидные структуры данных широко применяются для представления информации, особенно при наличии в ней иерархических отношений.

Определение 2.1. *Дерево И/ИЛИ — древовидная структура данных, внутренние узлы которой относятся к одному из следующих типов узлов:*

- *ИЛИ-узел (узел с конечным множеством узлов-потомков, соответствует операции объединения множеств);*
- *И-узел (узел с конечным множеством узлов-потомков, ребра к которым графически объединяются дополнительной линией, соответствует операции декартова произведения множеств).*

Кроме того, структура дерева И/ИЛИ может содержать конечные узлы (листья), которые не содержат узлов-потомков. На рисунке 2.1 показано графическое изображение узлов дерева И/ИЛИ. При этом с помощью фигуры треугольника будем обозначать поддереву соответствующего узла. Отметим, что любое поддерево дерева И/ИЛИ также является деревом И/ИЛИ.

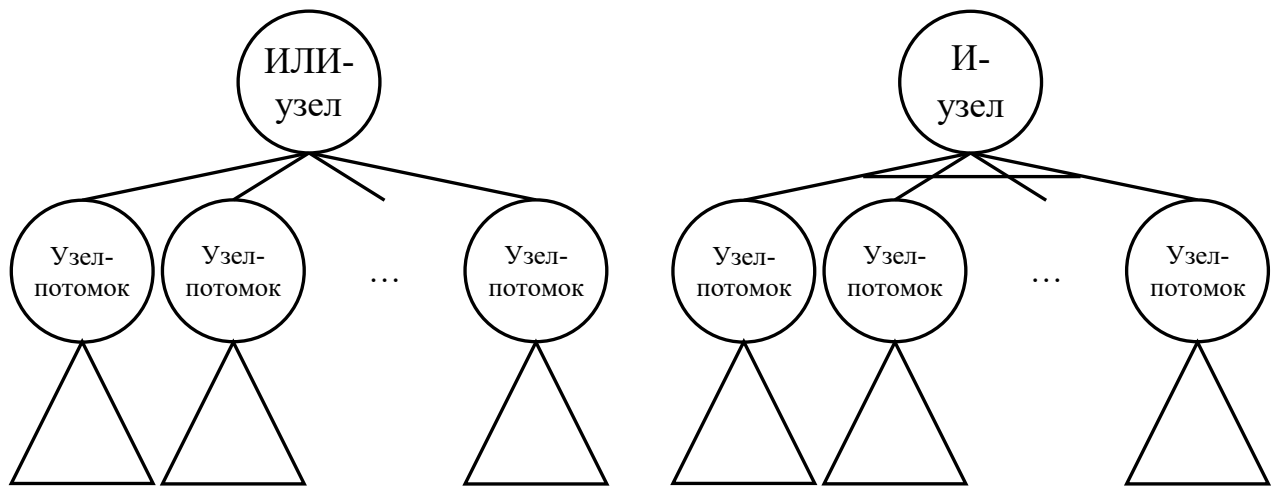


Рисунок 2.1 — Графическое изображение узлов дерева И/ИЛИ

В работах [137; 138; 170–172] введено понятие варианта дерева И/ИЛИ и предложен математический аппарат для работы с такими структурами в области разработки алгоритмов комбинаторной генерации.

Определение 2.2. *Вариант дерева И/ИЛИ — древовидная структура, полученная из исходной структуры дерева И/ИЛИ путем выбора одного узла-потомка для каждого ИЛИ-узла и удаления всех невыбранных узлов-потомков ИЛИ-узлов, включая их поддеревья.*

На рисунке 2.2 показан пример структуры дерева И/ИЛИ и все возможные варианты данного дерева И/ИЛИ. Для удобства отображения каждый узел дерева И/ИЛИ может быть помечен некоторой меткой.

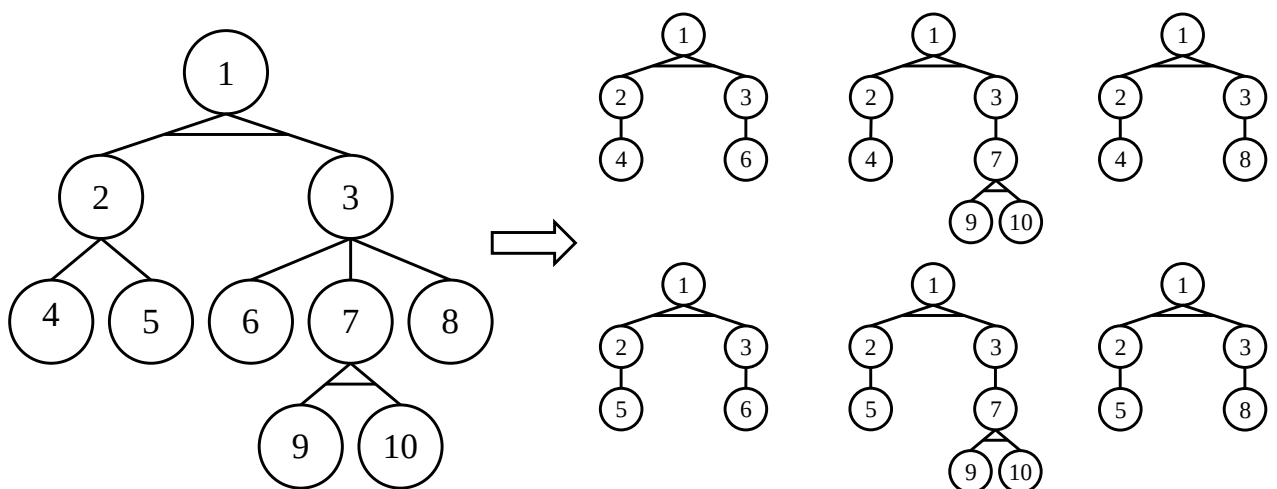


Рисунок 2.2 — Пример структуры дерева И/ИЛИ и все возможные варианты данного дерева И/ИЛИ

Таким образом, структура дерева И/ИЛИ задает конечное множество вариантов. При этом общее количество вариантов дерева И/ИЛИ определяется значением функции $w(root)$, где $root$ — корневой узел дерева И/ИЛИ. Чтобы вычислить количество вариантов в поддереве узла $node$, применяется следующая формула:

$$w(node) = \begin{cases} \prod_{i=1}^n w(child_i), & node \text{ — И-узел;} \\ \sum_{i=1}^n w(child_i), & node \text{ — ИЛИ-узел;} \\ 1, & node \text{ — лист.} \end{cases}$$

где $\{child_1, \dots, child_n\}$ — множество узлов-потомков узла $node$.

Если зафиксировать порядок узлов-потомков для каждого внутреннего узла дерева И/ИЛИ (то есть рассматривается упорядоченная структура дерева И/ИЛИ), тогда можно реализовать алгоритмы комбинаторной генерации для множества вариантов заданного дерева И/ИЛИ. Согласно оригинальной методологии [138], алгоритмы ранжирования и генерации по рангу для варианта рассматриваемой структуры дерева И/ИЛИ имеют рекурсивный характер, при этом вычисления на каждом уровне рекурсии определяются типом узла.

Например, алгоритмы 2.1 и 2.2 демонстрируют ранжирование и генерацию по рангам для узла варианта дерева И/ИЛИ, который является ИЛИ-узлом в дереве И/ИЛИ. В данных алгоритмах на вход подается структура ИЛИ-узла $node$, представленная в виде последовательности указателей на его узлы-потомки $(child_1, \dots, child_n)$. При этом структура варианта $variant$ поддерева ИЛИ-узла $node$ представляется как указатель на единственный выбранный узел-потомок $Vchild$, а запись $Vchild = child_i$ означает успешное сопоставление выбранного узла с узлом $child_i$ (например, путем проверки совпадения с уникальной меткой узла). Вычисление ранга для ИЛИ-узла основано на подсчете количества вариантов поддеревьев узлов-потомков, расположенных левее выбранного узла $Vchild$.

Алгоритмы 2.3 и 2.4 демонстрируют ранжирование и генерацию по рангам для узла варианта дерева И/ИЛИ, который является И-узлом в дереве И/ИЛИ. Вычисление ранга для И-узла основано на представлении рангов узлов-потомков в виде числа с использованием позиционной системы счисления, где количество разрядов равно количеству узлов-потомков и каждый разряд имеет свое основание, которое определяется значением количества вариантов в поддереве узла-потомка. При этом младшему разряду соответствует левый узел-потомок $child_1$.

Алгоритм 2.1: Алгоритм ранжирования варианта поддерева ИЛИ-узла

```

1 Rank_OR( $variant = (Vchild), node = (child_1, \dots, child_n)$ )
2 begin
3    $sum := 0$ 
4   for  $i := 1$  to  $n$  do           // Поиск выбранного узла  $child_I$ 
5     if  $Vchild = child_i$  then
6        $I := i$ 
7       break
8     end
9      $sum := sum + w(child_i)$ 
10  end
11   $l := \text{Rank\_Variant}(Vchild, child_I)$ 
12   $r := sum + l$ 
13  return  $r$ 
14 end

```

Алгоритм 2.2: Алгоритм генерации по рангу варианта поддерева ИЛИ-узла

```

1 Unrank_OR( $r, node = (child_1, \dots, child_n)$ )
2 begin
3    $sum := 0$ 
4   for  $i := 1$  to  $n$  do           // Поиск выбранного узла  $child_I$ 
5     if  $sum + w(child_i) > r$  then
6        $l := r - sum$ 
7        $I := i$ 
8       break
9     end
10     $sum := sum + w(child_i)$ 
11  end
12   $Vchild := \text{Unrank\_Variant}(l, child_I)$ 
13   $variant := (Vchild)$ 
14  return  $variant$ 
15 end

```

Алгоритм 2.3: Алгоритм ранжирования варианта поддерева И-узла

```

1 Rank_AND( $variant = (Vchild_1, \dots, Vchild_n), node = (child_1, \dots, child_n)$ )
2 begin
3    $r := 0$ 
4   for  $i := n$  to 1 by  $-1$  do      // Для каждого узла-потомка  $child_i$ 
5      $l_i := \text{Rank\_Variant}(Vchild_i, child_i)$ 
6      $r := l_i + w(child_i) \cdot r$ 
7   end
8   return  $r$ 
9 end

```

Алгоритм 2.4: Алгоритм генерации по рангу варианта поддерева И-узла

```

1 Unrank_AND( $r, node = (child_1, \dots, child_n)$ )
2 begin
3   for  $i := 1$  to  $n$  do      // Для каждого узла-потомка  $child_i$ 
4      $l_i := r \bmod w(child_i)$ 
5      $r := \left\lfloor \frac{r}{w(child_i)} \right\rfloor$ 
6      $Vchild_i := \text{Unrank\_Variant}(l_i, child_i)$ 
7   end
8    $variant := (Vchild_1, \dots, Vchild_n)$ 
9   return  $variant$ 
10 end

```

Отдельно рассматривается случай конечного узла дерева И/ИЛИ, то есть $node$ является листом *leaf* (алгоритмы 2.5 и 2.6). В таком случае поддерево листа является пустым, что соответствует единственному варианту поддерева листа.

Алгоритм 2.5: Алгоритм ранжирования варианта поддерева листа

```

1 Rank_LEAF( $variant = leaf, node = leaf$ )
2 begin
3    $r := 0$ 
4   return  $r$ 
5 end

```

Алгоритм 2.6: Алгоритм генерации по рангу варианта поддерева листа

```

1 Unrank_LEAF( $r, node = leaf$ )
2 begin
3    $variant := leaf$ 
4   return  $variant$ 
5 end
```

Если в структуре дерева И/ИЛИ имеется ИЛИ-узел, который содержит n узлов-потомков, каждый из которых является листом, то алгоритмы ранжирования и генерации по рангу примут следующий вид (алгоритмы 2.7 и 2.8):

Алгоритм 2.7: Алгоритм ранжирования варианта поддерева ИЛИ-узла
с листьями

```

1 Rank_NUM( $variant = (Vleaf), node = (leaf_1, \dots, leaf_n)$ )
2 begin
3   for  $i := 1$  to  $n$  do                                // Поиск выбранного узла  $leaf_I$ 
4     if  $Vleaf = leaf_i$  then
5        $I := i$ 
6       break
7     end
8   end
9    $r := I - 1$ 
10  return  $r$ 
11 end
```

Алгоритм 2.8: Алгоритм генерации по рангу варианта поддерева
ИЛИ-узла с листьями

```

1 Unrank_NUM( $r, node = (leaf_1, \dots, leaf_n)$ )
2 begin
3    $I := r + 1$ 
4    $Vleaf := leaf_I$ 
5    $variant := (Vleaf)$ 
6   return  $variant$ 
7 end
```

Кроме того, в качестве поддерева узла дерева И/ИЛИ допускается рекурсивный вызов исходной структуры дерева И/ИЛИ (алгоритмы 2.9 и 2.10). В данном случае для обеспечения остановки рекурсивных вызовов требуется указание дополнительных параметров структуры дерева И/ИЛИ, значение которых меняется на каждом уровне рекурсии. Если на текущем уровне рекурсии значение параметров соответствует одному из множества условий остановки рекурсивных вызовов, то поддеревом является конечный узел.

Алгоритм 2.9: Алгоритм ранжирования варианта поддереву узла с рекурсивным вызовом

```

1 Rank_REC(variant, node, parameters)
2 begin
3   if parameters = условие остановки рекурсии then r := 0
4   else
5     parameters* := Изменение значений параметров parameters
6     r := Rank_Variant(variant, node, parameters*)
7   end
8   return r
9 end
```

Алгоритм 2.10: Алгоритм генерации по рангу варианта поддереву узла с рекурсивным вызовом

```

1 Unrank_REC(r, node, parameters)
2 begin
3   if parameters = условие остановки рекурсии then variant := leaf
4   else
5     parameters* := Изменение значений параметров parameters
6     variant := Unrank_Variant(r, node, parameters*)
7   end
8   return variant
9 end
```

Таким образом, из представленных выше алгоритмов строятся общие алгоритмы ранжирования и генерации по рангу для варианта дерева И/ИЛИ (алгоритмы 2.11 и 2.12).

Алгоритм 2.11: Алгоритм ранжирования варианта поддерева узла

```

1 Rank_Variant(variant, node, parameters)
2 begin
3   switch Тип узла node do
4     case ИЛИ-узел do
5       return Rank_OR(variant, node, parameters)
6     case И-узел do
7       return Rank_AND(variant, node, parameters)
8     case Лист do
9       return Rank_LEAF(variant, node, parameters)
10    case ИЛИ-узел с листьями do
11      return Rank_NUM(variant, node, parameters)
12    case Рекурсивный вызов do
13      return Rank_REC(variant, node, parameters)
14   end
15 end

```

Алгоритм 2.12: Алгоритм генерации по рангу варианта поддерева узла

```

1 Unrank_Variant(r, node, parameters)
2 begin
3   switch Тип узла node do
4     case ИЛИ-узел do
5       return Unrank_OR(r, node, parameters)
6     case И-узел do
7       return Unrank_AND(r, node, parameters)
8     case Лист do
9       return Unrank_LEAF(r, node, parameters)
10    case ИЛИ-узел с листьями do
11      return Unrank_NUM(r, node, parameters)
12    case Рекурсивный вызов do
13      return Unrank_REC(r, node, parameters)
14   end
15 end

```

Также заметим, что в данном случае все алгоритмы были дополнены передачей параметров узла *variant*, так как эта информация может потребоваться при раскрытии структуры поддеревьев узлов-потомков на других уровнях рекурсии.

Алгоритмы 2.11 и 2.12 позволяют выполнить ранжирование и генерацию по рангу для множества вариантов любой структуры дерева И/ИЛИ. Если для множества вариантов дерева И/ИЛИ известно биективное отображение на множество других дискретных структур, то данные алгоритмы также могут быть использованы для их ранжирования и генерации по рангу. Следовательно, для исследуемого комбинаторного множества необходимо построить структуру дерева И/ИЛИ, количество вариантов которого совпадает с количеством объектов в комбинаторном множестве.

В работах [137; 138] показано, что если функция мощности комбинаторного множества удовлетворяет алгебре $\{+, \times, \mathbb{Z}_{>0}, R\}$, то можно построить соответствующую структуру дерева И/ИЛИ, количество вариантов которой равно мощности комбинаторного множества. Для этого необходимо выполнить следующие действия:

- каждая операция сложения $+$ из функции мощности комбинаторного множества представляется в виде ИЛИ-узла, количество узлов-потомков которого равно количеству слагаемых и каждое слагаемое определяет структуру поддерева соответствующего узла-потомка;
- каждая операция умножения $\times$ из функции мощности комбинаторного множества представляется в виде И-узла, количество узлов-потомков которого равно количеству множителей и каждый множитель определяет структуру поддерева соответствующего узла-потомка;
- каждое положительное целое число из функции мощности комбинаторного множества представляется в виде ИЛИ-узла, количество узлов-потомков которого равно значению числа и каждый узел-потомок является листом;
- каждый рекурсивный вызов из примитивно рекурсивной функции мощности комбинаторного множества представляется в виде узла, структура поддерева которого определяется исходной структурой дерева И/ИЛИ с измененными значениями его параметров.

Таким образом, описанная выше алгебра $\{+, \times, \mathbb{Z}_{>0}, R\}$ ограничивает множество допустимых функций мощности комбинаторных множеств, для которых можно построить структуру дерева И/ИЛИ и реализовать алгоритмы комбинаторной генерации.

2.2 Расширение алгебры функций мощности комбинаторных множеств

При моделировании реальных комбинаторных объектов возникают ситуации, когда функция мощности комбинаторного множества не может быть выражена в операциях алгебры $\{+, \times, \mathbb{Z}_{>0}, R\}$. В частности, в дискретной математике часто встречаются конструкции, где количество слагаемых или множителей не является константой, а зависит от некоторого дополнительного параметра. В результате это приводит к необходимости использования операторов суммирования и произведения следующего вида, в рамках которых динамически определяется количество узлов-потомков при построении частной структуры для заданных значений параметров:

$$f(n) = \sum_{i=0}^{n-1} f(i),$$

$$f(n) = \prod_{i=0}^{n-1} f(i).$$

Также функции мощности часто содержат условные выражения вида

$$f(n) = \begin{cases} f_1(n), & \text{если условие } P_1(n) \text{ истинно;} \\ f_2(n), & \text{если условие } P_2(n) \text{ истинно;} \\ \dots \end{cases}$$

где выражение функции мощности раскрывается в зависимости от конкретных значений параметров. В том числе при определенных условиях функция мощности может принимать значение 0, что соответствует пустому комбинаторному множеству. При этом алгебра $\{+, \times, \mathbb{Z}_{>0}, R\}$ не поддерживает работу с условным оператором и с нулевыми значениями. Кроме того, при обработке сложных дискретных структур, сформированных из более простых, функция мощности рассматриваемого комбинаторного множества может содержать в себе выражения функций мощности других множеств. Следовательно, требуется обработка случая композиции функций, в то время как оригинальная методология рассматривает только возможность рекурсивного вызова для случая примитивно рекурсивной функции.

Еще одним важным недостатком, который сильно ограничивает возможности построения эффективных алгоритмов комбинаторной генерации с использованием алгебры $\{+, \times, \mathbb{Z}_{>0}, R\}$, является отсутствие способов взаимодействия с другими существующими алгоритмами, которые получены без использования структур деревьев И/ИЛИ. Для преодоления указанных ограничений рассмотрим расширение алгебры $\{+, \times, \mathbb{Z}_{>0}, R\}$ новыми операторами и конструкциями.

Введем следующую расширенную алгебру, которая учитывает все указанные выше операции над функциями мощности комбинаторных множеств:

$$\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\},$$

где:

- « $+$ » — операция сложения;
- « Σ » — оператор суммирования;
- « $\times$ » — операция умножения;
- « Π » — оператор произведения;
- « $\mathbb{Z}_{\geq 0}$ » — неотрицательные целые числа;
- « I » — условный оператор;
- « C_{tree} » — операция композиции функций мощности комбинаторных множеств, для которых известны структуры деревьев И/ИЛИ;
- « C_{alg} » — операция композиции функций мощности комбинаторных множеств, для которых известны алгоритмы ранжирования и генерации по рангу.

Далее докажем существование структуры дерева И/ИЛИ, а также существование алгоритмов ранжирования и генерации по рангу для каждой компоненты расширенной алгебры $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$.

2.2.1 Композиция функций с известными структурами деревьев И/ИЛИ

Теорема 2.1. Пусть функция мощности f комбинаторного множества представлена в виде композиции функций $f = h \circ g$, где h и g являются функциями мощности других комбинаторных множеств. При этом функции h и g удовлетворяют требованиям для построения деревьев И/ИЛИ. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы $Rank_Tf$ и $Unrank_Tf$ для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. По условию теоремы, функции h и g удовлетворяют требованиям для построения деревьев И/ИЛИ. Следовательно, существуют:

- структура дерева И/ИЛИ T_h , количество вариантов которой равно значению функции h для заданных аргументов;
- структура дерева И/ИЛИ T_g , количество вариантов которой равно значению функции g для заданных аргументов;
- алгоритмы Rank_Th и Unrank_Th для дерева И/ИЛИ T_h ;
- алгоритмы Rank_Tg и Unrank_Tg для дерева И/ИЛИ T_g .

Одним из основных свойств структур деревьев И/ИЛИ является то, что любое поддереву дерева И/ИЛИ также является деревом И/ИЛИ. Из этого свойства следует возможность композиции деревьев И/ИЛИ: узел произвольного дерева И/ИЛИ может быть заменен на корень другого дерева И/ИЛИ, и результирующая структура останется корректным деревом И/ИЛИ.

Функции h соответствует структура дерева И/ИЛИ T_h , которое содержит И-узлы, ИЛИ-узлы и листья. Следовательно, функция h может содержать только операции умножения, сложения и положительные целые числа. В таком случае в качестве аргумента функции h выступает один из параметров, который обозначим как n и который принимает положительные целые значения, то есть $h := h(n)$ и $n \in \mathbb{Z}_{>0}$. Для каждого появления n в выражении функции $h(n)$ в структуре дерева И/ИЛИ T_h соответствует ИЛИ-узел с n листьями, то есть количество вариантов в поддереве такого ИЛИ-узла равно n . Если заменить каждый ИЛИ-узел для n вместе с его листьями в структуре дерева И/ИЛИ T_h на дерево И/ИЛИ T_g , то количество вариантов в поддереве трансформированного узла изменится с n на значение функции g , что также соответствует замене в записи функции h переменной n на значение функции g . Следовательно, получаем структуру дерева И/ИЛИ T_f , количество вариантов которой равно значению композиции функций $f = h \circ g$.

Если функция h зависит от нескольких переменных с положительными целыми значениями, то есть $h := h(n_1, n_2, \dots)$, тогда каждая из них может быть заменена значением функции g или функцией мощности любого другого комбинаторного множества, для которого существует соответствующая структура дерева И/ИЛИ. Следовательно, аналогичным образом получаем структуру дерева И/ИЛИ T_f для функции f , являющейся результатом более сложной композиции функций. При этом количество вариантов такого дерева И/ИЛИ T_f также равно значению функции f .

Для построения алгоритмов ранжирования и генерации по рангу для дерева И/ИЛИ T_f необходимо воспользоваться алгоритмами Rank_Th и Unrank_Th для дерева И/ИЛИ T_h . Однако при достижении ИЛИ-узла с листьями, который был заменен на корень дерева И/ИЛИ T_g , вместо алгоритмов Rank_NUM и Unrank_NUM выполняются алгоритмы Rank_Tg и Unrank_Tg. $\square$

Рассмотрим пример композиции функций мощности комбинаторных множеств и построим соответствующую композицию структур деревьев И/ИЛИ. Функция $h(n, m) = n \cdot m$ при заданных значениях n и m используется для вычисления мощности множества кортежей вида $\langle a, b \rangle$, где $a \in \{a_1, \dots, a_n\}$ и $b \in \{b_1, \dots, b_m\}$. На рисунке 2.3 представлена структура дерева И/ИЛИ T_h , построенная по функции $h(n, m) = n \cdot m$.

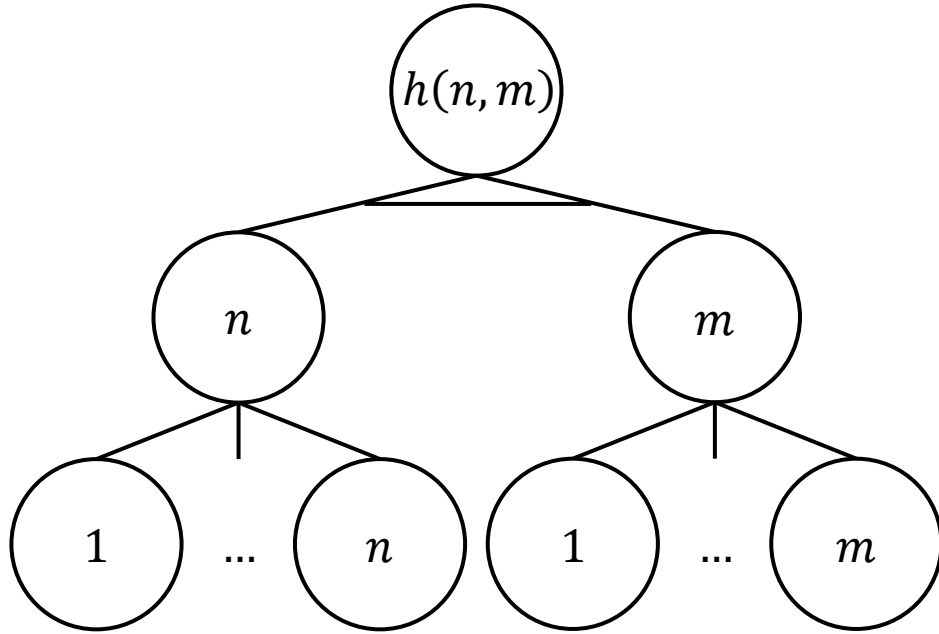


Рисунок 2.3 — Структура дерева И/ИЛИ T_h для $h(n, m) = n \cdot m$

Комбинация алгоритмов Rank_AND и Unrank_AND с алгоритмами Rank_NUM и Unrank_NUM позволяет реализовать ранжирование и генерацию по рангу для множества вариантов дерева И/ИЛИ T_h (алгоритмы 2.13 и 2.14). В данных алгоритмах вариант дерева И/ИЛИ T_h представляется как последовательность указателей на два узла-потомка корневого узла ($Vchild_n$ является указателем на узел, помеченный n в дерева И/ИЛИ T_h , а $Vchild_m$ является указателем на узел, помеченный m), при этом алгоритмы Rank_NUM и Unrank_NUM используются для обработки этих узлов-потомков. Также в качестве параметров алгоритмы принимают заданные значения n и m .

Алгоритм 2.13: Алгоритм ранжирования варианта дерева И/ИЛИ T_h для $h(n, m) = n \cdot m$

```

1 Rank_Th(variant = (Vchildn, Vchildm), n, m)
2 begin
3    $l_1 := \text{Rank\_NUM}(\textit{Vchild}_n, n)$ 
4    $l_2 := \text{Rank\_NUM}(\textit{Vchild}_m, m)$ 
5    $r := l_1 + n \cdot l_2$ 
6   return r
7 end
```

Алгоритм 2.14: Алгоритм генерации по рангу варианта дерева И/ИЛИ T_h для $h(n, m) = n \cdot m$

```

1 Unrank_Th(r, n, m)
2 begin
3    $l_1 := r \bmod n$ 
4    $l_2 := \lfloor \frac{r}{n} \rfloor$ 
5    $\textit{Vchild}_n := \text{Unrank\_NUM}(l_1, n)$ 
6    $\textit{Vchild}_m := \text{Unrank\_NUM}(l_2, m)$ 
7   variant := (Vchildn, Vchildm)
8   return variant
9 end
```

Также рассмотрим функцию $g_1(n, m)$, которая используется для вычисления мощности множества сочетаний из n элементов по m , и функцию $g_2(n)$, которая используется для вычисления мощности множества перестановок n элементов. Используя рекуррентную формулу $g_1(n, m) = g_1(n - 1, m - 1) + g_1(n - 1, m)$, получаем структуру дерева И/ИЛИ T_{g_1} (рисунок 2.4), где условием останковки рекурсивных вызовов является достижение значений параметров $m = 0$ или $m = n$, так как при таких условиях $g_1(n, m) = 1$. Используя рекуррентную формулу $g_2(n) = n \cdot g_2(n - 1)$, получаем структуру дерева И/ИЛИ T_{g_2} (рисунок 2.5), где $g_2(0) = 1$. Кроме того, на основе общих правил построения алгоритмов ранжирования и генерации по рангу вариантов деревьев И/ИЛИ можно записать алгоритмы Rank_Tg1 и Unrank_Tg1 для дерева И/ИЛИ T_{g_1} , а также алгоритмы Rank_Tg2 и Unrank_Tg2 для дерева И/ИЛИ T_{g_2} .

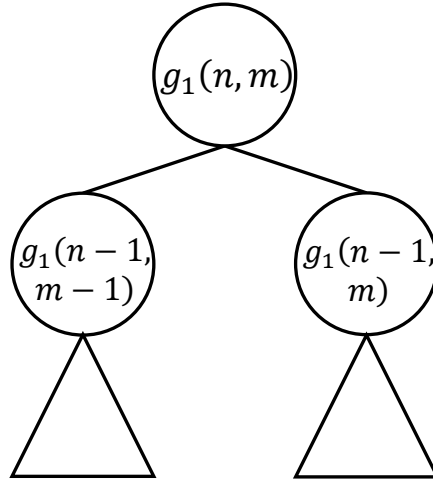


Рисунок 2.4 — Структура дерева И/ИЛИ T_{g_1} для $g_1(n, m) = g_1(n-1, m-1) + g_1(n-1, m)$

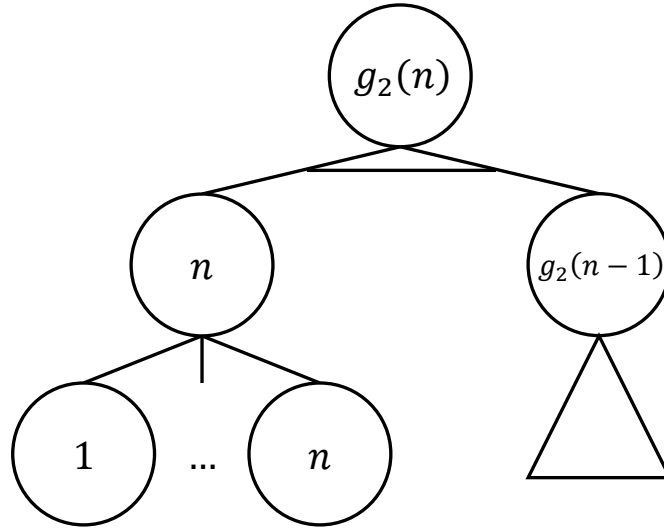


Рисунок 2.5 — Структура дерева И/ИЛИ T_{g_2} для $g_2(n) = n \cdot g_2(n-1)$

Композиция функций $f(n, m) = h(g_1(n, m), g_2(m))$ представляет собой функцию мощности нового комбинаторного множества, связанного с комбинаторными множествами для $g_1(n, m)$ и $g_2(m)$. Следовательно, в дереве И/ИЛИ T_h узел с меткой n заменяется на корень дерева И/ИЛИ T_{g_1} с параметрами $n := n$ и $m := m$, а узел с меткой m заменяется на корень дерева И/ИЛИ T_{g_2} с параметром $n := m$. Полученная структура дерева И/ИЛИ T_f для $f(n, m) = h(g_1(n, m), g_2(m))$ представлена на рисунке 2.6. Алгоритмы ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f строятся путем замены вызова алгоритмов Rank_NUM и Unrank_NUM в алгоритмах 2.13 и 2.14 на соответствующие алгоритмы для деревьев И/ИЛИ T_{g_1} и T_{g_2} .

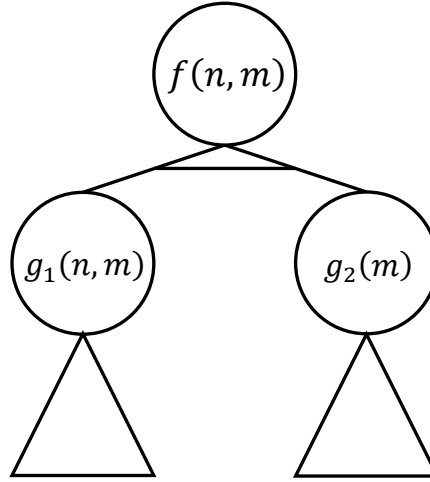


Рисунок 2.6 — Структура дерева И/ИЛИ T_f для $f(n, m) = h(g_1(n, m), g_2(m))$

Алгоритм 2.15: Алгоритм ранжирования варианта дерева И/ИЛИ T_f для $f(n, m) = h(g_1(n, m), g_2(m))$

```

1 Rank_Tf(variant = (Vchildn, Vchildm), n, m)
2 begin
3    $l_1 := \text{Rank\_Tg1}(\textit{Vchild}_n, n, m)$ 
4    $l_2 := \text{Rank\_Tg2}(\textit{Vchild}_m, m)$ 
5    $r := l_1 + g_1(n, m) \cdot l_2$ 
6   return r
7 end
```

Алгоритм 2.16: Алгоритм генерации по рангу варианта дерева И/ИЛИ T_f для $f(n, m) = h(g_1(n, m), g_2(m))$

```

1 Unrank_Tf(r, n, m)
2 begin
3    $l_1 := r \bmod g_1(n, m)$ 
4    $l_2 := \left\lfloor \frac{r}{g_1(n, m)} \right\rfloor$ 
5    $\textit{Vchild}_n := \text{Unrank\_Tg1}(l_1, n, m)$ 
6    $\textit{Vchild}_m := \text{Unrank\_Tg2}(l_2, m)$ 
7   variant := (Vchildn, Vchildm)
8   return variant
9 end
```

2.2.2 Операторы суммирования и произведения

Теорема 2.2. Пусть функция мощности f комбинаторного множества представлена в виде оператора суммирования следующего вида:

$$f(\bar{p}) = \sum_{i=a(\bar{p})}^{b(\bar{p})} f_i(\bar{p}),$$

где $\bar{p}$ — вектор значений параметров элементов комбинаторного множества; i — переменная суммирования; $a(\bar{p})$ и $b(\bar{p})$ — целочисленные выражения, определяющие начальное и конечное значение переменной суммирования в зависимости от значений параметров; $f_i(\bar{p})$ — функции, которые удовлетворяют требованиям для построения деревьев И/ИЛИ. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы Rank_Tf и Unrank_Tf для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. Оператор суммирования представляет собой операцию сложения, где количество слагаемых определяется только после объявления конкретных значений параметров $\bar{p}$ и равно $b(\bar{p}) - a(\bar{p}) + 1$. Таким образом, оператор суммирования реализует логику работы ИЛИ-узла, который имеет $b(\bar{p}) - a(\bar{p}) + 1$ узлов-потомков. При этом каждый узел-потомок соответствует установке одного из возможных значений для переменной суммирования i , а поддереву такого узла определяется слагаемым $f_i(\bar{p})$.

По условию теоремы, функции f_i удовлетворяют требованиям для построения деревьев И/ИЛИ. Следовательно, существуют:

- структуры деревьев И/ИЛИ T_{f_i} , количество вариантов которых равно значениям функций f_i для заданных аргументов;
- алгоритмы Rank_Tf_i и Unrank_Tf_i для деревьев И/ИЛИ T_{f_i} .

В результате получаем структуру дерева И/ИЛИ T_f , корень которой является ИЛИ-узлом с $b(\bar{p}) - a(\bar{p}) + 1$ узлами-потомками, чьи поддеревья представляют собой деревья И/ИЛИ T_{f_i} (рисунок 2.7).

Вычислим количество вариантов дерева И/ИЛИ T_f :

$$w(T_f) = \sum_{i=a(\bar{p})}^{b(\bar{p})} w(T_{f_i}) = \sum_{i=a(\bar{p})}^{b(\bar{p})} f_i(\bar{p}) = f(\bar{p}).$$

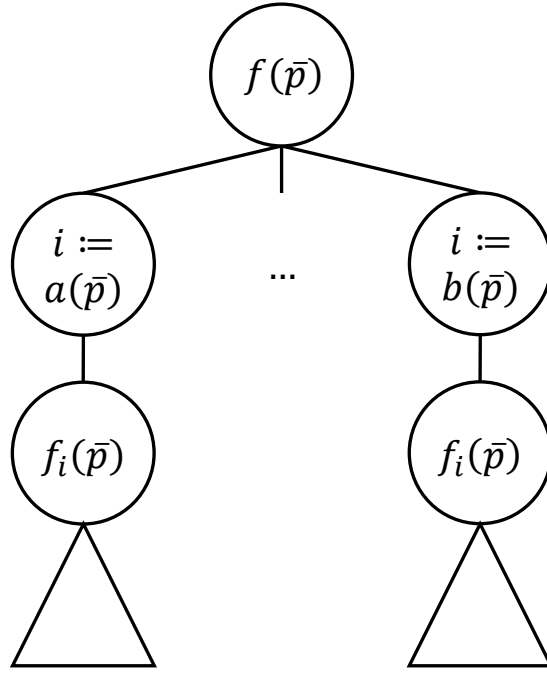


Рисунок 2.7 — Структура дерева И/ИЛИ T_f для оператора суммирования

Таким образом, для рассматриваемой функции f построена структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f . Так как полученная структура дерева И/ИЛИ состоит из ИЛИ-узла, а также операции композиции, для которых определены правила построения алгоритмов ранжирования и генерации по рангу (алгоритмы 2.1 и 2.2, теорема 2.1), то существуют и алгоритмы Rank_Tf и Unrank_Tf. $\square$

Отметим, что введение переменной суммирования позволяет объявить дополнительный локальный параметр, который может в дальнейшем быть использован при раскрытии поддеревьев узлов-потомков более высокого уровня.

Теорема 2.3. Пусть функция мощности f комбинаторного множества представлена в виде оператора произведения следующего вида:

$$f(\bar{p}) = \prod_{i=a(\bar{p})}^{b(\bar{p})} f_i(\bar{p}),$$

где $\bar{p}$ — вектор значений параметров элементов комбинаторного множества; i — переменная произведения; $a(\bar{p})$ и $b(\bar{p})$ — целочисленные выражения, определяющие начальное и конечное значение переменной произведения в зависимости от значений параметров; $f_i(\bar{p})$ — функции, которые удовлетворяют требованиям для построения деревьев И/ИЛИ. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы

$Rank_Tf$ и $Unrank_Tf$ для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. Доказательство строится аналогично доказательству теоремы 2.2, но в данном случае корень дерева И/ИЛИ T_f является И-узлом (рисунок 2.8).

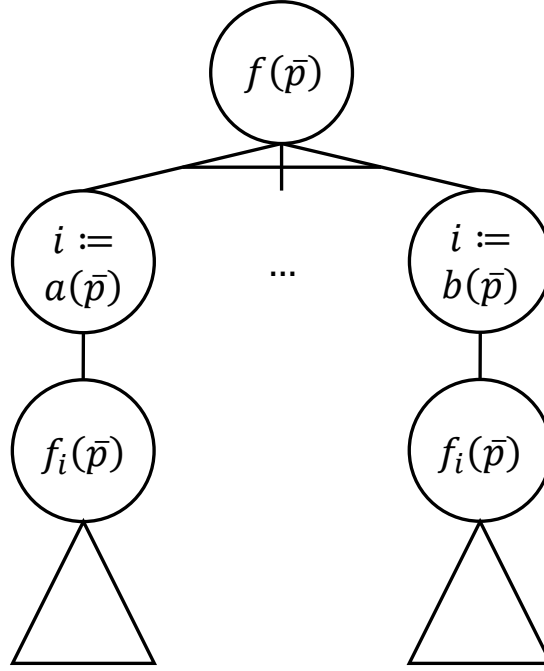


Рисунок 2.8 — Структура дерева И/ИЛИ T_f для оператора произведения

□

2.2.3 Условный оператор

При построении структуры дерева И/ИЛИ для рекурсивной функции f применяются условия остановки рекурсивных вызовов, которые явно не обозначаются в самой структуре дерева И/ИЛИ. Для самой функции это выражается в ее представлении как кусочно-заданной функцией с объявлением предикатов $P_i(\bar{p})$ на множестве значений параметров $\bar{p}$:

$$f(\bar{p}) = \begin{cases} f_1(\bar{p}), & \text{если условие } P_1(\bar{p}) \text{ истинно;} \\ f_2(\bar{p}), & \text{если условие } P_2(\bar{p}) \text{ истинно;} \\ \dots & \end{cases}$$

Для исправления данного недостатка предлагается ввести соответствующее обозначение условного оператора в графическое представление структуры дерева И/ИЛИ: узел с фигурой треугольника и меткой «IF», из которого выходят ребра к узлам-потомкам, каждый из которых помечен предикатом $P_i(\bar{p})$ и имеет поддерево для $f_i(\bar{p})$. Пример представления такого узла с условным оператором показан на рисунке 2.9.

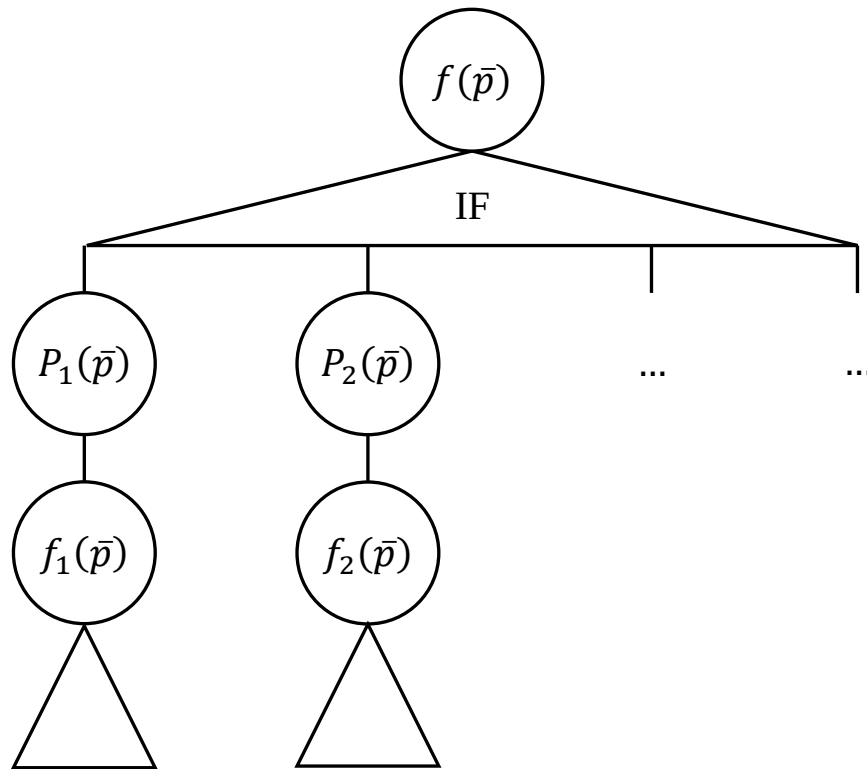


Рисунок 2.9 — Пример представления узла с условным оператором

С точки зрения интерпретации узел с условным оператором используется только для структуры дерева И/ИЛИ, в которой еще не объявлены конкретные значения параметров $\bar{p}$ (общая структура дерева И/ИЛИ). Когда параметры принимают конкретные значения, то остается только один узел-потомок, выбор которого определяется истинным значением предиката (частная структура дерева И/ИЛИ). Таким образом, предлагаемое обозначение позволяет наглядно отобразить все условия остановки рекурсивных вызовов. Кроме того, это позволяет задать структуру дерева И/ИЛИ для кусочно-заданных функций мощности комбинаторных множеств, то есть когда функция ведет себя по-разному в зависимости от текущих значений параметров (например, разное поведение функции для четных и нечетных значений параметров).

Теорема 2.4. Пусть функция мощности f комбинаторного множества представлена в виде кусочно-заданной функции следующего вида:

$$f(\bar{p}) = \begin{cases} f_1(\bar{p}), & \text{если условие } P_1(\bar{p}) \text{ истинно;} \\ f_2(\bar{p}), & \text{если условие } P_2(\bar{p}) \text{ истинно;} \\ \dots \end{cases}$$

где $\bar{p}$ — вектор значений параметров элементов комбинаторного множества; $f_i(\bar{p})$ — функции, которые удовлетворяют требованиям для построения деревьев И/ИЛИ; $P_i(\bar{p})$ — предикаты на множестве значений параметров. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы Rank_Tf и Unrank_Tf для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. По условию теоремы, функции f_i удовлетворяют требованиям для построения деревьев И/ИЛИ. Следовательно, существуют:

- структуры деревьев И/ИЛИ T_{f_i} , количество вариантов которых равно значениям функций f_i для заданных аргументов;
- алгоритмы Rank_Tf_i и Unrank_Tf_i для деревьев И/ИЛИ T_{f_i} .

В результате получаем структуру дерева И/ИЛИ T_f , корень которой является узлом с условным оператором, количество узлов-потомков которого определяется количеством предикатов в формуле для $f(\bar{p})$. При этом поддеревья узлов-потомков представляют собой деревья И/ИЛИ T_{f_i} (рисунок 2.9).

Вычислим количество вариантов дерева И/ИЛИ T_f :

$$\begin{aligned} w(T_f) &= \begin{cases} w(T_{f_1}), & \text{если условие } P_1(\bar{p}) \text{ истинно;} \\ w(T_{f_2}), & \text{если условие } P_2(\bar{p}) \text{ истинно;} \\ \dots \end{cases} = \\ &= \begin{cases} f_1(\bar{p}), & \text{если условие } P_1(\bar{p}) \text{ истинно;} \\ f_2(\bar{p}), & \text{если условие } P_2(\bar{p}) \text{ истинно;} \\ \dots \end{cases} = f(\bar{p}). \end{aligned}$$

Таким образом, для рассматриваемой функции f построена структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f . Так как полученная структура дерева И/ИЛИ состоит из узла с условным оператором, то значения

параметров определяют выбор конкретного узла-потомка и далее реализуются алгоритмы комбинаторной генерации для его поддеревя. Следовательно, получаем алгоритмы Rank_Tf и Unrank_Tf для дерева И/ИЛИ T_f (алгоритмы 2.17 и 2.18).

Алгоритм 2.17: Алгоритм ранжирования варианта поддеревя узла
с условным оператором

```

1 Rank_IF(variant,  $\bar{p}$ )
2 begin
3   if  $P_1(\bar{p})$  then
4      $r := \text{Rank\_F1}(\textit{variant}, \bar{p})$ 
5     return  $r$ 
6   end
7   if  $P_2(\bar{p})$  then
8      $r := \text{Rank\_F2}(\textit{variant}, \bar{p})$ 
9     return  $r$ 
10  end
11  ...
12 end
```

Алгоритм 2.18: Алгоритм генерации по рангу варианта поддеревя узла
с условным оператором

```

1 Unrank_IF( $r$ ,  $\bar{p}$ )
2 begin
3   if  $P_1(\bar{p})$  then
4      $\textit{variant} := \text{Unrank\_F1}(r, \bar{p})$ 
5     return  $\textit{variant}$ 
6   end
7   if  $P_2(\bar{p})$  then
8      $\textit{variant} := \text{Unrank\_F2}(r, \bar{p})$ 
9     return  $\textit{variant}$ 
10  end
11  ...
12 end
```

2.2.4 Нулевое значение

При определенных значениях параметров комбинаторное множество может быть пустым, то есть оно содержит 0 элементов, следовательно $f(\bar{p}) = 0$. Оригинальная методология деревьев И/ИЛИ не позволяет обрабатывать такие ситуации. Для исправления данного недостатка предлагается ввести соответствующее обозначение пустого узла в графическое представление структуры дерева И/ИЛИ: узел серого цвета, для которого количество вариантов равняется 0, то есть $w(\text{Пустой узел}) = 0$. Пример представления такого пустого узла показан на рисунке 2.10.

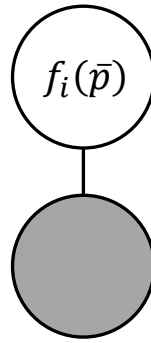


Рисунок 2.10 — Пример представления пустого узла

Теорема 2.5. Пусть функция мощности f комбинаторного множества представлена в виде кусочно-заданной функции следующего вида:

$$f(\bar{p}) = \begin{cases} f_1(\bar{p}), & \text{если условие } P_1(\bar{p}) \text{ истинно;} \\ f_2(\bar{p}), & \text{если условие } P_2(\bar{p}) \text{ истинно;} \\ \dots \end{cases}$$

где $\bar{p}$ — вектор значений параметров элементов комбинаторного множества; $f_i(\bar{p})$ — функции, которые удовлетворяют требованиям для построения деревьев И/ИЛИ или которые равны 0; $P_i(\bar{p})$ — предикаты на множестве значений параметров. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы $\text{Rank_}T_f$ и $\text{Unrank_}T_f$ для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. Функция $f_i(\bar{p}) = 0$ определяет условие, при котором невозможно построение какого-либо комбинаторного объекта, то есть при заданных значениях параметров формируется пустое комбинаторное множество. По определению пустого узла, если $f_i(\bar{p}) = 0$, то $w(T_{f_i}) = 0$. Следовательно, для функции

f_i построена структура дерева И/ИЛИ T_{f_i} , количество вариантов которого равно значению f_i . Объединяя данный вывод с результатом теоремы 2.4, получаем, что для функции f также существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f .

Если рассматривается формула следующего вида:

$$f(\bar{p}) = f_1(\bar{p}) + \dots + f_n(\bar{p}) + g(\bar{p}),$$

где $g(\bar{p}) = 0$, тогда

$$f(\bar{p}) = f_1(\bar{p}) + \dots + f_n(\bar{p}) + 0 = f_1(\bar{p}) + \dots + f_n(\bar{p}).$$

В таком случае пустой узел является потомком ИЛИИ-узла. При построении частной структуры дерева И/ИЛИ, а также при построении варианта дерева И/ИЛИ пустой узел должен быть удален, так как он не создает каких-либо структур. В результате ИЛИИ-узел перестанет содержать пустые узлы и количество его вариантов равно

$$w(T_f) = w(T_{f_1}) + \dots + w(T_{f_n}) = f_1(\bar{p}) + \dots + f_n(\bar{p}) = f(\bar{p}).$$

Если рассматривается формула следующего вида:

$$f(\bar{p}) = f_1(\bar{p}) \cdot \dots \cdot f_n(\bar{p}) \cdot g(\bar{p}),$$

где $g(\bar{p}) = 0$, тогда

$$f(\bar{p}) = f_1(\bar{p}) \cdot \dots \cdot f_n(\bar{p}) \cdot 0 = 0.$$

В таком случае пустой узел является потомком И-узла. При построении частной структуры дерева И/ИЛИ, а также при построении варианта дерева И/ИЛИ такой И-узел должен быть заменен на пустой узел. В результате И-узел станет пустым узлом и количество его вариантов равно

$$w(T_f) = 0 = f(\bar{p}).$$

Действия по обработке пустого узла необходимо повторить для родительского узла до тех пор, пока пустой узел не будет удален в рамках узлов-потомков какого-либо ИЛИИ-узла или пока не будет достигнут корневой узел.

Так как при построении варианта дерева И/ИЛИ все ветви с пустыми узлами удаляются, то в результате остается структура дерева И/ИЛИ с И-узлами, ИЛИИ-узлами и листьями. Следовательно, для алгоритмов ранжирования и генерации по рангу могут использоваться общие алгоритмы из параграфа 2.1. $\square$

2.2.5 Композиция функций с известными алгоритмами ранжирования и генерации по рангу

Теорема 2.6. Пусть функция мощности f комбинаторного множества представлена в виде композиции функций $f = h \circ g$, где h и g являются функциями мощности других комбинаторных множеств. При этом функция h удовлетворяет требованиям для построения деревьев И/ИЛИ, а для функции g существуют алгоритмы Rank_Tg и Unrank_Tg для ранжирования и генерации по рангу элементов соответствующего комбинаторного множества. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы Rank_Tf и Unrank_Tf для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .

Доказательство. Если функция g удовлетворяет требованиям для построения деревьев И/ИЛИ, то искомая структура дерева И/ИЛИ T_f , а также алгоритмы Rank_Tf и Unrank_Tf можно получить на основе результатов теоремы 2.1.

Если функция g не удовлетворяет требованиям для построения деревьев И/ИЛИ, то строится структура дерева И/ИЛИ следующего вида: корень является ИЛИ-узлом, который имеет g узлов-потомков в виде листьев, помеченных соответствующим комбинаторным объектом (рисунок 2.11).

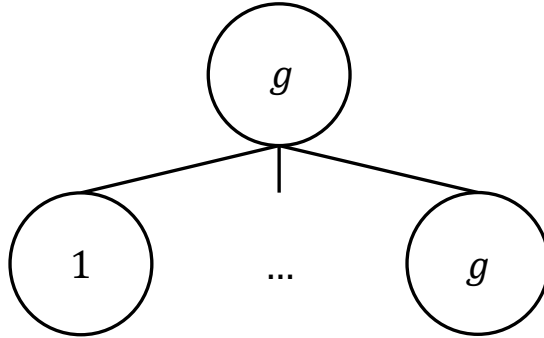


Рисунок 2.11 — Структура дерева И/ИЛИ T_g для g , которая не удовлетворяет требованиям для построения деревьев И/ИЛИ

Количество вариантов такого дерева И/ИЛИ T_g равно g , а алгоритмы Rank_Tg и Unrank_Tg позволяют преобразовать выбранный узел-потомок в значение ранга и выполнить обратное построение узла-потомка из значения ранга. Следовательно, приходим к ситуации композиции функций, рассмотренной в теореме 2.1. $\square$

Таким образом, теорема 2.6 позволяет использовать эффективные реализации алгоритмов комбинаторной генерации, в том числе полученные с использованием совершенно другого математического аппарата, внутри алгоритмов комбинаторной генерации для более сложных структур.

2.2.6 Рекурсивные функции

Оригинальная методология позволяет строить структуры деревьев И/ИЛИ для случая примитивно рекурсивной функции. Примитивно рекурсивные функции соответствуют следующей схеме организации вычислений: запускается цикл с заранее известным количеством итераций и на каждой итерации возможно применение арифметических операций и условных операторов. Таким образом, количество итераций цикла определяется количеством рекурсивных вызовов. Если же точное количество итераций цикла заранее не известно, тогда такая функция будет принадлежать классу частично рекурсивных функций. Процесс динамического построения частной структуры дерева И/ИЛИ и его вариантов в зависимости от текущих значений параметров не требует предварительного указания конкретного количества узлов древовидной структуры. Следовательно, становится возможным построение структуры дерева И/ИЛИ для случая частично рекурсивных функций.

Например, ярким примером частично рекурсивной функции является функция Аккермана [173], которая может быть определена следующей формулой, состоящей только из рекурсивных вызовов:

$$A(n, m) = \begin{cases} m + 1, & n = 0; \\ A(n - 1, 1), & n > 0 \text{ и } m = 0; \\ A(n - 1, A(n, m - 1)), & n > 0 \text{ и } m > 0. \end{cases}$$

На рисунке 2.12 показана общая структура дерева И/ИЛИ для функции Аккермана $A(n, m)$.

Рассмотрим частную структуру дерева И/ИЛИ для $A(1, 1)$. Это будет корневой узел, помеченный $A(1, 1)$, у которого будет единственный узел-потомок, помеченный $A(0, A(1, 0))$. Чтобы далее раскрыть структуру дерева И/ИЛИ, необходимо вычислить значение параметра $m := A(1, 0) = 2$. Следовательно, этот узел будет помечен уже как $A(0, 2)$ и будет представлять собой ИЛИ-узел с $m + 1 = 2 + 1 = 3$ узлами-потомками, которые являются листьями.

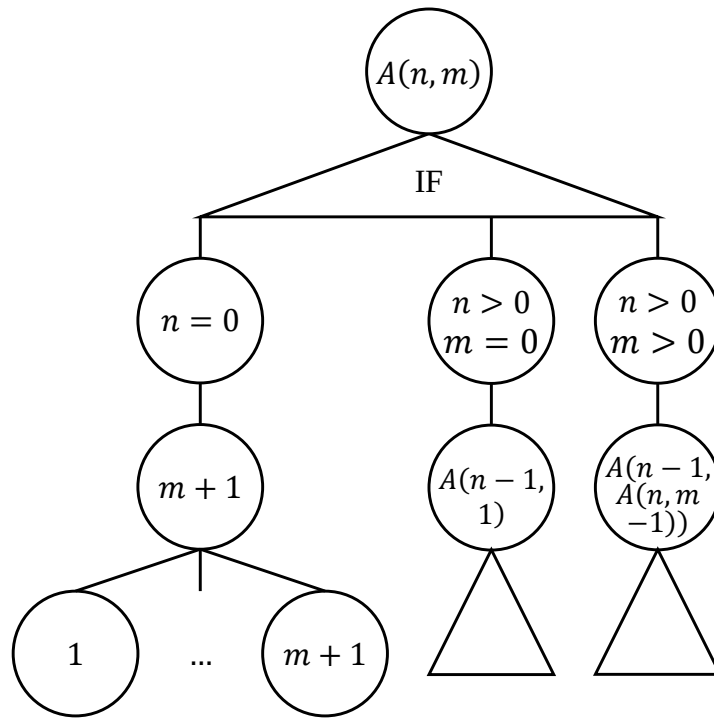


Рисунок 2.12 — Структура дерева И/ИЛИ для функции Аккермана $A(n, m)$

На рисунке 2.13 показано построение частной структуры дерева И/ИЛИ для функции Аккермана $A(1, 1)$. Количество вариантов такого дерева равно 3, что совпадает со значением функции $A(1, 1) = 3$.

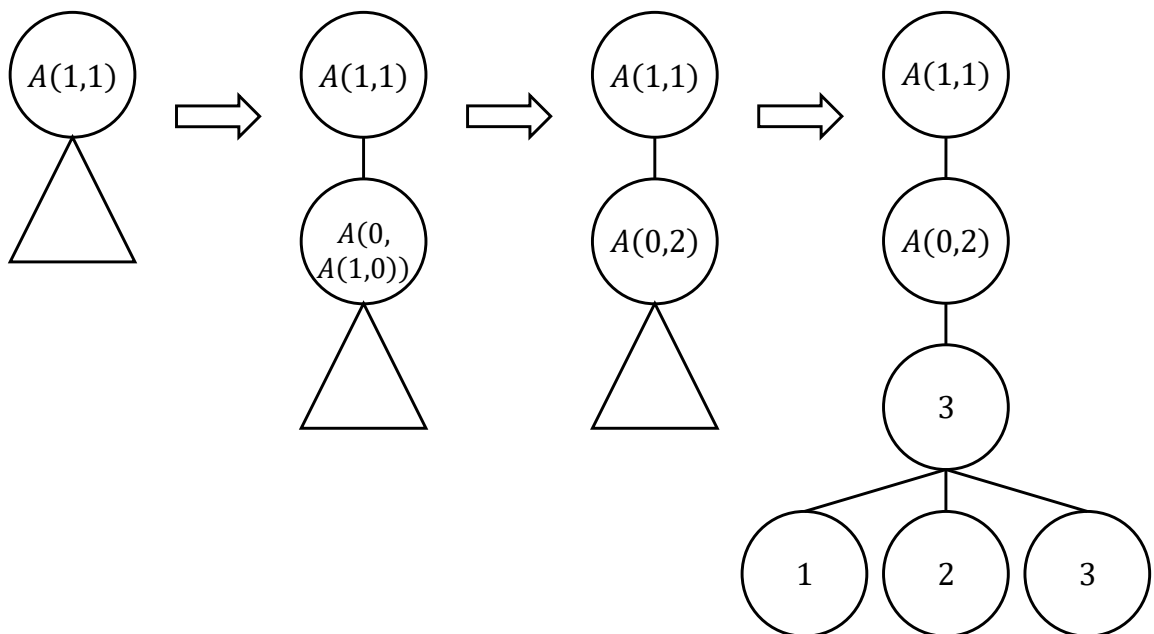


Рисунок 2.13 — Структура дерева И/ИЛИ для функции Аккермана $A(1, 1)$

Таким образом, наглядно продемонстрирован пример построения структуры дерева И/ИЛИ для частично рекурсивной функции.

2.2.7 Расширенная алгебра функций мощности комбинаторных множеств

Объединяя правила работы со структурами деревьев И/ИЛИ из оригинальной методологии (алгоритмы 2.1–2.10 для алгебры $\{+, \times, \mathbb{Z}_{>0}, R\}$) и предложенные дополнения (теоремы 2.1–2.6), получаем расширенную алгебру функций мощности комбинаторных множеств $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$.

Теорема 2.7. *Пусть функция мощности f комбинаторного множества представлена через операции алгебры $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$. Тогда существует структура дерева И/ИЛИ T_f , количество вариантов которого равно значению f , и существуют алгоритмы $Rank_Tf$ и $Unrank_Tf$ для ранжирования и генерации по рангу вариантов дерева И/ИЛИ T_f .*

Доказательство. Доказательство данной теоремы следует из совокупности ранее полученных результатов в виде алгоритмов 2.1–2.10 для алгебры $\{+, \times, \mathbb{Z}_{>0}, R\}$ и теорем 2.1 для C_{tree} , 2.2 для Σ , 2.3 для Π , 2.4 для I , 2.5 для 0 , 2.6 для C_{alg} . $\square$

Применение расширенной алгебры функций мощности комбинаторных множеств $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$ обеспечивает возможность построения рекурсивных композиций структур деревьев И/ИЛИ и алгоритмов комбинаторной генерации для множеств их вариантов. Кроме того, расширенная алгебра позволяет обрабатывать структуры, которые задаются в общем виде с использованием целочисленных параметров, при этом алгоритмы комбинаторной генерации динамически подстраиваются под частную структуру дерева И/ИЛИ для конкретных значений параметров. Общий метод, позволяющий строить алгоритмы ранжирования и генерации по рангу для заданной функции мощности f комбинаторного множества, можно описать последовательностью выполнения следующих шагов:

1. Проверить функцию мощности f комбинаторного множества на принадлежность алгебре $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$;
2. Если f удовлетворяет алгебре $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$, то построить соответствующую структуру дерева И/ИЛИ T_f . Иначе построение алгоритмов комбинаторной генерации на основе структур деревьев И/ИЛИ не допустимо;
3. Определить правила биективного отображения между комбинаторным множеством и множеством вариантов дерева И/ИЛИ T_f ;
4. Записать алгоритмы ранжирования и генерации по рангу на основе правил обработки каждого типа используемой операции в f .

Использование структур деревьев И/ИЛИ и алгоритмов комбинаторной генерации на их основе позволяет задать промежуточную форму представления информационных объектов при решении задач их кодирования. На рисунке 2.14 схематично изображен процесс кодирования информационного объекта a , принадлежащего комбинаторному множеству A , значением ранга r , полученным с помощью алгоритма ранжирования для соответствующего представления объекта в виде варианта v дерева И/ИЛИ.

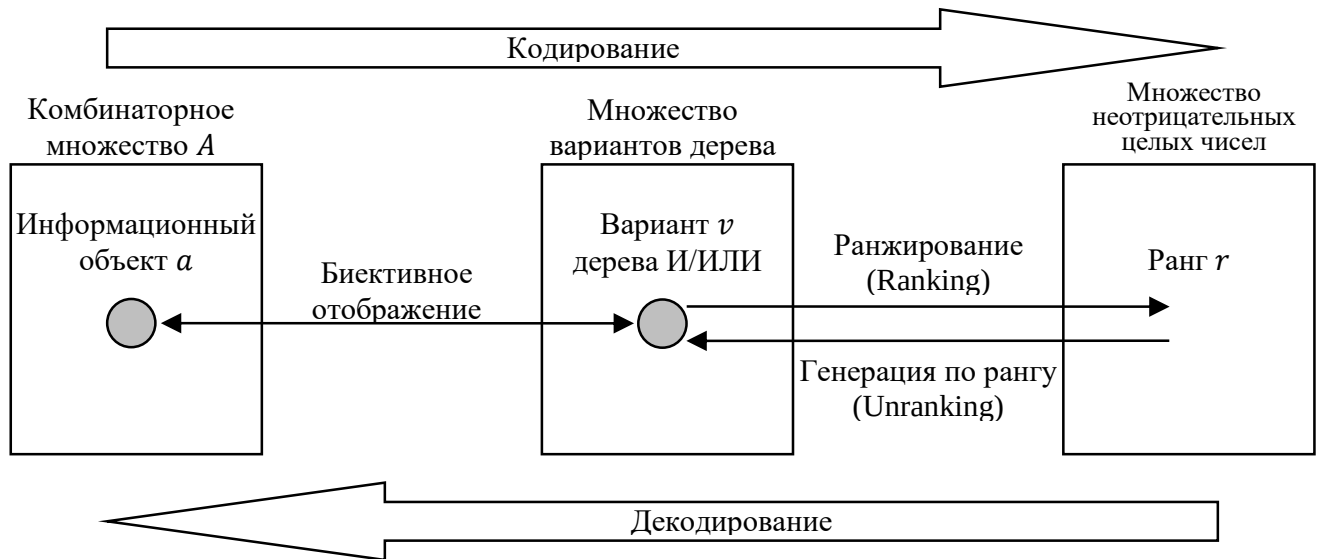


Рисунок 2.14 — Кодирование информации с помощью алгоритмов комбинаторной генерации с использованием промежуточного представления в виде структур деревьев И/ИЛИ

Таким образом, на основе описанных базовых правил построения структур деревьев И/ИЛИ (алгоритмы 2.1–2.10), доказанных теорем о расширении алгебры функций мощности комбинаторных множеств (теоремы 2.1–2.6) и алгоритмах их применения, а также с использованием общей схемы кодирования (рисунок 2.14) формируется общая методология построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ. Особенностью данной методологии является промежуточное представление кодируемого информационного объекта в виде варианта соответствующей структуры дерева И/ИЛИ, что позволяет выполнить декомпозицию сложной дискретной структуры на более простые и реализовать комбинацию наиболее эффективных алгоритмов комбинаторной генерации. В том числе появляется возможность комбинирования известных алгоритмов, полученных с использованием совершенно другого математического аппарата.

2.3 Рекурсивные композиции структур деревьев И/ИЛИ для дискретных структур, связанных с числами Фубини

Примерами информационных объектов, которые позволяет кодировать предложенная методология, служат как общие математические структуры, так и структуры, связанные с прикладным программным обеспечением. Далее в качестве демонстрации возможностей предложенной методологии рассмотрим задачу построения алгоритмов комбинаторной генерации для разных рекурсивных композиций структур деревьев И/ИЛИ одного комбинаторного множества.

Числа Фубини (или упорядоченные числа Белла), обозначаемые F_n , представляют собой следующую последовательность целых чисел для $n \in \mathbb{Z}_{\geq 0}$ (последовательность A000670 в OEIS [174]):

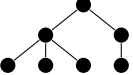
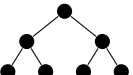
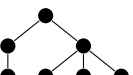
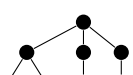
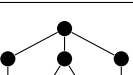
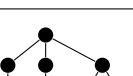
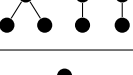
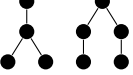
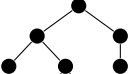
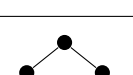
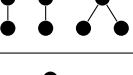
$$1, 1, 3, 13, 75, 541, 4683, 47293, 545835, 7087261, 102247563, \dots$$

Числа Фубини имеют важное значение в области дискретной математики, поскольку они имеют целый набор различных комбинаторных интерпретаций. Например, число Фубини F_n определяет мощность множества следующих комбинаторных объектов:

- рейтинг n элементов с возможностью ничьи, который можно представить в виде последовательности $(a_1, \dots, a_n)$, где каждый a_i показывает место i -го элемента в рейтинге [175];
- нестрогое упорядочивание n помеченных элементов, которое можно представить в виде последовательности n меток элементов, разделенных символами «>» и/или «=» [176];
- разбиение множества n элементов на подмножества, которые упорядочены между собой [177; 178];
- упорядоченные деревья с $n + 1$ листьями, где пути от корня к листьям имеют одинаковую длину и количество узлов на расстоянии i от корня строго меньше количества узлов на расстоянии $i + 1$ [179].

В таблице 2.1 представлены примеры комбинаторных множеств, мощность которых равна числу Фубини $F_3 = 13$. Числа Фубини также связаны со многими другими специальными числами [180–184], что позволяет использовать связанные с числами Фубина комбинаторные объекты в качестве подструктур более сложных дискретных структур.

Таблица 2.1 — Примеры комбинаторных множеств, мощность которых равна числу Фубини $F_3 = 13$

Рейтинг 3 элементов	Нестрогое упорядочивание 3 элементов	Разбиение множества 3 элементов	Упорядоченные деревья с 4 листьями
(1, 1, 1)	$a = b = c$	$\{a, b, c\}$	
(1, 1, 2)	$a = b > c$	$\{a, b\}, \{c\}$	
(1, 2, 1)	$a = c > b$	$\{a, c\}, \{b\}$	
(2, 1, 1)	$b = c > a$	$\{b, c\}, \{a\}$	
(1, 2, 2)	$a > b = c$	$\{a\}, \{b, c\}$	
(2, 1, 2)	$b > a = c$	$\{b\}, \{a, c\}$	
(2, 2, 1)	$c > a = b$	$\{c\}, \{a, b\}$	
(1, 2, 3)	$a > b > c$	$\{a\}, \{b\}, \{c\}$	
(1, 3, 2)	$a > c > b$	$\{a\}, \{c\}, \{b\}$	
(2, 1, 3)	$b > a > c$	$\{b\}, \{a\}, \{c\}$	
(2, 3, 1)	$c > a > b$	$\{c\}, \{a\}, \{b\}$	
(3, 1, 2)	$b > c > a$	$\{b\}, \{c\}, \{a\}$	
(3, 2, 1)	$c > b > a$	$\{c\}, \{b\}, \{a\}$	

Известны следующие формулы для вычисления чисел Фубини, каждая из которых удовлетворяет алгебре $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$:

– на основе биномиальных коэффициентов C_n^m [177]:

$$F_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0; \\ \sum_{i=1}^n C_n^i \cdot F_{n-i}, & n > 0, \end{cases} \quad (2.1)$$

где

$$C_n^m = \begin{cases} 1, & n \geq m = 0 \text{ или } m = n \geq 0; \\ C_{n-1}^m + C_{n-1}^{m-1}, & n > m > 0; \\ 0, & \text{иначе.} \end{cases} \quad (2.2)$$

– на основе чисел Стирлинга второго рода S_n^m [178]:

$$F_n = \sum_{i=0}^n i! \cdot S_n^i = \sum_{i=0}^n P_i \cdot S_n^i, \quad (2.3)$$

где

$$S_n^m = \begin{cases} 1, & n \geq m = 1 \text{ или } m = n \geq 0; \\ m \cdot S_{n-1}^m + S_{n-1}^{m-1}, & n > m > 1, \\ 0, & \text{иначе,} \end{cases} \quad (2.4)$$

$$P_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0; \\ n \cdot P_{n-1} & n > 0. \end{cases} \quad (2.5)$$

– на основе чисел Эйлера первого рода E_n^m [185]:

$$F_n = \sum_{i=0}^n 2^i \cdot E_n^i = \sum_{i=0}^n Q_i \cdot E_n^i, \quad (2.6)$$

где

$$E_n^m = \begin{cases} 1, & n \geq m = 0 \text{ или } m + 1 = n \geq 0; \\ (m + 1) \cdot E_{n-1}^m + (n - m) \cdot E_{n-1}^{m-1}, & n - 1 > m > 0; \\ 0, & \text{иначе,} \end{cases} \quad (2.7)$$

$$Q_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0; \\ 2 \cdot Q_{n-1} & n > 0. \end{cases} \quad (2.8)$$

Поскольку каждая из представленных функций для вычисления чисел Фубини удовлетворяет алгебре $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$, то можно построить соответствующие рекурсивные композиции структур деревьев И/ИЛИ для F_n .

2.3.1 Рекурсивная композиция структур деревьев И/ИЛИ на основе биномиальных коэффициентов

На рисунке 2.15 представлена структура дерева И/ИЛИ для F_n при использовании формулы (2.1).

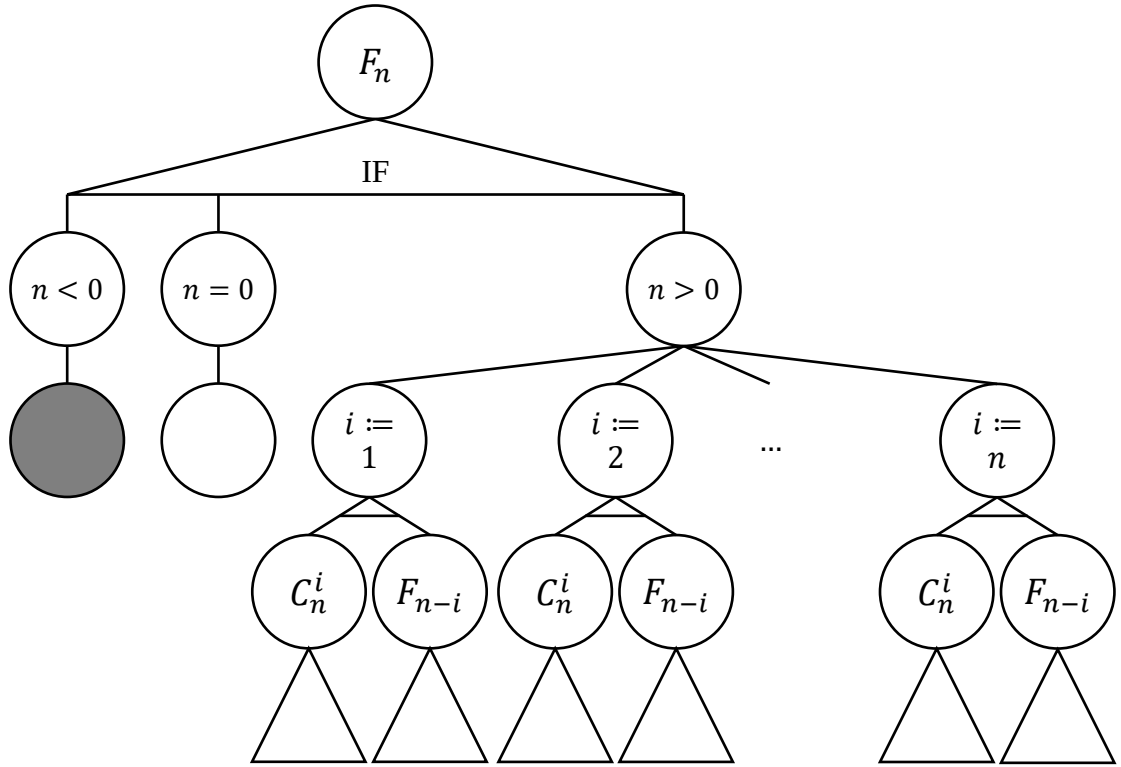


Рисунок 2.15 — Структура дерева И/ИЛИ для F_n при использовании формулы (2.1)

Также требуется структура дерева И/ИЛИ для C_n^m (рисунок 2.16).

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой F_0 является листом в дереве И/ИЛИ для F_n ;
- каждый узел с меткой C_n^0 является листом в дереве И/ИЛИ для C_n^m ;
- каждый узел с меткой C_n^n является листом в дереве И/ИЛИ для C_n^m .

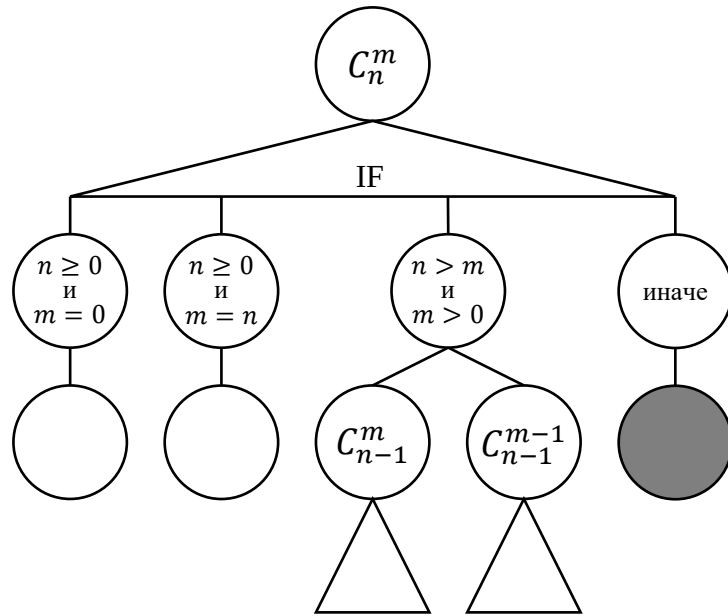


Рисунок 2.16 — Структура дерева И/ИЛИ для C_n^m при использовании формулы (2.2)

На рисунке 2.17 представлен пример структуры дерева И/ИЛИ для F_n , где $n = 3$. Общее количество его вариантов равно $F_3 = 13$.

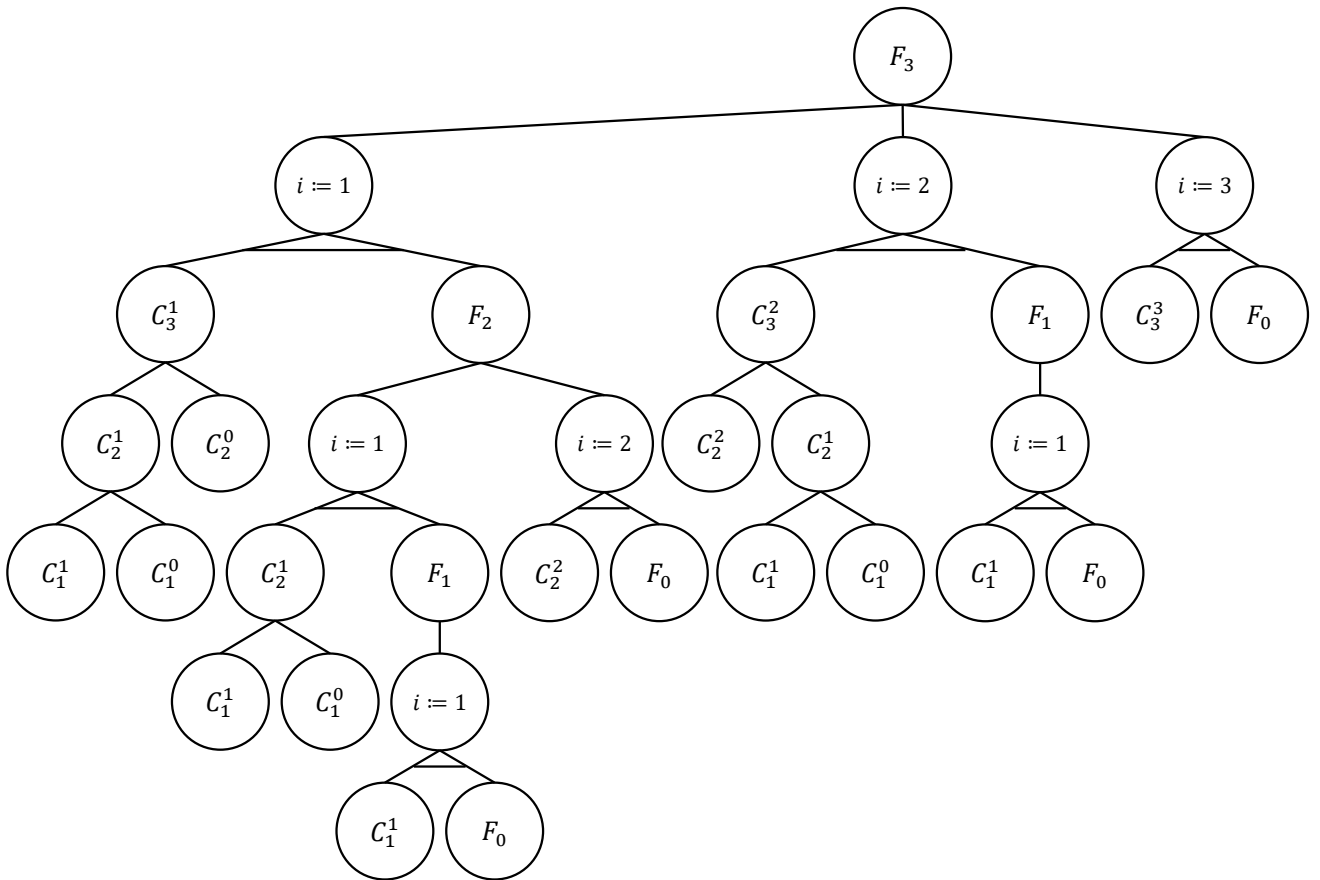


Рисунок 2.17 — Структура дерева И/ИЛИ для F_3 при использовании формулы (2.1)

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для F_n последовательностью $v = (I, v_C, v_F)$, где:

- пустая последовательность $v = ()$ соответствует выбору листа с меткой F_0 ;
- I соответствует выбранному значению i в дереве И/ИЛИ для F_n ;
- v_C соответствует варианту поддерева узла с меткой C_n^I (левое поддерево);
- v_F соответствует варианту поддерева узла с меткой F_{n-I} .

В свою очередь, вариант поддерева узла с меткой C_n^m кодируется последовательностью $v_C = (v_{C1}, v_{C2}, \dots)$, где:

- пустая последовательность $v_C = ()$ соответствует выбору листа с меткой C_n^n или C_n^0 ;
- $v_{Cj} = 0$ соответствует выбору левого потомка ИЛИ-узла на j -м уровне поддерева узла с меткой C_n^m ;
- $v_{Cj} = 1$ соответствует выбору правого потомка ИЛИ-узла на j -м уровне поддерева узла с меткой C_n^m ;

Теорема 2.8. *Существует биекция между множеством нестрогих упорядочиваний n помеченных элементов и множеством вариантов дерева И/ИЛИ для F_n , представленного на рисунке 2.15.*

Доказательство. Основываясь на комбинаторном доказательстве для (2.15), биективное отображение между множеством вариантов дерева И/ИЛИ для F_n и множеством нестрогих упорядочиваний определяется по следующим правилам:

- выбранный потомок ИЛИ-узла с меткой F_n определяет количество I элементов, которые вместе занимают последнее место в нестрогом упорядочивании n помеченных элементов, и значение I определяется меткой $i := I$ выбранного узла;
- поддерево узла с меткой C_n^I (левое поддерево) определяет, какие I элементов из n элементов вместе занимают последнее место в нестрогом упорядочивании;
- поддерево узла с меткой F_{n-I} определяет нестрогое упорядочивание оставшихся $n - I$ помеченных элементов;
- лист с меткой F_0 определяет пустое нестрогое упорядочивание.

□

При этом для варианта поддерева узла с меткой C_n^m :

- лист с меткой C_n^n означает, что выбраны все n элементов;
- лист с меткой C_n^0 означает, что все n элементов не выбраны;
- $v_{Cj} = 0$ на j -м уровне означает, что $(n - j + 1)$ -й элемент не выбран;
- $v_{Cj} = 1$ на j -м уровне означает, что $(n - j + 1)$ -й элемент выбран.

Аналогично можно определить биекцию между множеством вариантов дерева И/ИЛИ для F_n , представленного на рисунке 2.15, и любым другим комбинаторным множеством, связанным с числами Фубини.

Используя предложенную методологию, разработаны алгоритмы ранжирования и генерации по рангу вариантов дерева И/ИЛИ для F_n (алгоритмы 2.19 и 2.21), и дерева И/ИЛИ для C_n^m (алгоритмы 2.20 и 2.22).

Алгоритм 2.19: Алгоритмы ранжирования варианта дерева И/ИЛИ для F_n при использовании формулы (2.1)

```

1 Rank_F( $v = (I, v_C, v_F), n$ )
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5      $sum := \sum_{i=1}^{I-1} C_n^i \cdot F_{n-i}$ 
6      $w_1 := C_n^I$ 
7      $l_1 := \text{Rank}_C(v_C, n, I)$ 
8      $l_2 := \text{Rank}_F(v_F, n - I)$ 
9      $r := sum + l_1 + w_1 \cdot l_2$ 
10  end
11  return  $r$ 
12 end
```

Алгоритм 2.20: Алгоритмы ранжирования варианта дерева И/ИЛИ для C_n^m при использовании формулы (2.2)

```

1 Rank_C( $v = (v_1, v_2, \dots), n, m$ )
2 begin
3   if  $m = 0$  или  $m = n$  then  $r := 0$ 
4   else
5     if  $v_1 = 0$  then  $r := \text{Rank}_C((v_2, \dots), n - 1, m)$ 
6     else  $r := C_{n-1}^m + \text{Rank}_C((v_2, \dots), n - 1, m - 1)$ 
7   end
8   return  $r$ 
9 end
```

В таблице 2.2 показан пример представления нестрогих упорядочиваний 3 помеченных элементов вариантами дерева И/ИЛИ для F_3 , представленного на рисунке 2.17, и результат их ранжирования с помощью разработанных алгоритмов.

Алгоритм 2.21: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для F_n при использовании формулы (2.1)

```

1  Unrank_F( $r, n$ )
2  begin
3      if  $n = 0$  then  $v := ()$ 
4      else
5           $sum := 0$ 
6          for  $i := 1$  to  $n$  do
7               $s := C_n^i \cdot F_{n-i}$ 
8              if  $sum + s > r$  then
9                   $r := r - sum$ 
10                  $I := i$ 
11                 break
12             end
13              $sum := sum + s$ 
14         end
15          $w_1 := C_n^I$ 
16          $l_1 := r \bmod w_1$ 
17          $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
18          $v_C := \text{Unrank\_C}(l_1, n, I)$ 
19          $v_F := \text{Unrank\_F}(l_2, n - I)$ 
20          $v := (I, v_C, v_F)$ 
21     end
22     return  $v$ 
23 end

```

Алгоритм 2.22: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для C_n^m при использовании формулы (2.2)

```

1  Unrank_C( $r, n, m$ )
2  begin
3      if  $m = 0$  или  $m = n$  then  $v := ()$ 
4      else
5          if  $r < C_{n-1}^m$  then  $v := \text{concat}((0), \text{Unrank\_C}(r, n - 1, m))$ 
6          else  $v := \text{concat}((1), \text{Unrank\_C}(r - C_{n-1}^m, n - 1, m - 1))$ 
7          end
8          return  $v$ 
9  end

```

Таблица 2.2 — Ранжирование множества нестрогих упорядочиваний
3 помеченных элементов с использованием алгоритма 2.19

Нестрогое упорядочивание 3 элементов a , b и c	Вариант дерева И/ИЛИ для F_3 , представленного на рисунке 2.17	Ранг
$a = b = c$	$(3, (), ())$	12
$a = b > c$	$(1, (1), (2, (), ()))$	8
$a = c > b$	$(1, (0, 1), (2, (), ()))$	7
$b = c > a$	$(1, (0, 0), (2, (), ()))$	6
$a > b = c$	$(2, (1, 1), (1, (), ()))$	11
$b > a = c$	$(2, (1, 0), (1, (), ()))$	10
$c > a = b$	$(2, (0), (1, (), ()))$	9
$a > b > c$	$(1, (1), (1, (1), (1, (), ())))$	5
$a > c > b$	$(1, (0, 1), (1, (1), (1, (), ())))$	4
$b > a > c$	$(1, (1), (1, (0), (1, (), ())))$	2
$c > a > b$	$(1, (0, 1), (1, (0), (1, (), ())))$	1
$b > c > a$	$(1, (0, 0), (1, (1), (1, (), ())))$	3
$c > b > a$	$(1, (0, 0), (1, (0), (1, (), ())))$	0

2.3.2 Рекурсивная композиция структур деревьев И/ИЛИ на основе чисел Стирлинга второго рода

Далее аналогичным образом рассмотрим построение структур деревьев И/ИЛИ и алгоритмов комбинаторной генерации с использованием формулы на основе чисел Стирлинга второго рода. На рисунке 2.18 представлена структура дерева И/ИЛИ для F_n при использовании формулы (2.3). Также требуются структуры деревьев И/ИЛИ для S_n^m (рисунок 2.19) при использовании формулы (2.4) и для P_n (рисунок 2.20) при использовании формулы (2.5).

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой S_n^1 является листом в дереве И/ИЛИ для S_n^m ;
- каждый узел с меткой S_n^n является листом в дереве И/ИЛИ для S_n^m ;
- каждый узел с меткой P_0 является листом в дереве И/ИЛИ для P_n .

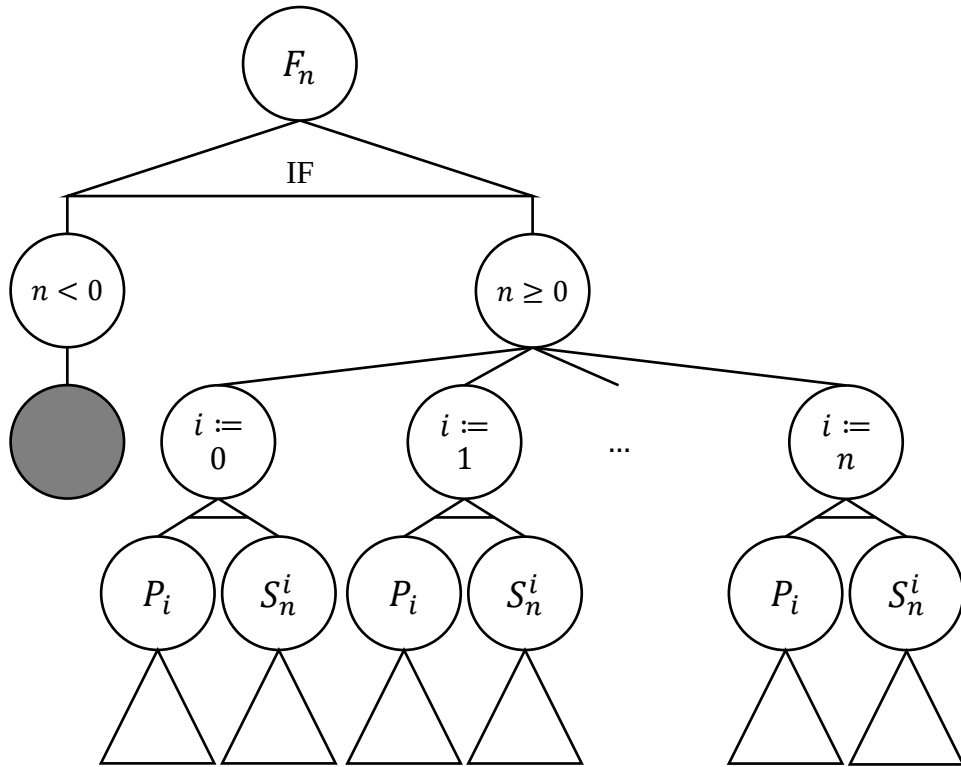


Рисунок 2.18 — Структура дерева И/ИЛИ для F_n при использовании формулы (2.3)

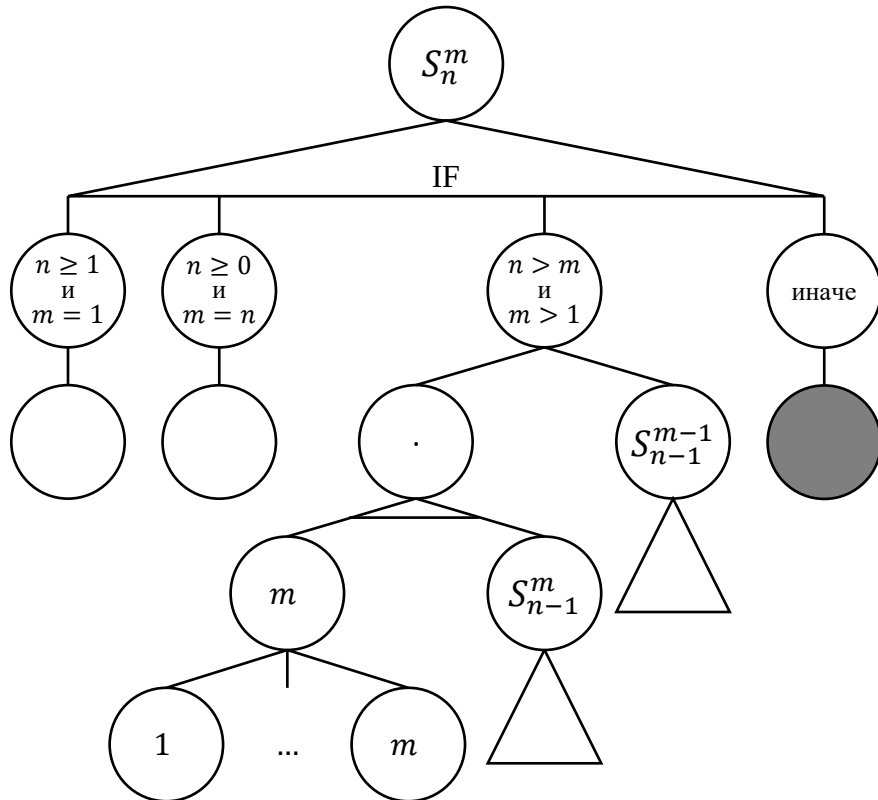


Рисунок 2.19 — Структура дерева И/ИЛИ для S_n^m при использовании формулы (2.4)

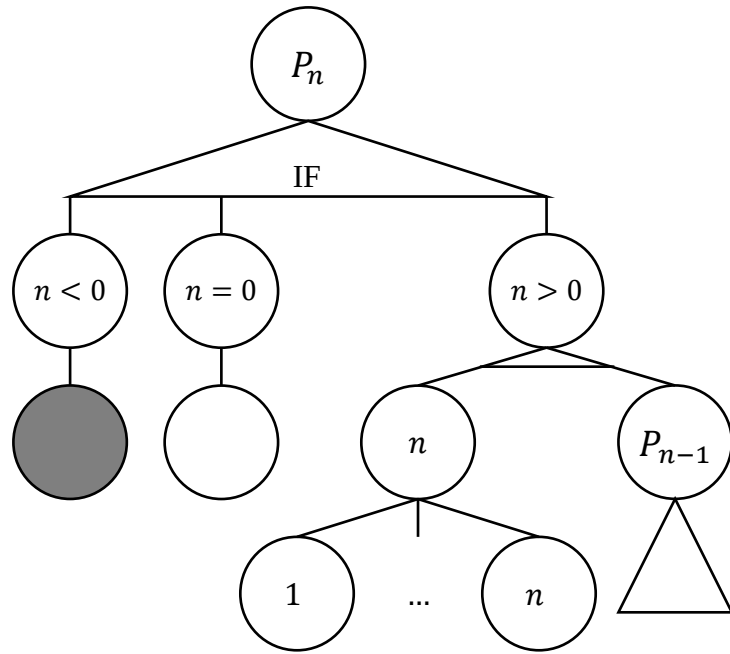


Рисунок 2.20 — Структура дерева И/ИЛИ для P_n при использовании формулы (2.5)

На рисунке 2.17 представлен пример структуры дерева И/ИЛИ для F_n , где $n = 3$. Общее количество его вариантов равно $F_3 = 13$.

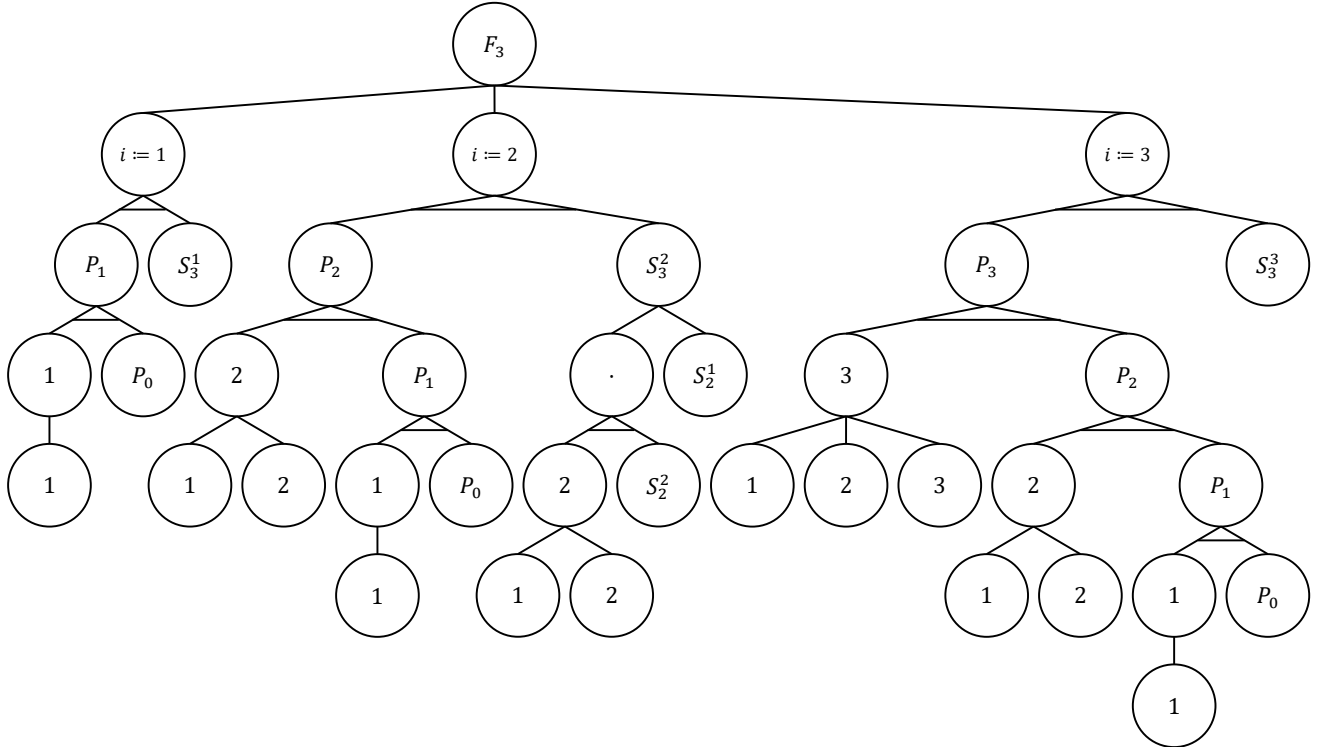


Рисунок 2.21 — Структура дерева И/ИЛИ для F_3 при использовании формулы (2.3)

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для F_n последовательностью $v = (I, v_P, v_S)$, где:

- I соответствует выбранному значению i в дереве И/ИЛИ для F_n ;
- v_P соответствует варианту поддерева узла с меткой P_I (левое поддерево);
- v_S соответствует варианту поддерева узла с меткой S_n^I (правое поддерево).

В свою очередь, вариант поддерева узла с меткой S_n^m кодируется последовательностью $v_S = (I_S, v_{SS})$, где:

- пустая последовательность $v_S = ()$ соответствует выбору листа с меткой S_n^1 или S_n^n ;
- если $I_S = 0$, то выбран правый потомок ИЛИ-узла с меткой S_n^m и v_{SS} соответствует варианту поддерева узла S_{n-1}^{m-1} ;
- если $I_S > 0$, то выбран левый потомок ИЛИ-узла с меткой S_n^m , I_S соответствует метке выбранного потомка ИЛИ-узла с меткой m и v_{SS} соответствует варианту поддерева узла S_{n-1}^m .

Также вариант поддерева узла с меткой P_n кодируется последовательностью $v_P = (v_1, \dots, v_n)$, где v_j соответствует метке выбранного потомка ИЛИ-узла на j -м уровне.

Теорема 2.9. *Существует биекция между множеством нестрогих упорядочиваний n помеченных элементов и множеством вариантов дерева И/ИЛИ для F_n , представленного на рисунке 2.18.*

Доказательство. Основываясь на комбинаторном доказательстве для (2.18), биективное отображение между множеством вариантов дерева И/ИЛИ для F_n и множеством нестрогих упорядочиваний определяется по следующим правилам:

- выбранный потомок ИЛИ-узла с меткой F_n определяет количество I подмножеств множества n элементов, где элементы подмножества вместе занимают одинаковое место в нестрогом упорядочивании, и значение I определяется меткой $i := I$ выбранного узла;
- поддерево узла с меткой S_n^I (правое поддерево) определяет способ разбиения n элементов на I непустых подмножеств;
- поддерево узла с меткой P_I (левое поддерево) определяет способ упорядочивания I подмножеств как перестановку I элементов.

При этом для варианта поддерева узла с меткой S_n^m :

- лист с меткой S_n^1 означает, что все n элементов разделены на n одноэлементных подмножеств;

- лист с меткой S_n^n означает, что n элементов объединены в одно множество;
- $I_S = 0$ для узла с меткой S_n^m означает, что n -й элемент в одиночку образует m -е подмножество, а поддерево узла с меткой S_{n-1}^{m-1} определяет разбиение оставшихся $n - 1$ элементов на остальные $m - 1$ подмножеств;
- $I_S > 0$ для узла с меткой S_n^m означает, что n -й элемент принадлежит I_S -му подмножеству среди m подмножеств, а поддерево узла с меткой S_{n-1}^m определяет разбиение оставшихся $n - 1$ элементов на m подмножеств.

Также для варианта поддерева узла с меткой P_n :

- лист с меткой P_0 определяет пустую перестановку элементов;
- v_j на j -м уровне поддерева узла с меткой P_n означает, что n -й элемент занимает v_j -ю позицию в перестановке n элементов, а поддерево узла с меткой P_{n-1} определяет перестановку оставшихся $n - 1$ элементов на оставшихся $n - 1$ позициях.

□

Аналогично можно определить биекцию между множеством вариантов дерева И/ИЛИ для F_n , представленного на рисунке 2.18, и любым другим комбинаторным множеством, связанным с числами Фубини.

Используя предложенную методологию, разработаны алгоритмы ранжирования и генерации по рангу вариантов дерева И/ИЛИ для F_n (алгоритмы 2.23 и 2.26), дерева И/ИЛИ для P_n (алгоритмы 2.24 и 2.27) и дерева И/ИЛИ для S_n^m (алгоритмы 2.25 и 2.28).

Алгоритм 2.23: Алгоритмы ранжирования варианта дерева И/ИЛИ для F_n при использовании формулы (2.3)

```

1 Rank_F( $v = (I, v_P, v_S), n$ )
2 begin
3    $sum := \sum_{i=0}^{I-1} P_i \cdot S_n^i$ 
4    $w_1 := P_I$ 
5    $l_1 := \text{Rank\_P}(v_P, I)$ 
6    $l_2 := \text{Rank\_S}(v_S, n, I)$ 
7    $r := sum + l_1 + w_1 \cdot l_2$ 
8   return  $r$ 
9 end
```

Алгоритм 2.24: Алгоритмы ранжирования варианта дерева И/ИЛИ
для P_n при использовании формулы (2.5)

```

1 Rank_P( $v = (v_1, \dots, v_n), n$ )
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5      $w_1 := n$ 
6      $l_1 := v_1 - 1$ 
7      $l_2 := \text{Rank\_P}((v_2, \dots, v_n), n - 1)$ 
8      $r := l_1 + w_1 \cdot l_2$ 
9   end
10  return  $r$ 
11 end

```

Алгоритм 2.25: Алгоритмы ранжирования варианта дерева И/ИЛИ
для S_n^m при использовании формулы (2.4)

```

1 Rank_S( $v = (I_S, v_{SS}), n, m$ )
2 begin
3   if  $m = 1$  или  $m = n$  then  $r := 0$ 
4   else
5     if  $I_S > 0$  then
6        $w_1 := m$ 
7        $l_1 := I_S - 1$ 
8        $l_2 := \text{Rank\_S}(v_{SS}, n - 1, m)$ 
9        $r := l_1 + w_1 \cdot l_2$ 
10    end
11    else
12       $sum := m \cdot S_{n-1}^m$ 
13       $l_3 := \text{Rank\_S}(v_{SS}, n - 1, m - 1)$ 
14       $r := sum + l_3$ 
15    end
16  end
17  return  $r$ 
18 end

```

Алгоритм 2.26: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для F_n при использовании формулы (2.3)

```

1 Unrank_F( $r, n$ )
2 begin
3    $sum := 0$ 
4   for  $i := 0$  to  $n$  do
5      $s := P_i \cdot S_n^i$ 
6     if  $sum + s > r$  then
7        $r := r - sum$ 
8        $I := i$ 
9       break
10    end
11     $sum := sum + s$ 
12  end
13   $w_1 := P_I$ 
14   $l_1 := r \bmod w_1$ 
15   $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
16   $v_P := \text{Unrank\_P}(l_1, I)$ 
17   $v_S := \text{Unrank\_S}(l_2, n, I)$ 
18   $v := (I, v_P, v_S)$ 
19  return  $v$ 
20 end

```

Алгоритм 2.27: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для P_n при использовании формулы (2.5)

```

1 Unrank_P( $r, n$ )
2 begin
3   if  $n = 0$  then  $v := ()$ 
4   else
5      $w_1 := n$ 
6      $l_1 := r \bmod w_1$ 
7      $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
8      $v := \text{concat}((l_1 + 1), \text{Unrank\_P}(l_2, n - 1))$ 
9   end
10  return  $v$ 
11 end

```

Алгоритм 2.28: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для S_n^m при использовании формулы (2.4)

```

1  Unrank_S( $r, n, m$ )
2  begin
3      if  $m = 1$  или  $m = n$  then  $v := ()$ 
4      else
5           $sum := m \cdot S_{n-1}^m$ 
6          if  $r < sum$  then
7               $w_1 := m$ 
8               $l_1 := r \bmod w_1$ 
9               $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
10              $I_S := l_1 + 1$ 
11              $v_{SS} := \text{Unrank\_S}(l_2, n - 1, m)$ 
12         end
13         else
14              $l_3 := r - sum$ 
15              $I_S := 0$ 
16              $v_{SS} := \text{Unrank\_S}(l_3, n - 1, m - 1)$ 
17         end
18          $v := (I_S, v_{SS})$ 
19     end
20     return  $v$ 
21 end

```

В таблице 2.3 показан пример представления нестрогих упорядочиваний 3 помеченных элементов вариантами дерева И/ИЛИ для F_3 , представленного на рисунке 2.21, и результат их ранжирования с помощью разработанных алгоритмов. В отличие от результатов, представленных в таблице 2.2, применение алгоритма 2.23 приводит к другим значениям рангов. Следовательно, наблюдается изменение распределения рангов, что в дальнейшем может отразиться на результатах применения статистических методов сжатия данных к файлам с рангами комбинаторных объектов.

Таблица 2.3 — Ранжирование множества нестрогих упорядочиваний
3 помеченных элементов с использованием алгоритма 2.23

Нестрогое упорядочивание 3 элементов a , b и c	Вариант дерева И/ИЛИ для F_3 , представленного на рисунке 2.21	Ранг
$a = b = c$	$(1, (1), ())$	0
$a = b > c$	$(2, (2, 1), (0, ()))$	6
$a = c > b$	$(2, (2, 1), (1, ()))$	2
$b = c > a$	$(2, (1, 1), (2, ()))$	3
$a > b = c$	$(2, (2, 1), (2, ()))$	4
$b > a = c$	$(2, (1, 1), (1, ()))$	1
$c > a = b$	$(2, (1, 1), (0, ()))$	5
$a > b > c$	$(3, (3, 2, 1), ())$	12
$a > c > b$	$(3, (2, 2, 1), ())$	11
$b > a > c$	$(3, (3, 1, 1), ())$	9
$c > a > b$	$(3, (1, 2, 1), ())$	10
$b > c > a$	$(3, (2, 1, 1), ())$	8
$c > b > a$	$(3, (1, 1, 1), ())$	7

2.3.3 Рекурсивная композиция структур деревьев И/ИЛИ на основе чисел Эйлера первого рода

Также рассмотрим построение структур деревьев И/ИЛИ и алгоритмов комбинаторной генерации с использованием формулы на основе чисел Эйлера первого рода. На рисунке 2.22 представлена структура дерева И/ИЛИ для F_n при использовании формулы (2.6). Также требуются структуры деревьев И/ИЛИ для E_n^m (рисунок 2.23) при использовании формулы (2.7) и для Q_n (рисунок 2.24) при использовании формулы (2.8).

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой E_n^0 является листом в дереве И/ИЛИ для E_n^m ;
- каждый узел с меткой E_n^{n-1} является листом в дереве И/ИЛИ для E_n^m ;
- каждый узел с меткой Q_0 является листом в дереве И/ИЛИ для Q_n .

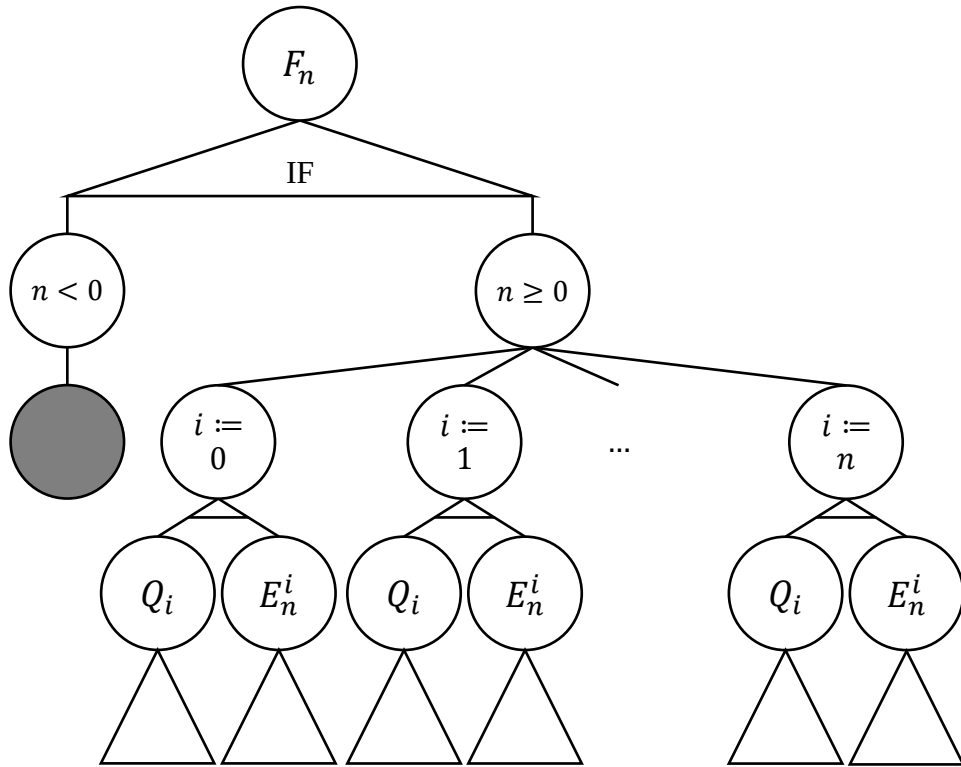


Рисунок 2.22 — Структура дерева И/ИЛИ для F_n при использовании формулы (2.6)

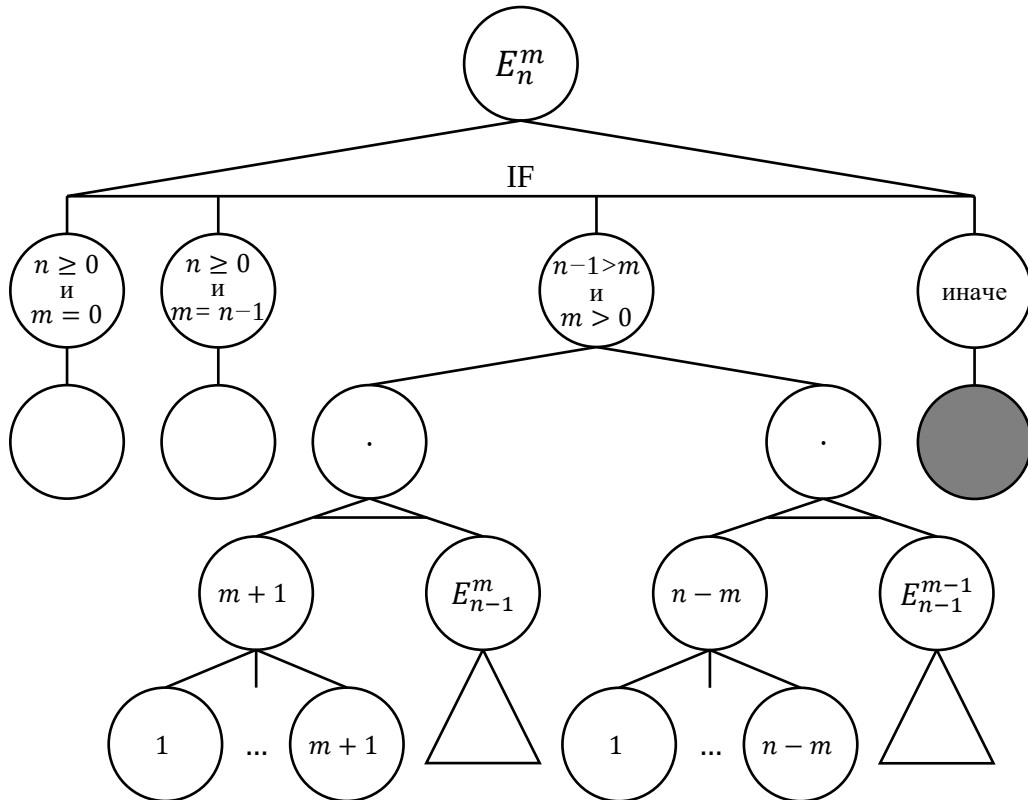


Рисунок 2.23 — Структура дерева И/ИЛИ для E_n^m при использовании формулы (2.7)

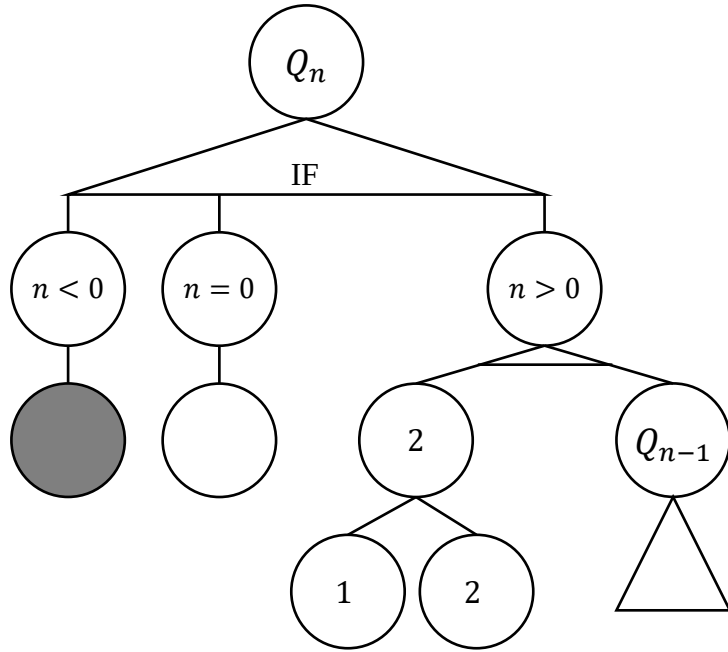


Рисунок 2.24 — Структура дерева И/ИЛИ для Q_n при использовании формулы (2.8)

На рисунке 2.25 представлен пример структуры дерева И/ИЛИ для F_n , где $n = 3$. Общее количество его вариантов равно $F_3 = 13$.

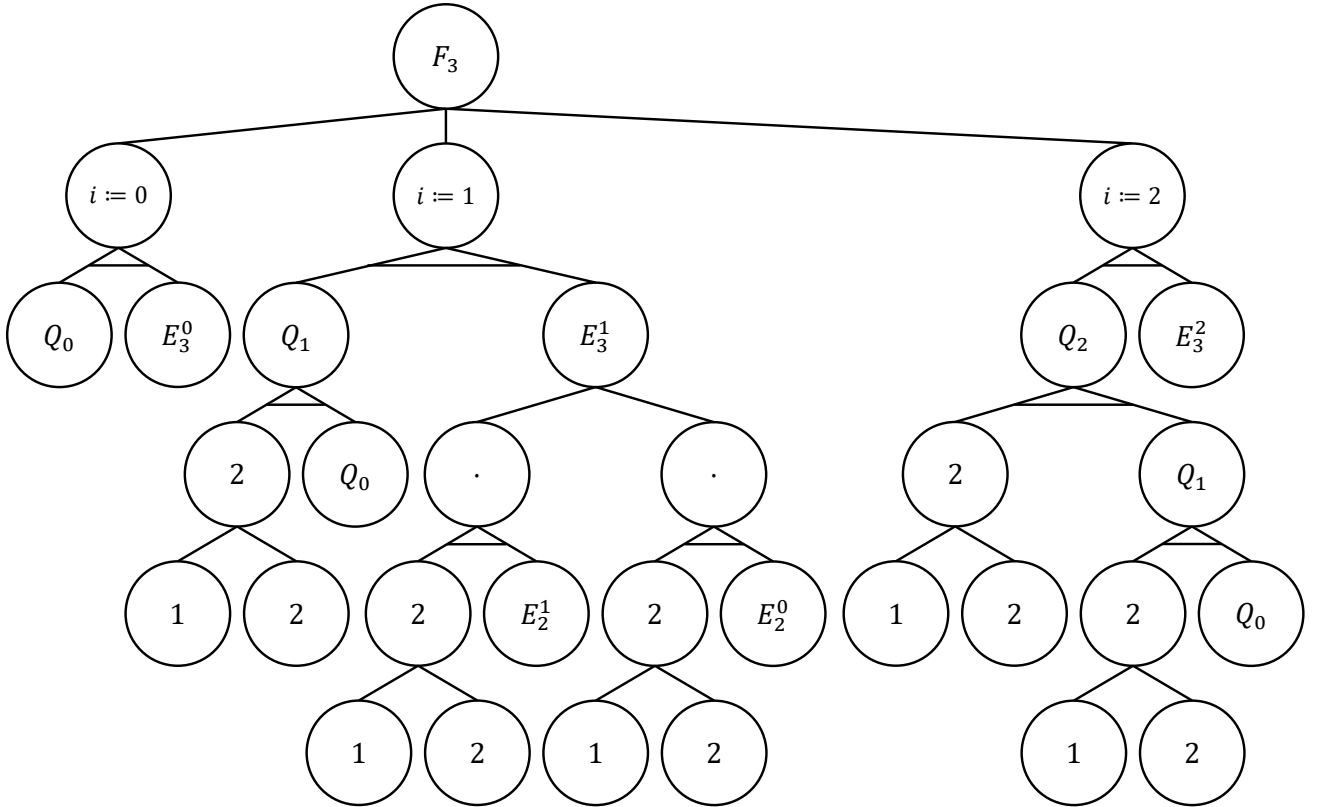


Рисунок 2.25 — Структура дерева И/ИЛИ для F_3 при использовании формулы (2.6)

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для F_n последовательностью $v = (I, v_Q, v_E)$, где:

- I соответствует выбранному значению i в дереве И/ИЛИ для F_n ;
- v_Q соответствует варианту поддерева узла с меткой Q_I (левое поддерево);
- v_E соответствует варианту поддерева узла с меткой E_n^I (правое поддерево).

В свою очередь, вариант поддерева узла с меткой E_n^m кодируется последовательностью $v_E = (I_{E1}, I_{E2}, v_{EE})$, где:

- пустая последовательность $v_E = ()$ соответствует выбору листа с меткой E_n^0 или E_n^{n-1} ;
- если $I_{E1} = 0$, то выбран левый потомок ИЛИ-узла с меткой E_n^m , I_{E2} соответствует метке выбранного потомка ИЛИ-узла с меткой $m + 1$ и v_{EE} соответствует варианту поддерева узла E_{n-1}^m ;
- если $I_{E1} = 1$, то выбран правый потомок ИЛИ-узла с меткой E_n^m , I_{E2} соответствует метке выбранного потомка ИЛИ-узла с меткой $n - m$ и v_{EE} соответствует варианту поддерева узла E_{n-1}^{m-1} .

Также вариант поддерева узла с меткой Q_n кодируется последовательностью $v_Q = (v_1, \dots, v_n)$, где v_j соответствует метке выбранного потомка ИЛИ-узла на j -м уровне.

Теорема 2.10. *Существует биекция между множеством нестрогих упорядочиваний n помеченных элементов и множеством вариантов дерева И/ИЛИ для F_n , представленного на рисунке 2.22.*

Доказательство. Основываясь на комбинаторном доказательстве для (2.22), биективное отображение между множеством вариантов дерева И/ИЛИ для F_n и множеством нестрогих упорядочиваний определяется по следующим правилам:

- выбранный потомок ИЛИ-узла с меткой F_n определяет количество I подъемов в перестановке n элементов, которая определяет порядок элементов в нестрогом упорядочивании, и значение I определяется меткой $i := I$ выбранного узла;
- поддерево узла с меткой E_n^I (правое поддерево) определяет способ перестановки n элементов с I подъемами;
- поддерево узла с меткой Q_I (левое поддерево) определяет необходимость приравнивания элементов, расположенных на местах I подъемов.

При этом для варианта поддерева узла с меткой E_n^m :

- лист с меткой E_n^0 означает, что n элементов расположены в порядке убывания;
- лист с меткой E_n^{n-1} означает, что n элементов расположены в порядке возрастания;
- $I_{E1} = 0$ для узла с меткой E_n^m означает, что n -й элемент не добавляет подъем в перестановку оставшихся $n - 1$ элементов, I_{E2} определяет позицию n -го элемента в перестановке оставшихся $n - 1$ элементов (имеется всего $m + 1$ подходящих позиций), а поддерево узла с меткой E_{n-1}^m определяет перестановку оставшихся $n - 1$ элементов с m подъемами;
- $I_{E1} = 1$ для узла с меткой E_n^m означает, что n -й элемент добавляет подъем в перестановку оставшихся $n - 1$ элементов, I_{E2} определяет позицию n -го элемента в перестановке оставшихся $n - 1$ элементов (имеется всего $n - m$ подходящих позиций), а поддерево узла с меткой E_{n-1}^{m-1} определяет перестановку оставшихся $n - 1$ элементов с $m - 1$ подъемом.

Также для варианта поддерева узла с меткой Q_n :

- лист с меткой Q_0 означает отсутствие действий с перестановкой, так как в перестановке нет подъемов;
- $v_j = 1$ на j -м уровне поддерева узла с меткой 2 означает, что элементы, расположенные на месте j -го подъема, имеют разные места в нестрогом упорядочивании (между ними стоит символ «>»;
- $v_j = 2$ на j -м уровне поддерева узла с меткой 2 означает, что элементы, расположенные на месте j -го подъема, имеют одинаковое место в нестрогом упорядочивании (между ними стоит символ «=».

□

Используя предложенную методологию, разработаны алгоритмы ранжирования и генерации по рангу вариантов дерева И/ИЛИ для F_n (алгоритмы 2.29 и 2.32), дерева И/ИЛИ для E_n^m (алгоритмы 2.30 и 2.33) и дерева И/ИЛИ для Q_n (алгоритмы 2.31 и 2.34).

В таблице 2.4 показан пример представления нестрогих упорядочиваний 3 помеченных элементов вариантами дерева И/ИЛИ для F_3 , представленного на рисунке 2.25, и результат их ранжирования с помощью разработанных алгоритмов.

Алгоритм 2.29: Алгоритмы ранжирования варианта дерева И/ИЛИ
для F_n при использовании формулы (2.6)

```

1 Rank_F( $v = (I, v_Q, v_E), n$ )
2 begin
3    $sum := \sum_{i=0}^{I-1} Q_i \cdot E_n^i$ 
4    $w_1 := Q_I$ 
5    $l_1 := \text{Rank\_Q}(v_Q, I)$ 
6    $l_2 := \text{Rank\_E}(v_E, n, I)$ 
7    $r := sum + l_1 + w_1 \cdot l_2$ 
8   return  $r$ 
9 end
```

Алгоритм 2.30: Алгоритмы ранжирования варианта дерева И/ИЛИ
для E_n^m при использовании формулы (2.7)

```

1 Rank_E( $v = (I_{E1}, I_{E2}, v_{EE}), n, m$ )
2 begin
3   if  $m = 0$  или  $m = n - 1$  then  $r := 0$ 
4   else
5     if  $I_{E1} = 0$  then
6        $w_1 := m + 1$ 
7        $l_1 := I_{E2} - 1$ 
8        $l_2 := \text{Rank\_E}(v_{EE}, n - 1, m)$ 
9        $r := l_1 + w_1 \cdot l_2$ 
10    end
11    else
12       $sum := (m + 1) \cdot E_{n-1}^m$ 
13       $w_3 := n - m$ 
14       $l_3 := I_{E2} - 1$ 
15       $l_4 := \text{Rank\_E}(v_{EE}, n - 1, m - 1)$ 
16       $r := sum + l_3 + w_3 \cdot l_4$ 
17    end
18  end
19  return  $r$ 
20 end
```

Алгоритм 2.31: Алгоритмы ранжирования варианта дерева И/ИЛИ
для Q_n при использовании формулы (2.8)

```

1 Rank_Q( $v = (v_1, \dots, v_n), n$ )
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5      $w_1 := 2$ 
6      $l_1 := v_1 - 1$ 
7      $l_2 := \text{Rank\_Q}((v_2, \dots, v_n), n - 1)$ 
8      $r := l_1 + w_1 \cdot l_2$ 
9   end
10  return  $r$ 
11 end

```

Алгоритм 2.32: Алгоритмы генерации по рангу варианта дерева И/ИЛИ
для F_n при использовании формулы (2.6)

```

1 Unrank_F( $r, n$ )
2 begin
3    $sum := 0$ 
4   for  $i := 0$  to  $n$  do
5      $s := Q_i \cdot E_n^i$ 
6     if  $sum + s > r$  then
7        $r := r - sum$ 
8        $I := i$ 
9       break
10    end
11     $sum := sum + s$ 
12  end
13   $w_1 := Q_I$ 
14   $l_1 := r \bmod w_1$ 
15   $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
16   $v_Q := \text{Unrank\_Q}(l_1, I)$ 
17   $v_E := \text{Unrank\_E}(l_2, n, I)$ 
18   $v := (I, v_Q, v_E)$ 
19  return  $v$ 
20 end

```

Алгоритм 2.33: Алгоритмы генерации по рангу варианта дерева И/ИЛИ
для E_n^m при использовании формулы (2.7)

```

1  Unrank_E( $r, n, m$ )
2  begin
3      if  $m = 0$  или  $m = n - 1$  then  $v := ()$ 
4      else
5           $sum := (m + 1) \cdot E_{n-1}^m$ 
6          if  $r < sum$  then
7               $w_1 := m + 1$ 
8               $l_1 := r \bmod w_1$ 
9               $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
10              $I_{E1} := 0$ 
11              $I_{E2} := l_1 + 1$ 
12              $v_{EE} := \text{Unrank\_E}(l_2, n - 1, m)$ 
13         end
14         else
15              $r := r - sum$ 
16              $w_3 := n - m$ 
17              $l_3 := r \bmod w_3$ 
18              $l_4 := \left\lfloor \frac{r}{w_3} \right\rfloor$ 
19              $I_{E1} := 1$ 
20              $I_{E2} := l_3 + 1$ 
21              $v_{EE} := \text{Unrank\_E}(l_4, n - 1, m - 1)$ 
22         end
23          $v := (I_{E1}, I_{E2}, v_{EE})$ 
24     end
25     return  $v$ 
26 end
```

Алгоритм 2.34: Алгоритмы генерации по рангу варианта дерева И/ИЛИ для Q_n при использовании формулы (2.8)

```

1 Unrank_Q( $r, n$ )
2 begin
3   if  $n = 0$  then  $v := ()$ 
4   else
5      $w_1 := 2$ 
6      $l_1 := r \bmod w_1$ 
7      $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
8      $v := \text{concat}((l_1 + 1), \text{Unrank\_Q}(l_2, n - 1))$ 
9   end
10  return  $v$ 
11 end

```

Таблица 2.4 — Ранжирование множества нестрогих упорядочиваний
3 помеченных элементов с использованием алгоритма 2.29

Нестрогое упорядочивание 3 элементов a, b и c	Вариант дерева И/ИЛИ для F_3 , представленного на рисунке 2.21	Ранг
$a = b = c$	$(2, (2, 2), ())$	12
$a = b > c$	$(2, (2, 1), ())$	10
$a = c > b$	$(1, (2), (0, 2, ()))$	4
$b = c > a$	$(1, (2), (1, 1, ()))$	6
$a > b = c$	$(2, (1, 2), ())$	11
$b > a = c$	$(1, (2), (1, 2, ()))$	8
$c > a = b$	$(1, (2), (0, 1, ()))$	2
$a > b > c$	$(2, (1, 1), ())$	9
$a > c > b$	$(1, (1), (0, 2, ()))$	3
$b > a > c$	$(1, (1), (1, 2, ()))$	7
$c > a > b$	$(1, (1), (0, 1, ()))$	1
$b > c > a$	$(1, (1), (1, 1, ()))$	5
$c > b > a$	$(0, (), ())$	0

2.4 Выводы по главе

В данной главе проведен анализ ограничений алгебры функций мощности комбинаторных множеств $\{+, \times, \mathbb{Z}_{>0}, R\}$ при построении алгоритмов ранжирования и генерации по рангу с использованием математического аппарата структур деревьев И/ИЛИ и их вариантов. В частности, алгебра $\{+, \times, \mathbb{Z}_{>0}, R\}$:

- не учитывает применение операторов суммирования и произведения;
- не способна обрабатывать кусочно-заданные функции, требующие условных операторов и учета нулевых значений;
- не позволяет выполнить композицию функций мощности различных комбинаторных множеств.

Для преодоления указанных ограничений в диссертационном исследовании предложена расширенная алгебра функций $\{+, \Sigma, \times, \Pi, \mathbb{Z}_{\geq 0}, I, C_{tree}, C_{alg}\}$. Доказана фундаментальная возможность построения структур деревьев И/ИЛИ, а также соответствующих алгоритмов ранжирования и генерации по рангу для любого комбинаторного множества, функция мощности которого выражима в терминах расширенной алгебры. Апробация предложенных теоретических основ проведена на классе комбинаторных объектов, связанных с числами Фубини: рассмотрены три функции мощности комбинаторного множества (на основе биномиальных коэффициентов, чисел Стирлинга второго рода и чисел Эйлера первого рода), построены соответствующие структуры деревьев И/ИЛИ, реализованы алгоритмы ранжирования и генерации по рангу.

На основе полученных результатов сформулирована общая методология построения алгоритмов кодирования информационных объектов, основанная на рекурсивных композициях структур деревьев И/ИЛИ. Методология формируется из базовых правил построения структур деревьев И/ИЛИ (алгоритмы 2.1–2.10), доказанных теорем о расширении алгебры функций мощности комбинаторных множеств (теоремы 2.1–2.6) и алгоритмов их применения, а также на основе схемы кодирования с использованием промежуточного представления информационного объекта в виде структуры варианта дерева И/ИЛИ (рисунок 2.14). Ключевой особенностью методологии является возможность комбинирования известных алгоритмов, полученных с использованием совершенно другого математического аппарата.

Результаты данной главы опубликованы в [186–190].

Глава 3. Кодирование информационных объектов на основе операций над производящими функциями

В данной главе представлены результаты исследования связи производящих функций с алгоритмами комбинаторной генерации. Используя формулы из параграфа 1.4 для вычисления мощности комбинаторных множеств, сформированных в результате выполнения операций над производящими функциями, строятся соответствующие рекурсивные композиции структур деревьев И/ИЛИ и алгоритмы комбинаторной генерации. Кроме того, решается задача кодирования информационных объектов, мощность множества которых определяется алгебраической производящей функцией. В качестве апробации рассматриваются частные случаи алгебраических производящих функций для классических числовых последовательностей (числа Фибоначчи, Пелля, Каталана, Моцкина, Шредера).

3.1 Рекурсивные композиции структур деревьев И/ИЛИ для комбинаторных множеств, сформированных в результате выполнения операций над производящими функциями

В параграфе 1.4 исследовано влияние операций над производящими функциями на связанные с ними множества информационных объектов. В частности, если рассмотреть комбинаторные множества A_n и B_n , которые содержат объекты размера n (или любого другого количественного параметра), то выполнение операций над производящими функциями $A(x)$ и $B(x)$ для последовательностей значений их мощностей $|A_n|$ и $|B_n|$ формирует новые комбинаторные множества, связанные с элементами исходных множеств. Поскольку формулы (1.1), (1.3), (1.4) и (1.5) удовлетворяют требованиям предложенной методологии, то для таких комбинаторных множеств можно построить соответствующие структуры дерева И/ИЛИ, а также алгоритмы комбинаторной генерации на их основе. Таким образом, становится возможной разработка алгоритмов кодирования информационных объектов, мощность множества которых определяется производящей функцией.

Кроме того, на основе декомпозиции производящей функции для сложной дискретной структуры определяется способ комбинирования алгоритмов

комбинаторной генерации для ее подструктур. Это, в свою очередь, позволяет использовать существующие алгоритмы, в основе которых лежит произвольный метод построения и которые обладают наилучшими показателями эффективности.

Далее рассмотрим следующие операции над производящими функциями и соответствующие им рекурсивные композиции структур деревьев И/ИЛИ, а также алгоритмы комбинаторной генерации.

3.1.1 Сложение производящих функций

Выполнение операции сложения производящих функций $A(x) + B(x)$ формирует новое комбинаторное множество H_n , связанное с элементами комбинаторных множеств A_n и B_n . Мощность комбинаторного множества H_n определяется формулой (1.1), которой соответствует композиция структур деревьев И/ИЛИ, представленная на рисунке 3.1. Полученная древовидная структура объединяет в себе соответствующие структуры деревьев И/ИЛИ для комбинаторных множеств A_n и B_n .

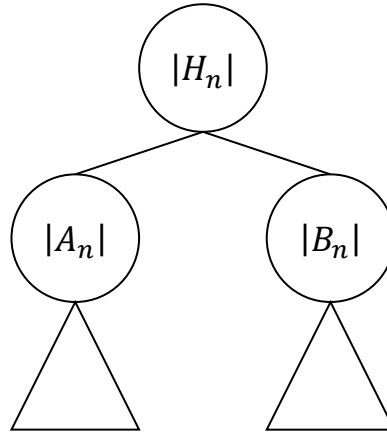


Рисунок 3.1 — Структура дерева И/ИЛИ для $|H_n| = [x^n] (A(x) + B(x))$

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 3.1) и генерации по рангу (алгоритм 3.2) вариантов дерева И/ИЛИ для $|H_n|$, представленного на рисунке 3.1. Разработанные алгоритмы базируются на применении алгоритмов ранжирования и генерации по рангу для комбинаторного множества A_n (алгоритмы $\text{Rank_A}(\text{object}, n)$ и $\text{Unrank_A}(r, n)$) и комбинаторного множеств B_n (алгоритмы $\text{Rank_B}(\text{object}, n)$ и $\text{Unrank_B}(r, n)$). Данные алгоритмы могут быть произвольной природы происхождения, что позволяет комбинировать алгоритмы комбинаторной генерации с наилучшими показателями эффективности. Также в этих алгоритмах используется обозначение ε для пустого объекта, то есть объекта размера $n = 0$.

Алгоритм 3.1: Алгоритм ранжирования для комбинаторного множества H_n при $|H_n| = [x^n](A(x) + B(x))$

```

1 Rank_Hsum(object, n)
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5     if  $\textit{object} \in A_n$  then  $r := \text{Rank\_A}(\textit{object}, n)$ 
6     else  $r := |A_n| + \text{Rank\_B}(\textit{object}, n)$ 
7   end
8   return  $r$ 
9 end
```

Алгоритм 3.2: Алгоритм генерации по рангу для комбинаторного множества H_n при $|H_n| = [x^n](A(x) + B(x))$

```

1 Unrank_Hsum( $r$ ,  $n$ )
2 begin
3   if  $n = 0$  then  $\textit{object} := \varepsilon$ 
4   else
5     if  $r < |A_n|$  then  $\textit{object} := \text{Unrank\_A}(r, n)$ 
6     else  $\textit{object} := \text{Unrank\_B}(r - |A_n|, n)$ 
7   end
8   return  $\textit{object}$ 
9 end
```

3.1.2 Умножение производящих функций

Выполнение операции умножения производящих функций $A(x) \cdot B(x)$ формирует новое комбинаторное множество H_n , связанное с элементами комбинаторных множеств A_n и B_n . Мощность комбинаторного множества H_n определяется формулой (1.3), которой соответствует композиция структур деревьев И/ИЛИ, представленная на рисунке 3.2. Полученная древовидная структура также объединяет в себе соответствующие структуры деревьев И/ИЛИ для комбинаторных множеств A_i и B_{n-i} , где $i \in \{0, 1, \dots, n\}$.

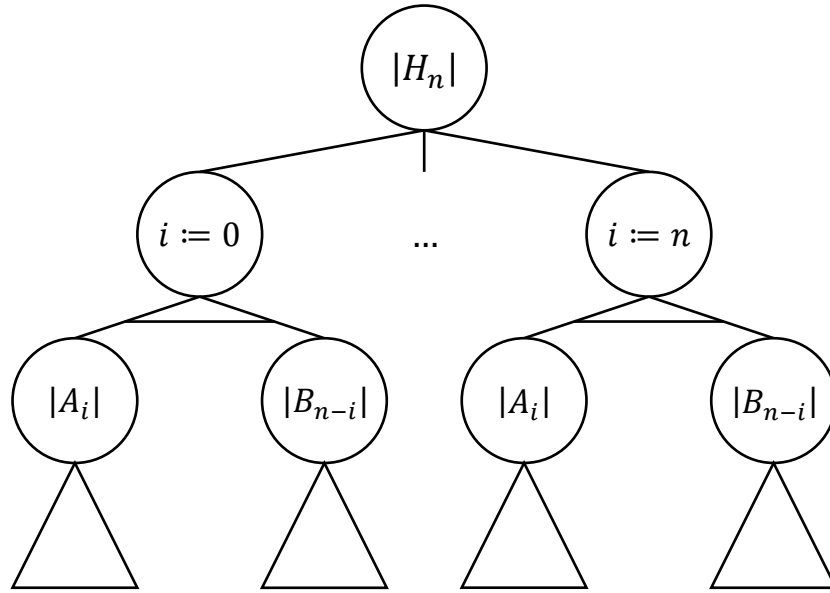


Рисунок 3.2 — Структура дерева И/ИЛИ для $|H_n| = [x^n] (A(x) \cdot B(x))$

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 3.3) и генерации по рангу (алгоритм 3.4) вариантов дерева И/ИЛИ для $|H_n|$, представленного на рисунке 3.2. Функция $\text{size}(\text{object})$ в алгоритме 3.3 реализует вычисление размера объекта object .

Алгоритм 3.3: Алгоритм ранжирования для комбинаторного множества H_n при $|H_n| = [x^n] (A(x) \cdot B(x))$

```

1 Rank_Hmul(object = (objecta, objectb), n)
2 begin
3   if n = 0 then r := 0
4   else
5     I := size(objecta)
6     sum :=  $\sum_{i=0}^{I-1} |A_i| \cdot |B_{n-i}|$ 
7     w1 := |AI|
8     l1 := Rank_A(objecta, I)
9     l2 := Rank_B(objectb, n - I)
10    r := sum + l1 + w1l2
11  end
12  return r
13 end

```

Алгоритм 3.4: Алгоритм генерации по рангу для комбинаторного множества H_n при $|H_n| = [x^n](A(x) \cdot B(x))$

```

1 Unrank_Hmul( $r, n$ )
2 begin
3   if  $n = 0$  then  $object := \varepsilon$ 
4   else
5      $sum := 0$ 
6     for  $i := 0$  to  $n$  do
7        $s := |A_i| \cdot |B_{n-i}|$ 
8       if  $sum + s > r$  then
9          $r := r - sum$ 
10         $I := i$ 
11        break
12      end
13       $sum := sum + s$ 
14    end
15     $w_1 := |A_I|$ 
16     $l_1 := r \bmod w_1$ 
17     $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
18     $object_a := \text{Unrank\_A}(l_1, I)$ 
19     $object_b := \text{Unrank\_B}(l_2, n - I)$ 
20     $object := (object_a, object_b)$ 
21  end
22  return  $object$ 
23 end

```

3.1.3 Возведение производящей функции в степень

Выполнение операции возведения производящей функции в k -ю степень $A(x)^k$ формирует новое комбинаторное множество $H_{n,k}$, связанное с элементами комбинаторного множества A_n . Мощность комбинаторного множества $H_{n,k}$ определяется формулой (1.4), которой соответствует рекурсивная композиция структур деревьев И/ИЛИ, представленная на рисунке 3.3.

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 3.5) и генерации по рангу (алгоритм 3.6) вариантов дерева И/ИЛИ для $|H_{n,k}|$, представленного на рисунке 3.3. Функция $\text{concat}(a, b)$ в алгоритме 3.6 реализует конкатенацию кортежей.

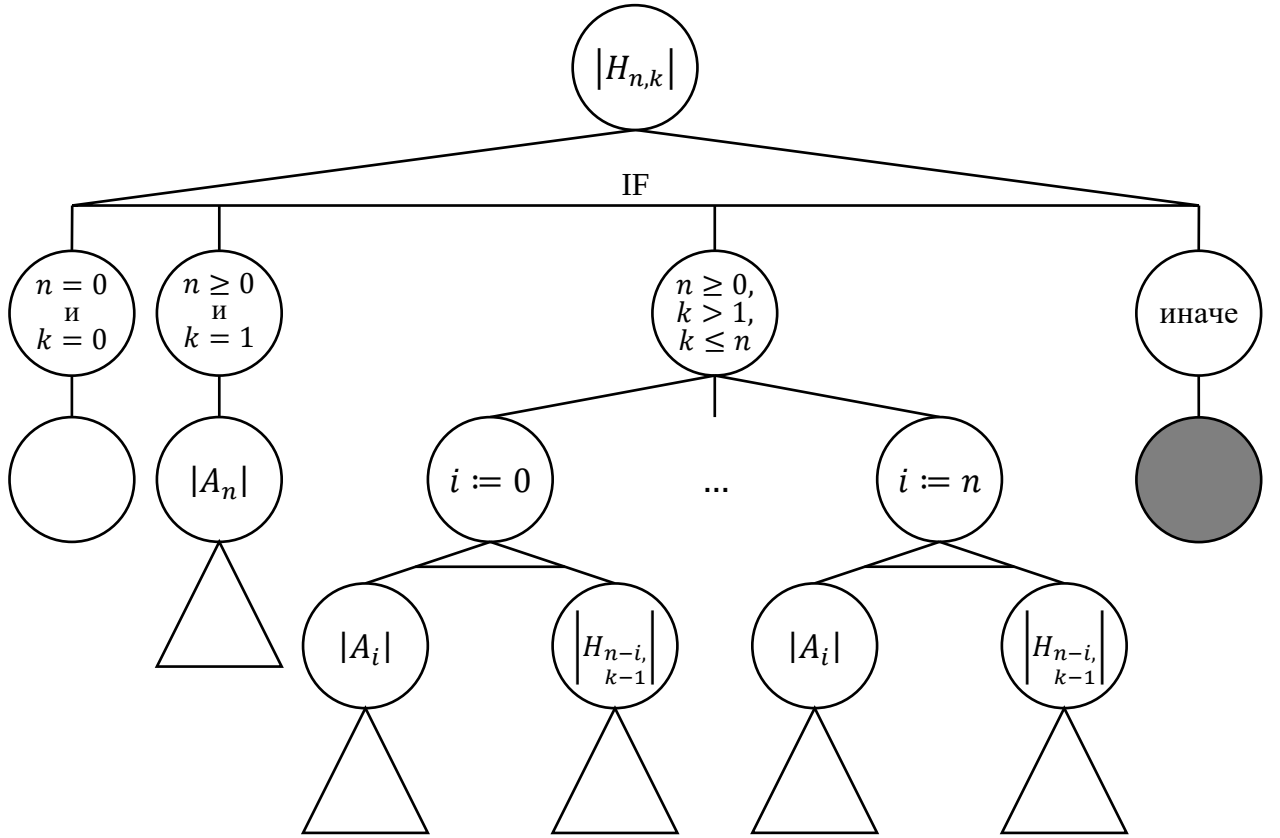


Рисунок 3.3 — Структура дерева И/ИЛИ для $|H_{n,k}| = [x^n] (A(x)^k)$

Алгоритм 3.5: Алгоритм ранжирования для комбинаторного множества $H_{n,k}$ при $|H_{n,k}| = [x^n] (A(x)^k)$

```

1 Rank_Hpow(object = (objecta, objecth), n, k)
2 begin
3   if n = 0 и k = 0 then r := 0
4   else if k = 1 then r := Rank_A(object, n)
5   else
6     I := size(objecta)
7     sum :=  $\sum_{i=0}^{I-1} |A_i| \cdot |H_{n-i, k-1}|$ 
8     w1 := |AI|
9     l1 := Rank_A(objecta, I)
10    l2 := Rank_Hpow(objecth, n - I, k - 1)
11    r := sum + l1 + w1l2
12  end
13  return r
14 end

```

Алгоритм 3.6: Алгоритм генерации по рангу для комбинаторного множества $H_{n,k}$ при $|H_{n,k}| = [x^n] (A(x)^k)$

```

1 Unrank_Hpow( $r, n, k$ )
2 begin
3   if  $n = 0$  u  $k = 0$  then  $object := \varepsilon$ 
4   else if  $k = 1$  then  $object := \text{Unrank\_A}(r, n)$ 
5   else
6      $sum := 0$ 
7     for  $i := 0$  to  $n$  do
8        $s := |A_i| \cdot |H_{n-i,k-1}|$ 
9       if  $sum + s > r$  then
10         $r := r - sum$ 
11         $I := i$ 
12        break
13      end
14       $sum := sum + s$ 
15    end
16     $w_1 := |A_I|$ 
17     $l_1 := r \bmod w_1$ 
18     $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
19     $object_a := \text{Unrank\_A}(l_1, I)$ 
20     $object_h := \text{Unrank\_Hpow}(l_2, n - I, k - 1)$ 
21     $object := \text{concat}((object_a), object_h)$ 
22  end
23  return  $object$ 
24 end

```

3.1.4 Композиция производящих функций

Выполнение операции композиции производящих функций $A(B(x))$ формирует новое комбинаторное множество H_n , связанное с элементами комбинаторных множеств A_n и B_n . Мощность комбинаторного множества H_n определяется формулой (1.5), которой соответствует рекурсивная композиция структур деревьев И/ИЛИ, представленная на рисунке 3.4.

Полученная древовидная структура содержит в себе структуру дерева И/ИЛИ для комбинаторного множества $B_{n,i}$ при $|B_{n,i}| = [x^n] (B(x)^i)$.

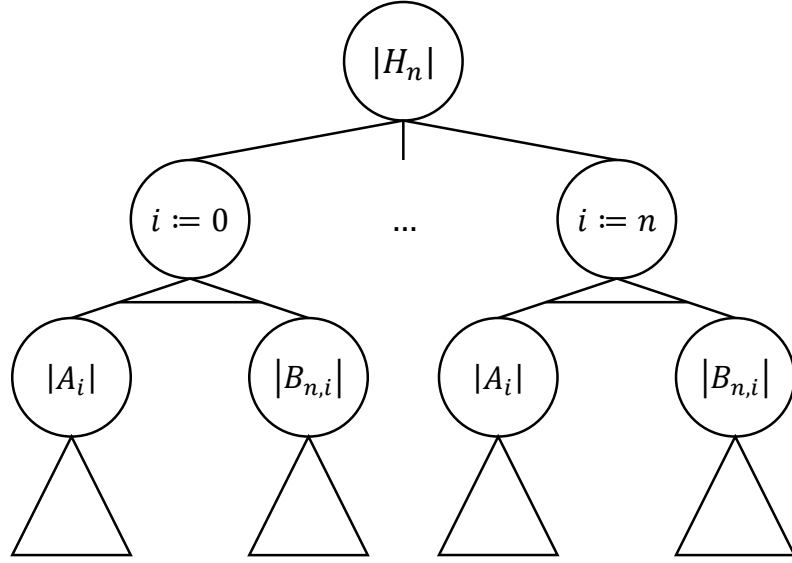


Рисунок 3.4 — Структура дерева И/ИЛИ для $|H_n| = [x^n] (A(B(x)))$

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 3.7) и генерации по рангу (алгоритм 3.8) вариантов дерева И/ИЛИ для $|H_n|$, представленного на рисунке 3.4.

Алгоритм 3.7: Алгоритм ранжирования для комбинаторного множества H_n при $|H_n| = [x^n] (A(B(x)))$

```

1 Rank_Hcomp(object = (objecta, objectb), n)
2 begin
3   if n = 0 then r := 0
4   else
5     I := size(objecta)
6     sum :=  $\sum_{i=0}^{I-1} |A_i| \cdot |B_{n,i}|$ 
7     w1 := |AI|
8     l1 := Rank_A(objecta, I)
9     l2 := Rank_Bpow(objectb, n, I)
10    r := sum + l1 + w1l2
11  end
12  return r
13 end

```

Алгоритм 3.8: Алгоритм генерации по рангу для комбинаторного множества H_n при $|H_n| = [x^n] (A(B(x)))$

```

1  Unrank_Hcomp( $r, n$ )
2  begin
3      if  $n = 0$  then  $object := \varepsilon$ 
4      else
5           $sum := 0$ 
6          for  $i := 0$  to  $n$  do
7               $s := |A_i| \cdot |B_{n,i}|$ 
8              if  $sum + s > r$  then
9                   $r := r - sum$ 
10                  $I := i$ 
11                 break
12             end
13              $sum := sum + s$ 
14         end
15          $w_1 := |A_I|$ 
16          $l_1 := r \bmod w_1$ 
17          $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
18          $object_a := \text{Unrank\_A}(l_1, I)$ 
19          $object_b := \text{Unrank\_Bpow}(l_2, n, I)$ 
20          $object := (object_a, object_b)$ 
21     end
22     return  $object$ 
23 end
```

Также заметим, что полученные алгоритмы комбинаторной генерации основаны на применении соответствующих алгоритмов $\text{Rank_Bpow}(object, n, k)$ и $\text{Unrnk_Bpow}(r, n, k)$ для комбинаторного множества $B_{n,k}$, полученного путем возведения производящей функции $B(x)$ в k -ю степень, то есть $|B_{n,k}| = [x^n] (B(x)^k)$. Содержание таких алгоритмов аналогично содержанию алгоритмов 3.5 и 3.6 для случая комбинаторного множества $H_{n,k}$ при $|H_{n,k}| = [x^n] (A(x)^k)$.

3.2 Кодирование информационных объектов, мощность множества которых определяется алгебраической производящей функцией

Основываясь на результатах предыдущего параграфа получаем следующий вывод: декомпозиция производящей функции задает способ комбинирования алгоритмов комбинаторной генерации для простых структур, из которых формируется исследуемая сложная структура. В свою очередь, это расширяет возможности разработки новых алгоритмов комбинаторной генерации, поскольку определяет дополнительный путь к решению данной задачи. В частности, на идее декомпозиции производящей функции основан символьный метод построения алгоритмов комбинаторной генерации, представленный в цикле работ [127–130]. Однако, если для производящей функции отсутствует аналитическое выражение, то решение задачи декомпозиции сильно усложняется. Например, такая ситуация возникает для производящей функции, заданной в виде функционального уравнения.

Рассмотрим класс алгебраических производящих функций [147].

Определение 3.1. *Производящая функция $A(x)$ называется алгебраической, если существуют многочлены вида $P_i(x) = \sum_{j=0}^{n_i} p_i(j) x^j$, где $i \in \{0, 1, \dots, m\}$, которые не все равны нулю и для которых выполняется*

$$P_0(x) + P_1(x) \cdot A(x) + P_2(x) \cdot A(x)^2 + \dots + P_m(x) \cdot A(x)^m = 0. \quad (3.1)$$

Если $p_1(0) = -1$, то функциональное уравнение (3.1) может быть представлено в следующем виде:

$$P_0(x) + P_{1*}(x) \cdot A(x) + P_2(x) \cdot A(x)^2 + \dots + P_m(x) \cdot A(x)^m = A(x),$$

где $P_{1*}(x) = P_1(x) + 1$, то есть

$$p_{1*}(j) = \begin{cases} 0, & j = 0; \\ p_1(j), & j > 0. \end{cases}$$

Чтобы разработать алгоритмы комбинаторной генерации для комбинаторного множества, мощность которого определяется алгебраической производящей функцией, требуется решить задачу вычисления ее коэффициентов.

Теорема 3.1. Для алгебраической производящей функции

$$A(x) = \sum_{n>0} a(n) x^n,$$

удовлетворяющей функциональному уравнению

$$A(x) = \sum_{i=0}^m P_i(x) \cdot A(x)^i, \quad (3.2)$$

где $P_i(x) = \sum_{j=0}^{n_i} p_i(j) x^j$ и $p_1(0) = 0$, справедлива следующая рекуррентная формула:

$$a(n) = \begin{cases} 0, & n \leq 0; \\ \sum_{i=0}^m \left(\sum_{\substack{j+j_1+\dots+j_i=n; \\ j \geq 0; j_1, \dots, j_i \geq 1}} p_i(j) \cdot a(j_1) \cdot \dots \cdot a(j_i) \right), & n > 0. \end{cases} \quad (3.3)$$

Доказательство. Условие $p_1(0) = 0$ позволяет выполнить преобразование функционального уравнения (3.2) к виду функционального уравнения (3.1), что подтверждает соответствие $A(x)$ классу алгебраических производящих функций.

По условию задано, что у производящей функции $A(x)$ отсутствует свободный коэффициент, то есть $a(0) = 0$ и $A(0) = 0$. Также по условию у производящей функции $A(x)$ отсутствует переменная x с отрицательным значением показателя степени, поэтому $a(n) = 0$ для $n < 0$.

Уравнение (3.2) можно представить в следующем виде:

$$\begin{aligned} \sum_{n>0} a(n) x^n &= \left(\sum_{j=0}^{n_0} p_0(j) x^j \right) + \left(\sum_{j=0}^{n_1} p_1(j) x^j \right) \cdot \left(\sum_{j_1>0} a(j_1) x^{j_1} \right) + \dots + \\ &+ \left(\sum_{j=0}^{n_m} p_m(j) x^j \right) \cdot \left(\sum_{j_1>0} a(j_1) x^{j_1} \right) \cdot \dots \cdot \left(\sum_{j_m>0} a(j_m) x^{j_m} \right) \end{aligned}$$

или

$$\sum_{n>0} a(n) x^n = \sum_{i=0}^m \left(\left(\sum_{j=0}^{n_i} p_i(j) x^j \right) \cdot \left(\sum_{j_1>0} a(j_1) x^{j_1} \right) \cdot \dots \cdot \left(\sum_{j_i>0} a(j_i) x^{j_i} \right) \right).$$

Слева коэффициент $a(n)$ расположен при x^n , значит в правой части уравнения необходимо определить коэффициент при такой же степени переменной.

Справа имеется $m + 1$ слагаемых, что соответствует количеству многочленов $P_i(x)$, где $i \in \{0, \dots, m\}$, поэтому в рамках каждого слагаемого нужно найти коэффициент при x^n и просуммировать их. По условию у всех многочленов $P_i(x)$ и у производящей функции $A(x)$ отсутствует переменная x с отрицательным значением показателя степени. Следовательно, в рамках каждого слагаемого требуется собрать комбинацию из $i + 1$ одночленов (один одночлен из $P_i(x)$ и i одночленов из $A(x)$) так, чтобы сумма показателей степеней переменной x была равна n . Таким образом, требуется рассмотреть все композиции числа n длиной $i + 1$, то есть $j + j_1 + \dots + j_i = n$. При этом по условию $j \in \{0, \dots, n_i\}$, а из условия $a(0) = 0$ следует, что $j_1, \dots, j_i \in \{1, \dots, n - 1\}$. В результате получаем искомую рекуррентную формулу (3.3) для вычисления коэффициентов $a(n)$ производящей функции $A(x)$. $\square$

Также отметим, что условие $p_1(0) = 0$ гарантирует отсутствие в правой части уравнения (3.2) слагаемого с производящей функцией $A(x)$ без умножения на переменную x , то есть $p_1(0) \cdot a(n) = 0$. Кроме того, условие $a(0) = 0$ гарантирует отсутствие $a(n)$ в $p_i(j) \cdot a(j_1) \cdot \dots \cdot a(j_i)$ для $i > 1$. В противном случае будет получено некорректное рекуррентное соотношение, в рамках которого вычисление коэффициента $a(n)$ потребует значения коэффициента $a(n)$.

Однако комбинаторным множествам свойственно использование пустого объекта для случая объекта размера $n = 0$. В таком случае связанная с данным комбинаторным множеством производящая функция будет иметь единичный свободный коэффициент ($a(0) = 1$) и становится невозможным применение рекуррентной формулы (3.3). Тогда для получения требуемого вида нужно преобразовать производящую функцию путем вычитания ее свободного коэффициента, то есть $A(x) := A(x) - a(0)$, либо путем домножения на переменную x , то есть $A(x) := x \cdot A(x)$.

Применение рекуррентной формулы (3.3) является неэффективным с точки зрения вычислительной сложности. В частности, на каждом уровне рекурсии при вычислении значения $a(n)$ требуется для каждого $i \in \{0, \dots, m\}$ определить все композиции числа n длиной $i + 1$ в виде суммы $j + j_1 + \dots + j_i = n$, где $j \in \{0, \dots, n_i\}$ и $j_1, \dots, j_i \in \{1, \dots, n - 1\}$. Временная сложность перебора всех искомым композиций числа n равна $O(\sum_{i=0}^m \binom{n}{i})$ при условии наличия алгоритма генерации каждой последующей композиции с постоянной временной сложностью $O(1)$. Для $m < \frac{n}{2}$ эта оценка не превышает $O(n^m)$, а для худшего

случая $m = n$ не превышает $O(2^n)$. В то же время каждая композиция генерирует множество новых рекурсивных вызовов для вычисления значений $a(j_1), \dots, a(j_i)$, при этом максимальная глубина рекурсии равна n . Таким образом, итоговая сложность вычисления значения $a(n)$ по рекуррентной формуле (3.3) является экспоненциальной, что делает ее неприменимой при решении реальных задач.

Чтобы исключить дублирование вычислений $a(1), \dots, a(n-1)$ на разных уровнях рекурсивных вызовов, можно организовать их хранение с использованием $O(n)$ дополнительной памяти. Тогда временная сложность вычисления значения $a(n)$ по рекуррентной формуле (3.3) станет равна $O(m \cdot n^{m+1})$:

- требуется вычислить n значений $a(1), \dots, a(n)$;
- для каждого из них требуется рассмотреть $m + 1$ значение параметра i ;
- для каждого случая параметра i требуется определить все композиции числа n , количество которых не превышает n^m .

Отметим, что в данном случае предполагается модель вычислений с использованием равномерной функции стоимости [191], то есть стоимость каждой машинной операции вне зависимости от размера обрабатываемых чисел принимается за постоянную величину.

Если выполняется условие $p_i(j) \in \mathbb{Z}_{\geq 0}$ для $i \in \{0, \dots, m\}$ и $j \in \{0, \dots, n_i\}$, то по рекуррентной формуле (3.3) можно построить соответствующую рекурсивную структуру дерева И/ИЛИ для $n > 0$ (рисунок 3.5).

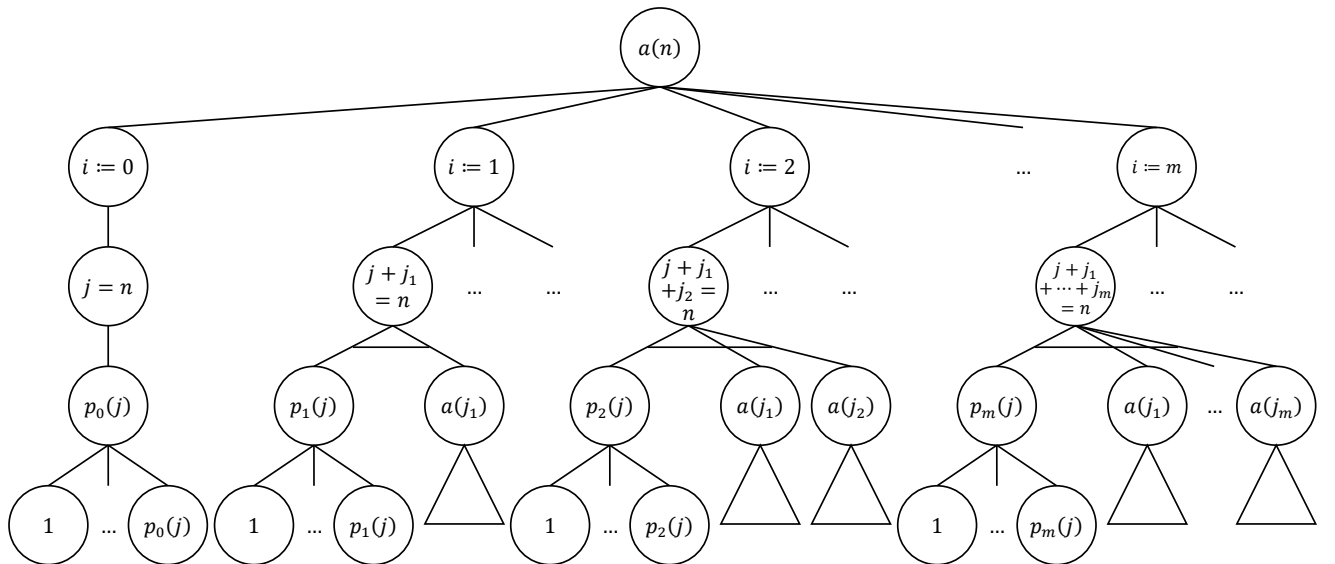


Рисунок 3.5 — Структура дерева И/ИЛИ для $a(n)$ при использовании формулы (3.3)

Полученная структура дерева И/ИЛИ характеризуется экспоненциальным ростом в ширину при увеличении уровня дерева. В свою очередь, это приводит к экспоненциальной временной сложности алгоритма генерации по рангу, разработанного на основе такой древовидной структуры.

Для уменьшения вычислительных затрат предлагается рассмотреть коэффициенты k -й степени производящей функции $A(x)$.

Теорема 3.2. *Для алгебраической производящей функции*

$$A(x) = \sum_{n \geq 0} a(n) x^n,$$

удовлетворяющей функциональному уравнению

$$A(x) = \sum_{i=0}^m P_i(x) \cdot A(x)^i, \quad (3.4)$$

где $P_i(x) = \sum_{j=0}^{n_i} p_i(j) x^j$, $p_i(j) \in \mathbb{Z}_{\geq 0}$ и $p_1(0) = 0$, и ее k -й степени

$$A(x)^k = \sum_{n \geq k} a(n, k) x^n,$$

где $k \geq 0$, справедлива следующая рекуррентная формула:

$$a(n) = \begin{cases} 0, & n \leq 0; \\ p_0(1), & n = 1; \\ \sum_{i=0}^m \sum_{j=0}^{\min(n_i, n-i)} p_i(j) \cdot a(n-j, i), & n > 1, \end{cases} \quad (3.5)$$

где

$$a(n, k) = \begin{cases} 1, & n = k = 0; \\ p_0(1)^n, & n = k > 0; \\ \sum_{i=0}^m \sum_{j=0}^{\min(n_i, n-k+1-i)} p_i(j) \cdot a(n-j, k-1+i), & n > k > 0; \\ 0, & \text{иначе.} \end{cases} \quad (3.6)$$

Доказательство. Значение $a(n) = 0$ для $n \leq 0$ следует из доказательства теоремы 3.1.

Основываясь на формуле (3.4), производящая функция $A(x)$ может быть представлена как сложение $m + 1$ производящих функций вида

$$B_i(x) = \sum_{n \geq 0} b_i(n) x^n = P_i(x) \cdot A(x)^i,$$

где $i \in \{0, \dots, m\}$, то есть

$$A(x) = B_0(x) + B_1(x) + \dots + B_m(x) = \sum_{i=0}^m B_i(x).$$

Следовательно, коэффициенты $a(n)$ связаны с коэффициентами $b_i(n)$ следующим выражением для случая $n > 0$:

$$a(n) = b_0(n) + b_1(n) + \dots + b_m(n) = \sum_{i=0}^m b_i(n).$$

В свою очередь, каждая производящая функция $B_i(x)$ является результатом умножения двух производящих функций $P_i(x)$ и $A(x)^i$:

$$B_i(x) = \sum_{n \geq 0} b_i(n) x^n = P_i(x) \cdot A(x)^i = \left(\sum_{n \geq 0} p_i(n) x^n \right) \cdot \left(\sum_{n \geq i} a(n, i) x^n \right).$$

Следовательно, коэффициенты $b_i(n)$ связаны с коэффициентами $p_i(n)$ и $a(n, i)$ следующим выражением для случая $n > 0$:

$$b_i(n) = \sum_{j=0}^n p_i(j) \cdot a(n-j, i).$$

Комбинируя формулы для $a(n)$ и $b_i(n)$, для случая $n > 0$ получаем

$$a(n) = \sum_{i=0}^m \sum_{j=0}^n p_i(j) \cdot a(n-j, i). \quad (3.7)$$

Учитывая тот факт, что $a(0) = 0$ для производящей функции $A(x)$, то в разложении в степенной ряд для ее k -й степени отсутствуют переменные с показателем степени меньше k , то есть $a(n, k) = 0$ для $n < k$. Тогда для $a(n-j, i)$ в формуле (3.7) имеем ограничение сверху для параметра j , а именно $j \leq n-i$. Также по условию $p_i(j) = 0$ для $j > n_i$, поэтому для параметра j в формуле (3.7) имеем дополнительное ограничение сверху, а именно $j \leq n_i$.

Отдельно рассмотрим случай для $a(1)$. Согласно формуле (3.7) с учетом указанных выше ограничений сверху для параметра j , получаем

$$a(1) = p_0(0) \cdot a(1, 0) + p_0(1) \cdot a(0, 0) + p_1(0) \cdot a(1, 1).$$

По определению $A(x)^0 = 1$, поэтому $a(0, 0) = 1$ и $a(n, 0) = 0$ для $n \neq 0$. Также по условию задано, что $p_1(0) = 0$. Упрощая выражение для $a(1)$, получаем $a(1) = p_0(1)$. Таким образом, комбинация полученных формул предоставляет искомую рекуррентную формулу (3.5) для вычисления коэффициентов $a(n)$ производящей функции $A(x)$.

Чтобы получить рекуррентную формулу (3.6) для вычисления коэффициентов $a(n, k)$ производящей функции $A(x)^k$, умножим левую и правую части уравнения (3.4) на $A(x)^{k-1}$, тогда получим

$$A(x)^k = \sum_{i=0}^m P_i(x) \cdot A(x)^{k-1+i}.$$

Основываясь на данной формуле, производящая функция $A(x)^k$ может быть представлена как сложение $m + 1$ производящих функций, каждая из которых, в свою очередь, является результатом умножения двух производящих функций $P_i(x)$ и $A(x)^{k-1+i}$. Следовательно, по аналогии с выводом формулы (3.7), для случая $n > 0$ и $k > 0$ получаем

$$a(n, k) = \sum_{i=0}^m \sum_{j=0}^n p_i(j) \cdot a(n - j, k - 1 + i). \quad (3.8)$$

Кроме того, ранее было показано, что $a(n, k) = 0$ для $n < k$, а также что $a(n, k) = 1$ для $n = k = 0$ и $a(n, k) = 0$ для $n \neq k = 0$. Тогда для $a(n - j, k - 1 + i)$ в формуле (3.8) имеем ограничение сверху для параметра j , а именно $j \leq n - k + 1 - i$.

Отдельно рассмотрим случай для $a(n, k)$, где $n = k$. Согласно формуле (3.8) с учетом указанных выше ограничений сверху для параметра j , получаем

$$a(n, n) = p_0(0) \cdot a(n, n - 1) + p_0(1) \cdot a(n - 1, n - 1) + p_1(0) \cdot a(n, n).$$

Учитывая тот факт, что $a(0) = 0$ для производящей функции $A(x)$, то из уравнения (3.4) следует, что $p_0(0) = 0$. Также по условию задано, что $p_1(0) = 0$. Упрощая выражение для $a(n, n)$, получаем

$$a(n, n) = p_0(1) \cdot a(n - 1, n - 1).$$

Далее данное выражение рекурсивно вызывается для $a(n-1, n-1)$ и повторяется до достижения коэффициента $a(0, 0) = 1$:

$$a(n, n) = p_0(1) \cdot a(n-1, n-1) = p_0(1) \cdot (p_0(1) \cdot a(n-2, n-2)) = \dots = p_0(1)^n.$$

Таким образом, комбинация полученных формул предоставляет искомую рекуррентную формулу (3.6) для вычисления коэффициентов $a(n, k)$ производящей функции $A(x)^k$. $\square$

Полученные рекуррентные формулы (3.5) и (3.6) характеризуются лучшими оценками вычислительной сложности по сравнению с использованием рекуррентной формулы (3.3).

Теорема 3.3. *Вычисление значения $a(n, k)$ по рекуррентной формуле (3.6) требует $O((n_1 + \dots + n_m + m) \cdot n^2)$ операций и $O(n^2)$ памяти.*

Доказательство. Реализуем последовательное вычисление значений $a(i, j)$ для $0 \leq j \leq i \leq n$ с сохранением полученных результатов. Таким образом, нужно вычислить $\frac{(n+1)^2}{2}$ значений $a(i, j)$, которые образуют числовой треугольник и могут храниться в таблице размера $n+1$ на $n+1$. Следовательно, требуется $O(n^2)$ памяти.

Согласно рекуррентной формуле (3.6), для вычисления каждого значения $a(i, j)$ требуется просуммировать $(m+1) \cdot (n+1)$ произведений вида $p_i(j) \cdot a(n-j, k-1+i)$. Учитывая тот факт, что $p_i(j) = 0$ для $j > n_i$, то остается не более $n_i + 1$ ненулевых коэффициентов $p_i(j)$ для каждого значения параметра $i \in \{0, \dots, m\}$. Следовательно, для вычисления каждого значения $a(i, j)$ требуется просуммировать $n_1 + \dots + n_m + m$ произведений вида $p_i(j) \cdot a(n-j, k-1+i)$. Тогда временная сложность вычисления значения $a(n, k)$ по рекуррентной формуле (3.6) равна $O((n_1 + \dots + n_m + m) \cdot n^2)$, так как:

- требуется вычислить $\frac{(n+1)^2}{2}$ значений $a(i, j)$;
- для каждого значения $a(i, j)$ требуется просуммировать $n_1 + \dots + n_m + m$ произведений вида $p_i(j) \cdot a(n-j, k-1+i)$. $\square$

Так как $a(n) = a(n, 1)$, а также для вычисления $a(n)$ по рекуррентной формуле (3.5) требуются значения $a(n-j, i)$, поэтому вычисление значения $a(n)$ характеризуется аналогичными оценками вычислительной сложности.

Следствие 3.1. *Вычисление значения $a(n)$ по рекуррентной формуле (3.5) требует $O((n_1 + \dots + n_m + m) \cdot n^2)$ операций и $O(n^2)$ памяти.*

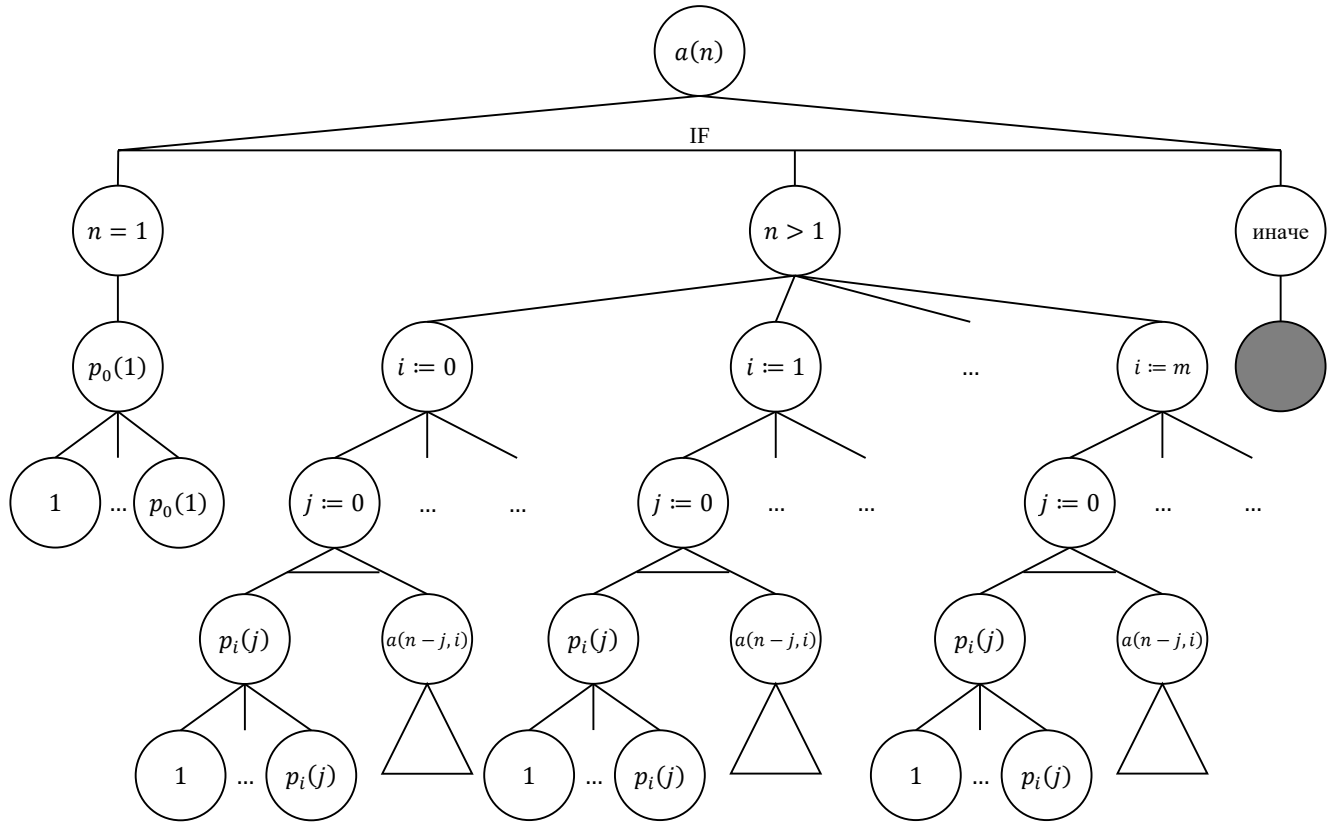


Рисунок 3.6 — Структура дерева И/ИЛИ для $a(n)$ при использовании формулы (3.5)

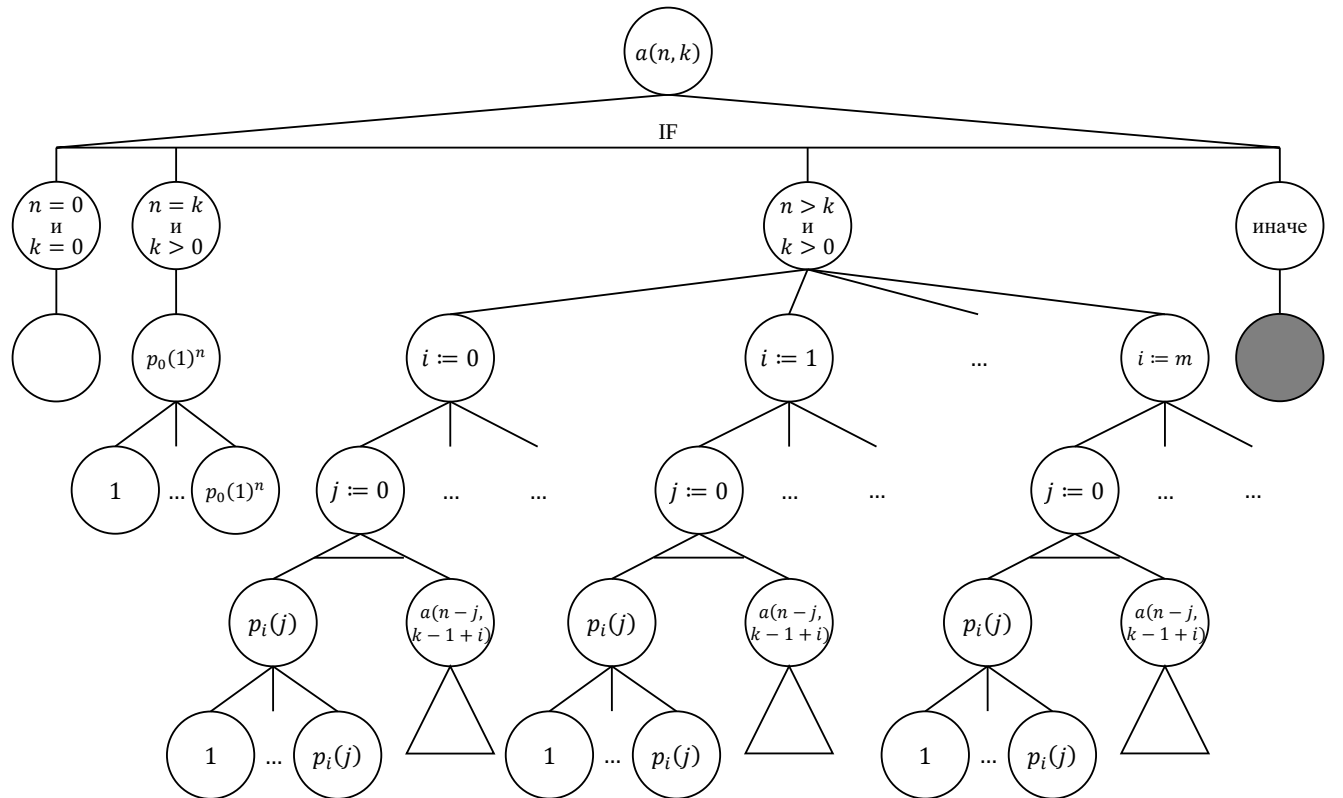


Рисунок 3.7 — Структуры дерева И/ИЛИ для $a(n, k)$ при использовании формулы (3.6)

Если выполняется условие $p_i(j) \in \mathbb{Z}_{\geq 0}$ для $i \in \{0, \dots, m\}$ и $j \in \{0, \dots, n_i\}$, то по рекуррентным формулам (3.5) и (3.6) можно построить соответствующие рекурсивные композиции структур деревьев И/ИЛИ (рисунки 3.6 и 3.7). Также структура дерева И/ИЛИ для $a(n)$ может быть дополнена случаем $a(0) = 1$, чтобы была возможность учета пустого объекта.

На основе полученных структур деревьев И/ИЛИ для $a(n)$ и $a(n, k)$ строятся рекурсивные алгоритмы ранжирования (алгоритмы 3.9 и 3.10) для соответствующих комбинаторных множеств. В данных алгоритмах используются обозначения для следующих вспомогательных функций, основанных на правилах построения конкретного комбинаторного объекта:

- функция $\text{GetRank}(object, p)$ определяет номер r правила из набора правил p , на основе которого был получен комбинаторный объект $object$;
- функция $\text{GetRule}(object)$ определяет набор правил p , на основе которого был получен комбинаторный объект $object$;
- функция $\text{ConvertSubobject}(r, p, object)$ определяет комбинаторный объект, из которого на основе правила под номером r из набора правил p был получен комбинаторный объект $object$.

Алгоритм 3.9: Алгоритм ранжирования для $a(n)$ при использовании формулы (3.5)

```

1 Rank_A(object, n)
2 begin
3   if n = 0 then r := 0
4   else if n = 1 then r := GetRank(object, p_0(1))
5   else
6     (I, J) := GetRule(object)
7     sum :=  $\sum_{i=0}^{I-1} \sum_{j=0}^{\min(n_i, n-i)} p_i(j) \cdot a(n-j, i) + \sum_{j=0}^{J-1} p_I(j) \cdot a(n-j, I)$ 
8     w_1 := p_I(J)
9     l_1 := GetRank(object, w_1)
10    subobject := ConvertSubobject(l_1, w_1, object)
11    l_2 := Rank_Apow(subobject, n - J, I)
12    r := sum + l_1 + w_1 l_2
13  end
14  return r
15 end

```

Алгоритм 3.10: Алгоритм ранжирования для $a(n, k)$ при использовании формулы (3.6)

```

1 Rank_Apow(object, n)
2 begin
3   if  $n = 0$  u  $k = 0$  then  $r := 0$ 
4   else if  $n = k$  then  $r := \text{GetRank}(\textit{object}, p_0(1)^n)$ 
5   else
6      $(I, J) := \text{GetRule}(\textit{object})$ 
7      $sum := \sum_{i=0}^{I-1} \sum_{j=0}^{\min(n_i, n-k+1-i)} p_i(j) \cdot a(n-j, k-1+i)$ 
8      $sum := sum + \sum_{j=0}^{J-1} p_I(j) \cdot a(n-j, k-1+I)$ 
9      $w_1 := p_I(J)$ 
10     $l_1 := \text{GetRank}(\textit{object}, w_1)$ 
11     $subobject := \text{ConvertSubobject}(l_1, w_1, \textit{object})$ 
12     $l_2 := \text{Rank\_Apow}(subobject, n-J, k-1+I)$ 
13     $r := sum + l_1 + w_1 l_2$ 
14  end
15  return  $r$ 
16 end

```

Вычисление ранга r в алгоритмах 3.9 и 3.10 на каждом уровне рекурсии связано с обработкой соответствующей структуры дерева И/ИЛИ:

- определяется выбор ветви дерева И/ИЛИ, в рамках которой задаются параметры $i := I$ и $j := J$ (строка 6 алгоритмов 3.9 и 3.10), что также соответствует выбору одного из слагаемых в правой части формулы (3.6) для алгоритма 3.9 или формулы (3.8) для алгоритма 3.10;

- определяется количество вариантов дерева И/ИЛИ, содержащихся левее выбранной ветви (вычисление sum в строке 7 алгоритма 3.9 и строках 7–8 алгоритма 3.10), что также соответствует сумме слагаемых в правой части формулы (3.6) для алгоритма 3.9 или формулы (3.8) для алгоритма 3.10, которые расположены левее выбранного слагаемого;

- определяется выбор одного из $p_I(J)$ узлов-потомков ИЛИ-узла с меткой $p_I(J)$ (вычисление l_1 в строке 9 алгоритма 3.9 и строке 10 алгоритма 3.10);

– определяется ранг поддерева на следующем уровне рекурсии (вычисление l_2 в строке 11 алгоритма 3.9 и строке 12 алгоритма 3.10).

Также на основе структур деревьев И/ИЛИ для $a(n)$ и $a(n, k)$ строятся рекурсивные алгоритмы генерации по рангу (алгоритмы 3.11 и 3.12) для соответствующих комбинаторных множеств.

Алгоритм 3.11: Алгоритм генерации по рангу для $a(n)$ при использовании формулы (3.5)

```

1 Unrank_A( $r, n$ )
2 begin
3   if  $n = 0$  then  $object := \varepsilon$ 
4   if  $n = 1$  then  $object := \text{GetObject}(r, p_0(1))$ 
5   else
6      $sum := 0$ 
7     for  $i := 0$  to  $m$  do
8       for  $j := 0$  to  $\min(n_i, n - k + 1 - i)$  do
9          $s := p_i(j) \cdot a(n - j, i)$ 
10        if  $sum + s > r$  then
11           $r := r - sum$ 
12           $I := i$ 
13           $J := j$ 
14          break
15        end
16         $sum := sum + s$ 
17      end
18    end
19     $w_1 := p_I(J)$ 
20     $l_1 := r \bmod w_1$ 
21     $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
22     $subobject := \text{Unrank_Apow}(l_2, n - J, I)$ 
23     $object := \text{ConvertObject}(l_1, w_1, subobject)$ 
24  end
25  return  $object$ 
26 end

```

Алгоритм 3.12: Алгоритм генерации по рангу для $a(n, k)$ при использовании формулы (3.6)

```

1  Unrank_Apow( $r, n, k$ )
2  begin
3      if  $n = 0$  u  $k = 0$  then  $object := \varepsilon$ 
4      else if  $n = k$  then  $object := \text{GetObject}(r, p_0(1)^n)$ 
5      else
6           $sum := 0$ 
7          for  $i := 0$  to  $m$  do
8              for  $j := 0$  to  $\min(n_i, n - k + 1 - i)$  do
9                   $s := p_i(j) \cdot a(n - j, k - 1 + i)$ 
10                 if  $sum + s > r$  then
11                      $r := r - sum$ 
12                      $I := i$ 
13                      $J := j$ 
14                     break
15                 end
16                  $sum := sum + s$ 
17             end
18         end
19          $w_1 := p_I(J)$ 
20          $l_1 := r \bmod w_1$ 
21          $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
22          $subobject := \text{Unrank\_Apow}(l_2, n - J, k - 1 + I)$ 
23          $object := \text{ConvertObject}(l_1, w_1, subobject)$ 
24     end
25     return  $object$ 
26 end

```

В данных алгоритмах генерации по рангу используются обозначения для следующих вспомогательных функций, реализующих правила построения конкретного комбинаторного объекта:

- функция $\text{GetObject}(r, p)$ реализует построение комбинаторного объекта на основе правила под номером r из набора правил p ;

– функция $\text{ConvertObject}(r, p, \text{object})$ реализует правило преобразования комбинаторного объекта object на основе правила под номером r из набора правил p .

Построение объекта object в алгоритмах 3.11 и 3.12 на каждом уровне рекурсии связано с обработкой соответствующей структуры дерева И/ИЛИ:

- определяется выбор ветви дерева И/ИЛИ, в рамках которой задаются параметры $i := I$ и $j := J$ (строки 6–18 алгоритмов 3.11 и 3.12);
- определяется выбор одного из $p_I(J)$ узлов-потомков ИЛИ-узла с меткой $p_I(J)$ (вычисление l_1 в строке 20 алгоритмов 3.11 и 3.12);
- определяется ранг поддеревя на следующем уровне рекурсии (вычисление l_2 в строке 21 алгоритмов 3.11 и 3.12).

Временная сложность алгоритмов равна $O((n_1 + \dots + n_m + m) \cdot O(a(n, k)) \cdot n)$, так как:

- каждый рекурсивный вызов требует $n_1 + \dots + n_m + m$ раз вычислить значение $a(i, j)$ в ходе выполнения двух вложенных циклов `for`;
- глубина рекурсии не превышает n , так как останавливается при достижении условия $n = 0$ или $n = k$.

В данном случае предполагается, что на каждом рекурсивном вызове вычисление вспомогательных функций требует меньше вычислительных ресурсов, чем вычисление $n_1 + \dots + n_m + m$ раз значения $a(i, j)$. В противном случае в оценке временной сложности алгоритмов требуется учесть дополнительное слагаемое.

Если для вычисления значения $a(n)$ использовать рекуррентную формулу (3.5) с временной сложностью $O((n_1 + \dots + n_m + m) \cdot n^2)$, то разработанные алгоритмы будут иметь временную сложность $O((n_1 + \dots + n_m + m)^2 \cdot n^3)$. Дополнительное повышение скорости работы алгоритмов может быть достигнуто за счет получения явной формулы для вычисления значения $a(n)$.

Таким образом, получаемые алгоритмы комбинаторной генерации обладают полиномиальной вычислительной сложностью, что делает их применимыми при решении реальных задач. Кроме того, коэффициенты $a(n, k)$ расширяют возможности кодирования и генерации по сравнению с коэффициентами $a(n)$, так как позволяют строить алгоритмы не только для исходного комбинаторного множества, связанного с производящей функцией $A(x)$, но и для кортежей таких объектов для случая $k > 1$.

3.3 Алгоритмы кодирования для частных случаев алгебраических производящих функций

Рассмотрим несколько примеров разработки алгоритмов ранжирования и генерации по рангу для частных случаев алгебраических производящих функций, удовлетворяющих требованиям теоремы 3.2.

Пример 1. Рассмотрим функциональное уравнение, которое определяет производящую функцию для последовательности чисел Фибоначчи (числовая последовательность A000045 в OEIS [174]):

$$A(x) = x + (x + x^2) \cdot A(x).$$

Согласно теореме 3.2 получаем следующую рекуррентную формулу для коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n-1, k-1) + a(n-1, k) + a(n-2, k), & n > k > 0; \\ 0, & \text{иначе.} \end{cases} \quad (3.9)$$

Для частного случая $a(n) = a(n, 1)$ получаем известную рекуррентную формулу для чисел Фибоначчи:

$$a(n) = a(n-1) + a(n-2), \quad a(1) = 1, \quad a(0) = 0.$$

Воспользуемся одной из известных комбинаторных интерпретаций для чисел Фибоначчи: $a(n)$ показывает количество композиций числа $n-1$ с использованием только единиц и двоек. Тогда $a(n, k)$ показывает количество кортежей длины k , где каждый элемент представляет собой композицию числа n_i-1 с использованием только единиц и двоек, причем $n_1 + n_2 + \dots + n_k = n$ и общая сумма единиц и двоек равна $n-k$.

Используя общие алгоритмы 3.10 и 3.12 для формулы (3.9), получаем соответствующие алгоритмы для ранжирования и генерации таких кортежей (алгоритмы 3.13 и 3.14). В данных алгоритмах используются следующие правила построения и преобразования объекта:

- если выполняется условие $n = k$ (строка 3 алгоритма 3.14), то в кортеже длины k элементов каждый элемент представляет собой пустой объект ε ;

– если выполняется условие $r < a(n - 1, k - 1)$ (строка 5 алгоритма 3.14), то к кортежу длины $k - 1$ элементов в конец добавляется новый элемент, представляющий собой пустой объект ε ;

– если выполняется условие $r - a(n - 1, k - 1) < a(n - 1, k)$ (строка 11 алгоритма 3.14), то в кортеже длины k элементов к последнему элементу добавляется единица к записи композиции числа;

– иначе выполняется условие $r - a(n - 1, k - 1) - a(n - 1, k) < a(n - 2, k)$ (строка 15 алгоритма 3.14), тогда в кортеже длины k элементов к последнему элементу добавляется двойка к записи композиции числа.

Таким образом, логика работы алгоритмов 3.10 и 3.12 заключается в выборе одного из слагаемых в правой части формулы (3.9), что также соответствует одному из представленных выше правил построения и преобразования объекта.

Алгоритм 3.13: Алгоритм ранжирования для $a(n, k)$ при использовании формулы (3.9)

```

1 Rank_Apow(object = (object1, ..., objectk), n, k)
2 begin
3   if n = k then r := 0
4   else
5     if objectk =  $\varepsilon$  then
6       | r := Rank_Apow(object1, ..., objectk-1), n - 1, k - 1)
7     end
8     else if objectk = subobject + 1 then
9       | r :=  $a(n - 1, k - 1)$ 
10      | r := r + Rank_Apow(object1, ..., objectk-1, subobject), n - 1, k)
11    end
12    else
13      | r :=  $a(n - 1, k - 1) + a(n - 1, k)$ 
14      | r := r + Rank_Apow(object1, ..., objectk-1, subobject), n - 2, k)
15    end
16  end
17  return r
18 end

```

Алгоритм 3.14: Алгоритм генерации по рангу для $a(n, k)$ при использовании формулы (3.9)

```

1 Unrank_Apow( $r, n, k$ )
2 begin
3   if  $n = k$  then  $object := (object_1, \dots, object_k) = (\varepsilon, \dots, \varepsilon)$ 
4   else
5     if  $r < a(n - 1, k - 1)$  then
6        $(object_1, \dots, object_{k-1}) := \text{Unrank\_Apow}(r, n - 1, k - 1)$ 
7        $object := (object_1, \dots, object_{k-1}, \varepsilon)$ 
8     end
9     else
10       $r := r - a(n - 1, k - 1)$ 
11      if  $r < a(n - 1, k)$  then
12         $(object_1, \dots, object_k) := \text{Unrank\_Apow}(r, n - 1, k)$ 
13         $object := (object_1, \dots, object_k + 1)$ 
14      end
15      else
16         $r := r - a(n - 1, k)$ 
17         $(object_1, \dots, object_k) := \text{Unrank\_Apow}(r, n - 2, k)$ 
18         $object := (object_1, \dots, object_k + 2)$ 
19      end
20    end
21  end
22  return  $object$ 
23 end

```

Для компактности представления в данных алгоритмах используется запись $object_k + 1$ или $object_k + 2$, которая обозначает добавление единицы или двойки в конец комбинаторного объекта $object_k$.

В таблице 3.1 представлены примеры результатов генерации комбинаторных объектов путем запуска алгоритма 3.14 для $n = 5$ и $k \in \{1, 2, 3, 4\}$. Таким образом, применение алгоритма 3.13 к рассмотренному множеству объектов реализует их кодирование в виде рангов, а применение алгоритма 3.14 позволяет восстановить исходные объекты.

Таблица 3.1 — Результаты генерации объектов с помощью алгоритма 3.14 для $n = 5$ и $k \in \{1, 2, 3, 4\}$

Ранг	Мощность комбинаторного множества и его элементы			
	$a(5, 1) = 5$	$a(5, 2) = 10$	$a(5, 3) = 9$	$a(5, 4) = 4$
$r = 0$	$(1 + 1 + 1 + 1)$	$(1 + 1 + 1, \varepsilon)$	$(1 + 1, \varepsilon, \varepsilon)$	$(1, \varepsilon, \varepsilon, \varepsilon)$
$r = 1$	$(2 + 1 + 1)$	$(2 + 1, \varepsilon)$	$(2, \varepsilon, \varepsilon)$	$(\varepsilon, 1, \varepsilon, \varepsilon)$
$r = 2$	$(1 + 2 + 1)$	$(1 + 2, \varepsilon)$	$(1, 1, \varepsilon)$	$(\varepsilon, \varepsilon, 1, \varepsilon)$
$r = 3$	$(1 + 1 + 2)$	$(1 + 1, 1)$	$(\varepsilon, 1 + 1, \varepsilon)$	$(\varepsilon, \varepsilon, \varepsilon, 1)$
$r = 4$	$(2 + 2)$	$(2, 1)$	$(\varepsilon, 2, \varepsilon)$	—
$r = 5$	—	$(1, 1 + 1)$	$(1, \varepsilon, 1)$	—
$r = 6$	—	$(\varepsilon, 1 + 1 + 1)$	$(\varepsilon, 1, 1)$	—
$r = 7$	—	$(\varepsilon, 2 + 1)$	$(\varepsilon, \varepsilon, 1 + 1)$	—
$r = 8$	—	$(1, 2)$	$(\varepsilon, \varepsilon, 2)$	—
$r = 9$	—	$(\varepsilon, 1 + 2)$	—	—

Пример 2. Рассмотрим функциональное уравнение, которое определяет производящую функцию для последовательности чисел Пелля (числовая последовательность A000129 в OEIS [174]):

$$A(x) = x + (2x + x^2) \cdot A(x).$$

Согласно теореме 3.2 получаем следующую рекуррентную формулу для коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n - 1, k - 1) + 2 \cdot a(n - 1, k) + a(n - 2, k), & n > k > 0; \\ 0, & \text{иначе.} \end{cases} \quad (3.10)$$

Воспользуемся одной из известных комбинаторных интерпретаций для чисел Пелля: $a(n)$ показывает количество композиций числа $n - 1$ с использованием только единиц и двоек, причем имеется два вида единиц (например, единицы двух цветов). Тогда $a(n, k)$ показывает количество кортежей длины k , где каждый элемент представляет собой композицию числа $n_i - 1$ с использованием только единиц (двух видов) и двоек, причем $n_1 + n_2 + \dots + n_k = n$ и общая сумма единиц и двоек равна $n - k$. Используя общие алгоритмы 3.10 и 3.12 для формулы (3.10), получаем соответствующие алгоритмы для ранжирования и генерации таких кортежей (алгоритмы 3.15 и 3.16).

Поскольку выбранная комбинаторная интерпретация для чисел Пелля схожа с комбинаторной интерпретацией для чисел Фибоначчи из предыдущего примера, в данных алгоритмах используются аналогичные правила построения и преобразования объекта. Однако в случае для чисел Пелля к записи композиции числа можно добавлять единицы двух видов, которые отличаются цветом, то есть используется запись $object_k + \textcolor{red}{1}$ или $object_k + \textcolor{blue}{1}$. Это расширение правил преобразования напрямую влияет на процесс генерации, увеличивая разнообразие получаемых объектов.

Алгоритм 3.15: Алгоритм ранжирования для $a(n, k)$ при использовании формулы (3.10)

```

1 Rank_Apow( $object = (object_1, \dots, object_k)$ ,  $n, k$ )
2 begin
3   if  $n = k$  then  $r := 0$ 
4   else
5     if  $object_k = \varepsilon$  then
6        $r := \text{Rank\_Apow}((object_1, \dots, object_{k-1}), n - 1, k - 1)$ 
7     end
8     else if  $object_k = subobject + \textcolor{red}{1}$  then
9        $r := a(n - 1, k - 1)$ 
10       $r := r + 2 \cdot \text{Rank\_Apow}((object_1, \dots, object_{k-1}, subobject), n - 1, k)$ 
11    end
12    else if  $object_k = subobject + \textcolor{blue}{1}$  then
13       $r := a(n - 1, k - 1) + 1$ 
14       $r := r + 2 \cdot \text{Rank\_Apow}((object_1, \dots, object_{k-1}, subobject), n - 1, k)$ 
15    end
16    else
17       $r := a(n - 1, k - 1) + 2 \cdot a(n - 1, k)$ 
18       $r := r + \text{Rank\_Apow}((object_1, \dots, object_{k-1}, subobject), n - 2, k)$ 
19    end
20  end
21  return  $r$ 
22 end
```

Алгоритм 3.16: Алгоритм генерации по рангу для $a(n, k)$ при использовании формулы (3.10)

```

1  Unrank_Apow( $r, n, k$ )
2  begin
3      if  $n = k$  then  $object := (object_1, \dots, object_k) = (\varepsilon, \dots, \varepsilon)$ 
4      else
5          if  $r < a(n - 1, k - 1)$  then
6               $(object_1, \dots, object_{k-1}) := \text{Unrank\_Apow}(r, n - 1, k - 1)$ 
7               $object := (object_1, \dots, object_{k-1}, \varepsilon)$ 
8          end
9          else
10              $r := r - a(n - 1, k - 1)$ 
11             if  $r < 2 \cdot a(n - 1, k)$  then
12                  $r_1 := r \bmod 2$ 
13                  $r_2 := \lfloor \frac{r}{2} \rfloor$ 
14                  $(object_1, \dots, object_k) := \text{Unrank\_Apow}(r_2, n - 1, k)$ 
15                 if  $r_1 = 0$  then  $object := (object_1, \dots, object_k + 1)$ 
16                 else  $object := (object_1, \dots, object_k + 1)$ 
17             end
18             else
19                  $r := r - 2 \cdot a(n - 1, k)$ 
20                  $(object_1, \dots, object_k) := \text{Unrank\_Apow}(r, n - 2, k)$ 
21                  $object := (object_1, \dots, object_k + 2)$ 
22             end
23         end
24     end
25     return  $object$ 
26 end

```

В таблице 3.2 представлены примеры результатов генерации комбинаторных объектов путем запуска алгоритма 3.16 для $n = 5$ и $k \in \{1, 2, 3, 4\}$. Таким образом, применение алгоритма 3.15 к рассмотренному множеству объектов реализует их кодирование в виде рангов, а применение алгоритма 3.16 позволяет восстановить исходные объекты.

Таблица 3.2 — Результаты генерации объектов с помощью алгоритма 3.16 для $n = 4$ и $k \in \{1, 2, 3, 4\}$

Ранг	Мощность комбинаторного множества и его элементы			
	$a(4, 1) = 12$	$a(4, 2) = 14$	$a(4, 3) = 6$	$a(4, 4) = 1$
$r = 0$	(1 + 1 + 1)	(1 + 1, ε)	(1, ε , ε)	(ε , ε , ε , ε)
$r = 1$	(1 + 1 + 1)	(1 + 1, ε)	(1, ε , ε)	—
$r = 2$	(1 + 1 + 1)	(1 + 1, ε)	(ε , 1, ε)	—
$r = 3$	(1 + 1 + 1)	(1 + 1, ε)	(ε , 1, ε)	—
$r = 4$	(1 + 1 + 1)	(2, ε)	(ε , ε , 1)	—
$r = 5$	(1 + 1 + 1)	(1, 1)	(ε , ε , 1)	—
$r = 6$	(1 + 1 + 1)	(1, 1)	—	—
$r = 7$	(1 + 1 + 1)	(1, 1)	—	—
$r = 8$	(2 + 1)	(1, 1)	—	—
$r = 9$	(2 + 1)	(ε , 1 + 1)	—	—
$r = 10$	(1 + 2)	(ε , 1 + 1)	—	—
$r = 11$	(1 + 2)	(ε , 1 + 1)	—	—
$r = 12$	—	(ε , 1 + 1)	—	—
$r = 13$	—	(ε , 2)	—	—

Пример 3. Рассмотрим функциональное уравнение, которое определяет производящую функцию для последовательности чисел Каталана (числовая последовательность A000108 в OEIS [174]), которые разделены нулями (числовая последовательность A126120 в OEIS [174]):

$$A(x) = x + x \cdot A(x)^2.$$

Согласно теореме 3.2 получаем следующую рекуррентную формулу для коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n-1, k-1) + a(n-1, k+1), & n > k > 0; \\ 0, & \text{иначе.} \end{cases} \quad (3.11)$$

Воспользуемся одной из известных комбинаторных интерпретаций для чисел Каталана: $a(n)$ количество правильных скобочных последовательностей длины $n-1$. Тогда $a(n, k)$ показывает количество кортежей длины k , где каждый элемент

представляет собой правильную скобочную последовательность длины $n_i - 1$, причем $n_1 + n_2 + \dots + n_k = n$ и общее количество открывающих и закрывающих скобок равно $n - k$. Также заметим, что $a(n, k) = 0$ для нечетных значений $n - k$.

Используя общие алгоритмы 3.10 и 3.12 для формулы (3.11), получаем соответствующие алгоритмы для ранжирования и генерации таких кортежей (алгоритмы 3.17 и 3.18). В данных алгоритмах используются следующие правила построения и преобразования объекта:

- если выполняется условие $n = k$ (строка 3 алгоритма 3.18), то в кортеже длины k элементов каждый элемент представляет собой пустой объект ε ;
- если выполняется условие $r < a(n - 1, k - 1)$ (строка 5 алгоритма 3.18), то к кортежу длины $k - 1$ элементов в конец добавляется новый элемент, представляющий собой пустой объект ε ;
- иначе выполняется условие $r - a(n - 1, k - 1) < a(n - 1, k + 1)$ (строка 9 алгоритма 3.18), тогда в кортеже длины $k + 1$ элементов последние два элемента объединяются в один путем конкатенации вида $\text{concat}("(", object_k, ")")$, $object_{k+1}$.

Алгоритм 3.17: Алгоритм ранжирования для $a(n, k)$ при использовании формулы (3.11)

```

1 Rank_Apow(object = (object1, ..., objectk), n, k)
2 begin
3   if n = k then r := 0
4   else
5     if objectk =  $\varepsilon$  then
6       | r := Rank_Apow((object1, ..., objectk-1), n - 1, k - 1)
7     end
8     else
9       | (subobject1, subobject2) := Parse(objectk)
10      | newobject := (object1, ..., objectk-1, subobject1, subobject2)
11      | r := a(n - 1, k - 1) + Rank_Apow(newobject, n - 1, k + 1)
12    end
13  end
14  return r
15 end

```

Алгоритм 3.18: Алгоритм генерации по рангу для $a(n, k)$ при использовании формулы (3.11)

```

1 Unrank_Apow( $r, n, k$ )
2 begin
3   if  $n = k$  then  $object := (object_1, \dots, object_k) = (\varepsilon, \dots, \varepsilon)$ 
4   else
5     if  $r < a(n - 1, k - 1)$  then
6        $(object_1, \dots, object_{k-1}) := \text{Unrank\_Apow}(r, n - 1, k - 1)$ 
7        $object := (object_1, \dots, object_{k-1}, \varepsilon)$ 
8     end
9     else
10       $r := r - a(n - 1, k - 1)$ 
11       $(object_1, \dots, object_{k+1}) := \text{Unrank\_Apow}(r, n - 1, k + 1)$ 
12       $object := (object_1, \dots, \text{concat}(", object_k, ")", object_{k+1})$ 
13    end
14  end
15  return  $object$ 
16 end

```

Используемая в алгоритме 3.17 функция $\text{Parse}(object)$ реализует синтаксический анализ заданного объекта, в результате которого определяются два объекта $subobject_1$ и $subobject_2$, для которых выполняется следующее:

$$\text{concat}(", subobject_1, ")", subobject_2) = object.$$

Кроме того, чтобы повысить скорость работы алгоритмов, можно применить следующую явную формулу для вычисления значений $a(n, k)$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ \frac{2k}{n+k} \binom{n-1}{\frac{n-k}{2}}, & n > k > 0 \text{ и } (n - k) \text{ четное}; \\ 0, & \text{иначе.} \end{cases} \quad (3.12)$$

Явная формула (3.12) получена путем применения математического аппарата композит производящих функций. Аналитическим решением функционального уравнения (3.11) является следующее выражение:

$$A(x) = \frac{1 - \sqrt{1 - 4x^2}}{2x},$$

которое может быть представлено в виде композиции производящих функций

$$A(x) = \frac{C(B(x))}{x},$$

где

$$C(x) = \frac{1 - \sqrt{1 - 4x}}{2},$$

$$B(x) = x^2.$$

Тогда коэффициенты $a(n, k) = [x^n]A(x)^k$ связаны с коэффициентами $b(n, k) = [x^n]B(x)^k$ и $c(n, k) = [x^n]C(x)^k$ через формулу

$$a(n, k) = \sum_{i=k}^{n+k} b(n+k, i) \cdot c(i, k),$$

где

$$b(n, k) = [x^n]B(x)^k = [x^n]x^{2k} = \delta(n, 2k),$$

$$c(n, k) = [x^n]C(x)^k = [x^n] \left(\frac{1 - \sqrt{1 - 4x}}{2} \right)^k = \frac{k}{n} \binom{2n - k - 1}{n - 1}.$$

В результате, комбинируя полученные выражения, получаем явную формулу (3.12).

В таблице 3.3 представлены примеры результатов генерации комбинаторных объектов путем запуска алгоритма 3.18. Таким образом, применение алгоритма 3.17 к рассмотренному множеству объектов реализует их кодирование в виде рангов, а применение алгоритма 3.18 позволяет восстановить исходные объекты.

Аналогичным образом получаем алгоритмы ранжирования и генерации по рангу для других примеров алгебраических производящих функций:

– для функционального уравнения

$$A(x) = x + x \cdot A(x) + x \cdot A(x)^2,$$

которое определяет производящую функцию для последовательности чисел Моцкина (числовая последовательность A001006 в OEIS [174]), получаем следующую рекуррентную формулу для коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n-1, k-1) + a(n-1, k) + a(n-1, k+1), & n > k > 0; \\ 0, & \text{иначе;} \end{cases}$$

Таблица 3.3 — Результаты генерации объектов с помощью алгоритма 3.18

Ранг	Мощность комбинаторного множества и его элементы			
	$a(9, 1) = 14$	$a(8, 2) = 14$	$a(7, 3) = 9$	$a(8, 4) = 14$
$r = 0$	$(\text{"}(((\text{()}\text{)}))\text{"})$	$(\text{"}((\text{()}\text{)})\text{", } \varepsilon)$	$(\text{"}(\text{()}\text{)}\text{", } \varepsilon, \varepsilon)$	$(\text{"}(\text{()}\text{)}\text{", } \varepsilon, \varepsilon, \varepsilon)$
$r = 1$	$(\text{"}((\text{()}\text{()}\text{)})\text{"})$	$(\text{"}(\text{()}\text{()}\text{)}\text{", } \varepsilon)$	$(\text{"}()\text{()}\text{", } \varepsilon, \varepsilon)$	$(\text{"}()\text{()}\text{", } \varepsilon, \varepsilon, \varepsilon)$
$r = 2$	$(\text{"}((\text{()}\text{()}\text{()}\text{)})\text{"})$	$(\text{"}(\text{()}\text{()}\text{()}\text{)}\text{", } \varepsilon)$	$(\text{"}()\text{", } \text{"}()\text{", } \varepsilon)$	$(\text{"}()\text{", } \text{"}()\text{", } \varepsilon, \varepsilon)$
$r = 3$	$(\text{"}(\text{()}\text{()}\text{()}\text{()}\text{)})\text{"})$	$(\text{"}()\text{()}\text{()}\text{", } \varepsilon)$	$(\varepsilon, \text{"}(\text{()}\text{)}\text{", } \varepsilon)$	$(\varepsilon, \text{"}(\text{()}\text{)}\text{", } \varepsilon, \varepsilon)$
$r = 4$	$(\text{"}(\text{()}\text{()}\text{()}\text{()}\text{()}\text{)})\text{"})$	$(\text{"}()\text{()}\text{()}\text{", } \varepsilon)$	$(\varepsilon, \text{"}()\text{()}\text{", } \varepsilon)$	$(\varepsilon, \text{"}()\text{()}\text{", } \varepsilon, \varepsilon)$
$r = 5$	$(\text{"}((\text{()}\text{()}\text{()}\text{()}\text{)})\text{")}$	$(\text{"}(\text{()}\text{()}\text{")}\text{", } \text{"}()\text{")}$	$(\text{"}()\text{", } \varepsilon, \text{"}()\text{")}$	$(\text{"}()\text{", } \varepsilon, \text{"}()\text{", } \varepsilon)$
$r = 6$	$(\text{"}(\text{()}\text{()}\text{()}\text{()}\text{()}\text{")}$	$(\text{"}()\text{()}\text{", } \text{"}()\text{")}$	$(\varepsilon, \text{"}()\text{", } \text{"}()\text{")}$	$(\varepsilon, \text{"}()\text{", } \text{"}()\text{", } \varepsilon)$
$r = 7$	$(\text{"}(\text{()}\text{()}\text{()}\text{()}\text{")}$	$(\text{"}()\text{", } \text{"}(\text{()}\text{")}$	$(\varepsilon, \varepsilon, \text{"}(\text{()}\text{")}$	$(\varepsilon, \varepsilon, \text{"}(\text{()}\text{")}\text{", } \varepsilon)$
$r = 8$	$(\text{"}()\text{()}\text{()}\text{()}\text{")}$	$(\varepsilon, \text{"}((\text{()}\text{)})\text{")}$	$(\varepsilon, \varepsilon, \text{"}()\text{()}\text{")}$	$(\varepsilon, \varepsilon, \text{"}()\text{()}\text{", } \varepsilon)$
$r = 9$	$(\text{"}()\text{()}\text{()}\text{()}\text{")}$	$(\varepsilon, \text{"}(\text{()}\text{()}\text{")}$	—	$(\text{"}()\text{", } \varepsilon, \varepsilon, \text{"}()\text{")}$
$r = 10$	$(\text{"}(\text{()}\text{()}\text{()}\text{()}\text{")}$	$(\text{"}()\text{", } \text{"}()\text{()}\text{")}$	—	$(\varepsilon, \text{"}()\text{", } \varepsilon, \text{"}()\text{")}$
$r = 11$	$(\text{"}()\text{()}\text{()}\text{()}\text{")}$	$(\varepsilon, \text{"}(\text{()}\text{()}\text{()}\text{")}$	—	$(\varepsilon, \varepsilon, \text{"}()\text{", } \text{"}()\text{")}$
$r = 12$	$(\text{"}()\text{()}\text{()}\text{()}\text{")}$	$(\varepsilon, \text{"}()\text{()}\text{()}\text{")}$	—	$(\varepsilon, \varepsilon, \varepsilon, \text{"}(\text{()}\text{")}$
$r = 13$	$(\text{"}()\text{()}\text{()}\text{()}\text{")}$	$(\varepsilon, \text{"}()\text{()}\text{()}\text{")}$	—	$(\varepsilon, \varepsilon, \varepsilon, \text{"}()\text{()}\text{")}$

— для функционального уравнения

$$A(x) = x + x \cdot A(x) + A(x)^2,$$

которое определяет производящую функцию для последовательности больших чисел Шредера (числовая последовательность A006318 в OEIS [174]), получаем следующую рекуррентную формулу для коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n-1, k-1) + a(n-1, k) + a(n, k+1), & n > k > 0; \\ 0, & \text{иначе;} \end{cases}$$

— для функционального уравнения

$$A(x) = x + A(x)^2 + A(x)^3,$$

которое определяет производящую функцию для числовой последовательности A001002 в OEIS [174], получаем следующую рекуррентную формулу для

коэффициентов производящей функции $A(x)^k$:

$$a(n, k) = \begin{cases} 1, & n = k; \\ a(n-1, k-1) + a(n, k+1) + a(n, k+2), & n > k > 0; \\ 0, & \text{иначе.} \end{cases}$$

3.4 Выводы по главе

В данной главе показана связь рекурсивных композиций структур деревьев И/ИЛИ с результатом выполнения операций (таких как сложение, умножение, возведение в степень и композиция) над производящими функциями для комбинаторных множеств. В частности, декомпозиция производящей функции с использованием указанных операций задает способ комбинирования алгоритмов комбинаторной генерации для простых структур, из которых формируется исследуемая сложная структура. Следовательно, определяется дополнительный путь к решению задачи разработки новых алгоритмов комбинаторной генерации на основе обработки соответствующих производящих функций.

Также рассмотрен класс алгебраических производящих функций, которые задаются в виде функционального уравнения. Используя операцию возведения в степень для таких производящих функций, решена задача вычисления их коэффициентов с полиномиальной оценкой вычислительной сложности как с точки зрения временных затрат, так и с точки зрения затрат по памяти. На основе доказанных рекуррентных формул формируется общий метод кодирования информационных объектов, мощность множества которых определяется коэффициентами алгебраических производящих функций. При этом основные шаги данного метода заключаются в следующем:

1. Согласно теореме 3.2 для функционального уравнения вида (3.4) для всех $p_i(j) \neq 0$ формируется сумма слагаемых вида $p_i(j) \cdot a(n-j, k-1+i)$;
2. Каждое слагаемое вида $p_i(j) \cdot a(n-j, k-1+i)$ определяет правило преобразования кортежа $k-1+i$ элементов с суммарным размером $n-j$ в кортеж k элементов с суммарным размером n . При этом преобразование объекта может быть реализовано $p_i(j)$ способами;

3. Используя общие алгоритмы 3.10 и 3.12 строятся соответствующие алгоритмы ранжирования и генерации по рангу для кортежей рассматриваемых комбинаторных объектов.

Продемонстрированные примеры получения рекуррентных формул и алгоритмов ранжирования и генерации по рангу на их основе для классических числовых последовательностей (числа Фибоначчи, Пелля, Каталана, Моцкина, Шредера) подтверждают применимость и универсальность предложенного метода. Кроме того, использование коэффициентов степеней производящих функций расширяет возможности генерации, так как это позволяет строить не только объекты исходного множества, связанного с производящей функцией, но и кортежи таких объектов. Перспективными направлениями дальнейших исследований может стать расширение класса поддерживаемых функциональных уравнений, а также адаптация подхода к задачам с использованием вероятностных моделей. Кроме того, актуальной является направление оптимизации создаваемых алгоритмов комбинаторной генерации за счет реализации параллельных вычислений.

Результаты данной главы опубликованы в [192–194].

Глава 4. Кодирование информационных объектов, представленных решеточными путями

В данной главе изложены результаты апробации предложенной методологии на примере построения алгоритмов кодирования информационных объектов, представленных решеточными путями. Рассматриваются частные случаи решеточных путей, такие как северо-восточные решеточные пути, а также пути Дика, Деланнуа, Шредера и Моцкина. Кроме того, решается задача кодирования множеств решеточных путей для общего случая, а именно: разработка алгоритмов ранжирования и генерации по рангу для множества направленных решеточных путей на плоскости.

4.1 Комбинаторные множества решеточных путей

Решеточные пути представляют собой фундаментальную структуру в дискретной математике и находят применение в различных прикладных областях. Решеточный путь можно определить как последовательность перемещений на целочисленной решетке, обычно ограниченную типами допустимых шагов, из которых и формируется путь от исходной точки до конечной точки. Важным свойством решеточных путей является возможность их наглядного визуального представления. В то же время исследуемое множество решеточных путей может быть отображено в множество более сложных дискретных структур.

Определение 4.1. *Решеточный путь P — это последовательность точек $(P_0, P_1, \dots, P_l)$ в d -мерной целочисленной решетке.*

Таким образом, задается упорядоченный набор точек $P_i = (p_{i,1}, p_{i,2}, \dots, p_{i,d})$, где $p_{i,j} \in \mathbb{Z}$ и $P_i \in \mathbb{Z}^d$. При этом точка P_0 является начальной, а точка P_l — конечной. Для простоты изложения в рамках диссертационного исследования рассматривается класс решеточных путей на плоскости, то есть когда $d = 2$, однако все полученные результаты также могут быть обобщены на случай решеточных путей при $d > 2$.

Кроме того, для формирования решеточного пути $P = (P_0, P_1, \dots, P_l)$ требуется указать множество допустимых шагов $S = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k\}$, где каждый шаг $\mathbf{s}_i$ является вектором в d -мерной целочисленной решетке, то есть $\mathbf{s}_i \in \mathbb{Z}^d$. Для двумерного случая имеем $\mathbf{s}_i = (a_i; b_i)$, где $a_i, b_i \in \mathbb{Z}$. Тогда решеточный путь $P = (P_0, P_1, \dots, P_l)$ может быть представлен как последовательность выполненных шагов:

$$P = (\overrightarrow{P_0 P_1}, \dots, \overrightarrow{P_{l-1} P_l}) = (\mathbf{ps}_1, \mathbf{ps}_2, \dots, \mathbf{ps}_l),$$

где $\overrightarrow{P_{i-1} P_i} = \mathbf{ps}_i \in S$.

На рисунке 4.1 представлен пример решеточного пути $P = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_1, \mathbf{s}_2)$, который начинается в точке $P_0 = (0; 0)$ и построен с помощью множества допустимых шагов $S = \{\mathbf{s}_1 = (1; 1), \mathbf{s}_2 = (2; 0), \mathbf{s}_3 = (-1; 1)\}$.

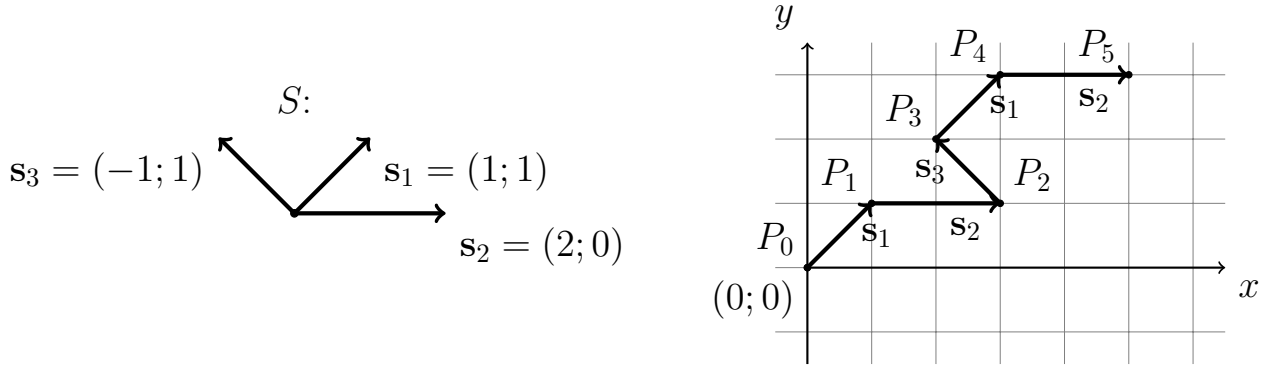


Рисунок 4.1 — Пример множества допустимых шагов $S = \{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3\}$ и возможный решеточный путь на их основе $P = (\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_1, \mathbf{s}_2)$

Решеточные пути широко используются в комбинаторике, так как хорошо подходят для кодирования других комбинаторных объектов. Это позволяет изучать свойства сложных дискретных структур, основываясь на свойствах соответствующих решеточных путей. Краткий исторический обзор исследований, связанных с решеточными путями, представлен в работе [195]. Подробное описание основных методов и результатов перечисления решеточных путей приведено в работе [196].

Анализ научных работ показывает, что решеточные пути чаще всего используются для решения задач перечислительной комбинаторики, в том числе для других более сложных дискретных структур с биективным отображением на структуры решеточных путей. В то же время решеточные пути используются для решения задач комбинаторной генерации с целью получения алгоритмов генерации таких структур. Например, в работе [197] разработаны алгоритмы ранжирования и

генерации по рангу для множества решеточных путей в $(t + 1)$ -мерной целочисленной решетке, которые начинаются в точке $(n_0; n_1; \dots; n_t)$, заканчиваются в точке начала координат и не опускаются ниже гиперплоскости $x_0 = \sum_{i=1}^t (k_i - 1)x_i$. Использование таких решеточных путей позволило получить алгоритмы комбинаторной генерации для множества упорядоченных деревьев с $n_0 + 1$ листьями и n_i внутренними узлами, имеющими k_i дочерних узлов, где $i = 1, \dots, t$. В работе [198] предложены алгоритмы ранжирования и генерации по рангу для множества бинарных деревьев с n -узлами за счет биективного отображения на множество n -путей Дика. При этом использовалось упорядочивание комбинаторного множества, в рамках которого соседние деревья отличаются на один оборот, и изучалось влияние таких поворотов на связанные решеточные пути. Работа [199] также посвящена исследованию бинарных деревьев с помощью соответствующих им n -путей Дика. Здесь предложен эффективный алгоритм их исчерпывающей генерации, использующий $O(n)$ памяти и $O(1)$ времени в среднем на одно дерево.

Кроме того, существуют исследования, в которых рассматривается разработка алгоритмов комбинаторной генерации непосредственно для решеточных путей. Например, в работах [200; 201] представлены алгоритмы последовательной генерации положительных путей Моцкина и Шредера, а также их префиксов. Алгоритмы генерации для множества северо-восточных решеточных путей с поворотами можно найти в работе [202]. Но наиболее известным и изученным как с точки зрения перечислительной комбинаторики, так и комбинаторной генерации является класс решеточных путей Дика. Это обусловлено их связью с числами Каталана, что позволяет строить биективное отображение с множествами других дискретных структур. Примеры исследований, направленных на разработку алгоритмов комбинаторной генерации для путей Дика, можно найти в работах [203–205].

Таким образом, разработка алгоритмов комбинаторной генерации для множеств решеточных путей является актуальной исследовательской задачей, так как это позволит кодировать информационные объекты, представленные решеточными путями. Также отметим отсутствие решений, претендующих на универсальность их применения, то есть существующие алгоритмы затрагивают лишь частные примеры решеточных путей с конкретным множеством допустимых шагов.

4.2 Алгоритмы кодирования для частных случаев решеточных путей

Рассмотрим применение предложенной методологии для частных случаев решеточных путей [206], а именно: северо-восточные решеточные пути [207], пути Дика [208], пути Деланнуа [209], пути Шредера [210] и пути Моцкина [211].

4.2.1 Северо-восточные решеточные пути

Определение 4.2. Северо-восточный решеточный путь из $(0; 0)$ в $(n; m)$ — это решеточный путь на плоскости, который начинается в точке $(0; 0)$, заканчивается в точке $(n; m)$ и состоит из шагов $(0; 1)$ и $(1; 0)$.

Шаг $(0; 1)$ называется северным шагом и обозначается **N**, шаг $(1; 0)$ называется восточным шагом и обозначается **E**. На рисунке 4.2 показаны все возможные 20 вариантов северо-восточных решеточных путей для $n = 3$ и $m = 3$.

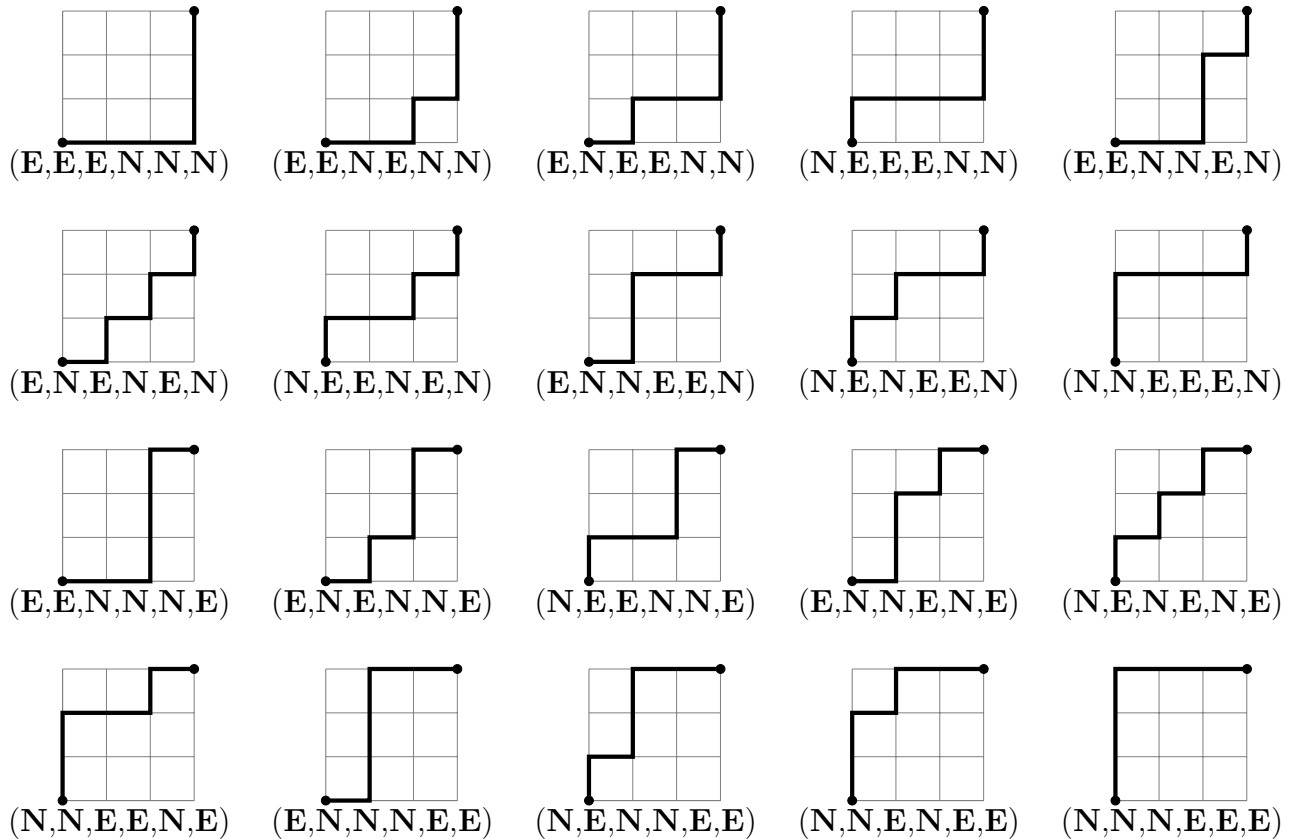


Рисунок 4.2 — Все северо-восточные решеточные пути, начинающиеся в точке $(0; 0)$ и заканчивающиеся в точке $(3; 3)$

Общее количество северо-восточных решеточных путей из $(0; 0)$ в $(n; m)$ для $n, m \in \mathbb{Z}_{\geq 0}$ определяется следующим биномиальным коэффициентом (последовательность A007318 в OEIS [174]):

$$L_n^m = \binom{n+m}{m} = \binom{n+m}{n}. \quad (4.1)$$

Значение L_n^m также вычисляется, используя рекуррентную формулу

$$L_n^m = \begin{cases} 0, & n < 0 \text{ или } m < 0; \\ 1, & n \geq m = 0 \text{ или } m \geq n = 0; \\ L_n^{m-1} + L_{n-1}^m, & \text{иначе.} \end{cases} \quad (4.2)$$

Кроме того, известна производящая функция

$$\sum_{n \geq 0} \sum_{m \geq 0} L_n^m x^n y^m = \frac{1}{1-x-y}.$$

Поскольку формула (4.2) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для L_n^m (рисунок 4.3).

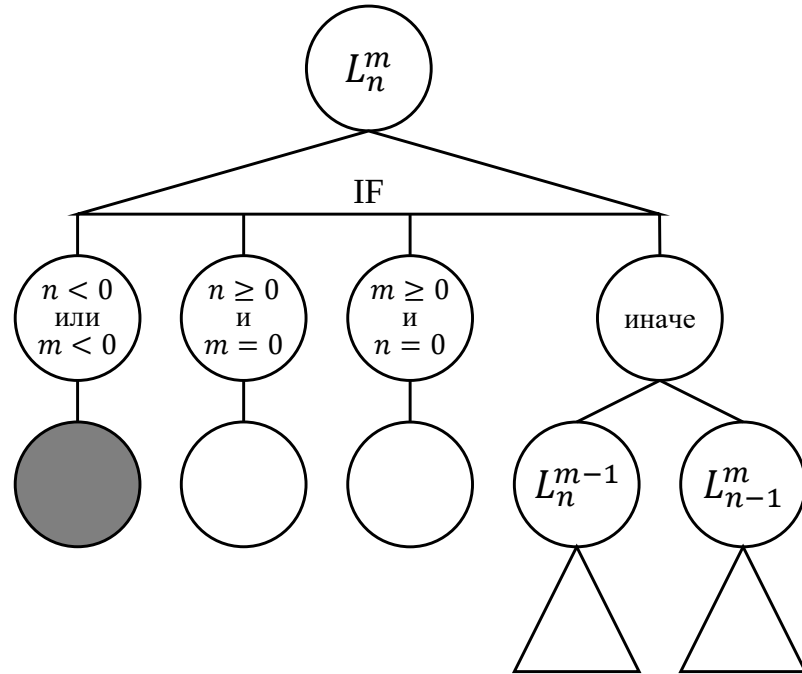


Рисунок 4.3 — Структура дерева И/ИЛИ для L_n^m

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой L_n^0 является листом в дереве И/ИЛИ для L_n^m ;
- каждый узел с меткой L_0^m является листом в дереве И/ИЛИ для L_n^m .

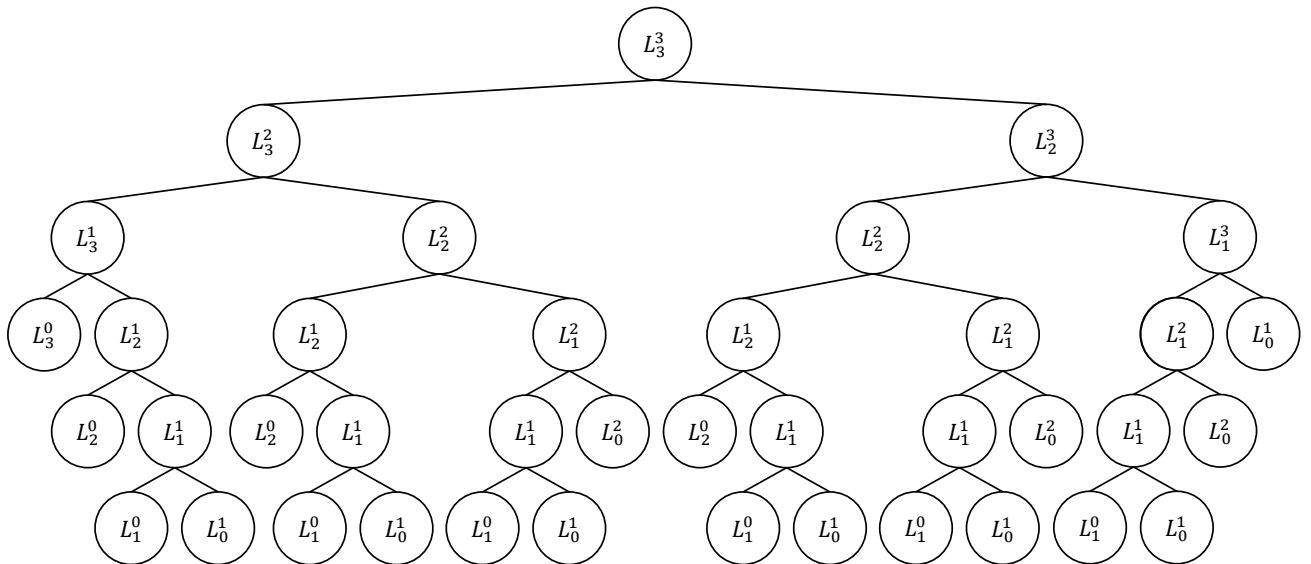


Рисунок 4.4 — Структура дерева И/ИЛИ для L_3^3

На рисунке 4.4 представлен пример структуры дерева И/ИЛИ для L_n^m , где $n = 3$ и $m = 3$. Общее количество его вариантов равно $L_3^3 = 20$.

Поскольку полученная структура дерева И/ИЛИ не содержит И-узлов, то каждый вариант такого дерева представляет собой путь от корня к листу. Поэтому будем кодировать вариант дерева И/ИЛИ для L_n^m последовательностью $v = (v_1, v_2, \dots)$ выбранных потомков ИЛИ-узлов в этом дереве (выбору левого потомка соответствует $v_i = 1$, а выбору правого потомка — $v_i = 2$).

Теорема 4.1. *Существует биективное отображение между множеством северо-восточных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, и множеством вариантов дерева И/ИЛИ для L_n^m , представленного на рисунке 4.3.*

Доказательство. Общее количество северо-восточных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, равно L_n^m . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для L_n^m , представленного на рисунке 4.3, равно значению L_n^m . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.2), биективное отображение между множеством вариантов дерева И/ИЛИ для L_n^m и множеством северо-восточных решеточных путей из $(0; 0)$ в $(n; m)$ определяется по следующим правилам:

- выбранный левый потомок ИЛИ-узла с меткой L_n^m определяет добавление одного северного шага к северо-восточному решеточному пути, полученному поддеревом узла с меткой L_n^{m-1} : результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_{n+m-1}, \mathbf{N})$;
- выбранный правый потомок ИЛИ-узла с меткой L_n^m определяет добавление одного восточного шага к северо-восточному решеточному пути, полученному поддеревом узла с меткой L_{n-1}^m : результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_{n+m-1}, \mathbf{E})$;
- лист с меткой L_n^0 определяет решеточный путь от точки $(0; 0)$ до точки $(n; 0)$, состоящий из n восточных шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_n) = (\mathbf{E}, \dots, \mathbf{E})$;
- лист с меткой L_0^m определяет решеточный путь от точки $(0; 0)$ до точки $(0; m)$, состоящий из m северных шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_m) = (\mathbf{N}, \dots, \mathbf{N})$.

Таким образом, из заданного варианта v дерева И/ИЛИ для L_n^m рекурсивно формируется северо-восточный решеточный путь вида $(\mathbf{ps}_1, \dots, \mathbf{ps}_{n+m})$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ.

Докажем биективность данного отображения. Для обеспечения инъективности отображения требуется, чтобы разным вариантам дерева И/ИЛИ соответствовали разные северо-восточные решеточные пути. Пусть V_1 и V_2 — два разных варианта дерева И/ИЛИ для L_n^m . Тогда существует ИЛИ-узел, для которого в V_1 выбран один его потомок, а в V_2 выбран другой его потомок. Пусть P_1 — решеточный путь, полученный на основе V_1 , а P_2 — решеточный путь, полученный на основе V_2 . Так как выбор потомка ИЛИ-узла влияет на тип добавляемого шага в решеточном пути, то различие в выборе узла-потомка для V_1 и V_2 приводит к выбору разных типов шагов для P_1 и P_2 на соответствующем шаге рекурсии. Следовательно, $P_1 \neq P_2$ для $V_1 \neq V_2$.

Для обеспечения сюръективности отображения требуется, чтобы для любого северо-восточного решеточного пути существовал вариант дерева И/ИЛИ. Применим метод математической индукции, для этого введем параметр $k = n + m$. В качестве базы индукции проверим существование варианта дерева И/ИЛИ для северо-восточного решеточного пути при $k \in \{0, 1, 2\}$. Условие $k = 0$ выполняется только для $n = m = 0$, что приводит к единственной возможной ситуации пустого решеточного пути. Этому случаю соответствует вариант дерева И/ИЛИ,

представляющий собой лист с меткой L_0^0 . Условие $k = 1$ выполняется для $n = 1$ и $m = 0$, что приводит к единственной возможной ситуации решеточного пути **(Е)**. Этому случаю соответствует вариант дерева И/ИЛИ, представляющий собой лист с меткой L_1^0 . Также условие $k = 1$ выполняется для $n = 0$ и $m = 1$, что приводит к единственной возможной ситуации решеточного пути **(N)**. Этому случаю соответствует вариант дерева И/ИЛИ, представляющий собой лист с меткой L_0^1 . Условие $k = 2$ выполняется для $n = 2$ и $m = 0$, что приводит к единственной возможной ситуации решеточного пути **(Е,Е)**. Этому случаю соответствует вариант дерева И/ИЛИ, представляющий собой лист с меткой L_2^0 . Также условие $k = 2$ выполняется для $n = 0$ и $m = 2$, что приводит к единственной возможной ситуации решеточного пути **(N,N)**. Этому случаю соответствует вариант дерева И/ИЛИ, представляющий собой лист с меткой L_0^2 . Кроме того, условие $k = 2$ выполняется для $n = 1$ и $m = 1$, что приводит к решеточным путям **(Е,N)** и **(N,Е)**. Этому случаю соответствуют варианты дерева И/ИЛИ, закодированные в виде последовательностей (1) и (2). Допустим, что вариант дерева И/ИЛИ существует для любого северо-восточного решеточного пути при произвольном $k = n + m$. Докажем справедливость утверждения для случая $k + 1$. Случаю $k + 1 = n + m + 1$ соответствует решеточный путь, заканчивающийся в точке $(n; m + 1)$ или точке $(n + 1; m)$. Попасть в точку $(n; m + 1)$ можно либо сделав северный шаг из точки $(n; m)$, либо сделав восточный шаг из точки $(n - 1; m + 1)$. Выбору типа шага сопоставим выбор потомка ИЛИ-узла в дереве И/ИЛИ. Пути из точки $(0; 0)$ в указанные точки являются северо-восточными решеточными путями, для которых, согласно предположению для $k = n + m$, существуют варианты дерева И/ИЛИ. Соответствующий вариант дерева И/ИЛИ формирует поддерево выбранного потомка ИЛИ-узла. Следовательно, получаем вариант дерева И/ИЛИ для северо-восточного решеточного пути, заканчивающегося в точке $(n; m + 1)$. Аналогично получаем вариант дерева И/ИЛИ для северо-восточного решеточного пути, заканчивающегося в точке $(n + 1; m)$.

Таким образом, отображение является одновременно инъективным и сюръективным, то есть отображение биективно. $\square$

Алгоритмы, реализующие разработанные правила биекции, имеют линейную временную сложность $O(n + m)$, так как требуют одного прохода для заполнения последовательности из $n + m$ элементов. Пример применения этих правил биекции представлен в таблице 4.1.

Таблица 4.1 — Ранжирование множества северо-восточных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$

Решеточный путь	Вариант дерева И/ИЛИ	Ранг
(E,E,E,N,N,N)	(1,1,1)	0
(E,E,N,E,N,N)	(1,1,2,1)	1
(E,N,E,E,N,N)	(1,1,2,2,1)	2
(N,E,E,E,N,N)	(1,1,2,2,2)	3
(E,E,N,N,E,N)	(1,2,1,1)	4
(E,N,E,N,E,N)	(1,2,1,2,1)	5
(N,E,E,N,E,N)	(1,2,1,2,2)	6
(E,N,N,E,E,N)	(1,2,2,1,1)	7
(N,E,N,E,E,N)	(1,2,2,1,2)	8
(N,N,E,E,E,N)	(1,2,2,2)	9
(E,E,N,N,N,E)	(2,1,1,1)	10
(E,N,E,N,N,E)	(2,1,1,2,1)	11
(N,E,E,N,N,E)	(2,1,1,2,2)	12
(E,N,N,E,N,E)	(2,1,2,1,1)	13
(N,E,N,E,N,E)	(2,1,2,1,2)	14
(N,N,E,E,N,E)	(2,1,2,2)	15
(E,N,N,N,E,E)	(2,2,1,1,1)	16
(N,E,N,N,E,E)	(2,2,1,1,2)	17
(N,N,E,N,E,E)	(2,2,1,2)	18
(N,N,N,E,E,E)	(2,2,2)	19

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.1) и генерации по рангу (алгоритм 4.2) вариантов дерева И/ИЛИ для L_n^m , представленного на рисунке 4.3. В этих алгоритмах $()$ обозначает пустую последовательность, а функция $\text{concat}(a,b)$ обозначает конкатенацию последовательностей a и b . В таблице 4.1 представлен пример ранжирования комбинаторного множества всех северо-восточных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$.

Алгоритм 4.1: Алгоритм ранжирования варианта дерева И/ИЛИ для L_n^m

```

1 Rank_L( $v = (v_1, v_2, \dots)$ ,  $n$ ,  $m$ )
2 begin
3   if  $n = 0$  или  $m = 0$  then  $r := 0$ 
4   else
5     if  $v_1 = 1$  then  $r := \text{Rank\_L}((v_2, \dots), n, m - 1)$ 
6     else  $r := L_n^{m-1} + \text{Rank\_L}((v_2, \dots), n - 1, m)$ 
7   end
8   return  $r$ 
9 end
```

Алгоритм 4.2: Алгоритм генерации по рангу варианта дерева И/ИЛИ
для L_n^m

```

1 Unrank_L( $r$ ,  $n$ ,  $m$ )
2 begin
3   if  $n = 0$  или  $m = 0$  then  $v := ()$ 
4   else
5     if  $r < L_n^{m-1}$  then  $v := \text{concat}((1), \text{Unrank\_L}(r, n, m - 1))$ 
6     else  $v := \text{concat}((2), \text{Unrank\_L}(r - L_n^{m-1}, n - 1, m))$ 
7   end
8   return  $v$ 
9 end
```

Разработанные алгоритмы имеют следующую вычислительную сложность:

- алгоритм 4.1 имеет не более m рекурсивных вызовов, где $v_1 = 1$ (каждый такой вызов требует одного присваивания), и не более n рекурсивных вызовов, где $v_1 = 2$ (каждый такой вызов требует вычисления L_n^{m-1}). Применяя формулу (4.1), вычисление L_n^m имеет линейную временную сложность $O(m)$ для $m < n$ и $O(n)$ для $m > n$. Следовательно, алгоритм 4.1 имеет полиномиальную временную сложность $O(nm)$ для $m < n$ и $O(m + n^2)$ для $m > n$;

- алгоритм 4.2 имеет не более $(n + m)$ рекурсивных вызовов, каждый из которых требует вычисления L_n^{m-1} . Следовательно, алгоритм 4.2 имеет полиномиальную временную сложность $O(m(n + m))$ для $m < n$ и $O(n(n + m))$ для $m > n$.

4.2.2 Решеточные пути Дика

Определение 4.3. *Путь Дика из $(0; 0)$ в $(n; n)$ — это решеточный путь на плоскости, который начинается в точке $(0; 0)$, заканчивается в точке $(n; n)$, состоит из шагов $(0; 1)$ и $(1; 0)$, и никогда не поднимается выше диагонали $y = x$.*

Множество путей Дика из $(0; 0)$ в $(n; n)$ является подмножеством северо-восточных решеточных путей из $(0; 0)$ в $(n; n)$. На рисунке 4.5 показаны все возможные 5 вариантов путей Дика для $n = 3$.

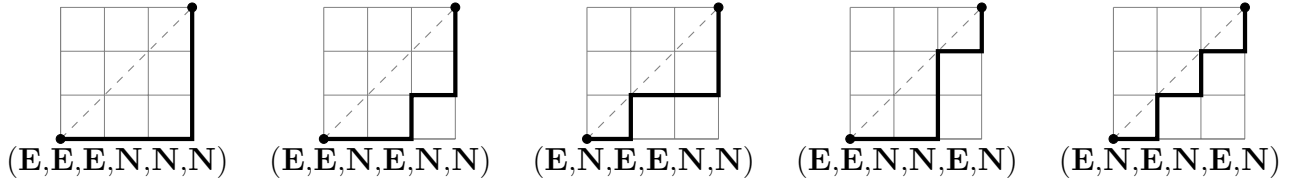


Рисунок 4.5 — Все пути Дика, начинающиеся в точке $(0; 0)$ и заканчивающиеся в точке $(3; 3)$

Общее количество путей Дика из $(0; 0)$ в $(n; n)$ для $n \in \mathbb{Z}_{\geq 0}$ определяется числом Каталана C_n (последовательность A000108 в OEIS [174]):

$$C_n = \frac{1}{n+1} \binom{2n}{n}. \quad (4.3)$$

Значение C_n также вычисляется, используя рекуррентную формулу

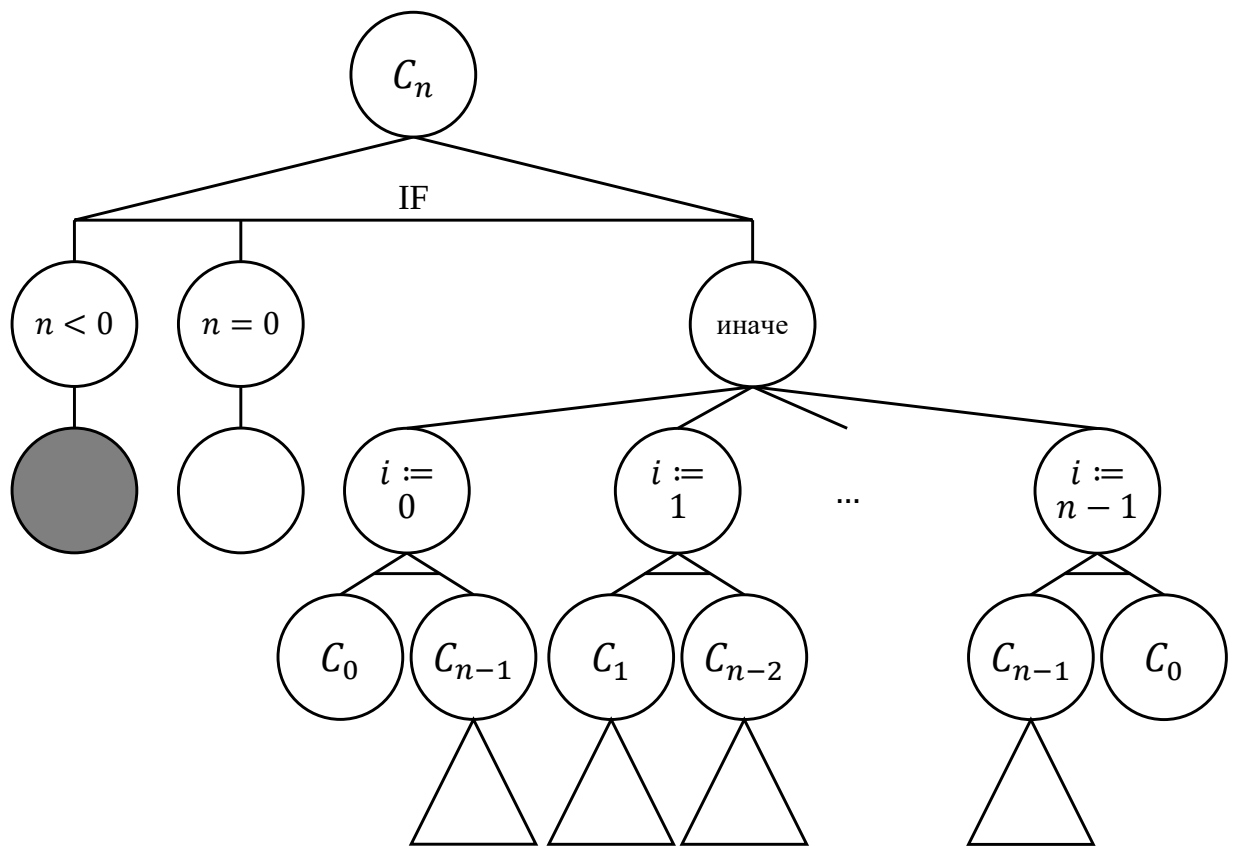
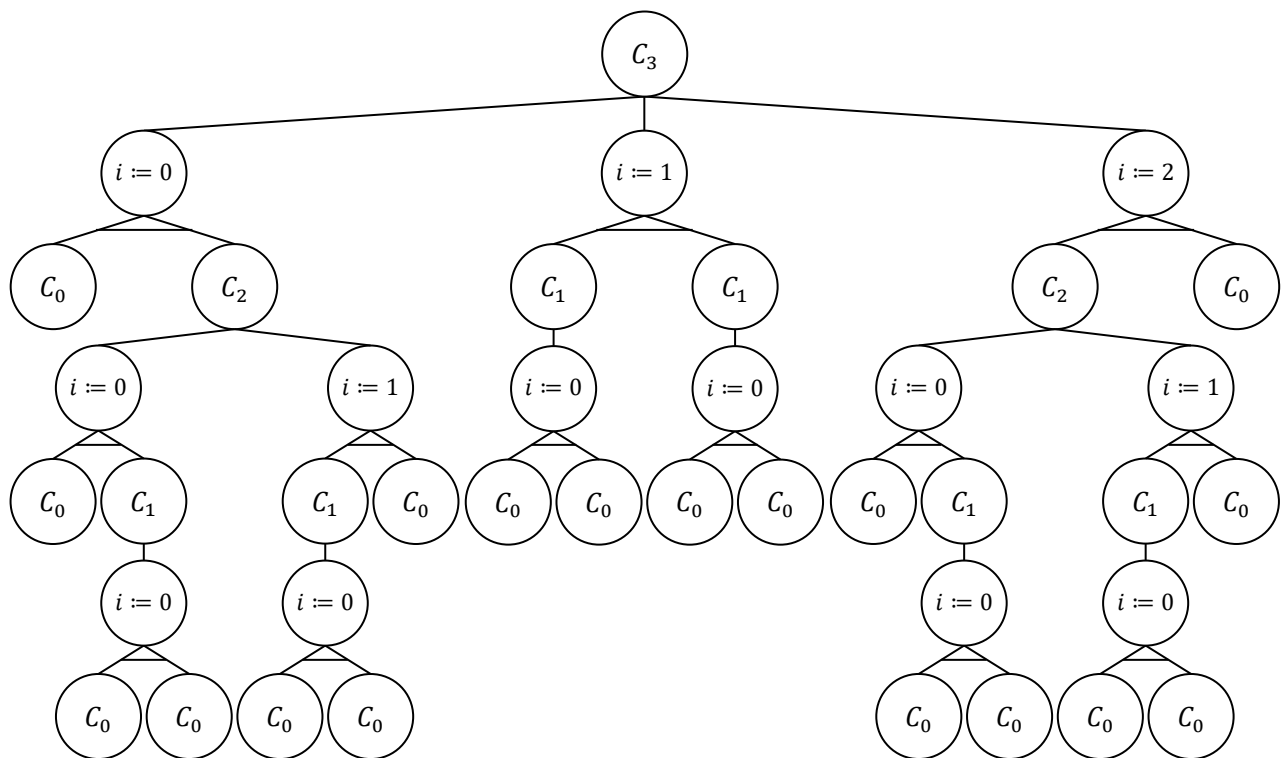
$$C_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0; \\ \sum_{i=0}^{n-1} C_i C_{n-1-i}, & \text{иначе.} \end{cases} \quad (4.4)$$

Кроме того, известна производящая функция

$$\sum_{n \geq 0} C_n x^n = \frac{1 - \sqrt{1 - 4x}}{2x}.$$

Поскольку формула (4.4) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для C_n (рисунок 4.6).

Для компактности представления воспользуемся следующим допущением: каждый узел с меткой C_0 является листом в дереве И/ИЛИ для C_n . На рисунке 4.7 представлен пример структуры дерева И/ИЛИ для C_n , где $n = 3$. Общее количество его вариантов равно $C_3 = 5$.

Рисунок 4.6 — Структура дерева И/ИЛИ для C_n Рисунок 4.7 — Структура дерева И/ИЛИ для C_3

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для C_n последовательностью $v = (I, v_1, v_2)$, где применимо следующее:

- пустая последовательность $v = ()$ соответствует выбору листа с меткой C_0 ;
- I соответствует выбранному значению i в дереве И/ИЛИ для C_n ;
- v_1 соответствует варианту поддерева узла с меткой C_I (левое поддерево);
- v_2 соответствует варианту поддерева узла с меткой C_{n-1-I} (правое поддерево).

Теорема 4.2. *Существует биективное отображение между множеством путей Дика, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, и множеством вариантов дерева И/ИЛИ для C_n , представленного на рисунке 4.6.*

Доказательство. Общее количество путей Дика, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, равно C_n . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для C_n , представленного на рисунке 4.6, равно значению C_n . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.4), биективное отображение между множеством вариантов дерева И/ИЛИ для C_n и множеством путей Дика из $(0; 0)$ в $(n; n)$ определяется по следующим правилам:

- выбранный потомок ИЛИ-узла с меткой C_n определяет добавление одного восточного шага и одного северного шага к пути Дика, который объединяет два пути Дика (из $(0; 0)$ в $(I; I)$ за $2I$ шагов и из $(I+1; I)$ в $(n; n-1)$ за $n-1-I$ шагов), где значение I определяется меткой $i := I$ выбранного узла: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_{2I}, \mathbf{E}, \mathbf{ps}_{2I+2}, \dots, \mathbf{ps}_{2n-1}, \mathbf{N})$;
- поддерево узла с меткой C_I (левое поддерево) определяет путь Дика из $(0; 0)$ в $(I; I)$;
- поддерево узла с меткой C_{n-1-I} (правое поддерево) определяет путь Дика из $(I+1; I)$ в $(n; n-1)$;
- лист с меткой C_0 определяет пустой решеточный путь $()$.

Таким образом, из заданного варианта v дерева И/ИЛИ для C_n рекурсивно формируется путь Дика вида $(\mathbf{ps}_1, \dots, \mathbf{ps}_{2n})$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ. Доказательство биективности данного отображения строится аналогично доказательству теоремы 4.1. □

Алгоритмы, реализующие разработанные правила биекции, имеют полиномиальную временную сложность $O(n^2)$, так как делают $2n$ рекурсивных вызовов, каждый из которых требует одного прохода для заполнения последовательности из $2n$ элементов. Пример применения этих правил биекции представлен в таблице 4.2.

Таблица 4.2 — Ранжирование множества путей Дика, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$

Решеточный путь	Вариант дерева И/ИЛИ	Ранг
(E,E,E,N,N,N)	$(0,(),(0,(),(0,(),())))$	0
(E,E,N,E,N,N)	$(0,(),(1,(0,(),()),()))$	1
(E,N,E,E,N,N)	$(1,(0,(),()),(0,(),()))$	2
(E,E,N,N,E,N)	$(2,(0,(),(0,(),())),())$	3
(E,N,E,N,E,N)	$(2,(1,(0,(),()),()),())$	4

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.3) и генерации по рангу (алгоритм 4.4) вариантов дерева И/ИЛИ для C_n , представленного на рисунке 4.6. В таблице 4.2 представлен пример ранжирования комбинаторного множества всех путей Дика, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$.

Алгоритм 4.3: Алгоритм ранжирования варианта дерева И/ИЛИ для C_n

```

1 Rank_C( $v = (I, v_1, v_2)$ ,  $n$ )
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5      $sum := \sum_{i=0}^{I-1} C_i C_{n-1-i}$ 
6      $w_1 := C_I$ 
7      $l_1 := \text{Rank\_C}(v_1, I)$ 
8      $l_2 := \text{Rank\_C}(v_2, n - 1 - I)$ 
9      $r := sum + l_1 + w_1 l_2$ 
10  end
11  return  $r$ 
12 end
```

Алгоритм 4.4: Алгоритм генерации по рангу варианта дерева И/ИЛИ
для C_n

```

1 Unrank_C( $r, n$ )
2 begin
3   if  $n = 0$  then  $v := ()$ 
4   else
5      $sum := 0$ 
6     for  $i := 0$  to  $n - 1$  do
7        $s := C_i C_{n-1-i}$ 
8       if  $sum + s > r$  then
9          $r := r - sum$ 
10         $I := i$ 
11        break
12      end
13       $sum := sum + s$ 
14    end
15     $w_1 := C_I$ 
16     $l_1 := r \bmod w_1$ 
17     $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
18     $v_1 := \text{Unrank\_C}(l_1, I)$ 
19     $v_2 := \text{Unrank\_C}(l_2, n - 1 - I)$ 
20     $v := (I, v_1, v_2)$ 
21  end
22  return  $v$ 
23 end

```

Разработанные алгоритмы имеют следующую вычислительную сложность:

- алгоритм 4.3 имеет не более n рекурсивных вызовов, каждый из которых требует вычисления C_n максимум $(2n - 1)$ раз. Применяя Формулу (4.3), вычисление C_n имеет линейную временную сложность $O(n)$. Следовательно, алгоритм 4.3 имеет полиномиальную временную сложность $O(n^3)$;

- алгоритм 4.4 имеет не более n рекурсивных вызовов, каждый из которых требует вычисления C_n максимум $(2n + 1)$ раз. Следовательно, алгоритм 4.4 имеет полиномиальную временную сложность $O(n^3)$.

4.2.3 Решеточные пути Деланнуа

Определение 4.4. *Путь Деланнуа из $(0; 0)$ в $(n; m)$ — это решеточный путь на плоскости, который начинается в точке $(0; 0)$, заканчивается в точке $(n; m)$ и состоит из шагов $(0; 1)$, $(1; 0)$ и $(1; 1)$.*

Шаг $(1; 1)$ называется северо-восточным шагом и обозначается **NE**. Множество путей Деланнуа из $(0; 0)$ в $(n; m)$ является обобщением северо-восточных решеточных путей из $(0; 0)$ в $(n; m)$ путем добавления северо-восточных шагов. На рисунке 4.8 показаны все возможные 25 вариантов путей Деланнуа для $n = 3$ и $m = 2$.

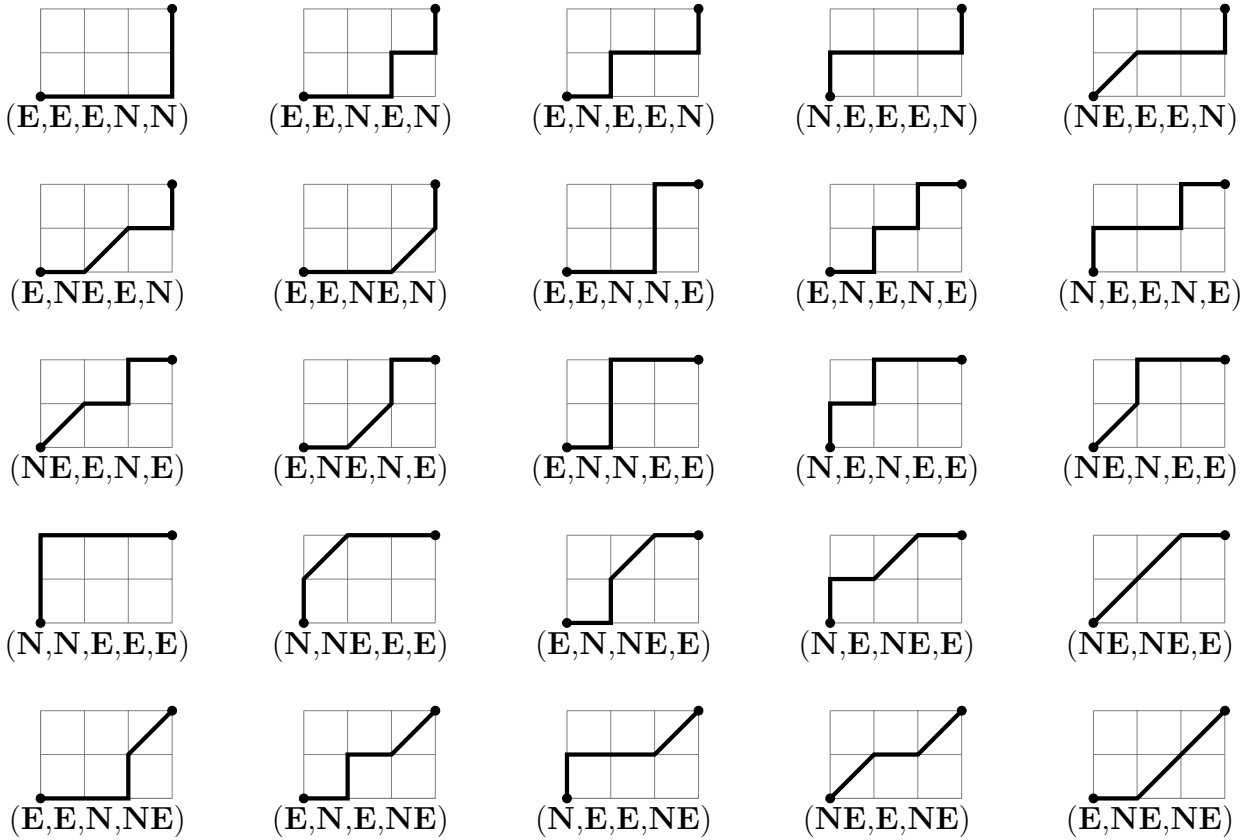


Рисунок 4.8 — Все пути Деланнуа, начинающиеся в точке $(0; 0)$ и заканчивающиеся в точке $(3; 2)$

Общее количество путей Деланнуа из $(0; 0)$ в $(n; m)$ для $n, m \in \mathbb{Z}_{\geq 0}$ определяется числом Деланнуа D_n^m (последовательность A008288 в OEIS [174]):

$$D_n^m = \sum_{i=1}^{\min(n,m)} \binom{n}{i} \binom{m}{i} 2^i. \quad (4.5)$$

Значение D_n^m также вычисляется, используя рекуррентную формулу

$$D_n^m = \begin{cases} 0, & n < 0 \text{ или } m < 0; \\ 1, & n \geq m = 0 \text{ или } m \geq n = 0; \\ D_n^{m-1} + D_{n-1}^m + D_{n-1}^{m-1}, & \text{иначе.} \end{cases} \quad (4.6)$$

Кроме того, известна производящая функция

$$\sum_{n \geq 0} \sum_{m \geq 0} D_n^m x^n y^m = \frac{1}{1 - x - y - xy}.$$

Поскольку формула (4.6) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для D_n^m (рисунок 4.9).

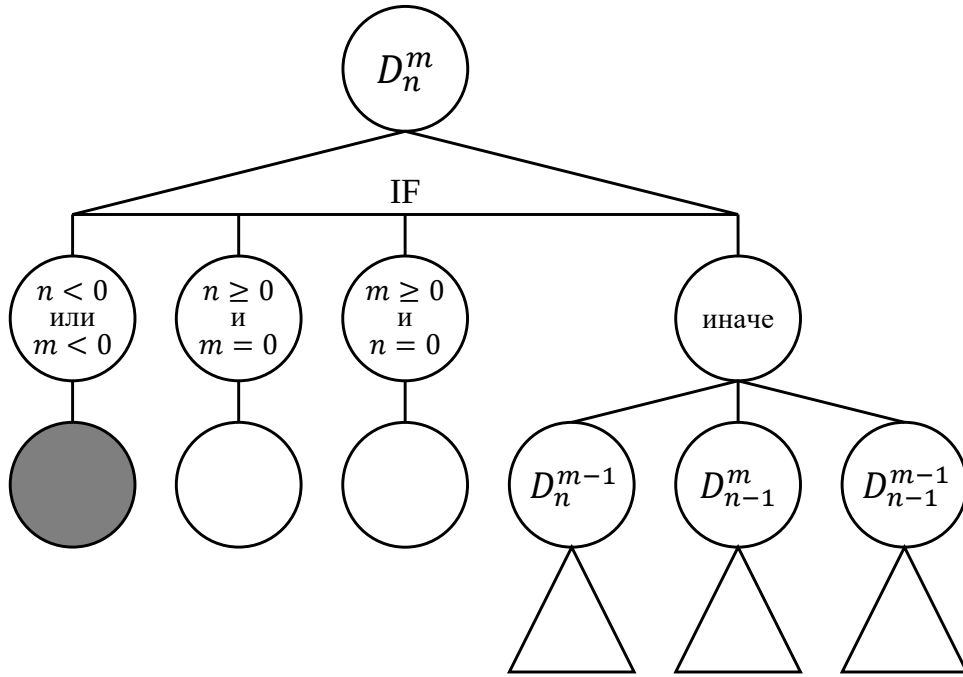


Рисунок 4.9 — Структура дерева И/ИЛИ для D_n^m

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой D_n^0 является листом в дереве И/ИЛИ для D_n^m ;
- каждый узел с меткой D_0^m является листом в дереве И/ИЛИ для D_n^m .

Поскольку полученная структура дерева И/ИЛИ не содержит И-узлов, то каждый вариант такого дерева представляет собой путь от корня к листу. Поэтому будем кодировать вариант дерева И/ИЛИ для D_n^m последовательностью $v = (v_1, v_2, \dots)$ выбранных потомков ИЛИ-узлов в этом дереве (выбору левого потомка соответствует $v_i = 1$, выбору среднего потомка — $v_i = 2$, а выбору правого потомка — $v_i = 3$).

На рисунке 4.10 представлен пример структуры дерева И/ИЛИ для D_n^m , где $n = 3$ и $m = 2$. Общее количество его вариантов равно $D_3^2 = 25$.

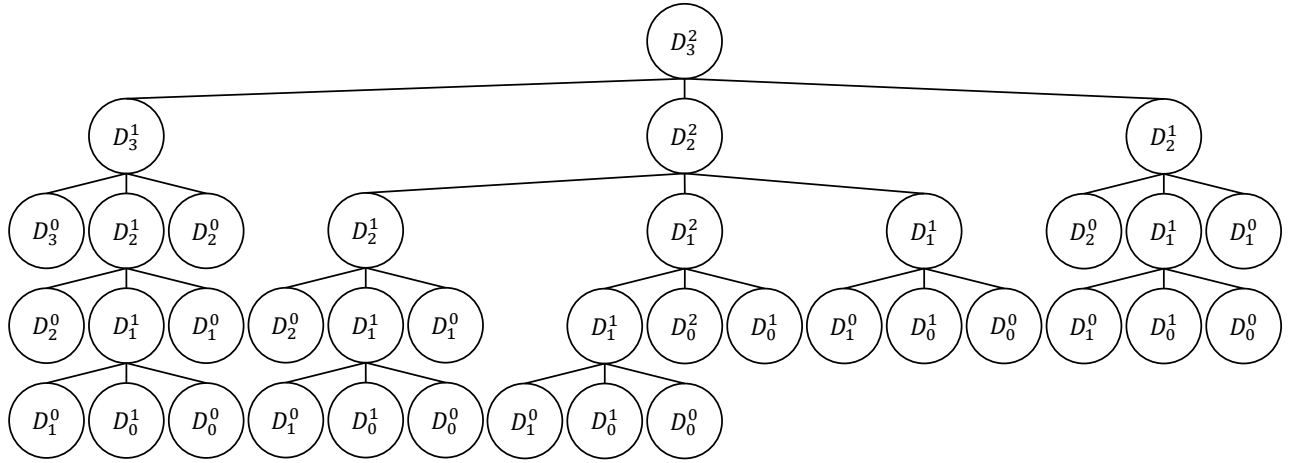


Рисунок 4.10 — Структура дерева И/ИЛИ для D_3^2

Теорема 4.3. *Существует биекция между множеством путей Деланнуа, начинающихся в точке $(0; 0)$ и заканчивающихся в $(n; m)$, и множеством вариантов дерева И/ИЛИ для D_n^m , представленного на рисунке 4.9.*

Доказательство. Общее количество путей Деланнуа, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, равно D_n^m . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для D_n^m , представленного на рисунке 4.9, равно значению D_n^m . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.6), биективное отображение между множеством вариантов дерева И/ИЛИ для D_n^m и множеством путей Деланнуа из $(0; 0)$ в $(n; m)$ определяется по следующим правилам:

- выбранный левый потомок ИЛИ-узла с меткой D_n^m определяет добавление одного северного шага к пути Деланнуа, полученному поддеревом узла с меткой D_n^{m-1} и содержащему k шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_k, \mathbf{N})$;

- выбранный средний потомок ИЛИ-узла с меткой D_n^m определяет добавление одного восточного шага к пути Деланнуа, полученному поддеревом узла с меткой D_{n-1}^m и содержащему k шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_k, \mathbf{E})$;

- выбранный правый потомок ИЛИ-узла с меткой D_n^m определяет добавление одного северо-восточного шага к пути Деланнуа, полученному поддеревом узла с меткой D_{n-1}^{m-1} и содержащему k шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_k, \mathbf{NE})$;
- лист с меткой D_n^0 определяет решеточный путь от точки $(0; 0)$ до точки $(n; 0)$, состоящий из n восточных шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_n) = (\mathbf{E}, \dots, \mathbf{E})$;
- лист с меткой D_0^m определяет решеточный путь от точки $(0, 0)$ до точки $(0, m)$, состоящий из m северных шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_m) = (\mathbf{N}, \dots, \mathbf{N})$.

Таким образом, из заданного варианта v дерева И/ИЛИ для D_n^m рекурсивно формируется путь Деланнуа вида $(\mathbf{ps}_1, \mathbf{ps}_2, \dots)$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ. Доказательство биективности данного отображения строится аналогично доказательству теоремы 4.1. $\square$

Алгоритмы, реализующие разработанные правила биекции, имеют линейную временную сложность $O(n + m)$, так как требуют одного прохода для заполнения последовательности из максимум $n + m$ элементов. Пример применения этих правил биекции представлен в таблице 4.3.

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.5) и генерации по рангу (алгоритм 4.6) вариантов дерева И/ИЛИ для D_n^m , представленного на рисунке 4.9. В таблице 4.3 представлен пример ранжирования комбинаторного множества всех путей Деланнуа, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 2)$.

Разработанные алгоритмы имеют следующую вычислительную сложность:

- алгоритм 4.5 имеет не более t рекурсивных вызовов, где $v_1 = 1$ (каждый такой вызов требует одного присваивания), не более n рекурсивных вызовов, где $v_1 = 2$ (каждый такой вызов требует вычисления D_n^{m-1}), и не более $\min(n, m)$ рекурсивных вызовов, где $v_1 = 3$ (каждый такой вызов требует вычисления D_n^{m-1} и D_{n-1}^m). Применяя формулу (4.5), вычисление D_n^m имеет полиномиальную временную сложность $O(m^2)$ для $m < n$ и $O(n^2)$ для $m > n$. Следовательно, алгоритм 4.5 имеет полиномиальную временную сложность $O(m^2(n + m))$ для $m < n$ и $O(m + n^3)$ для $m > n$;

Таблица 4.3 — Ранжирование множества путей Деланнуа, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 2)$

Решеточный путь	Вариант дерева И/ИЛИ	Ранг
(E,E,E,N,N)	(1,1)	0
(E,E,N,E,N)	(1,2,1)	1
(E,N,E,E,N)	(1,2,2,1)	2
(N,E,E,E,N)	(1,2,2,2)	3
(NE,E,E,N)	(1,2,2,3)	4
(E,NE,E,N)	(1,2,3)	5
(E,E,NE,N)	(1,3)	6
(E,E,N,N,E)	(2,1,1)	7
(E,N,E,N,E)	(2,1,2,1)	8
(N,E,E,N,E)	(2,1,2,2)	9
(NE,E,N,E)	(2,1,2,3)	10
(E,NE,N,E)	(2,1,3)	11
(E,N,N,E,E)	(2,2,1,1)	12
(N,E,N,E,E)	(2,2,1,2)	13
(NE,N,E,E)	(2,2,1,3)	14
(N,N,E,E,E)	(2,2,2)	15
(N,NE,E,E)	(2,2,3)	16
(E,N,NE,E)	(2,3,1)	17
(N,E,NE,E)	(2,3,2)	18
(NE,NE,E)	(2,3,3)	19
(E,E,N,NE)	(3,1)	20
(E,N,E,NE)	(3,2,1)	21
(N,E,E,NE)	(3,2,2)	22
(NE,E,NE)	(3,2,3)	23
(E,NE,NE)	(3,3)	24

Алгоритм 4.5: Алгоритм ранжирования варианта дерева И/ИЛИ
для D_n^m

```

1 Rank_D( $v = (v_1, v_2, \dots)$ ,  $n$ ,  $m$ )
2 begin
3   if  $n = 0$  или  $m = 0$  then  $r := 0$ 
4   else
5     if  $v_1 = 1$  then  $r := \text{Rank\_D}((v_2, \dots), n, m - 1)$ 
6     else if  $v_1 = 2$  then  $r := D_n^{m-1} + \text{Rank\_D}((v_2, \dots), n - 1, m)$ 
7     else  $r := D_n^{m-1} + D_{n-1}^m + \text{Rank\_D}((v_2, \dots), n - 1, m - 1)$ 
8   end
9   return  $r$ 
10 end

```

Алгоритм 4.6: Алгоритм генерации по рангу варианта дерева И/ИЛИ
для D_n^m

```

1 Unrank_D( $r$ ,  $n$ ,  $m$ )
2 begin
3   if  $n = 0$  или  $m = 0$  then  $v := ()$ 
4   else
5     if  $r < D_n^{m-1}$  then  $v := \text{concat}((1), \text{Unrank\_D}(r, n, m - 1))$ 
6     else if  $r < D_n^{m-1} + D_{n-1}^m$  then
7        $v := \text{concat}((2), \text{Unrank\_D}(r - D_n^{m-1}, n - 1, m))$ 
8     else  $v := \text{concat}((3), \text{Unrank\_D}(r - D_n^{m-1} - D_{n-1}^m, n - 1, m - 1))$ 
9   end
10  return  $v$ 
11 end

```

– алгоритм 4.6 имеет не более $(n + m)$ рекурсивных вызовов, каждый из которых требует вычисления D_n^{m-1} или D_{n-1}^m . Следовательно, алгоритм 4.6 имеет полиномиальную временную сложность $O(m^2(n + m))$ для $m < n$ и $O(n^2(n + m))$ для $m > n$.

4.2.4 Решеточные пути Шредера

Определение 4.5. *Путь Шредера из $(0; 0)$ в $(n; n)$ — это решеточный путь на плоскости, который начинается в точке $(0; 0)$, заканчивается в точке $(n; n)$, состоит из шагов $(0; 1)$, $(1; 0)$ и $(1; 1)$, и никогда не поднимается выше диагонали $y = x$.*

Множество путей Шредера из $(0; 0)$ в $(n; n)$ является подмножеством путей Деланнуа из $(0; 0)$ в $(n; n)$. Также множество путей Шредера из $(0; 0)$ в $(n; n)$ является обобщением путей Дика из $(0; 0)$ в $(n; n)$ путем добавления северо-восточных шагов. На рисунке 4.11 показаны все возможные 22 варианта путей Шредера для $n = 3$.

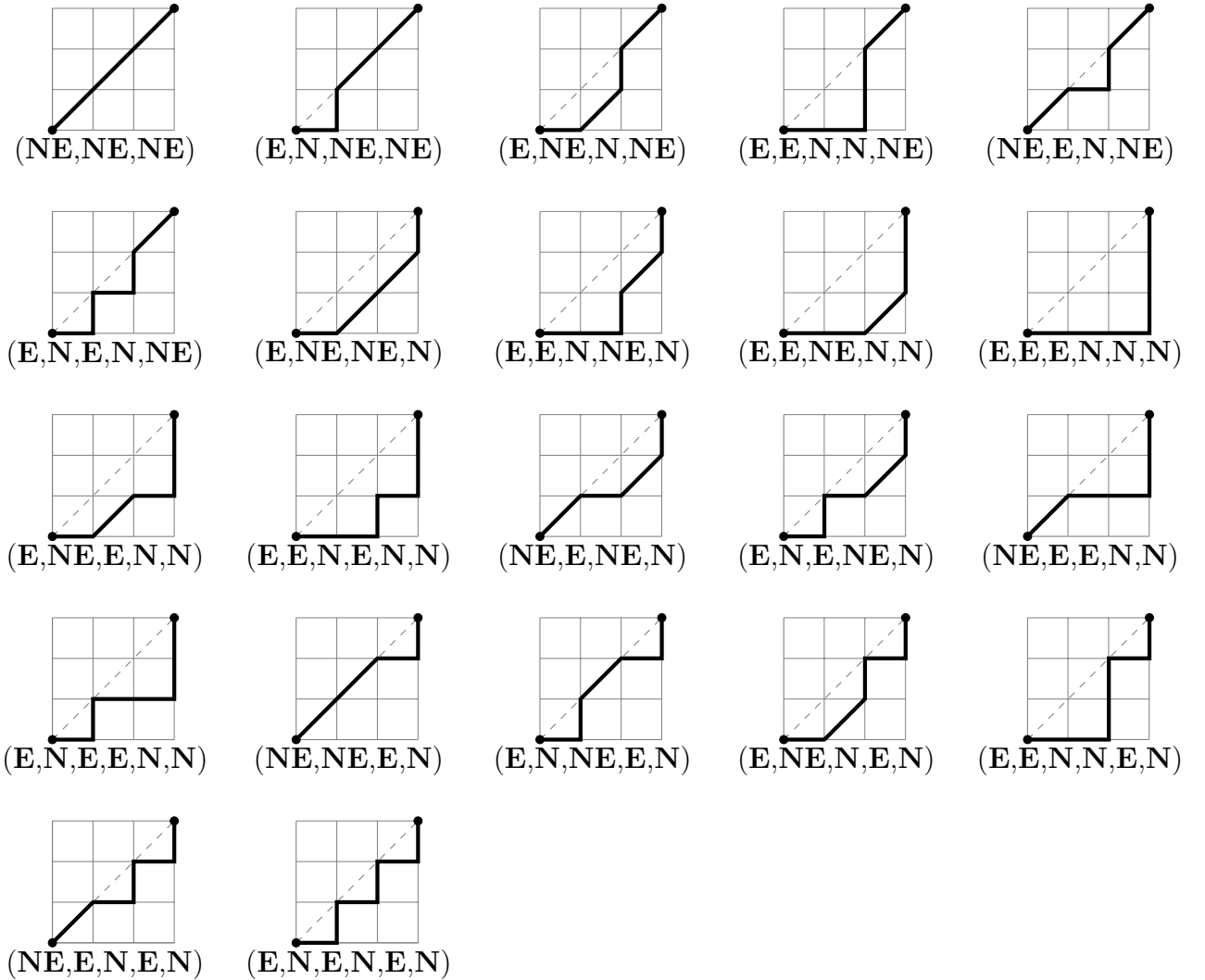


Рисунок 4.11 — Все пути Шредера, начинающиеся в точке $(0; 0)$ и заканчивающиеся в точке $(3; 3)$

Общее количество путей Шредера из $(0; 0)$ в $(n; n)$ для $n \in \mathbb{Z}_{\geq 0}$ определяется числом Шредера S_n (последовательность A006318 в OEIS [174]):

$$S_n = \sum_{i=0}^n \frac{1}{i+1} \binom{n+i}{i} \binom{n}{i}. \quad (4.7)$$

Значение S_n также вычисляется, используя рекуррентную формулу

$$S_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0; \\ S_{n-1} + \sum_{i=0}^{n-1} S_i S_{n-1-i}, & \text{иначе.} \end{cases} \quad (4.8)$$

Кроме того, известна производящая функция

$$\sum_{n \geq 0} S_n x^n = \frac{1 - x - \sqrt{1 - 6x + x^2}}{2x}.$$

Поскольку формула (4.8) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для S_n (рисунок 4.12).

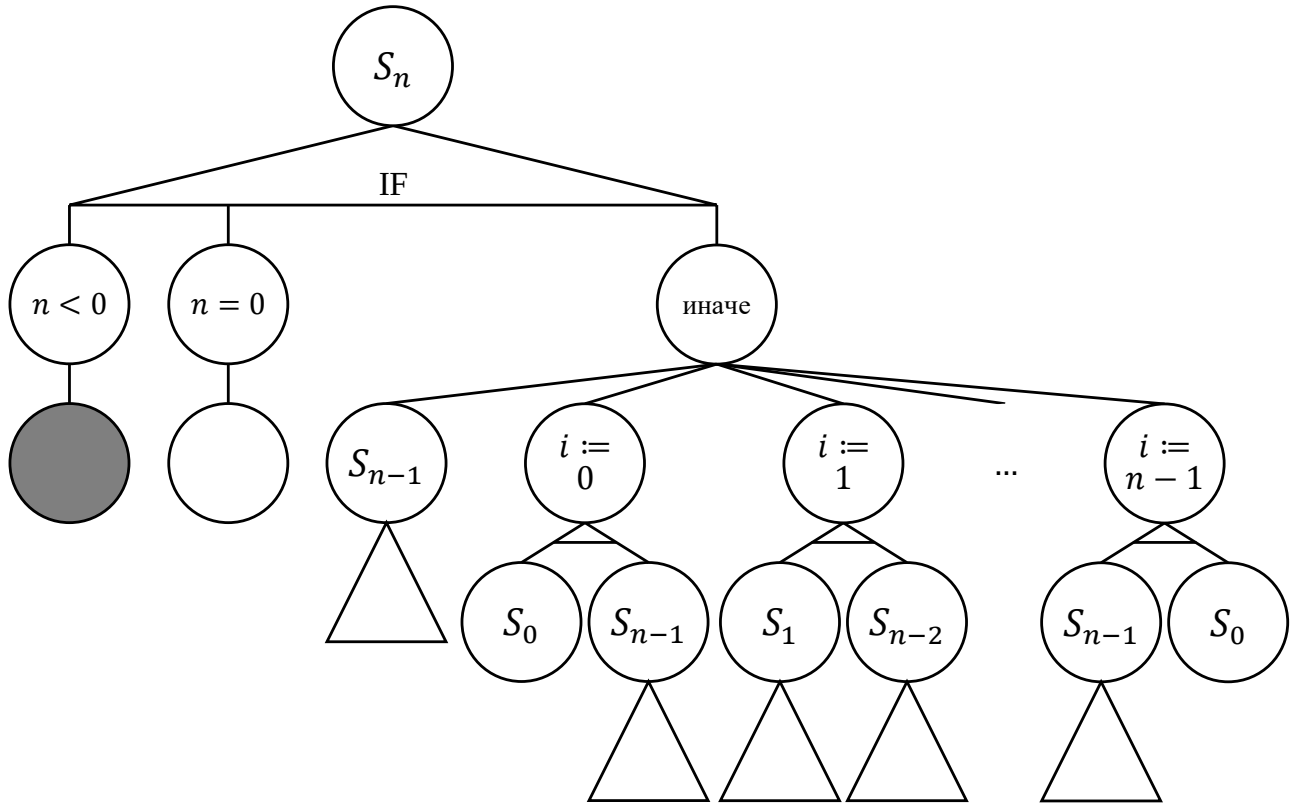


Рисунок 4.12 — Структура дерева И/ИЛИ для S_n

Для компактности представления воспользуемся следующим допущением: каждый узел с меткой S_0 является листом в дереве И/ИЛИ для S_n . На рисунке 4.13 представлен пример структуры дерева И/ИЛИ для S_n , где $n = 3$. Общее количество его вариантов равно $S_3 = 22$.

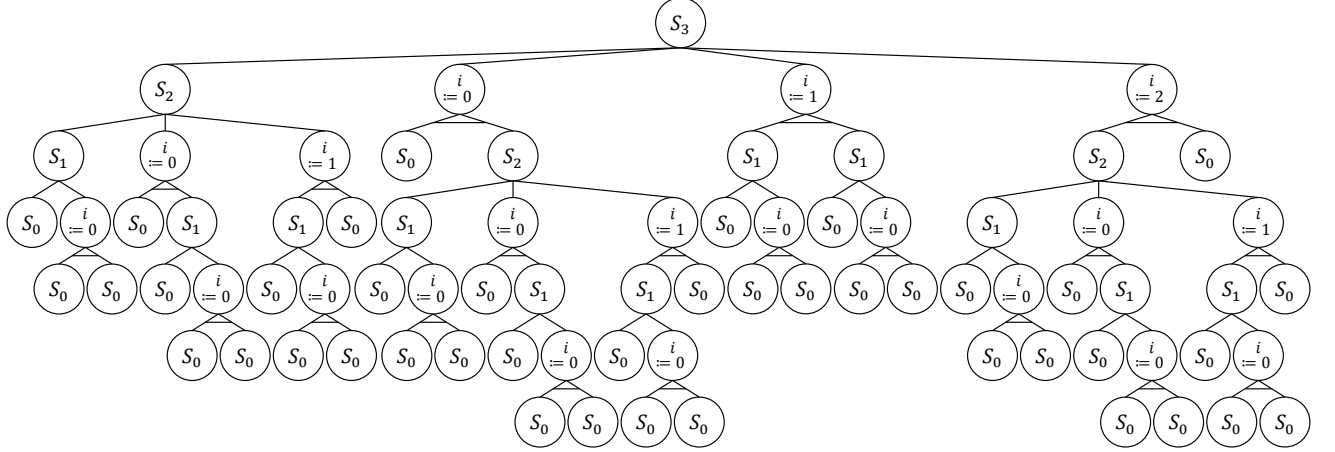


Рисунок 4.13 — Структура дерева И/ИЛИ для S_3

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для S_n последовательностью v , где применимо следующее:

- пустая последовательность $v = ()$ соответствует выбору листа с меткой S_0 ;
- если выбран левый потомок ИЛИ-узла с меткой S_n , то $v = (I, v_1)$, где:
 - $I = -1$;
 - v_1 соответствует варианту поддерева узла с меткой S_{n-1} ;
- иначе $v = (I, v_1, v_2)$, где:
 - I соответствует выбранному значению i в дереве И/ИЛИ для S_n ;
 - v_1 соответствует варианту поддерева узла с меткой S_I ;
 - v_2 соответствует варианту поддерева узла с меткой S_{n-1-I} .

Указанный способ кодирования основан на представлении древовидной структуры данных в виде правильной скобочной последовательности. При этом первый элемент внутри пары соответствующих скобок показывает одновременно способ организации последующих элементов и вспомогательную информацию для их построения.

Теорема 4.4. *Существует биективное отображение между множеством путей Шредера, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, и множеством вариантов дерева И/ИЛИ для S_n , представленного на рисунке 4.12.*

Доказательство. Общее количество путей Шредера, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, равно S_n . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для S_n , представленного на рисунке 4.12, равно значению S_n . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.8), биективное отображение между множеством вариантов дерева И/ИЛИ для S_n и множеством путей Шредера из $(0; 0)$ в $(n; n)$ определяется по следующим правилам:

- выбранный левый потомок ИЛИ-узла с меткой S_n определяет добавление одного северо-восточного шага к пути Шредера, полученному поддеревом узла с меткой S_{n-1} и состоящему из k шагов: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_k, \mathbf{NE})$;
- выбранный нелевый потомок ИЛИ-узла с меткой S_n определяет добавление одного восточного шага и одного северного шага к пути Шредера, который объединяет два пути Шредера (из $(0; 0)$ в $(I; I)$ за k_1 шагов и из $(I+1; I)$ в $(n; n-1)$ за k_2 шагов), где значение I определяется меткой $i := I$ выбранного узла: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_{k_1}, \mathbf{E}, \mathbf{ps}_{k_1+2}, \dots, \mathbf{ps}_{k_1+1+k_2}, \mathbf{N})$;
- поддерево узла с меткой S_I (левое поддерево) определяет путь Шредера из $(0; 0)$ в $(I; I)$;
- поддерево узла с меткой S_{n-1-I} (правое поддерево) определяет путь Шредера из $(I+1; I)$ в $(n; n-1)$;
- лист с меткой S_0 определяет пустой решеточный путь $()$.

Таким образом, из заданного варианта v дерева И/ИЛИ для S_n рекурсивно формируется путь Шредера вида $(\mathbf{ps}_1, \mathbf{ps}_2, \dots)$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ. Доказательство биективности данного отображения строится аналогично доказательству теоремы 4.1. $\square$

Алгоритмы, реализующие разработанные правила биекции, имеют полиномиальную временную сложность $O(n^2)$, так как делают n рекурсивных вызовов, каждый из которых требует одного прохода для заполнения последовательности из максимум $2n$ элементов. Пример применения этих правил биекции представлен в таблице 4.4.

Таблица 4.4 — Ранжирование множества путей Шредера, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$

Решеточный путь	Вариант дерева И/ИЛИ	Ранг
(NE,NE,NE)	$(-1,(-1,(-1,())))$	0
(E,N,NE,NE)	$(-1,(-1,(0,()),()))$	1
(E,NE,N,NE)	$(-1,(0,()),(-1,()))$	2
(E,E,N,N,NE)	$(-1,(0,()),(0,()),())$	3
(NE,E,N,NE)	$(-1,(1,(-1,()),()))$	4
(E,N,E,N,NE)	$(-1,(1,(0,()),()),())$	5
(E,NE,NE,N)	$(0,()),(-1,(-1,()))$	6
(E,E,N,NE,N)	$(0,()),(-1,(0,()),())$	7
(E,E,NE,N,N)	$(0,()),(0,()),(-1,())$	8
(E,E,E,N,N,N)	$(0,()),(0,()),(0,()),())$	9
(E,NE,E,N,N)	$(0,()),(1,(-1,()),())$	10
(E,E,N,E,N,N)	$(0,()),(1,(0,()),()),()$	11
(NE,E,NE,N)	$(1,(-1,()),(-1,()))$	12
(E,N,E,NE,N)	$(1,(0,()),()),(-1,())$	13
(NE,E,E,N,N)	$(1,(-1,()),(0,()),())$	14
(E,N,E,E,N,N)	$(1,(0,()),()),(0,()),()$	15
(NE,NE,E,N)	$(2,(-1,(-1,()),()),)$	16
(E,N,NE,E,N)	$(2,(-1,(0,()),()),())$	17
(E,NE,N,E,N)	$(2,(0,()),(-1,()),())$	18
(E,E,N,N,E,N)	$(2,(0,()),(0,()),()),()$	19
(NE,E,N,E,N)	$(2,(1,(-1,()),()),())$	20
(E,N,E,N,E,N)	$(2,(1,(0,()),()),()),()$	21

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.7) и генерации по рангу (алгоритм 4.8) вариантов дерева И/ИЛИ для S_n , представленного на рисунке 4.12. В таблице 4.4 представлен пример ранжирования комбинаторного множества всех путей Шредера, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$.

Разработанные алгоритмы имеют следующую вычислительную сложность:

– алгоритм 4.7 имеет не более n рекурсивных вызовов, где $I = -1$ (каждый такой вызов требует одного присваивания), и не более n рекурсивных вызовов, где $I \neq -1$ (каждый такой вызов требует вычисления S_n максимум $2n$ раз). Применяя формулу (4.7), вычисление S_n имеет полиномиальную временную сложность $O(n^2)$. Следовательно, алгоритм 4.7 имеет полиномиальную временную сложность $O(n^4)$;

– алгоритм 4.8 имеет не более n рекурсивных вызовов, где $I = -1$ (каждый такой вызов требует вычисления S_n), и не более n рекурсивных вызовов, где $I \neq -1$ (каждый такой вызов требует вычисления S_n максимум $2n + 2$ раз). Следовательно, алгоритм 4.8 имеет полиномиальную временную сложность $O(n^4)$.

Алгоритм 4.7: Алгоритм ранжирования варианта дерева И/ИЛИ для S_n

```

1 Rank_S( $v = (I, v_1, \dots), n$ )
2 begin
3   if  $n = 0$  then  $r := 0$ 
4   else
5     if  $I = -1$  then  $r := \text{Rank\_S}(v_1, n - 1)$ 
6     else
7        $sum := S_{n-1} + \sum_{i=0}^{I-1} S_i S_{n-1-i}$ 
8        $w_1 := S_I$ 
9        $l_1 := \text{Rank\_S}(v_1, I)$ 
10       $l_2 := \text{Rank\_S}(v_2, n - 1 - I)$ 
11       $r := sum + l_1 + w_1 l_2$ 
12    end
13  end
14  return  $r$ 
15 end
```

Алгоритм 4.8: Алгоритм генерации по рангу варианта дерева И/ИЛИ
для S_n

```

1  Unrank_S( $r, n$ )
2  begin
3      if  $n = 0$  then  $v := ()$ 
4      else
5          if  $r < S_{n-1}$  then
6               $I := -1$ 
7               $v_1 := \text{Unrank\_S}(r, n - 1)$ 
8               $v := (I, v_1)$ 
9          end
10         else
11              $r := r - S_{n-1}$ 
12              $sum := 0$ 
13             for  $i := 0$  to  $n - 1$  do
14                  $s := S_i S_{n-1-i}$ 
15                 if  $sum + s > r$  then
16                      $r := r - sum$ 
17                      $I := i$ 
18                     break
19                 end
20                  $sum := sum + s$ 
21             end
22              $w_1 := S_I$ 
23              $l_1 := r \bmod w_1$ 
24              $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
25              $v_1 := \text{Unrank\_S}(l_1, I)$ 
26              $v_2 := \text{Unrank\_S}(l_2, n - 1 - I)$ 
27              $v := (I, v_1, v_2)$ 
28         end
29     end
30     return  $v$ 
31 end

```

4.2.5 Решеточные пути Моцкина

Определение 4.6. *Путь Моцкина из $(0; 0)$ в $(n; n)$ — это решеточный путь на плоскости, который начинается в точке $(0; 0)$, заканчивается в точке $(n; n)$, состоит из шагов $(0; 2)$, $(2; 0)$ и $(1; 1)$, и никогда не поднимается выше диагонали $y = x$.*

Шаг $(0; 2)$ называется двойным северным шагом и состоит из двух последовательно выполненных северных шагов **N**. Шаг $(2; 0)$ называется двойным восточным шагом и состоит из двух последовательно выполненных восточных шагов **E**. Множество путей Моцкина из $(0; 0)$ в $(n; n)$ является подмножеством путей Шредера из $(0; 0)$ в $(n; n)$. На рисунке 4.14 показаны все возможные 4 варианта рассматриваемых путей Моцкина для $n = 3$.

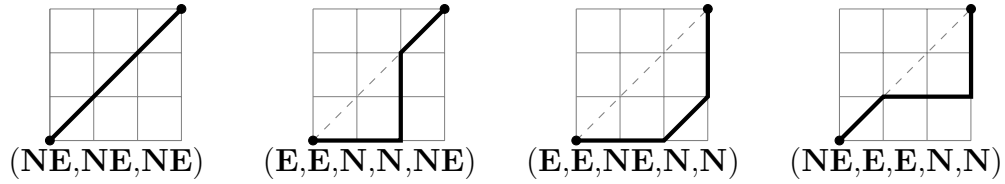


Рисунок 4.14 — Все пути Моцкина, начинающиеся в точке $(0; 0)$ и заканчивающиеся в точке $(3; 3)$

Общее количество путей Моцкина из $(0; 0)$ в $(n; n)$ определяется числом Моцкина M_n (последовательность A001006 в OEIS [174]):

$$M_n = \sum_{i=0}^{\lfloor \frac{n}{2} \rfloor} \frac{1}{i+1} \binom{n}{2i} \binom{2i}{i}. \quad (4.9)$$

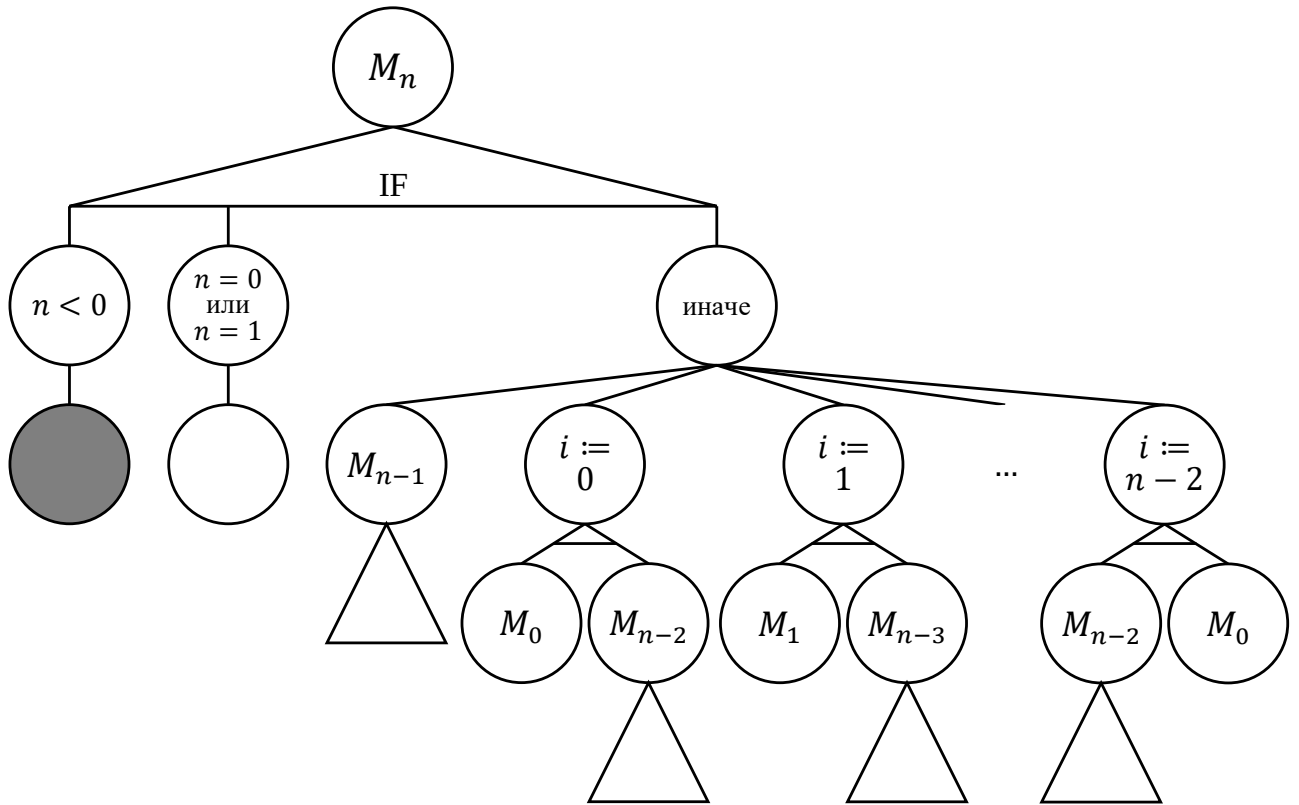
Значение M_n также вычисляется, используя рекуррентную формулу

$$M_n = \begin{cases} 0, & n < 0; \\ 1, & n = 0 \text{ или } n = 1; \\ M_{n-1} + \sum_{i=0}^{n-2} M_i M_{n-2-i}, & \text{иначе.} \end{cases} \quad (4.10)$$

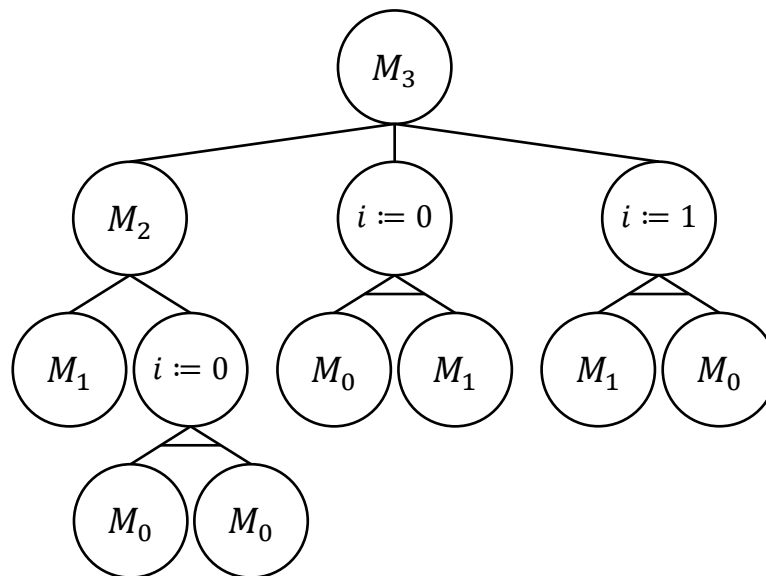
Кроме того, известна производящая функция

$$\sum_{n \geq 0} M_n x^n = \frac{1 - x - \sqrt{1 - 2x - 3x^2}}{2x^2}.$$

Поскольку формула (4.10) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для M_n (рисунок 4.15).

Рисунок 4.15 — Структура дерева И/ИЛИ для M_n

Для компактности представления воспользуемся следующим допущением: каждый узел с меткой M_0 или M_1 является листом в дереве И/ИЛИ для M_n . На рисунке 4.16 представлен пример структуры дерева И/ИЛИ для M_n , где $n = 3$. Общее количество его вариантов равно $M_3 = 4$.

Рисунок 4.16 — Структура дерева И/ИЛИ для M_3

Также для удобства программной реализации будем кодировать вариант дерева И/ИЛИ для M_n последовательностью v , где применимо следующее:

- пустая последовательность $v = ()$ соответствует выбору листа с меткой M_0 или M_1 ;
- если выбран левый потомок ИЛИ-узла с меткой M_n , то $v = (I, v_1)$, где:
 - $I = -1$;
 - v_1 соответствует варианту поддерева узла с меткой M_{n-1} ;
- иначе $v = (I, v_1, v_2)$, где:
 - I соответствует выбранному значению i в дереве И/ИЛИ для M_n ;
 - v_1 соответствует варианту поддерева узла с меткой M_I ;
 - v_2 соответствует варианту поддерева узла с меткой M_{n-2-I} .

Указанный способ кодирования основан на представлении древовидной структуры данных в виде правильной скобочной последовательности. При этом первый элемент внутри пары соответствующих скобок показывает одновременно способ организации последующих элементов и вспомогательную информацию для их построения.

Теорема 4.5. *Существует биективное отображение между множеством путей Моцкина, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, и множеством вариантов дерева И/ИЛИ для M_n , представленного на рисунке 4.15.*

Доказательство. Общее количество путей Моцкина, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; n)$, равно M_n . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для M_n , представленного на рисунке 4.15, равно значению M_n . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.10), биективное отображение между множеством вариантов дерева И/ИЛИ для M_n и множеством путей Моцкина из $(0; 0)$ в $(n; n)$ определяется по следующим правилам:

- выбранный левый потомок ИЛИ-узла с меткой M_n определяет добавление одного северо-восточного шага к пути Моцкина, полученному поддеревом узла с меткой M_{n-1} и состоящему из k шагов: результирующий решеточный путь равен $(ps_1, \dots, ps_k, \mathbf{NE})$;

- выбранный нелевый потомок ИЛИ-узла с меткой M_n определяет добавление двойного восточного шага и двойного северного шага к пути Моцкина, который объединяет два пути Моцкина (из $(0; 0)$ в $(I; I)$ за k_1 шагов и из $(I+2; I)$ в $(n; n-2)$ за k_2 шагов), где значение I определяется меткой $i := I$ выбранного узла: результирующий решеточный путь равен $(\mathbf{ps}_1, \dots, \mathbf{ps}_{k_1}, \mathbf{E}, \mathbf{E}, \mathbf{ps}_{k_1+2}, \dots, \mathbf{ps}_{k_1+1+k_2}, \mathbf{N}, \mathbf{N})$;
- поддерево узла с меткой M_I (левое поддерево) определяет путь Моцкина из $(0; 0)$ в $(I; I)$;
- поддерево узла с меткой M_{n-2-I} (правое поддерево) определяет путь Моцкина из $(I+2; I)$ в $(n; n-2)$;
- лист с меткой M_1 определяет решеточный путь от точки $(0; 0)$ до точки $(1; 1)$, состоящий из одного северо-восточного шага: результирующий решеточный путь равен $(\mathbf{ps}_1) = (\mathbf{NE})$;
- лист с меткой M_0 определяет пустой решеточный путь $()$.

Таким образом, из заданного варианта v дерева И/ИЛИ для M_n рекурсивно формируется путь Моцкина вида $(\mathbf{ps}_1, \mathbf{ps}_2, \dots)$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ. Доказательство биективности данного отображения строится аналогично доказательству теоремы 4.1. $\square$

Алгоритмы, реализующие разработанные правила биекции, имеют полиномиальную временную сложность $O(n^2)$, так как делают n рекурсивных вызовов, где каждый такой вызов требует одного прохода для заполнения последовательности из максимум $2n$ элементов. Пример применения этих правил биекции представлен в таблице 4.5.

Таблица 4.5 — Ранжирование множества путей Моцкина, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$

Решеточный путь	Вариант дерева И/ИЛИ	Ранг
$(\mathbf{NE}, \mathbf{NE}, \mathbf{NE})$	$(-1, (-1, (-1, ())))$	0
$(\mathbf{E}, \mathbf{E}, \mathbf{N}, \mathbf{N}, \mathbf{NE})$	$(-1, (0, ()), ())$	1
$(\mathbf{E}, \mathbf{E}, \mathbf{NE}, \mathbf{N}, \mathbf{N})$	$(0, ()), (-1, ())$	2
$(\mathbf{NE}, \mathbf{E}, \mathbf{E}, \mathbf{N}, \mathbf{N})$	$(1, (-1, ()), ())$	3

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.9) и генерации по рангу (алгоритм 4.10) вариантов дерева И/ИЛИ для M_n , представленного на рисунке 4.15. В таблице 4.5 представлен пример ранжирования комбинаторного множества всех путей Моцкина, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(3; 3)$.

Разработанные алгоритмы имеют следующую вычислительную сложность:

- алгоритм 4.9 имеет не более n рекурсивных вызовов, где $I = -1$ (каждый такой вызов требует одного присваивания), и не более n рекурсивных вызовов, где $I \neq -1$ (каждый такой вызов требует вычисления M_n максимум $2n - 2$ раз). Применяя формулу (4.9), вычисление M_n имеет полиномиальную временную сложность $O(n^2)$. Следовательно, алгоритм 4.9 имеет полиномиальную временную сложность $O(n^4)$;

- алгоритм 4.10 имеет не более n рекурсивных вызовов, где $I = -1$ (каждый такой вызов требует вычисления M_n) и не более n рекурсивных вызовов, где $I \neq -1$ (каждый такой вызов требует вычисления M_n максимум $2n$ раз). Следовательно, алгоритм 4.10 имеет полиномиальную временную сложность $O(n^4)$.

Алгоритм 4.9: Алгоритм ранжирования варианта дерева И/ИЛИ для M_n

```

1 Rank_M( $v = (I, v_1, \dots), n$ )
2 begin
3   if  $n = 0$  или  $n = 1$  then  $r := 0$ 
4   else
5     if  $I = -1$  then  $r := \text{Rank\_M}(v_1, n - 1)$ 
6     else
7        $sum := M_{n-1} + \sum_{i=0}^{I-1} M_i M_{n-2-i}$ 
8        $w_1 := M_I$ 
9        $l_1 := \text{Rank\_M}(v_1, I)$ 
10       $l_2 := \text{Rank\_M}(v_2, n - 2 - I)$ 
11       $r := sum + l_1 + w_1 l_2$ 
12    end
13  end
14  return  $r$ 
15 end
```

Алгоритм 4.10: Алгоритм генерации по рангу варианта дерева И/ИЛИ
для M_n

```

1  Unrank_M( $r, n$ )
2  begin
3      if  $n = 0$  или  $n = 1$  then  $v := ()$ 
4      else
5          if  $r < M_{n-1}$  then
6               $I := -1$ 
7               $v_1 := \text{Unrank\_M}(r, n - 1)$ 
8               $v := (I, v_1)$ 
9          end
10         else
11              $r := r - M_{n-1}$ 
12              $sum := 0$ 
13             for  $i := 0$  to  $n - 1$  do
14                  $s := M_i M_{n-2-i}$ 
15                 if  $sum + s > r$  then
16                      $r := r - sum$ 
17                      $I := i$ 
18                     break
19                 end
20                  $sum := sum + s$ 
21             end
22              $w_1 := M_I$ 
23              $l_1 := r \bmod w_1$ 
24              $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
25              $v_1 := \text{Unrank\_M}(l_1, I)$ 
26              $v_2 := \text{Unrank\_M}(l_2, n - 2 - I)$ 
27              $v := (I, v_1, v_2)$ 
28         end
29     end
30     return  $v$ 
31 end

```

4.2.6 Вычислительные эксперименты

С целью подтверждения полученных оценок временной сложности разработанных алгоритмов ранжирования и генерации по рангу для частных случаев решеточных путей, был проведен вычислительный эксперимент. Для этого были созданы программные реализации соответствующих алгоритмов на языке системы компьютерной алгебры «Maxima» [212]. Запуск программных реализаций осуществлялся на ноутбуке со следующими характеристиками: Intel i7-9750H, 2,6 ГГц, Windows 10, 64 бит. Суть вычислительного эксперимента заключалась в измерении времени работы алгоритмов ранжирования и генерации по рангу вариантов деревьев И/ИЛИ для L_n^m , C_n , D_n^m , S_n и M_n с разными значениями параметров n и m . Все расчеты проводились для вариантов деревьев И/ИЛИ с максимальным значением ранга, поскольку это обычно самый сложный случай для вычислений из-за необходимости обхода всей структуры дерева И/ИЛИ.

Для частного случая максимального значения ранга $r := L_n^m - 1$ алгоритмы 4.1 и 4.2 имеют полиномиальную временную сложность $O(nm)$ при $m < n$ и $O(n^2)$ при $n < m$. На рисунке 4.17 показаны результаты измерения зависимости среднего значения времени генерации по рангу и ранжирования вариантов дерева И/ИЛИ для L_n^m от параметров n и m .

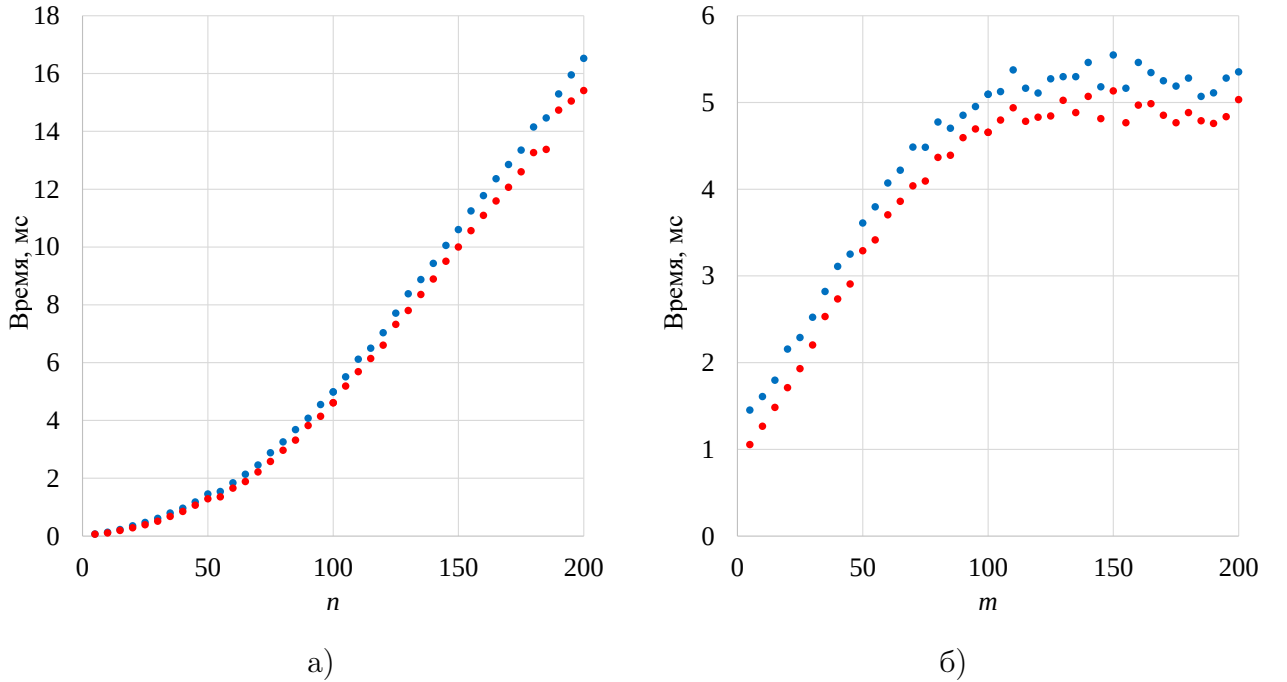


Рисунок 4.17 — Среднее время генерации по рангу (синий маркер) и ранжирования (красный маркер) вариантов дерева И/ИЛИ для L_n^m : а) зависимость от n при $m = 100$; б) зависимость от m при $n = 100$

Рисунок 4.17а подтверждает полиномиальную временную сложность $O(n^2)$ при $n < t$ и линейную временную сложность $O(n)$ при $n > t$ для фиксированного значения $t = 100$. Рисунок 4.17б подтверждает линейную временную сложность $O(t)$ для $t < n$ и постоянную временную сложность $O(1)$ для $t > n$ для фиксированного значения $n = 100$. При других фиксированных значениях t (или n) общий вид зависимости от n (или t) не меняется.

Для частного случая максимального значения ранга $r := C_n - 1$ алгоритмы 4.3 и 4.4 имеют полиномиальную временную сложность $O(n^3)$. На рисунке 4.18 показано среднее время генерации по рангу и ранжирования вариантов дерева И/ИЛИ для C_n , где n находится в диапазоне от 1 до 50 с шагом 1. Данный рисунок подтверждает полиномиальную временную сложность $O(n^3)$.

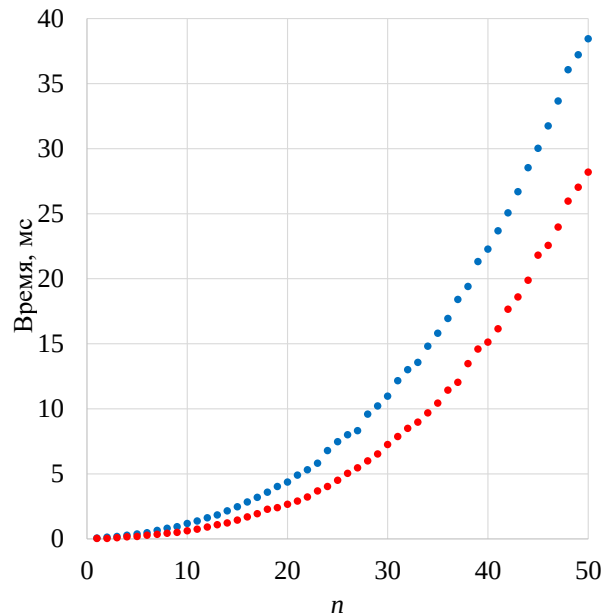
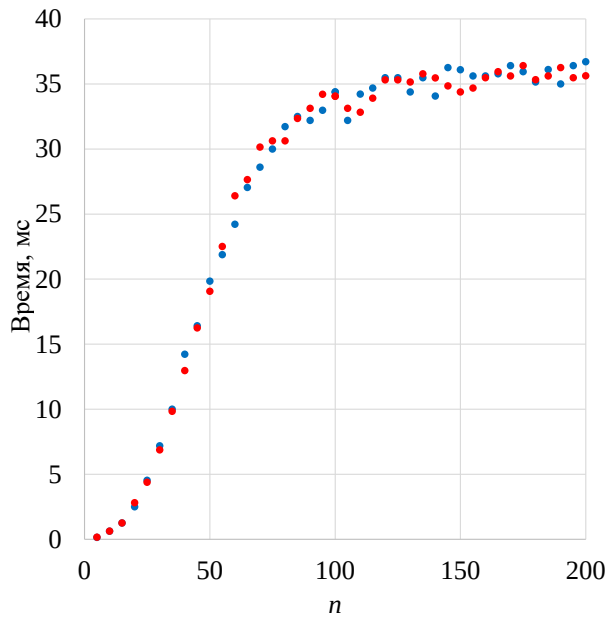
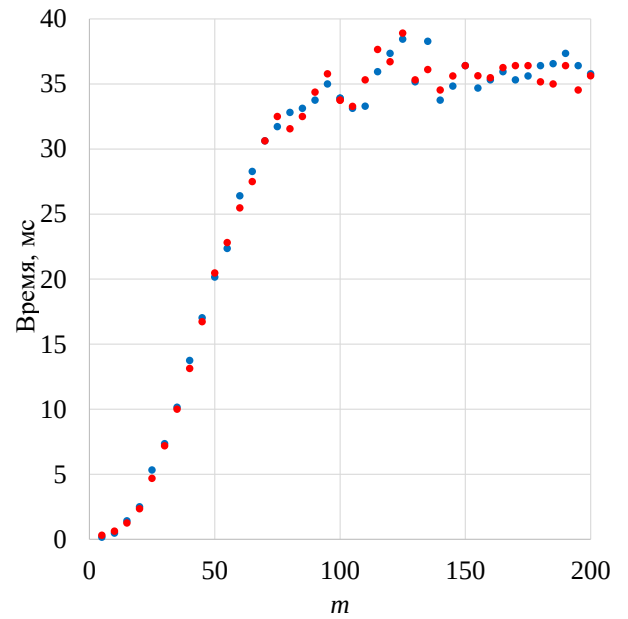


Рисунок 4.18 — Среднее время генерации по рангу (синий маркер) и ранжирования (красный маркер) вариантов дерева И/ИЛИ для C_n

Для частного случая максимального значения ранга $r := D_n^m - 1$ алгоритмы 4.5 и 4.6 имеют полиномиальную временную сложность $O(m^3)$ при $2m < n$, $O(m^2(n - m))$ при $m < n < 2m$, $O(n^2(m - n))$ при $n < m < 2n$ и $O(n^3)$ при $2n < m$. Разнообразие оценок временной сложности связано с особенностями вычисления биномиальных коэффициентов при использовании формулы (4.5). На рисунке 4.19 показаны результаты измерения зависимости среднего значения времени генерации по рангу и ранжирования вариантов дерева И/ИЛИ для D_n^m от параметров n и m . Форма наблюдаемой зависимости соответствует теоретическим оценкам временной сложности разработанных алгоритмов.



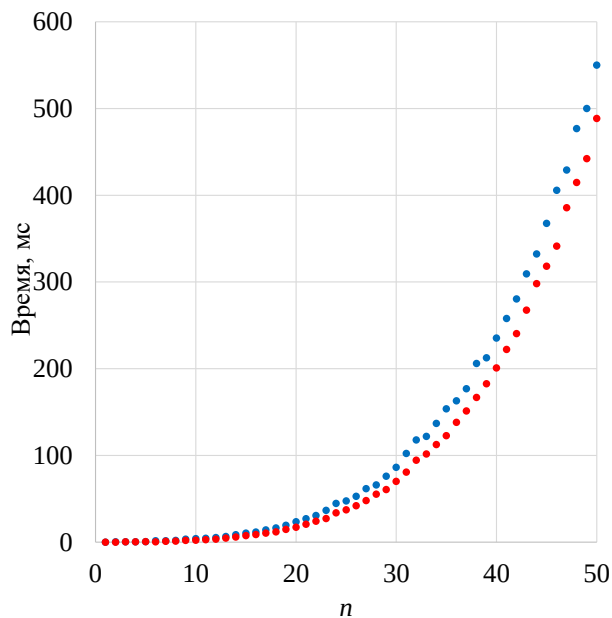
а)



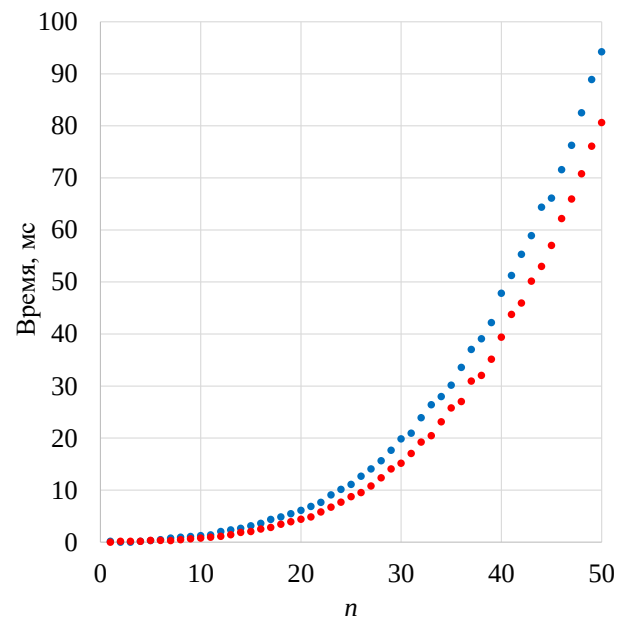
б)

Рисунок 4.19 — Среднее время генерации по рангу (синий маркер) и ранжирования (красный маркер) вариантов дерева И/ИЛИ для D_n^m :

а) зависимость от n при $m = 50$; б) зависимость от m при $n = 50$



а)



б)

Рисунок 4.20 — Среднее время генерации по рангу (синий маркер) и ранжирования (красный маркер) вариантов дерева И/ИЛИ:

а) для S_n ; б) для M_n

Для частного случая максимального значения ранга $r := S_n - 1$ алгоритмы 4.7 и 4.8 имеют полиномиальную временную сложность $O(n^4)$. На рисунке 4.20а показано среднее время генерации по рангу и ранжирования вариантов дерева И/ИЛИ для S_n , где n находится в диапазоне от 1 до 50 с шагом 1. Данный рисунок подтверждает полиномиальную временную сложность $O(n^4)$.

Для частного случая максимального значения ранга $r := M_n - 1$ алгоритмы 4.9 и 4.10 имеют полиномиальную временную сложность $O(n^4)$. На рисунке 4.20б показано среднее время генерации по рангу и ранжирования вариантов дерева И/ИЛИ для M_n , где n находится в диапазоне от 1 до 50 с шагом 1. Данный рисунок подтверждает полиномиальную временную сложность $O(n^4)$.

Таким образом, разработанные алгоритмы ранжирования и генерации по рангу для частных случаев решеточных путей имеют полиномиальную временную сложность, которая определяется максимальным количеством рекурсивных вызовов (высотой дерева И/ИЛИ), максимальным количеством потомков ИЛИ-узлов (шириной дерева И/ИЛИ), а также сложностью вычисления значения функции мощности комбинаторного множества. В предположении модели вычислений с использованием равномерной функции стоимости [191], вычислительные эксперименты подтвердили теоретические оценки временной сложности разработанных алгоритмов. В рамках данной модели вычислений принимается за постоянную величину стоимость каждой машинной операции вне зависимости от размера обрабатываемых чисел, то есть временная сложность арифметических операций с числами равна $O(1)$. Такой подход широко распространен в научных работах, связанных с разработкой алгоритмов комбинаторной генерации, и позволяет сравнить вычислительную сложность алгоритмов без привязки к архитектуре используемой вычислительной системы.

Для уменьшения временной сложности алгоритмов ранжирования и генерации по рангу можно добавить шаг предварительной обработки данных, на котором вычисляются и сохраняются все необходимые значения функции мощности. В таком случае вычислительная сложность алгоритмов будет определяться лишь размером структуры дерева И/ИЛИ, но потребуются дополнительные ресурсы памяти компьютера. Кроме того, если структура дерева И/ИЛИ содержит ИЛИ-узлы, в которых количество потомков зависит от параметров дерева И/ИЛИ, то вычислительная сложность алгоритмов может быть уменьшена путем реализации приближенных вычислений при выборе конкретного узла-потомка [86].

4.3 Алгоритмы кодирования для направленных решеточных путей

Алгоритмы комбинаторной генерации, представленные в параграфе 4.2, носят частный характер, так как решают задачи кодирования конкретных примеров решеточных путей (северо-восточные решеточные пути, а также пути Дика, Деланнуа, Шредера и Моцкина). Кроме того, разработка таких алгоритмов напрямую связана с наличием функции мощности, удовлетворяющей требованиям предложенной методологии. В то же время, любой решеточный путь можно представить в виде последовательности выполненных шагов $P = (\mathbf{ps}_1, \mathbf{ps}_2, \dots, \mathbf{ps}_l)$. Применяя идею метода поиска с возвратом для построения всех возможных решеточных путей при заданном множестве допустимых шагов $S = \{\mathbf{s}_1, \dots, \mathbf{s}_k\}$, а также заданных начальной и конечной точках, дополнительно решается задача перечисления формируемого комбинаторного множества. В свою очередь, это позволяет построить алгоритмы ранжирования и генерации по рангу для общего случая множеств решеточных путей [213].

В качестве примера рассмотрим широкий класс решеточных путей, называемых направленными решеточными путями [214].

Определение 4.7. *Направленный решеточный путь — это решеточный путь на плоскости, в котором для каждого шага $\mathbf{s}_i = (a_i; b_i)$ из множества допустимых шагов S выполняется $a_i > 0$.*

Для направленных решеточных путей существуют исследования, посвященные решению задач перечислительной комбинаторики. Например, в работе [214] получены производящие функции для перечисления направленных решеточных путей на основе применения ядерного метода (kernel method), предложенного Р. Flajolet [215]. В работе [216] проведен асимптотический анализ производящих функций, связанных с направленными решеточными путями. Используя модификацию ядерного метода, в работе [217] представлены результаты по перечислению и асимптотическому анализу для направленных решеточных путей, избегающих заданных шаблонов. Кроме того, некоторые ненаправленные решеточные пути могут быть преобразованы в направленные за счет изменения системы координат [218]. В свою очередь, это позволяет применять к ним соответствующие известные методы.

4.3.1 Простые направленные решеточные пути

Начнем с частного случая простых направленных решеточных путей.

Определение 4.8. *Простой направленный решеточный путь — это решеточный путь на плоскости, в котором для каждого шага $\mathbf{s}_i = (a_i; b_i)$ из множества допустимых шагов S выполняется $a_i = 1$.*

Если простой направленный решеточный путь начинается в точке $P_0 = (0; 0)$ и состоит из n шагов вида $\mathbf{s}_i = (1; b_i)$, то он заканчивается в точке $P_n = (n; m)$.

Для разработки алгоритмов ранжирования и генерации по рангу с использованием предложенной методологии необходимо иметь формулу для вычисления мощности множества простых направленных решеточных путей. Например, в работе [219] получены явные формулы для решеточных путей с возможными шагами вида $(1; b_i)$, где $b_i \in \{-h, \dots, -1, +1, \dots, +h\}$. Однако в рамках диссертационного исследования предлагается решение данной задачи в общем виде, то есть для произвольного множества допустимых шагов.

Теорема 4.6. *Количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$ и состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, вычисляется с помощью следующей рекуррентной формулы:*

$$W_n^m = \begin{cases} 0, & n < 0 \text{ или } n = 0 \text{ и } m \neq 0; \\ 1, & n = m = 0; \\ \sum_{i=1}^k W_{n-1}^{m-b_i}, & \text{иначе.} \end{cases} \quad (4.11)$$

Доказательство. Обозначим через W_n^m количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$ при заданном множестве допустимых шагов S . Рассмотрим конечную точку $(n; m)$ направленного решеточного пути. На основе заданного множества допустимых шагов $S = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k\} = \{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, конечная точка $(n; m)$ может быть достигнута из следующих k точек:

1. Из точки $(n-1; m-b_1)$, выполнив шаг $\mathbf{s}_1 = (1; b_1)$;
2. Из точки $(n-1; m-b_2)$, выполнив шаг $\mathbf{s}_2 = (1; b_2)$;
- $\vdots$
- k . Из точки $(n-1; m-b_k)$, выполнив шаг $\mathbf{s}_k = (1; b_k)$.

На рисунке 4.21 показано множество допустимых шагов $S = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k\}$ и все варианты достижения точки $(n; m)$, используя один шаг из S .

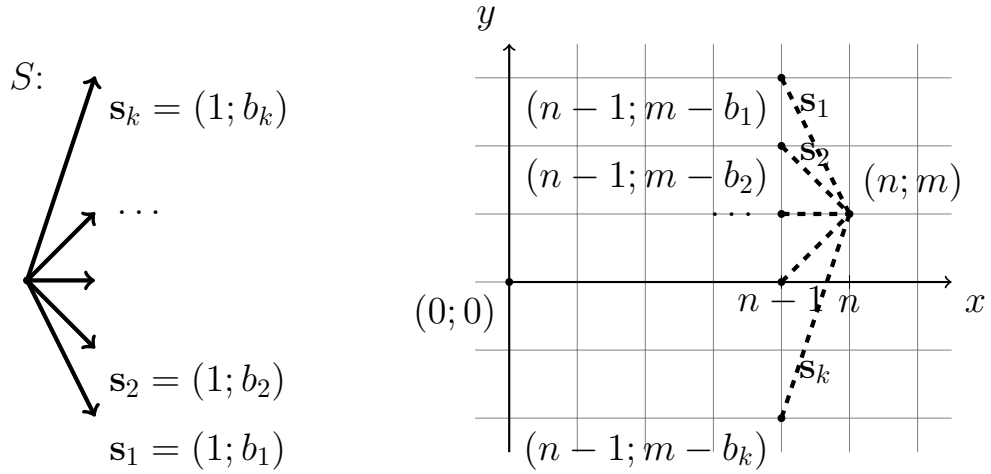


Рисунок 4.21 — Все варианты достижения точки $(n; m)$ за один шаг из $S = \{\mathbf{s}_1, \mathbf{s}_2, \dots, \mathbf{s}_k\}$

Согласно принятым обозначениям, количество всех простых направленных решеточных путей, начинающихся в $(0; 0)$, заканчивающихся в $(n-1; m-b_i)$ и состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, равно $W_{n-1}^{m-b_i}$. Из точки $(n-1; m-b_i)$ в точку $(n; m)$ можно попасть только одним способом: для этого необходимо выполнить шаг $\mathbf{s}_i = (1; b_i)$. Следовательно, количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$ и проходящих через точку $(n-1; m-b_i)$, тоже равно $W_{n-1}^{m-b_i}$. Чтобы вычислить количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, необходимо просуммировать количество всех простых направленных решеточных путей из точки $(0; 0)$ в каждую точку $(n-1; m-b_i)$, где $i \in \{1, \dots, k\}$. Таким образом, получаем искомую формулу (4.11).

Определим начальные условия для полученной рекуррентной формулы:

1. Поскольку простой направленный решеточный путь начинается в точке $(0; 0)$ и выполняется $a_i = 1$ для каждого шага $\mathbf{s}_i = (a_i, b_i)$, то есть координата по оси x всегда увеличивается, поэтому невозможно достичь точки $(n; m)$, где $n < 0$. Следовательно, получаем начальное условие $W_n^m = 0$ для $n < 0$.

2. Поскольку простой направленный решеточный путь начинается в точке $(0; 0)$ и выполняется $a_i = 1$ для каждого шага $\mathbf{s}_i = (a_i, b_i)$, поэтому невозможно достичь точки $(0; m)$, где $m \neq 0$. Следовательно, получаем начальное условие $W_0^m = 0$ для $m \neq 0$;

3. Значение W_0^0 показывает количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(0; 0)$. Существует только один такой решеточный путь: пустой решеточный путь, который не содержит выполненных шагов. Следовательно, получаем начальное условие $W_0^0 = 1$.

На рисунке 4.22 показано множество допустимых шагов $S = \{s_1, s_2, \dots, s_k\}$, а также все возможные и невозможные шаги для простого направленного решеточного пути, который начинается в точке $(0; 0)$ и заканчивается в точке $(1; m)$.

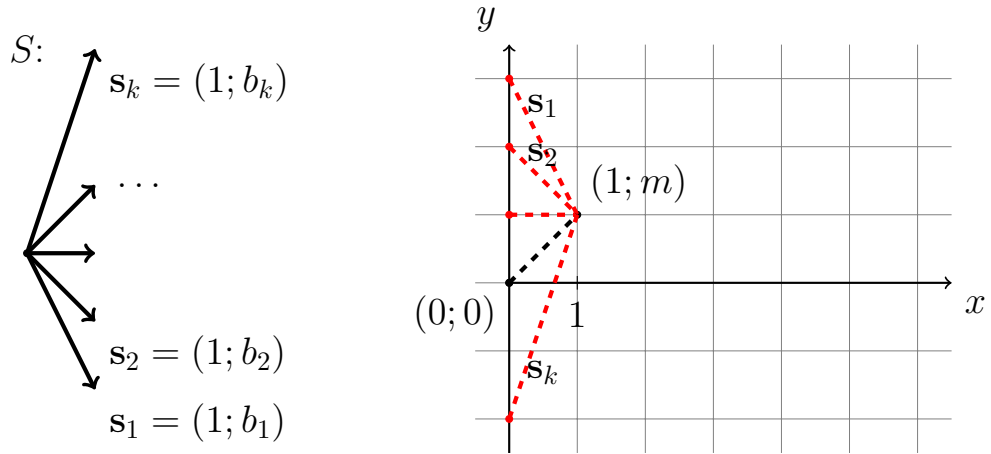


Рисунок 4.22 — Все возможные (черная линия) и невозможные (красная линия) шаги для простого направленного решеточного пути, который начинается в точке $(0; 0)$ и заканчивается в точке $(1; m)$

Следовательно, полученную рекуррентную формулу (4.11) с заданными начальными условиями можно применить для вычисления количества всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$ и состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$. $\square$

Эффективность вычисления W_n^m можно улучшить, добавив следующее условие, которое уменьшает количество рекурсивных вызовов:

$$W_n^m = 0 \text{ для } m > n \cdot \max_i b_i \text{ или } m < n \cdot \min_i b_i. \quad (4.12)$$

Кроме того, можно использовать следующее условие, которое также сокращает количество рекурсивных вызовов при вычислении W_n^m :

$$W_n^m = 1 \text{ для } m = n \cdot \max_i b_i \text{ или } m = n \cdot \min_i b_i. \quad (4.13)$$

На рисунке 4.23 показан пример множества шагов $S = \{(1; -1), (1; 0), (1; 1)\}$ и все рекурсивные вызовы при применении формулы (4.11) для вычисления

W_3^1 без (рисунок 4.23а) и с (рисунок 4.23б) использованием начальных условий (4.12) и (4.13).

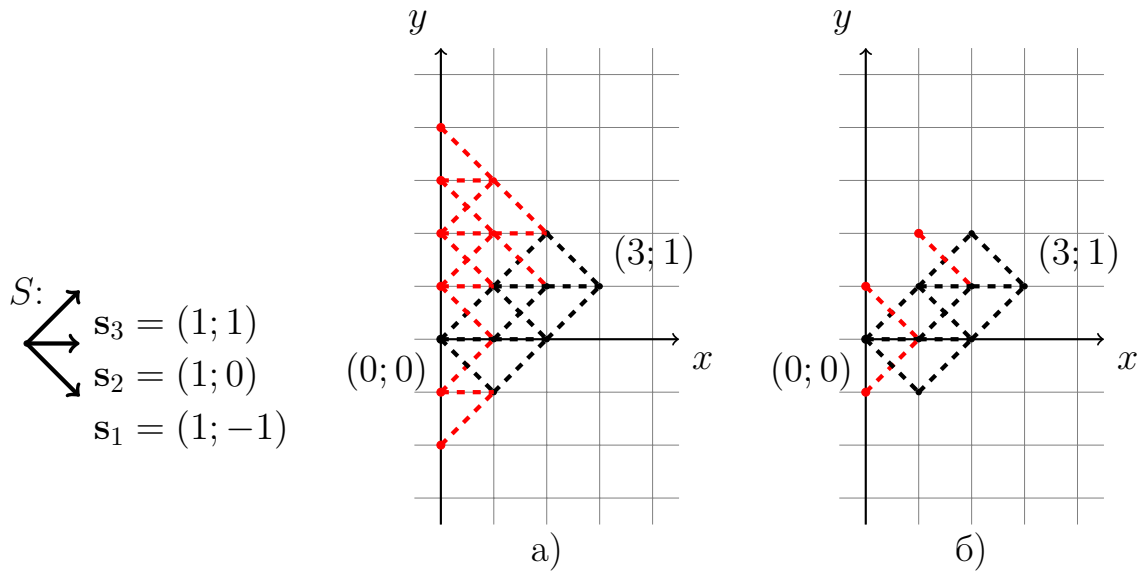


Рисунок 4.23 — Применение формулы (4.11) для вычисления W_3^1 :
а) без использования (4.12) и (4.13); б) с использованием (4.12) и (4.13)

Поскольку полученная рекуррентная формула (4.11) с использованием начальных условий (4.12) и (4.13) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для W_n^m (рисунок 4.24).

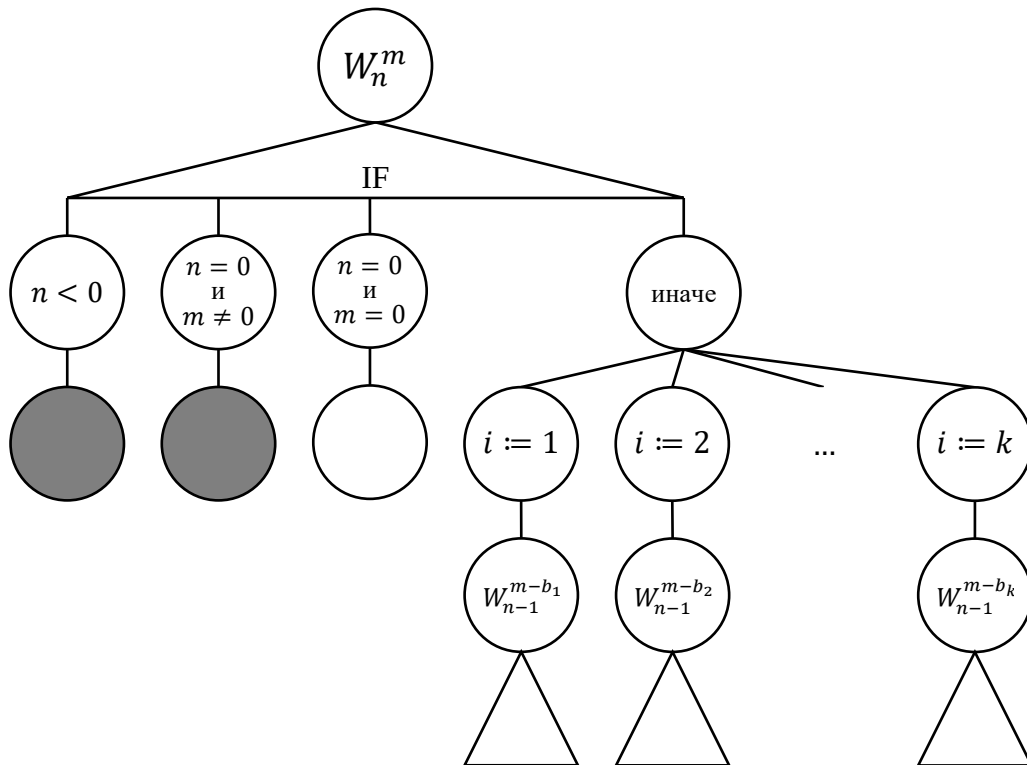


Рисунок 4.24 — Структура дерева И/ИЛИ для W_n^m на основе формулы (4.11)

Кроме того, данная структура дерева И/ИЛИ для W_n^m может быть дополнена условиями (4.12) и (4.13). Также важно отметить, что необходимо зафиксировать некоторый порядок на множестве допустимых шагов S , поскольку дерево И/ИЛИ является упорядоченной структурой.

Для компактности представления воспользуемся следующими допущениями:

- каждый узел с меткой W_0^0 является листом в дереве И/ИЛИ для W_n^m ;
- каждый узел с меткой W_n^m , где $m = n \cdot \max_i b_i$ или $m = n \cdot \min_i b_i$, является листом в дереве И/ИЛИ для W_n^m .

На рисунке 4.25 представлен пример структуры дерева И/ИЛИ для W_3^1 при $S = \{(1; -1), (1; 0), (1; 1)\}$. Общее количество его вариантов равно $W_3^1 = 6$.

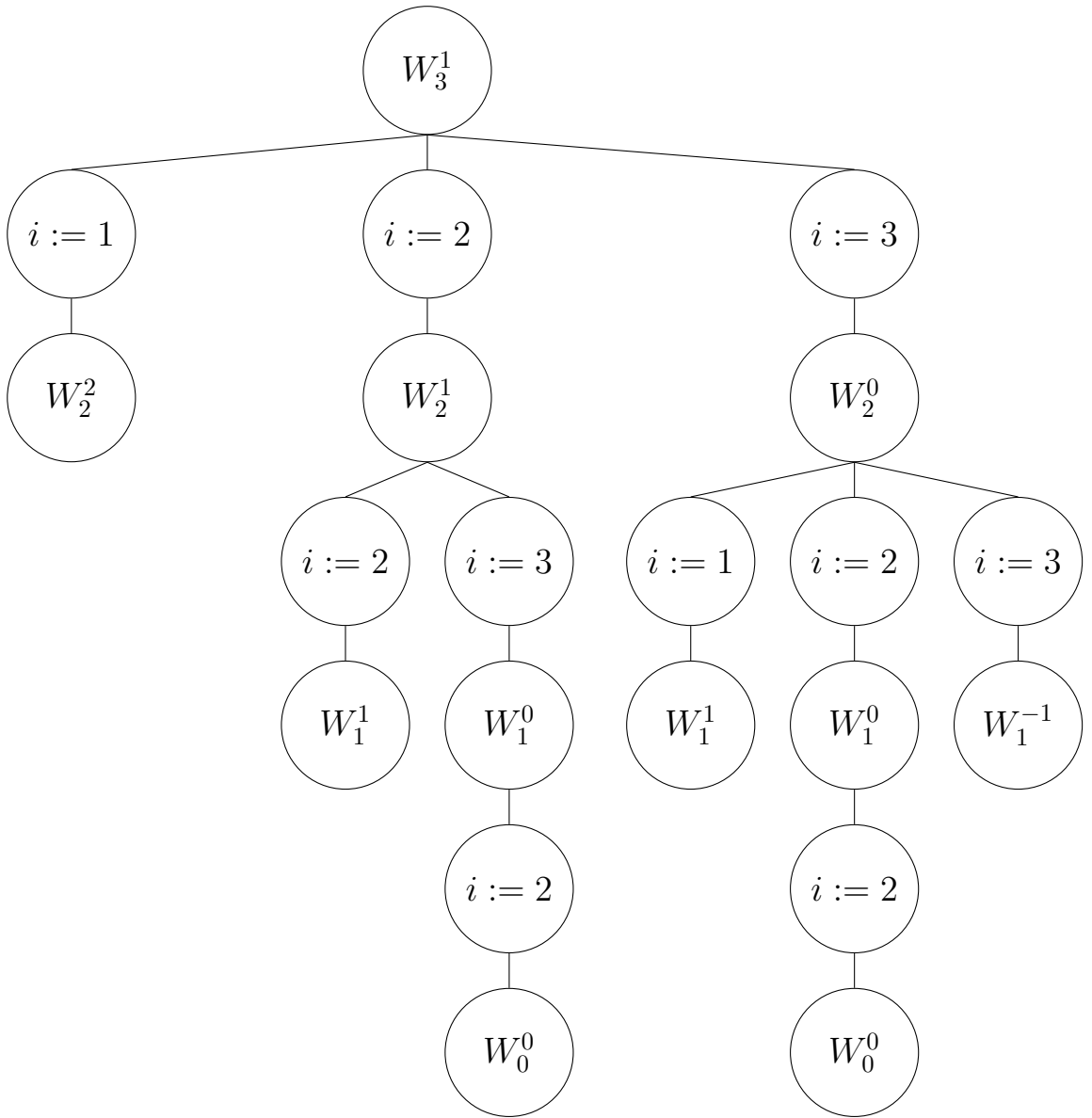


Рисунок 4.25 — Структура дерева И/ИЛИ для W_3^1 при $S = \{(1; -1), (1; 0), (1; 1)\}$

Поскольку полученная структура дерева И/ИЛИ не содержит И-узлов, то каждый вариант такого дерева представляет собой путь от корня к листу.

Теорема 4.7. *Существует биективное отображение между множеством простых направленных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, и множеством вариантов дерева И/ИЛИ для W_n^m , представленного на рисунке 4.24.*

Доказательство. Общее количество простых направленных решеточных путей, начинающихся в точке $(0; 0)$ и заканчивающихся в точке $(n; m)$, равно W_n^m . Согласно правилам построения деревьев И/ИЛИ и их вариантов, общее количество вариантов дерева И/ИЛИ для W_n^m , представленного на рисунке 4.24, равно значению W_n^m . Равенство значений мощности для двух комбинаторных множеств является необходимым условием для наличия биективного отображения между ними.

Основываясь на комбинаторном доказательстве формулы (4.11), биективное отображение между множеством вариантов дерева И/ИЛИ для W_n^m и множеством простых направленных решеточных путей из точки $(0; 0)$ в точку $(n; m)$ определяется по следующим правилам:

- выбор потомка ИЛИ-узла с меткой W_n^m (каждый такой узел-потомок помечен определенным значением параметра i) соответствует выбору шага $\mathbf{s}_i$ при достижении точки $(n; m)$, то есть $\mathbf{ps}_n = \mathbf{s}_i$ в искомом решеточном пути $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n)$;
- поддереву узла с меткой $W_{n-1}^{m-b_i}$ определяет оставшуюся часть простого направленного решеточного пути из точки $(0; 0)$ в точку $(n-1; m-b_i)$;
- лист с меткой W_n^m , где $m = n \cdot \max_i b_i$, означает достижение точки $(n; m)$ из точки $(0; 0)$ путем выполнения n шагов вида $(1; \max_i b_i)$, то есть $\mathbf{ps}_1 = \dots = \mathbf{ps}_n = (1; \max_i b_i)$;
- лист с меткой W_n^m , где $m = n \cdot \min_i b_i$, означает достижение точки $(n; m)$ из точки $(0; 0)$ путем выполнения n шагов вида $(1; \min_i b_i)$, то есть $\mathbf{ps}_1 = \dots = \mathbf{ps}_n = (1; \min_i b_i)$;
- лист с меткой W_0^0 определяет пустой решеточный путь $()$.

Таким образом, множеству допустимых шагов, выполнение которых приводит к достижению точки $(n; m)$ из точек $(n-1; m-b_i)$, сопоставляется множество потомков ИЛИ-узла с меткой W_n^m . В результате из заданного варианта v дерева И/ИЛИ для W_n^m рекурсивно формируется простой направленный решеточный путь вида $(\mathbf{ps}_1, \dots, \mathbf{ps}_n)$. Используя обратные преобразования из заданного решеточного пути формируется соответствующий вариант дерева И/ИЛИ. $\square$

Так как построенная структура дерева И/ИЛИ не содержит И-узлов, а выбор между потомками ИЛИ-узлов соответствует выбору допустимых шагов при построении решеточных путей, то эта структура аналогична генерирующему дереву, получаемому путем применения метода поиска с возвратом. Обход этого дерева позволяет осуществить последовательную генерацию всех рассматриваемых простых направленных решеточных путей. При этом начальные условия выполняют роль ограничивающих функций, которые заблаговременно отсекают ветви с невозможными шагами (узлы с меткой $W_n^m = 0$). Однако, в отличие от метода поиска с возвратом, методология на основе рекурсивных композиций структур деревьев И/ИЛИ позволяет построить другие классы алгоритмов комбинаторной генерации для множества простых направленных решеточных путей.

Используя предложенную методологию, разработаны алгоритмы ранжирования (алгоритм 4.11) и генерации по рангу (алгоритм 4.12) для простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$ и состоящих из шагов множества $S = \{(1; b_1), (1; b_2), \dots, (1; b_k)\}$. В этих алгоритмах $()$ обозначает пустой решеточный путь, а функция $\text{concat}(a, b)$ обозначает конкатенацию решеточных путей. В таблице 4.6 представлен пример ранжирования комбинаторного множества всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(3; 1)$ и состоящих из шагов множества $S = \{s_1, s_2, s_3\} = \{(1; -1), (1; 0), (1; 1)\}$.

Алгоритм 4.11: Алгоритм ранжирования решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(1; b_1), (1; b_2), \dots, (1; b_k)\}$

```

1 Rank_SDLP ( $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n), n, m$ )
2 begin
3   if  $(n = 0 \text{ и } m = 0)$  или  $m = n \cdot \max_i b_i$  или  $m = n \cdot \min_i b_i$  then  $r := 0$ 
4   else
5      $I :=$  порядковый номер  $\mathbf{ps}_n$  в упорядоченном множестве  $S$ 
6      $sum := \sum_{i=1}^{I-1} W_{n-1}^{m-b_i}$ 
7      $r := sum + \text{Rank\_SDLP}((\mathbf{ps}_1, \dots, \mathbf{ps}_{n-1}), n-1, m-b_I)$ 
8   end
9   return  $r$ 
10 end
```

Алгоритм 4.12: Алгоритм генерации по рангу решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(1; b_1), (1; b_2), \dots, (1; b_k)\}$

```

1 Unrank_SDLP( $r, n, m$ )
2 begin
3   if  $n = 0$  u  $m = 0$  then  $P := ()$ 
4   else if  $m = n \cdot \max_i b_i$  then
5      $P := (\mathbf{ps}_1, \dots, \mathbf{ps}_n) = ((1; \max_i b_i), \dots, (1; \max_i b_i))$ 
6   else if  $m = n \cdot \min_i b_i$  then
7      $P := (\mathbf{ps}_1, \dots, \mathbf{ps}_n) = ((1; \min_i b_i), \dots, (1; \min_i b_i))$ 
8   else
9      $sum := 0$ 
10    for  $i := 1$  to  $k$  do
11       $s := W_{n-1}^{m-b_i}$ 
12      if  $sum + s > r$  then
13         $r := r - sum$ 
14         $I := i$ 
15        break
16      end
17       $sum := sum + s$ 
18    end
19     $P := \text{concat}(\text{Unrank\_SDLP}(r, n-1, m-b_I), (s_I))$ 
20 end
```

Таблица 4.6 — Ранжирование множества решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(3; 1)$ и состоящих из шагов множества $S = \{\mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3\} = \{(1; -1), (1; 0), (1; 1)\}$

Решеточный путь	Ранг
$(\mathbf{s}_3, \mathbf{s}_3, \mathbf{s}_1) = ((1; 1), (1; 1), (1; -1))$	0
$(\mathbf{s}_3, \mathbf{s}_2, \mathbf{s}_2) = ((1; 1), (1; 0), (1; 0))$	1
$(\mathbf{s}_2, \mathbf{s}_3, \mathbf{s}_2) = ((1; 0), (1; 1), (1; 0))$	2
$(\mathbf{s}_3, \mathbf{s}_1, \mathbf{s}_3) = ((1; 1), (1; -1), (1; 1))$	3
$(\mathbf{s}_2, \mathbf{s}_2, \mathbf{s}_3) = ((1; 0), (1; 0), (1; 1))$	4
$(\mathbf{s}_1, \mathbf{s}_3, \mathbf{s}_3) = ((1; -1), (1; 1), (1; 1))$	5

Алгоритмы 4.11 и 4.12 имеют не более n рекурсивных вызовов, каждый из которых требует вычисления W_n^m максимум k раз. Следовательно, алгоритмы 4.11 и 4.12 имеют временную сложность $O(n \cdot k \cdot w(n, m, S))$, где $w(n, m, S)$ — это функция, описывающая сложность вычисления значения W_n^m в зависимости от параметров n и m , а также множества допустимых шагов S . Если для вычисления W_n^m использовать рекуррентную формулу (4.11), которая имеет временную сложность $O(k^n)$, то получаем экспоненциальную временную сложность $O(nk^n)$ для алгоритмов ранжирования и генерации по рангу. Чтобы повысить эффективность получаемых алгоритмов комбинаторной генерации, нужно использовать явные формулы для вычисления W_n^m , которые показывают лучшую оценку вычислительной сложности.

Таким образом, используя разработанные в общем виде алгоритмы ранжирования и генерации по рангу, можно кодировать любые простые направленные решеточные пути с произвольным набором допустимых шагов вида $\mathbf{s}_i = (1; b_i)$. Более того, некоторые ненаправленные решеточные пути могут быть преобразованы в форму простых направленных решеточных путей. Например, рассмотрим северо-восточные решеточные пути, которые состоят из шагов множества $S = \{\mathbf{s}_1, \mathbf{s}_2\} = \{(0; 1), (1; 0)\}$. Если изменить базис и использовать другую систему координат (красные оси x^* и y^* на рисунке 4.26), то северо-восточные решеточные пути преобразуются в простые направленные решеточные пути.

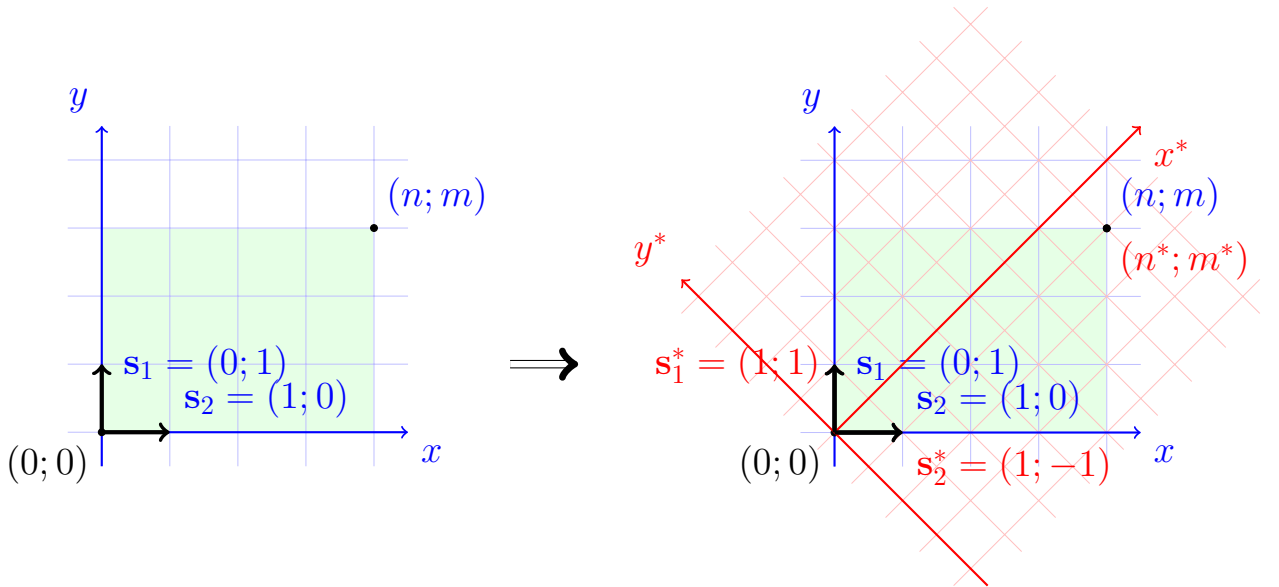


Рисунок 4.26 — Преобразование северо-восточных решеточных путей в простые направленные решеточные пути

Полученные решеточные пути начинаются в точке $(0; 0)$, заканчиваются в точке $(n^*; m^*)$ и состоят из шагов множества $S^* = \{\mathbf{s}_1^*, \mathbf{s}_2^*\} = \{(1; 1), (1; -1)\}$ в системе координат x^* и y^* . Количество таких решеточных путей определяется значением $W_{n^*}^{m^*}$. Следовательно, получаем связь с северо-восточными решеточными путями в исходной системе координат x и y через

$$L_n^m = W_{n^*}^{m^*} = W_{m+n}^{m-n}. \quad (4.14)$$

Применяя алгоритмы 4.11 и 4.12 для рассматриваемых простых направленных решеточных путей, получаем алгоритмы ранжирования (алгоритм 4.13) и генерации по рангу (алгоритм 4.14) для множества северо-восточных решеточных путей. Таким образом, чтобы выполнить кодирование северо-восточного решеточного пути $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n)$, который начинается в точке $(0; 0)$, заканчивается в точке $(n; m)$ и каждый выполненный шаг $\mathbf{ps}_i \in S = \{(0; 1), (1; 0)\}$, необходимо изменить базис и получить соответствующий простой направленный решеточный путь $P^* = (\mathbf{ps}_1^*, \dots, \mathbf{ps}_n^*)$, где $\mathbf{ps}_i^* \in S^* = \{(1; 1), (1; -1)\}$. Затем требуется запустить алгоритм 4.13 со следующими параметрами:

$$\text{Rank_SDLP_NE} (P^* = (\mathbf{ps}_1^*, \dots, \mathbf{ps}_n^*), n^* = m + n, m^* = m - n).$$

Алгоритм 4.13: Алгоритм ранжирования решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(1; 1), (1; -1)\}$

```

1 Rank_SDLP_L ( $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n)$ ,  $n$ ,  $m$ )
2 begin
3   if  $m = n$  или  $m = -n$  then  $r := 0$ 
4   else
5     if  $\mathbf{ps}_n = (1; 1)$  then
6        $r := \text{Rank\_SDLP\_L}((\mathbf{ps}_1, \dots, \mathbf{ps}_{n-1}), n-1, m-1)$ 
7     else  $r := W_{n-1}^{m-1} + \text{Rank\_SDLP\_L}((\mathbf{ps}_1, \dots, \mathbf{ps}_{n-1}), n-1, m+1)$ 
8   end
9   return  $r$ 
10 end
```

Алгоритм 4.14: Алгоритм генерации по рангу решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(1; 1), (1; -1)\}$

```

1 Unrank_SDLP_L( $r, n, m$ )
2 begin
3   if  $n = 0$  и  $m = 0$  then  $P := ()$ 
4   else if  $m = n$  then  $P := (\mathbf{ps}_1, \dots, \mathbf{ps}_n) = ((1; 1), \dots, (1; 1))$ 
5   else if  $m = -n$  then  $P := (\mathbf{ps}_1, \dots, \mathbf{ps}_n) = ((1; -1), \dots, (1; -1))$ 
6   else
7     if  $r < W_{n-1}^{m-1}$  then
8        $P := \text{concat}(\text{Unrank\_SDLP\_L}(r, n-1, m-1), ((1; 1)))$ 
9     else
10       $P := \text{concat}(\text{Unrank\_SDLP\_L}(r - W_{n-1}^{m-1}, n-1, m+1), ((1; -1)))$ 
11   end
12   return  $P$ 
13 end
```

Согласно теореме 4.6, а также начальным условиям (4.12) и (4.13), для случая $S = \{(1; 1), (1; -1)\}$ получаем

$$W_n^m = \begin{cases} 0, & m > n \text{ или } m < -n; \\ 1, & m = n \text{ или } m = -n; \\ W_{n-1}^{m-1} + W_{n-1}^{m+1}, & \text{иначе.} \end{cases} \quad (4.15)$$

Если использовать рекуррентную формулу (4.15) при выполнении алгоритмов 4.13 и 4.14, то получим экспоненциальную временную сложность $O(n \cdot 2^n)$. Значительное улучшение будет получено, если использовать явную формулу (4.1) и связь W_n^m с L_n^m через формулу (4.14):

$$W_n^m = \begin{cases} 0, & (n+m) \text{ нечетное;} \\ L_{\frac{n-m}{2}}^{\frac{n+m}{2}}, & \text{иначе} \end{cases} = \begin{cases} 0, & (n+m) \text{ нечетное;} \\ \binom{n}{\frac{n+m}{2}}, & \text{иначе.} \end{cases} \quad (4.16)$$

Тогда для алгоритмов 4.13 и 4.14 получаем полиномиальную временную сложность $O(n^2)$. Результаты вычислительных экспериментов с использованием программных реализаций данных алгоритмов подтверждают теоретическую оценку временной сложности.

4.3.2 Простые направленные решеточные пути с ограничениями

К направленным решеточным путям могут применяться различные ограничения, такие как фиксация конечной точки решеточного пути, запрет выхода за определенные границы, вычисление некоторой статистики решеточного пути и так далее. Например, в работе [214] выделяются следующие четыре типа направленных решеточных путей:

1. Направленный решеточный путь типа «Walk»: направленный решеточный путь, который заканчивается в точке $(n; m)$, где m не фиксируется;
2. Направленный решеточный путь типа «Bridge»: направленный решеточный путь, который заканчивается в точке $(n; 0)$;
3. Направленный решеточный путь типа «Meander»: направленный решеточный путь, который заканчивается в точке $(n; m)$, где m не фиксируется, и никогда не опускается ниже оси x ;
4. Направленный решеточный путь типа «Excursion»: направленный решеточный путь, который заканчивается в точке $(n; 0)$ и никогда не опускается ниже оси x .

Так как решеточные пути типа «Walk» и «Bridge» строятся на основе решеточных путей из теоремы 4.6, то для решения задачи их перечисления используются значения W_n^m .

Следствие 4.1. *Количество всех простых направленных решеточных путей типа «Walk», начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$, где m не фиксируется, и состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, вычисляется с помощью следующей формулы:*

$$W_n = \sum_{m=n \cdot \min b_i}^{n \cdot \max b_i} W_n^m. \quad (4.17)$$

Доказательство. Простые направленные решеточные пути типа «Walk» отличаются от решеточных путей из теоремы 4.6 тем, что m не фиксируется. Поэтому нужно рассмотреть все возможные значения m и просуммировать количество простых направленных решеточных путей для каждого из них. Минимально возможное значение m будет получено, если в простой направленный решеточный путь состоит из n шагов вида $(1; b_{\min})$, где $b_{\min} = \min_i b_i$. Максимально возможное значение m будет получено, если состоит из n шагов вида $(1; b_{\max})$, где $b_{\max} = \max_i b_i$. Объединяя все условия, получаем искомую формулу (4.17). $\square$

Следствие 4.2. *Количество всех простых направленных решеточных путей типа «Bridge», начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; 0)$ и состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, вычисляется с помощью следующей формулы:*

$$B_n = W_n^0.$$

Для вычисления количества простых направленных решеточных путей типа «Meander» и «Excursion» необходимо не допустить их падения ниже оси x , то есть требуется добавить соответствующее начальное условие к рекуррентной формуле (4.11).

Теорема 4.8. *Количество всех простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$, состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, и никогда не опускающихся ниже оси x , вычисляется с помощью следующей рекуррентной формулы:*

$$M_n^m = \begin{cases} 0, & n < 0 \text{ или } m < 0 \text{ или } n = 0 \text{ и } m \neq 0; \\ 1, & n = m = 0; \\ \sum_{i=1}^k M_{n-1}^{m-b_i}, & \text{иначе.} \end{cases}$$

Доказательство. В отличие от теоремы 4.6, для рассматриваемых простых направленных решеточных путей невозможно достичь точки $(0; m)$, где $m < 0$. Поэтому получаем дополнительное начальное условие, что $M_n^m = 0$ для $m < 0$. $\square$

Поскольку рассматриваемые простые направленные решеточные пути никогда не опускаются ниже оси x , то для вычисления значения M_n^m можно добавить только следующие дополнительные начальные условия:

$$M_n^m = 0 \text{ для } m > n \cdot \max_i b_i, \quad (4.18)$$

$$M_n^m = 1 \text{ для } m = n \cdot \max_i b_i. \quad (4.19)$$

Чтобы получить алгоритмы ранжирования и генерации по рангу для простых направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$, состоящих из шагов множества $S = \{s_1, s_2, \dots, s_k\} = \{(1; b_1), (1; b_2), \dots, (1; b_k)\}$ и никогда не опускающихся ниже оси x , необходимо преобразовать общие алгоритмы 4.11 и 4.12: заменить W_n^m на M_n^m и использовать начальные условия (4.18) и (4.19) взамен начальных условий (4.12) и (4.13).

Алгоритм 4.15: Алгоритм ранжирования решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$, состоящего из шагов множества $S = \{(1; b_1), \dots, (1; b_k)\}$ и не опускающегося ниже оси x

```

1 Rank_SDLPx ( $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n)$ ,  $n$ ,  $m$ )
2 begin
3   if  $(n = 0 \text{ и } m = 0)$  или  $m = n \cdot \max_i b_i$  then  $r := 0$ 
4   else
5      $I :=$  порядковый номер  $\mathbf{ps}_n$  в упорядоченном множестве  $S$ 
6      $sum := \sum_{i=1}^{I-1} M_{n-1}^{m-b_i}$ 
7      $r := sum + \text{Rank\_SDLPx}((\mathbf{ps}_1, \mathbf{ps}_2, \dots, \mathbf{ps}_{n-1}), n-1, m-b_I)$ 
8   end
9   return  $r$ 
10 end
```

Алгоритм 4.16: Алгоритм генерации по рангу решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$, состоящего из шагов множества $S = \{(1; b_1), \dots, (1; b_k)\}$ и не опускающегося ниже оси x

```

1 Unrank_SDLPx ( $r$ ,  $n$ ,  $m$ )
2 begin
3   if  $n = 0 \text{ и } m = 0$  then  $P := ()$ 
4   else if  $m = n \cdot \max_i b_i$  then
5      $P := (\mathbf{ps}_1, \dots, \mathbf{ps}_n) = ((1; \max_i b_i), \dots, (1; \max_i b_i))$ 
6   else
7      $sum := 0$ 
8     for  $i := 1$  to  $k$  do
9        $s := M_{n-1}^{m-b_i}$ 
10      if  $sum + s > r$  then
11         $r := r - sum$ 
12         $I := i$ 
13        break
14      end
15       $sum := sum + s$ 
16    end
17     $P := \text{concat}(\text{Unrank\_SDLPx}(r, n-1, m-b_I), (\mathbf{s}_I))$ 
18  end
19 return  $P$ 
20 end
```

Следствие 4.3. Количество всех простых направленных решеточных путей типа «Meander», начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$, где m не фиксируется, состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, и никогда не опускающихся ниже оси x , вычисляется с помощью следующей формулы:

$$M_n = \sum_{m=0}^{n \cdot \max b_i} M_n^m.$$

Доказательство. Доказательство аналогично доказательству следствия 4.1. $\square$

Следствие 4.4. Количество всех простых направленных решеточных путей типа «Excursion», начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; 0)$, состоящих из шагов множества $\{(1; b_1), (1; b_2), \dots, (1; b_k)\}$, $b_i \in \mathbb{Z}$, и никогда не опускающихся ниже оси x , вычисляется с помощью следующей формулы:

$$E_n = M_n^0.$$

В качестве примера рассмотрим использование алгоритмов 4.15 и 4.16 для кодирования решеточных путей Дика, которые состоят из шагов множества $S = \{s_1, s_2\} = \{(0; 1), (1; 0)\}$ и никогда не поднимаются выше диагонали $y = x$. Если изменить базис и использовать другую систему координат (красные оси x^* и y^* на рисунке 4.27), то пути Дика преобразуются в простые направленные решеточные пути с ограничениями.

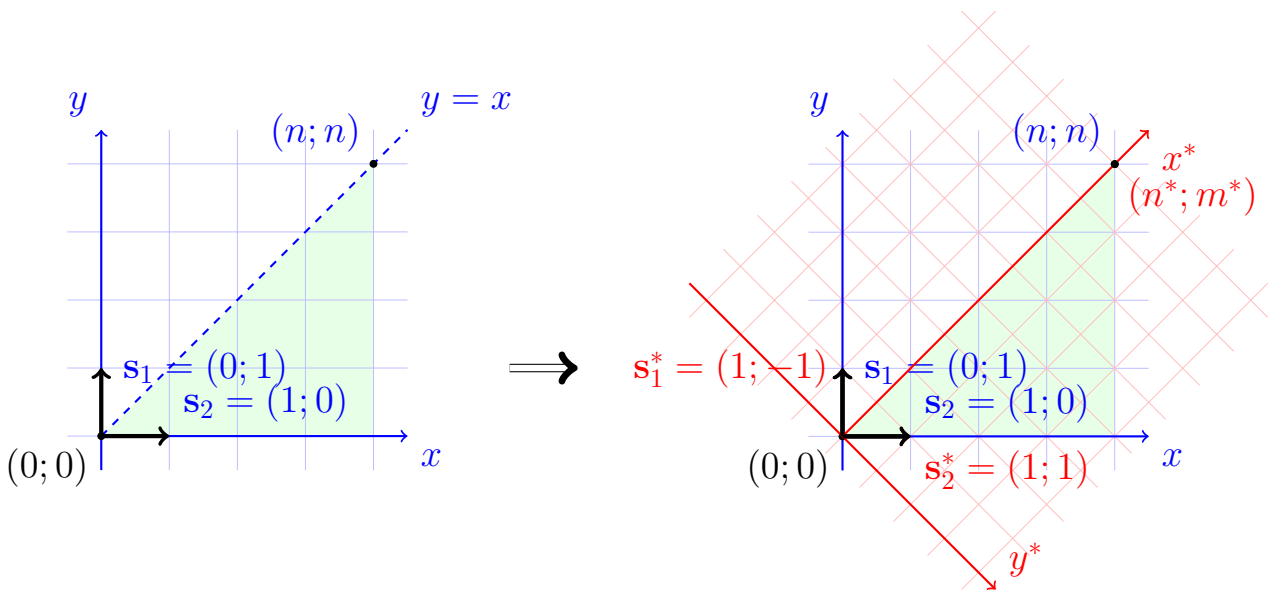


Рисунок 4.27 — Преобразование путей Дика в простые направленные решеточные пути

Полученные решеточные пути начинаются в точке $(0; 0)$, заканчиваются в точке $(n^*; m^*)$, состоят из шагов множества $S^* = \{\mathbf{s}_1^*, \mathbf{s}_2^*\} = \{(1; -1), (1; 1)\}$ и никогда не опускаются ниже оси x^* в новой системе координат. Количество таких решеточных путей определяется значением $M_{n^*}^{m^*}$. Следовательно, получаем связь с путями Дика в исходной системе координат x и y через

$$C_n = M_{n^*}^{m^*} = M_{2n}^0 = E_{2n}.$$

Применяя алгоритмы 4.15 и 4.16 для рассматриваемых простых направленных решеточных путей, получаем алгоритмы ранжирования и генерации по рангу для множества путей Дика. Таким образом, чтобы выполнить кодирование пути Дика $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_n)$, который начинается в точке $(0; 0)$, заканчивается в точке $(n; n)$ и каждый выполненный шаг $\mathbf{ps}_i \in S = \{(0; 1), (1; 0)\}$, необходимо изменить базис и получить соответствующий простой направленный решеточный путь $P^* = (\mathbf{ps}_1^*, \dots, \mathbf{ps}_n^*)$, где $\mathbf{ps}_i^* \in S^* = \{(1; -1), (1; 1)\}$. Затем требуется запустить алгоритм 4.15 со следующими параметрами;

$$\text{Rank_SDLPx} (P^* = (\mathbf{ps}_1^*, \mathbf{ps}_2^*, \dots, \mathbf{ps}_n^*), n^* = 2n, m^* = 0).$$

4.3.3 Направленные решеточные пути

Все полученные результаты по рекуррентным формулам и алгоритмам комбинаторной генерации для простых направленных решеточных путей могут быть обобщены на случай направленных решеточных путей. Аналогичные результаты по получению рекуррентных формул для направленных решеточных путей показаны в работе [220].

Теорема 4.9. *Количество всех направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$ и состоящих из шагов множества $\{(a_1; b_1), (a_2; b_2), \dots, (a_k; b_k)\}$, $a_i \in \mathbb{Z}_{>0}$, $b_i \in \mathbb{Z}$, вычисляется с помощью следующей рекуррентной формулы:*

$$\dot{W}_n^m = \begin{cases} 0, & n < 0 \text{ или } n = 0 \text{ и } m \neq 0; \\ 1, & n = m = 0; \\ \sum_{i=1}^k \dot{W}_{n-a_i}^{m-b_i}, & \text{иначе.} \end{cases} \quad (4.20)$$

Доказательство. Доказательство строится аналогично доказательству теоремы 4.6. В данном случае рассматривается достижение точки $(n; m)$ из k точек вида $(n - a_i; m - b_i)$. □

Теорема 4.10. *Количество всех направленных решеточных путей, начинающихся в точке $(0; 0)$, заканчивающихся в точке $(n; m)$, состоящих из шагов множества $\{(a_1; b_1), (a_2; b_2), \dots, (a_k; b_k)\}$, $a_i \in \mathbb{Z}_{>0}$, $b_i \in \mathbb{Z}$, и никогда не опускающихся ниже оси x , вычисляется с помощью следующей рекуррентной формулы:*

$$\dot{M}_n^m = \begin{cases} 0, & n < 0 \text{ или } m < 0 \text{ или } n = 0 \text{ и } m \neq 0; \\ 1, & n = m = 0; \\ \sum_{i=1}^k \dot{M}_{n-a_i}^{m-b_i}, & \text{иначе.} \end{cases} \quad (4.21)$$

Доказательство. Доказательство аналогично доказательству теоремы 4.8. $\square$

Рекуррентные формулы (4.20) и (4.21) также удовлетворяют требованиям предложенной методологии кодирования на основе рекурсивных композиций структур деревьев И/ИЛИ. Это позволяет строить алгоритмы комбинаторной генерации для более широкого класса дискретных структур — направленных решеточных путей. Таким образом, используя разработанные в общем виде алгоритмы ранжирования (алгоритм 4.17) и генерации по рангу (алгоритм 4.18), можно кодировать любые направленные решеточные пути с произвольным набором допустимых шагов вида $\mathbf{s}_i = (a_i; b_i)$, где $a_i > 0$.

Алгоритм 4.17: Алгоритм ранжирования решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(a_1; b_1), \dots, (a_k; b_k)\}$

```

1 Rank_DLP ( $P = (\mathbf{ps}_1, \dots, \mathbf{ps}_l), n, m$ )
2 begin
3   if  $n = 0$  и  $m = 0$  then  $r := 0$ 
4   else
5      $I :=$  порядковый номер  $\mathbf{ps}_l$  в упорядоченном множестве  $S$ 
6      $sum := \sum_{i=1}^{I-1} \dot{W}_{n-a_i}^{m-b_i}$ 
7      $r := sum + \text{Rank\_DLP}((\mathbf{ps}_1, \dots, \mathbf{ps}_{l-1}), n - a_I, m - b_I)$ 
8   end
9   return  $r$ 
10 end
```

Алгоритм 4.18: Алгоритм генерации по рангу решеточного пути, начинающегося в точке $(0; 0)$, заканчивающегося в точке $(n; m)$ и состоящего из шагов множества $S = \{(a_1; b_1), \dots, (a_k; b_k)\}$

```

1 Unrank_DLP( $r, n, m$ )
2 begin
3   if  $n = 0$  и  $m = 0$  then  $P := ()$ 
4   else
5      $sum := 0$ 
6     for  $i := 1$  to  $k$  do
7        $s := \dot{W}_{n-a_i}^{m-b_i}$ 
8       if  $sum + s > r$  then
9          $r := r - sum$ 
10         $I := i$ 
11        break
12      end
13       $sum := sum + s$ 
14    end
15     $P := \text{concat}(\text{Unrank\_DLP}(r, n - a_I, m - b_I), (s_I))$ 
16  end
17  return  $P$ 
18 end

```

Более того, некоторые ненаправленные решеточные пути могут быть преобразованы в форму направленных решеточных путей, например:

- пути Деланнуа, состоящие из шагов множества $S = \{(0; 1), (1; 0), (1; 1)\}$, преобразуются в направленные решеточные пути путем измерения базиса (красные оси x^* и y^* на рисунке 4.28). Тогда $D_n^m = \dot{W}_{m+n}^{m-n}$ для $S^* = \{(1; 1), (1; -1), (2; 0)\}$;
- пути Шредера, состоящие из шагов множества $S = \{(0; 1), (1; 0), (1; 1)\}$, преобразуются в направленные решеточные пути путем измерения базиса (красные оси x^* и y^* на рисунке 4.29). Тогда $S_n = \dot{M}_{2n}^0$ для $S^* = \{(1; -1), (1; 1), (2; 0)\}$;
- пути Моцкина, состоящие из шагов множества $S = \{(0; 2), (2; 0), (1; 1)\}$, преобразуются в направленные решеточные пути путем измерения базиса (красные оси x^* и y^* на рисунке 4.30). Тогда $M_n = \dot{M}_n^0$ для $S^* = \{(1; -1), (1; 1), (1; 0)\}$.

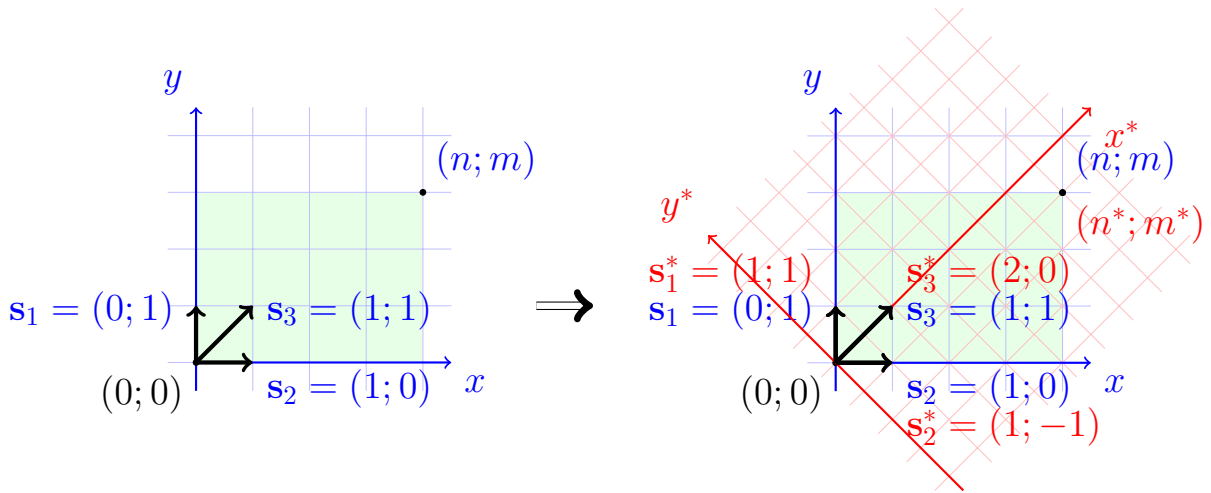


Рисунок 4.28 — Преобразование путей Деланнуа в направленные решеточные пути

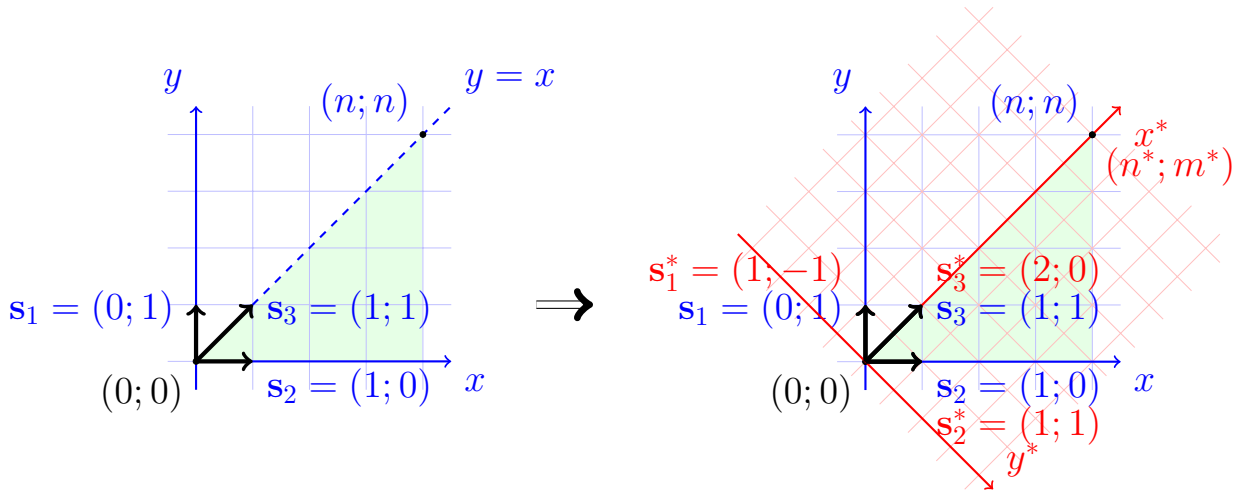


Рисунок 4.29 — Преобразование путей Шредера в направленные решеточные пути

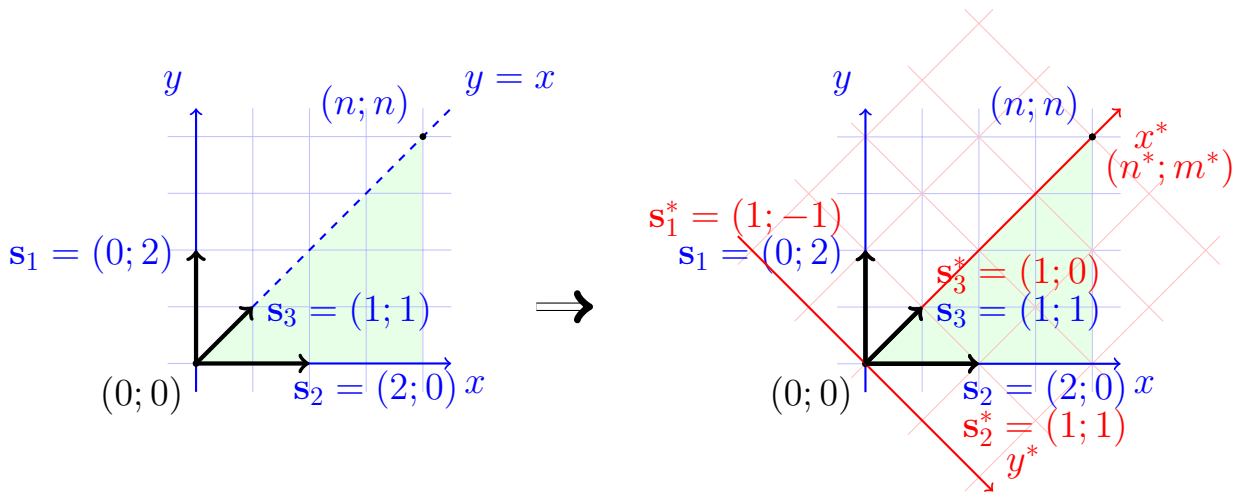


Рисунок 4.30 — Преобразование путей Моцкина в направленные решеточные пути

4.3.4 Направленные решеточные пути с ограничениями

Рассмотрим более сложные примеры направленных решеточных путей, в которых учитывается некоторая дополнительная статистика. Например, в работе [221] решается задача перечисления решеточных путей Дика из точки $(0; 0)$ в точку $(2n; 0)$ с использованием шагов $(1; 1)$ и $(1; -1)$, где каждый шаг $(1; -1)$ помечен уникальным значением от 1 до n и последовательность меток возвратных шагов (возвращение на ось x) содержит k подъемов. На рисунке 4.31 показан пример всех возможных вариантов рассматриваемых решеточных путей для $n = 3$ и $k = 1$.

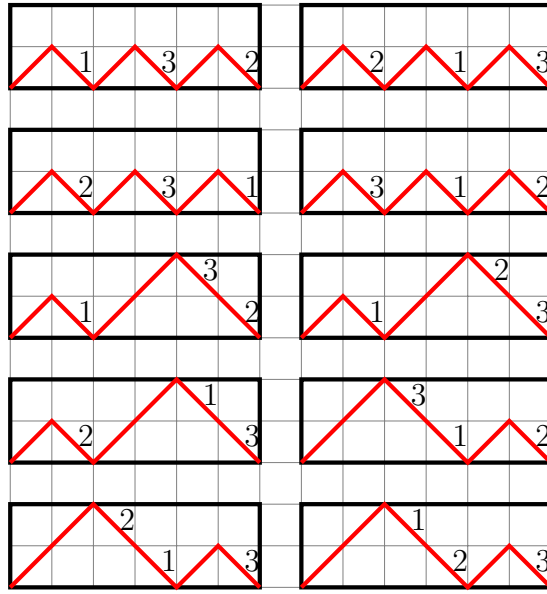


Рисунок 4.31 — Все пути Дика длины 6 с 1 подъемом на возвратных шагах

Основываясь на том, что такие решеточные пути комбинируют в себе свойства нескольких классических математических структур, получена следующая формула для их перечисления:

$$EC_n^k = \begin{cases} 0, & n < 0 \text{ или } k < 0 \text{ или } k > n; \\ 1, & n = k = 0; \\ \sum_{i=k+1}^n CT_n^i C_n^i E_i^k P_{n-i}, & \text{иначе.} \end{cases} \quad (4.22)$$

В данной формуле учитываются множества следующих структур:

– CT_n^i является элементом транспонированного треугольника Каталана (последовательность A033184 в OEIS [174]) и отвечает за количество путей Дика из $(0; 0)$ в $(2n; 0)$ с i возвратными шагами;

- C_n^i является элементом треугольника Паскаля (последовательность A007318 в OEIS [174]) и отвечает за количество вариантов выбора i меток для возвратных шагов;
- E_i^k является элементом треугольника Эйлера (последовательность A173018 в OEIS [174]) и отвечает за количество вариантов перестановки i меток возвратных шагов, чтобы их последовательность образовывала k подъемов;
- P_{n-i} отвечает за количество вариантов перестановки $n - i$ меток для остальных помеченных шагов.

Поскольку формула (4.22) и рекуррентные формулы для всех ее составляющих (CT_n^i , C_n^i , E_i^k и P_{n-i}) удовлетворяют требованиям методологии, то на их основе построена рекурсивная композиция структур деревьев И/ИЛИ и разработаны соответствующие алгоритмы комбинаторной генерации [222].

В работе [223] рассматривается другая статистика для решеточных путей Дика — количество пиков (выполнение шага $(1; -1)$ после шага $(1; 1)$). В ходе решения задачи перечисления решеточных путей Дика из точки $(0; 0)$ в точку $(2n; 0)$ с использованием шагов $(1; 1)$ и $(1; -1)$, полученных путем конкатенации l путей Дика с общим количеством m пиков, получена следующая рекуррентная формула:

$$N_{n,m,l} = N_{n-1,m-1,l-1} + N_{n-1,m-1,l} + N_{n-1,m,l} + N_{n-1,m,l+1} \quad (4.23)$$

для $n \geq m \geq l > 0$ и $N_{n,n,n} = 1$.

На рисунке 4.32 показан пример всех возможных вариантов рассматриваемых решеточных путей для $n = 4$, $m = 3$ и $l = 2$.

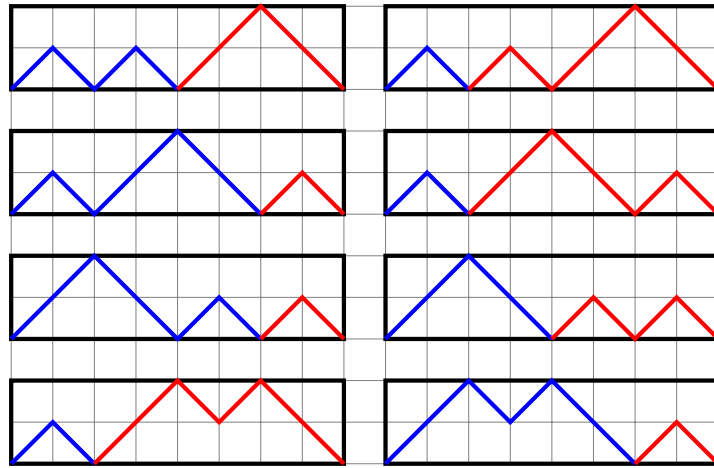


Рисунок 4.32 — Все пути Дика длины 8 с 3 пиками, образованные 2 путями Дика

Поскольку формула (4.23) удовлетворяет требованиям методологии, то на ее основе построена структура деревьев И/ИЛИ и разработаны соответствующие алгоритмы комбинаторной генерации [224].

Однако отсутствие формулы для подсчета направленных решеточных путей, в которых учитывается некоторая дополнительная статистика, не является проблемой с точки зрения разработки алгоритмов ранжирования и генерации по рангу. В таком случае решеточные пути, задача перечисления которых решается в рамках теорем 4.9 и 4.10, могут быть дополнены новыми параметрами, подсчитывающими требуемую статистику. Например, рассмотрим решеточные пути Дика из точки $(0; 0)$ в точку $(2n; 0)$ с использованием шагов $(1; 1)$ и $(1; -1)$, а также общим количеством k пиков. Общее количество таких решеточных путей для $n, k \in \mathbb{Z}_{\geq 0}$, $n \geq k$, определяется числом Нараяны N_n^k (последовательность A001263 в OEIS [174]):

$$N_n^k = \frac{1}{n} \binom{n}{k} \binom{n}{k-1}. \quad (4.24)$$

Формула (4.24) не удовлетворяет требованиям методологии и не может быть использована для построения структуры дерева И/ИЛИ. Рассмотрим решеточные пути из теоремы 4.10 с множеством допустимых шагов $\{(1; 1), (1; -1)\}$ и добавим к ним учет количества шаблонов вида подряд идущих шагов $(1; 1)$ и $(1; -1)$. В результате получаем следующую рекуррентную формулу перечисления таких решеточных путей для $n, m, k \geq 0$:

$$\dot{M}_n^m(k, l) = \begin{cases} 0, & m > n - 2k; \\ 1, & n = m = k = 0; \\ \dot{M}_{n-1}^{m-1}(k, 0) + \dot{M}_{n-1}^{m+1}(k, 1), & l = 0; \\ \dot{M}_{n-1}^{m-1}(k-1, 0) + \dot{M}_{n-1}^{m+1}(k, 1), & l = 1, \end{cases} \quad (4.25)$$

где параметр k отвечает за количество пиков, параметр l показывает является ли последующий шаг из точки $(n; m)$ шагом $(1; -1)$ (соответствует $l = 1$) или шагом $(1; 1)$ (соответствует $l = 0$).

Поскольку формула (4.25) удовлетворяет требованиям методологии, то можно построить соответствующую структуру дерева И/ИЛИ для $\dot{M}_n^m(k, l)$, а также разработать алгоритмы комбинаторной генерации. При этом решеточные пути Дика из точки $(0; 0)$ в точку $(2n; 0)$ с использованием шагов множества $\{(1; 1), (1; -1)\}$ и общим количеством k пиков соответствуют частному случаю решеточных путей, мощность множества которых определяется значением $\dot{M}_{2n}^0(k, 0)$. Таким образом, получаем связь с числами Нараяны через $N_n^k = \dot{M}_{2n}^0(k, 0)$, что обеспечивает построение соответствующих алгоритмов ранжирования и генерации по рангу.

Также можно рассмотреть комбинаторное множество вторичных структур РНК без псевдоузлов, состоящих из n нуклеотидов с k парами оснований. Общее количество таких вторичных структур РНК для $n, k \in \mathbb{Z}_{\geq 0}$, $n > 2k$, определяется числом S_n^k (последовательность A089732 в OEIS [174]):

$$S_n^k = \frac{1}{n-k} \binom{n-k}{k} \binom{n-k}{k+1}.$$

В работе [225] решена задача разработки алгоритмов комбинаторной генерации для данного комбинаторного множества путем построения структуры дерева И/ИЛИ по следующей известной рекуррентной формуле:

$$S_n^k = \begin{cases} 0, & k < 0 \text{ или } n < 2k; \\ 1, & k = 0 \text{ и } n \geq 0; \\ S_{n-1}^k + \sum_{i=0}^{k-1} \sum_{j=2i}^{n-2(k-i)-1} S_{n-2-j}^{k-1-i} S_j^i, & \text{иначе.} \end{cases}$$

В то же время, вторичные структуры РНК без псевдоузлов, состоящих из n нуклеотидов с k парами оснований, могут быть представлены направленными решеточными путями. Для этого рассмотрим решеточные пути из теоремы 4.10 с множеством допустимых шагов $\{(1; 1), (1; -1), (1; 0)\}$ и добавим к ним учет количества шагов $(1; -1)$, а также запрет шаблонов вида подряд идущих шагов $(1; 1)$ и $(1; -1)$. В результате получаем следующую рекуррентную формулу перечисления таких решеточных путей для $n, m, k \geq 0$:

$$\dot{M}_n^m(k, l) = \begin{cases} 0, & m > n - 2k; \\ 1, & n = m = k = 0; \\ \dot{M}_{n-1}^{m-1}(k, 0) + \dot{M}_{n-1}^m(k, 0) + \dot{M}_{n-1}^{m+1}(k-1, 1), & l = 0; \\ \dot{M}_{n-1}^m(k, 0) + \dot{M}_{n-1}^{m+1}(k-1, 1), & l = 1, \end{cases}$$

где параметр k отвечает за количество шагов $(1; 1)$, параметр l показывает является ли последующий шаг из точки $(n; m)$ шагом $(1; -1)$ (соответствует $l = 1$) или шагом $(1; 1)$ (соответствует $l = 0$).

Таким образом, получаем связь с количеством вторичных структур РНК через $S_n^k = \dot{M}_n^0(k, 0)$, что обеспечивает построение соответствующих алгоритмов ранжирования и генерации по рангу.

4.4 Выводы по главе

В данной главе изложены результаты апробации предложенной методологии на примере построения алгоритмов кодирования информационных объектов, представленных решеточными путями. Рассмотрены частные случаи решеточных путей, такие как северо-восточные решеточные пути, а также пути Дика, Деланнуа, Шредера и Моцкина. Для каждого из этих множеств решеточных путей построена рекурсивная композиция структур деревьев И/ИЛИ и разработаны алгоритмы комбинаторной генерации. Разработанные алгоритмы ранжирования и генерации по рангу имеют полиномиальную временную сложность, которая определяется максимальным количеством рекурсивных вызовов (высотой дерева И/ИЛИ), максимальным количеством потомков ИЛИ-узлов (шириной дерева И/ИЛИ), а также сложностью вычисления значения функции мощности комбинаторного множества. В предположении модели вычислений с использованием равномерной функции стоимости, вычислительные эксперименты подтвердили теоретические оценки временной сложности разработанных алгоритмов.

Кроме того, полученные результаты обобщены на случай направленных решеточных путей. Для этого доказаны теоремы о рекуррентных формулах для подсчета количества направленных решеточных путей. Полученные рекуррентные формулы удовлетворяют требованиям предложенной методологии кодирования на основе рекурсивных композиций структур деревьев И/ИЛИ. Используя рекуррентные формулы, построены рекурсивные композиции структур деревьев И/ИЛИ, определены правила биективного отображения множества вариантов дерева И/ИЛИ на множество направленных решеточных путей и разработаны алгоритмы кодирования направленных решеточных путей в общем виде. При этом биективное отображение базируется на сопоставлении допустимых шагов решеточного пути с потомками ИЛИ-узлов структуры дерева И/ИЛИ. Таким образом, используя разработанные в общем виде алгоритмы ранжирования (алгоритм 4.17) и генерации по рангу (алгоритм 4.18), можно кодировать любые направленные решеточные пути с произвольным набором допустимых шагов вида $\mathbf{s}_i = (a_i; b_i)$, где $a_i > 0$.

Дополнительно рассмотрена разработка алгоритмов ранжирования и генерации по рангу для более сложных направленных решеточных путей, в которых учитывается некоторая количественная статистика (например, учет количества

шаблонов определенного вида или их запрет). В качестве апробации исследованы решеточные пути, множество которых определяется числами Эйлера–Каталана, числами Нараяны, обобщенными числами Нараяны, а также решеточные пути, представляющие собой вторичные структуры РНК без псевдоузлов. Таким образом, методология позволяет получать алгоритмы кодирования для любого направленного решеточного пути. Результаты вычислительных экспериментов с использованием программных реализаций разработанных алгоритмов ранжирования и генерации по рангу подтвердили их корректность.

Таким образом, на основе полученных результатов в области разработки алгоритмов комбинаторной генерации для множеств направленных решеточных путей формируется общий метод кодирования информационных объектов, представленных решеточными путями на плоскости. При этом основные шаги данного метода заключаются в следующем:

1. Определить рекуррентную формулу для подсчета количества направленных решеточных путей с заданным множеством допустимых шагов:
 - если отсутствуют дополнительные ограничения, то воспользоваться результатами теоремы 4.9;
 - если имеются ограничения в виде запрета выхода за определенные границы, то добавить в рекуррентную формулу соответствующие начальные условия (по аналогии с примером решеточных путей из теоремы 4.10);
 - если имеются ограничения в виде учета количественной статистики, то добавить в рекуррентную формулу соответствующие параметры и правила их изменения (по аналогии с примером решеточных путей из формулы (4.25));
2. На основе рекуррентной формулы построить структуру дерева И/ИЛИ, в котором каждому допустимому шагу решеточного пути из заданной точки $(n; m)$ сопоставляется выбор потомка соответствующего ИЛИ-узла;
3. Разработать алгоритмы ранжирования и генерации по рангу вариантов построенной структуры дерева И/ИЛИ.

Также отметим, что вывод рекуррентных формул соответствует идее метода поиска с возвратом, что позволяет в дальнейшем обобщить все полученные результаты на случай любого решеточного пути, а не только направленного решеточного пути. При этом построение соответствующей структуры дерева И/ИЛИ позволяет разрабатывать алгоритмы ранжирования и генерации по рангу для таких решеточных путей.

Результаты данной главы опубликованы в [206; 213; 221–223; 225].

Глава 5. Кодирование информационных объектов, представленных словами контекстно-свободных грамматик

В данной главе изложены результаты апробации предложенной методологии на примере построения алгоритмов кодирования информационных объектов, представленных словами контекстно-свободных грамматик. Решается задача биективного отображения деревьев вывода слов контекстно-свободных грамматик на структуры вариантов деревьев И/ИЛИ, а также задача подсчета количества деревьев вывода слов заданной длины из заданного нетерминального символа однозначной контекстно-свободной грамматики. В качестве апробации рассматриваются частные случаи контекстно-свободных грамматик, в том числе грамматика, генерирующая скобочные представления для вторичных структур РНК без псевдоузлов.

5.1 Комбинаторные множества слов контекстно-свободных грамматик

Формальный язык представляет собой множество слов, полученных с помощью заданного конечного алфавита символов. Формальные языки широко используются в области теоретической информатики и различных ее приложениях. Например, получаемое с помощью теории формальных языков абстрактное описание исследуемых дискретных структур позволяет формализовать процессы их синтаксического и семантического анализа. В свою очередь, это позволяет достичь более глубокого понимания исследуемого объекта и делает возможным разработку эффективных алгоритмов обработки соответствующей информации. Ярким примером использования результатов теории формальных языков является область современных языков программирования и их компиляторов, где в качестве базового инструмента описания синтаксических структур языка программирования применяется математический аппарат формальных грамматик [226].

Порождающая грамматика позволяет сформировать множество всех слов соответствующего ей формального языка путем перебора возможных комбинаций применения правил вывода. Если при этом ограничить длину генерируемых слов, то задаваемое грамматикой множество будет являться конечным. Таким

образом, получаем комбинаторное множество, элементами которого являются слова заданной длины из соответствующего формального языка. Разработка алгоритмов комбинаторной генерации для множества слов формального языка, заданного некоторой порождающей грамматикой, позволяет решать следующие научные задачи: сжатие данных и моделирование сложных дискретных структур.

В рамках первого направления, в работе [18] предлагается подход к сжатию строк данных, которые представляют собой слова заданной длины из некоторого формального языка, за счет применения алгоритма ранжирования. Суть данного подхода к ранжированию заключается в вычислении количества слов, меньших заданного при лексикографическом упорядочивании всех слов соответствующего формального языка. В частности, представлены формулы для вычисления ранга с полиномиальной вычислительной сложностью при использовании однозначной контекстно-свободной грамматики. В работе [227] исследуются возможности обобщения алгоритмов ранжирования на неоднозначные контекстно-свободные грамматики, а работа [228] изучает вопросы распараллеливания вычислений при ранжировании. Дальнейшее развитие этой задачи представлено в работе [229], в которой представлен не только алгоритм ранжирования, но и алгоритм генерации по рангу слов заданной длины некоторого формального языка с полиномиальной вычислительной сложностью. Однако в данном случае исследуется задача сжатия слов только для случая формального языка, задаваемого однозначной контекстно-свободной грамматикой, где каждое слово представляет собой последовательность применения правил вывода другой контекстно-свободной грамматики. В работе [230] представлены алгоритмы ранжирования и генерации по рангу, которые могут работать с неоднозначными контекстно-свободными грамматиками, обрабатывая деревья вывода. Также в качестве примера рассмотрена задача сжатия исходного кода программы на языке программирования С. Дальнейшее развитие данного направления позволит создать общие алгоритмы сжатия данных на основе формальных грамматик [21].

Что касается моделирования сложных дискретных структур, то в работах [231; 232] рассматривается генерация случайного слова заданной длины путем применения правил вывода контекстно-свободной грамматики в случайном порядке. При этом, чтобы обеспечить равномерное распределение генерируемых слов, предлагается предварительное вычисление вероятностей применения каждого правила вывода, то есть формируется вероятностная контекстно-свободная грамматика. Если применять алгоритм генерации по рангу для случайной ге-

нерации, то равномерность распределения генерируемых слов обеспечивается использованием генератора случайных чисел. Также существуют решения задачи неравномерной генерации слов с использованием алгоритма генерации по рангу. Например, в работах [233; 234] такие алгоритмы комбинаторной генерации позволяют моделировать и сжимать датасеты вторичных структур РНК. Кроме того, алгоритмы комбинаторной генерации для контекстно-свободных языков применяются в области криптографии, например, для шифрования данных с сохранением формата [230; 235; 236].

Таким образом, разработка новых методов построения алгоритмов комбинаторной генерации для формальных языков является актуальной научной задачей, так как обладает как теоретической, так и практической значимостью благодаря имеющимся приложениям в задачах сжатия данных и моделирования. Также отметим, что существующие алгоритмы комбинаторной генерации обрабатывают только слова заданной длины из формальных языков, задаваемых контекстно-свободными грамматиками, что ограничивает область их применения.

Определение 5.1. *Формальная грамматика представляет собой четверку $G = (T, N, S, P)$, где:*

- T — множество терминальных символов грамматики;
- N — множество нетерминальных символов грамматики, $T \cap N = \emptyset$;
- S — начальный символ грамматики, $S \in N$;
- P — множество правил вывода вида $\alpha \rightarrow \beta$, где $\alpha \in (N \cup T)^* N (N \cup T)^*$ — последовательность символов грамматики с хотя бы одним нетерминальным символом, $\beta \in (N \cup T)^*$ — последовательность любых символов грамматики.

Путем последовательного применения правил вывода из начального символа грамматики $S \in N$ выводится последовательность терминальных символов (слово) $\omega \in T^*$, этот процесс обозначается как $S \Rightarrow^* \omega$.

Определение 5.2. *Язык $L(G)$ грамматики $G = (T, N, S, P)$ — множество всех слов $\omega \in T^*$, выводимых из начального символа грамматики $S \in N$, то есть $L(G) = \{\omega \in T^* | S \Rightarrow^* \omega\}$.*

В данном диссертационном исследовании рассматриваются формальные грамматики, относящиеся к типу контекстно-свободных грамматик. Такие грамматики содержат только правила вывода вида $\alpha \rightarrow \beta$, где $\alpha \in N$ и $\beta \in (N \cup T)^*$.

Рассмотрим ограничение на длину слов, выводимых с помощью заданной формальной грамматики G , то есть сформируем подмножество $L_n(G) \subseteq L(G)$, где каждое слово $\omega \in L_n(G)$ имеет фиксированную длину $|\omega| = n$. Следовательно, формальная грамматика $G = (T, N, S, P)$ задает конечное множество $L_n(G)$, содержащее $|L_n(G)|$ различных слов заданной длины n . Здесь и далее по тексту главы оператор прямых скобок используется для обозначения:

- если ω является словом (последовательность терминальных символов), то $|\omega|$ показывает длину слова ω (количество терминальных символов в ω);
- если L является множеством, то $|L|$ показывает мощность множества L (количество элементов в L);
- если A является нетерминальным символом формальной грамматики, то $|A|$ показывает количество деревьев вывода всех слова из A .

Применение алгоритма ранжирования для формального языка $L_n(G)$ позволяет перенумеровать каждое слово $\omega \in L_n(G)$, то есть задается функция $\text{Rank} : L_n(G) \rightarrow \mathbb{Z}_{\geq 0}$. Применение алгоритма генерации по рангу позволяет выполнить обратное к ранжированию преобразование, то есть задается функция $\text{Unrank} : \mathbb{Z}_{\geq 0} \rightarrow L_n(G)$. Алгоритмы ранжирования и генерации по рангу реализуют биективное отображение, поэтому $\text{Unrank}(\text{Rank}(\omega)) = \omega$. Следовательно, с помощью алгоритмов комбинаторной генерации реализуется кодирование слов формального языка, что также позволяет сжимать объем хранимых данных.

Процесс вывода слова контекстно-свободной грамматики требует выбора конкретного правила вывода для рассматриваемого на текущем шаге нетерминального символа, а также выполняет конкатенацию полученных символов. Этим действиям можно сопоставить логику работы ИЛИ-узлов (выбор конкретного потомка узла) и И-узлов (выбор всех потомков узла) в структурах деревьев И/ИЛИ. Следовательно, множество деревьев вывода слов контекстно-свободной грамматики биективно отображается на множество структур вариантов деревьев И/ИЛИ. Таким образом, становится возможной разработка алгоритмов ранжирования и генерации по рангу с целью кодирования информационных объектов, строящихся по правилам вывода контекстно-свободных грамматик. При этом методология на основе рекурсивных композиций структур деревьев И/ИЛИ не имеет ограничения на количество параметров, определяющих кодируемый объект. В свою очередь, это позволяет учитывать не только длину выводимых слов, но и дополнительные количественные характеристики объекта.

5.2 Метод кодирования слов контекстно-свободных грамматик

Пусть задана контекстно-свободная грамматика $G = (T, N, S, P)$. Основную идею предлагаемого метода кодирования слов заданной длины n из формального языка $L_n(G) = \{\omega \in T^* | S \Rightarrow^* \omega, |\omega| = n\}$ можно выразить совокупностью четырех шагов [237]. На первом шаге формируется комбинаторное множество (в частности, конечное множество слов заданной длины) путем модификации правил вывода грамматики, которые дополняются учетом новых параметров. Далее строится соответствующая комбинаторному множеству структура дерева И/ИЛИ (шаг 2) и реализуется взаимосвязь вариантов данного дерева И/ИЛИ с элементами комбинаторного множества (шаг 3). Основываясь на полученной взаимосвязи комбинаторного множества и структуры дерева И/ИЛИ, конечным этапом разрабатываются алгоритмы ранжирования и генерации по рангу (шаг 4). Далее представлено подробное описание каждого шага:

Шаг 1. Модификация правил вывода грамматики с целью учета длины генерируемого слова.

Если правила вывода грамматики $G = (T, N, S, P)$ не учитывают длину генерируемого слова ω ($S \Rightarrow^* \omega$), то необходимо выполнить следующие преобразования для каждого правила вывода вида $A \rightarrow \beta$ ($A \in N, \beta \in (N \cup T)^*$):

- в левой части заменить нетерминальный символ A на новый нетерминальный символ A_n , то есть преобразовать $A \rightarrow \beta$ в $A_n \rightarrow \beta$;
- если $\beta \in T^*$, то есть содержит только терминальные символы, тогда правило вывода $A_n \rightarrow \beta$ заменить на $A_t \rightarrow \beta$, где t равно количеству терминальных символов в β ;
- если $\beta = \varepsilon$, то есть выводится пустое слово нулевой длины, тогда правило вывода $A_n \rightarrow \varepsilon$ заменить на $A_0 \rightarrow \varepsilon$;
- если $\beta = \beta_1 B \beta_2$ ($\beta_1, \beta_2 \in T^*, B \in N$), то есть содержит один нетерминальный символ B , тогда правило вывода $A_n \rightarrow \beta_1 B \beta_2$ заменить на $A_n \rightarrow \beta_1 B_{n-t} \beta_2$, где t равно количеству терминальных символов в β ;
- если $\beta = \beta_1 B \beta_2 C \beta_3$ ($\beta_1, \beta_2, \beta_3 \in T^*, B, C \in N$), то есть содержит нетерминальные символы B и C , тогда правило вывода $A_n \rightarrow \beta_1 B \beta_2 C \beta_3$ заменить на набор правил вывода $A_n \rightarrow \beta_1 B_i \beta_2 C_{n-t-i} \beta_3$ для всех значений параметра i от 0 до $n - t$, где t равно количеству терминальных символов в β ;

- если β содержит более двух нетерминальных символов, тогда аналогичным образом перебираются все возможные разбиения их индексов;
- если какая-либо комбинация индексов приводит к невозможной ситуации (например, отрицательная длина генерируемого слова), тогда соответствующее правило вывода и все связанные с ним правила вывода необходимо удалить, то есть не должны остаться такие нетерминальные символы, которые ни разу не встречаются в левой части правил вывода.

В результате из заданной контекстно-свободной грамматики $G = (T, N, S, P)$ получаем новую контекстно-свободную грамматику $G' = (T, N', S_n, P')$, которая задает формальный язык $L_n(G) = \{\omega \in T^* | S_n \Rightarrow^* \omega\}$, где каждое выводимое слово ω имеет длину $|\omega| = n$. Кроме длины слова, аналогичным образом можно учитывать и другие количественные параметры выводимых слов, то есть нетерминальные символы правил вывода $A \rightarrow \beta$ дополняются новыми индексами, значения которых в β изменяются в зависимости от происходящих преобразований.

Шаг 2. Построение структуры дерева И/ИЛИ.

Для полученной на предыдущем шаге контекстно-свободной грамматики $G' = (T, N', S_n, P')$ формируется соответствующая структура дерева И/ИЛИ путем композиции деревьев И/ИЛИ, каждое из которых строится следующим образом по каждому правилу вывода вида $A \rightarrow \beta$ ($A \in N$, $\beta \in (N \cup T)^*$):

- создать корень дерева И/ИЛИ и пометить его символом A . Поддерево данного узла формируется на основе β ;
- если имеется набор из m правил вывода с одинаковой левой частью, то есть $A \rightarrow \beta_1, \dots, A \rightarrow \beta_m$ (краткая форма записи $A \rightarrow \beta_1 | \dots | \beta_m$), тогда корневой узел, помеченный A , становится ИЛИ-узлом с m узлами-потомками. Поддерево каждого из m узлов-потомков формируется на основе правых частей $\beta_1, \dots, \beta_m$;
- узел, формируемый на основе β , становится И-узлом, количество узлов-потомков которого определяется количеством терминальных и нетерминальных символов в β . Каждый узел-потомок помечается соответствующим символом;
- если узлу соответствует терминальный символ, в том числе пустое слово, тогда он становится листом;
- если узлу соответствует нетерминальный символ, тогда его поддерево определяется структурой соответствующего дерева И/ИЛИ (композиция деревьев И/ИЛИ);

– если для нетерминального символа A_n в зависимости от значения n применяются разные правила вывода, тогда используется структура дерева И/ИЛИ с условным оператором;

– если узлу соответствует нетерминальный символ, который ни разу не встречается в левой части правил вывода, тогда он становится пустым листом.

В результате получаем композицию деревьев И/ИЛИ с корневым узлом S_n и с количеством вариантов, равным количеству всех возможных деревьев вывода всех слов ω заданной длины n из формального языка $L_n(G)$.

Шаг 3. Сопоставление слов формального языка с вариантами структуры дерева И/ИЛИ.

Формирование варианта дерева И/ИЛИ сводится к выбору одного из узлов-потомков для каждого ИЛИ-узла. В полученной на предыдущем шаге структуре дерева И/ИЛИ каждый ИЛИ-узел соответствует одному из нетерминальных символов, а каждый узел-потомок ИЛИ-узла соответствует одному из правил вывода для соответствующего нетерминального символа. Следовательно, выбор в варианте дерева И/ИЛИ одного из узлов-потомков для каждого ИЛИ-узла соответствует применению конкретного правила вывода. Таким образом, вариант дерева И/ИЛИ имеет вид, схожий со структурой дерева вывода, а сама структура дерева И/ИЛИ описывает совокупность всех возможных деревьев вывода заданной контекстно-свободной грамматики. Аналогично дереву вывода, крона варианта дерева И/ИЛИ формирует выводимое слово. Чтобы задать обратное преобразование, то есть по заданному слову получить соответствующий вариант дерева И/ИЛИ, необходимо применить алгоритм синтаксического анализа (например, алгоритм Эрли [238]) и полученное дерево вывода использовать в качестве основы структуры варианта дерева И/ИЛИ.

Заметим, что если заданная контекстно-свободная грамматика является однозначной, то представленное выше сопоставление реализует биективное отображение (каждому слову из формального языка $L_n(G)$ соответствует только один вариант дерева И/ИЛИ). В случае неоднозначной контекстно-свободной грамматики одному слову может соответствовать множество различных деревьев вывода. Тогда количество вариантов дерева И/ИЛИ будет превышать количество слов из формального языка $L_n(G)$, то есть одному слову может соответствовать более одного варианта дерева И/ИЛИ.

Шаг 4. Построение алгоритмов ранжирования и генерации по рангу.

Для полученной структуры дерева И/ИЛИ применяются общие алгоритмы ранжирования и генерации по рангу вариантов деревьев И/ИЛИ. В результате получаем частные случаи алгоритмов комбинаторной генерации, учитывающие специфику заданной структуры дерева И/ИЛИ.

5.3 Алгоритмы кодирования для частных случаев контекстно-свободных грамматик

Рассмотрим апробацию разработанного метода на примере частных случаев контекстно-свободных грамматик.

Пример 1. Пусть задана следующая контекстно-свободная грамматика:

$$G = (T, N, S, P), \quad T = \{ (,), \varepsilon \}, \quad N = \{ S \}, \quad P = \{ S \rightarrow (S)S, S \rightarrow \varepsilon \}. \quad (5.1)$$

Данная грамматика задает формальный язык, каждое слово которого представляет собой правильную скобочную последовательность. Рассмотрим построение алгоритмов ранжирования и генерации по рангу для множества слов заданной длины n из формального языка, задаваемого грамматикой (5.1). Заметим, что мощность формируемого множества правильных скобочных последовательностей, состоящих из $n/2$ пар открывающей и закрывающей скобок, связано с числом Каталана $C_{n/2}$.

В результате проведения модификации правил вывода грамматики получаем следующее множество новых правил вывода:

$$S_n \rightarrow (S_i)S_{n-2-i} \text{ для } i = 0, 1, 2, \dots, n-2; \quad S_0 \rightarrow \varepsilon.$$

Однако в полученном наборе правил вывода содержатся такие комбинации индексов, которые приводят к невозможной ситуации. Например, для правила вывода из нетерминального символа S_1 получаем невозможную комбинацию индексов в правой части (отрицательная длина генерируемого слова), поэтому все правила вывода с S_1 в правой части должны быть удалены. Аналогичным образом приходим к необходимости удалить все правила вывода, которые содержат нетерминальный символ S_n для всех нечетных значений n .

Окончательно получаем следующее множество новых правил вывода P' :

$$S_n \rightarrow (S_i)S_{n-2-i} \text{ для } n = 2, 4, 6, \dots \text{ и } i = 0, 2, 4, \dots, n-2; \quad S_0 \rightarrow \varepsilon. \quad (5.2)$$

На основе полученных правил вывода формируется соответствующая структура дерева И/ИЛИ, представленная на рисунке 5.1. Частные случаи структуры дерева И/ИЛИ при $n = 0, \dots, 4$ представлены на рисунке 5.2.

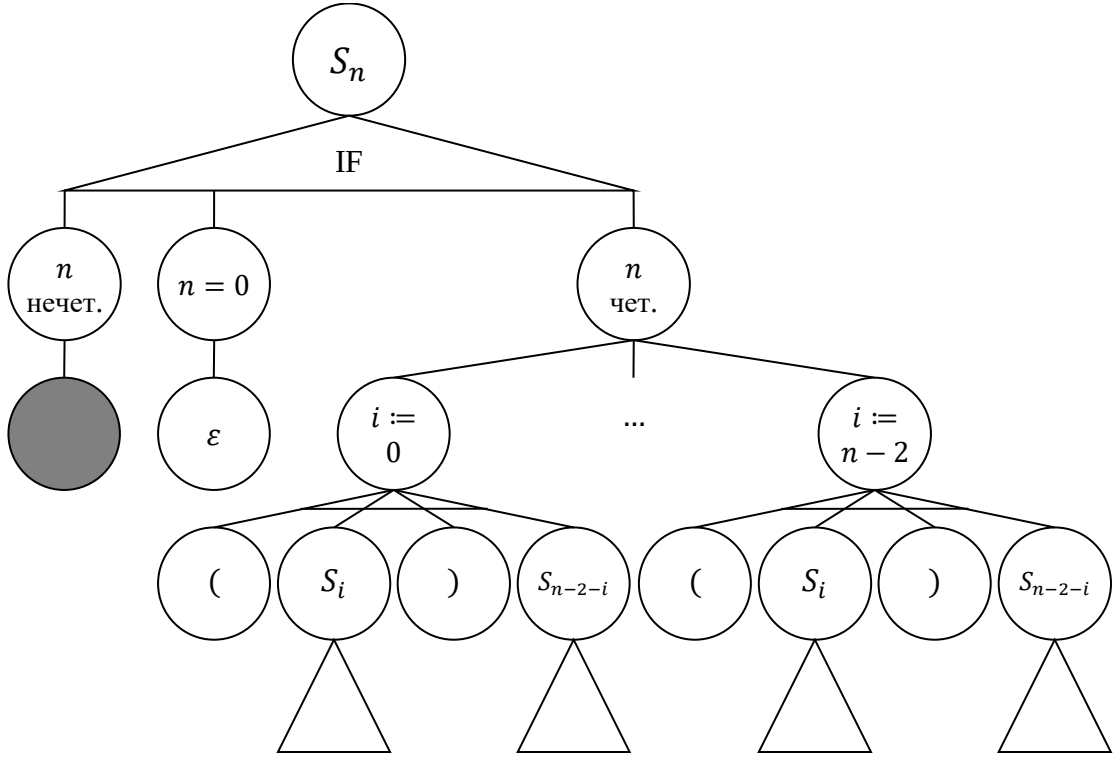


Рисунок 5.1 — Общая структура дерева И/ИЛИ для правил вывода (5.2)

Используя общие алгоритмы ранжирования и генерации по рангу для структуры дерева И/ИЛИ, представленной на рисунке 5.1, получаем соответствующие алгоритмы для слов ω длины n , строящихся по правилам вывода (5.2). Функция $\text{ParseW}(\omega, p)$ в алгоритме 5.1 реализует синтаксический анализ заданного слова ω , представляя его в виде $\omega = (\omega_1)\omega_2$, где ω_1 и ω_2 являются правильными скобочными последовательностями, и возвращает ω_1 при $p = 1$ и ω_2 при $p = 2$.

Заметим, что для повышения эффективности алгоритмов можно вычислить значения $|S_n|$ для правил вывода (5.2) через явную формулу для чисел Каталана:

$$|S_n| = \begin{cases} 0, & n \text{ нечетное;} \\ 1, & n = 0; \\ C_{n/2}, & n \text{ четное.} \end{cases} = \begin{cases} 0, & n \text{ нечетное;} \\ 1, & n = 0; \\ \frac{2}{n+2} \binom{n}{n/2}, & n \text{ четное.} \end{cases}$$

Алгоритм 5.1: Алгоритм ранжирования слова ω длины n , строящегося по правилам вывода (5.2)

```

1 Rank_w( $\omega, n$ )
2 begin
3   if  $n$  нечетное then  $r := \text{"ERROR"}$ 
4   else if  $n = 0$  then  $r := 0$ 
5   else
6      $\omega_1 := \text{ParseW}(\omega, 1)$ 
7      $\omega_2 := \text{ParseW}(\omega, 2)$ 
8      $I := |\omega_1|$ 
9      $sum := \sum_{i=0}^{I-1} |S_i| \cdot |S_{n-2-i}|$ 
10     $w_1 := |S_I|$ 
11     $l_1 := \text{Rank\_w}(\omega_1, I)$ 
12     $l_2 := \text{Rank\_w}(\omega_2, n - 2 - I)$ 
13     $r := sum + l_1 + w_1 l_2$ 
14  end
15  return  $r$ 
16 end

```

Алгоритм 5.2: Алгоритм генерации по рангу слова ω длины n , строящегося по правилам вывода (5.2)

```

1 Unrank_w( $r, n$ )
2 begin
3   if  $n$  нечетное then  $\omega := \text{"ERROR"}$ 
4   else if  $n = 0$  then  $\omega := \varepsilon$ 
5   else
6      $sum := 0$ 
7     for  $i := 0$  to  $n - 2$  do
8        $s := |S_i| \cdot |S_{n-2-i}|$ 
9       if  $sum + s > r$  then
10         $r := r - sum$ 
11         $I := i$ 
12        break
13      end
14       $sum := sum + s$ 
15    end
16     $w_1 := |S_I|$ 
17     $l_1 := r \bmod w_1$ 
18     $l_2 := \lfloor \frac{r}{w_1} \rfloor$ 
19     $\omega_1 := \text{Unrank\_w}(l_1, I)$ 
20     $\omega_2 := \text{Unrank\_w}(l_2, n - 2 - I)$ 
21     $\omega := (\omega_1)\omega_2$ 
22  end
23  return  $\omega$ 
24 end

```

Пример 2. Добавим еще один параметр в рассмотренный выше пример, а именно рассмотрим построение алгоритмов ранжирования и генерации по рангу для множества слов длины n , строящихся по правилам вывода (5.2), а также с учетом количества k шаблонов вида ().

В результате проведения модификации правил вывода (5.2) с учетом только возможных ситуаций получаем следующее множество новых правил вывода P'' :

$$\begin{aligned}
S_n^k &\rightarrow (S_i^j)S_{n-2-i}^{k-j} \text{ для } n = 2, 4, 6, \dots \geq 2k, i = 2, 4, \dots, n-2, \\
&\text{и } j = \max\left(1, k - \frac{n-2-i}{2}\right), \dots, \min\left(k, \frac{i}{2}\right); \\
S_n^k &\rightarrow (S_0^0)S_{n-2}^{k-1} \text{ для } n = 2, 4, 6, \dots \geq 2k; \quad S_0^0 \rightarrow \varepsilon.
\end{aligned} \tag{5.3}$$

На основе полученных правил вывода формируется соответствующая структура дерева И/ИЛИ, представленная на рисунке 5.3.

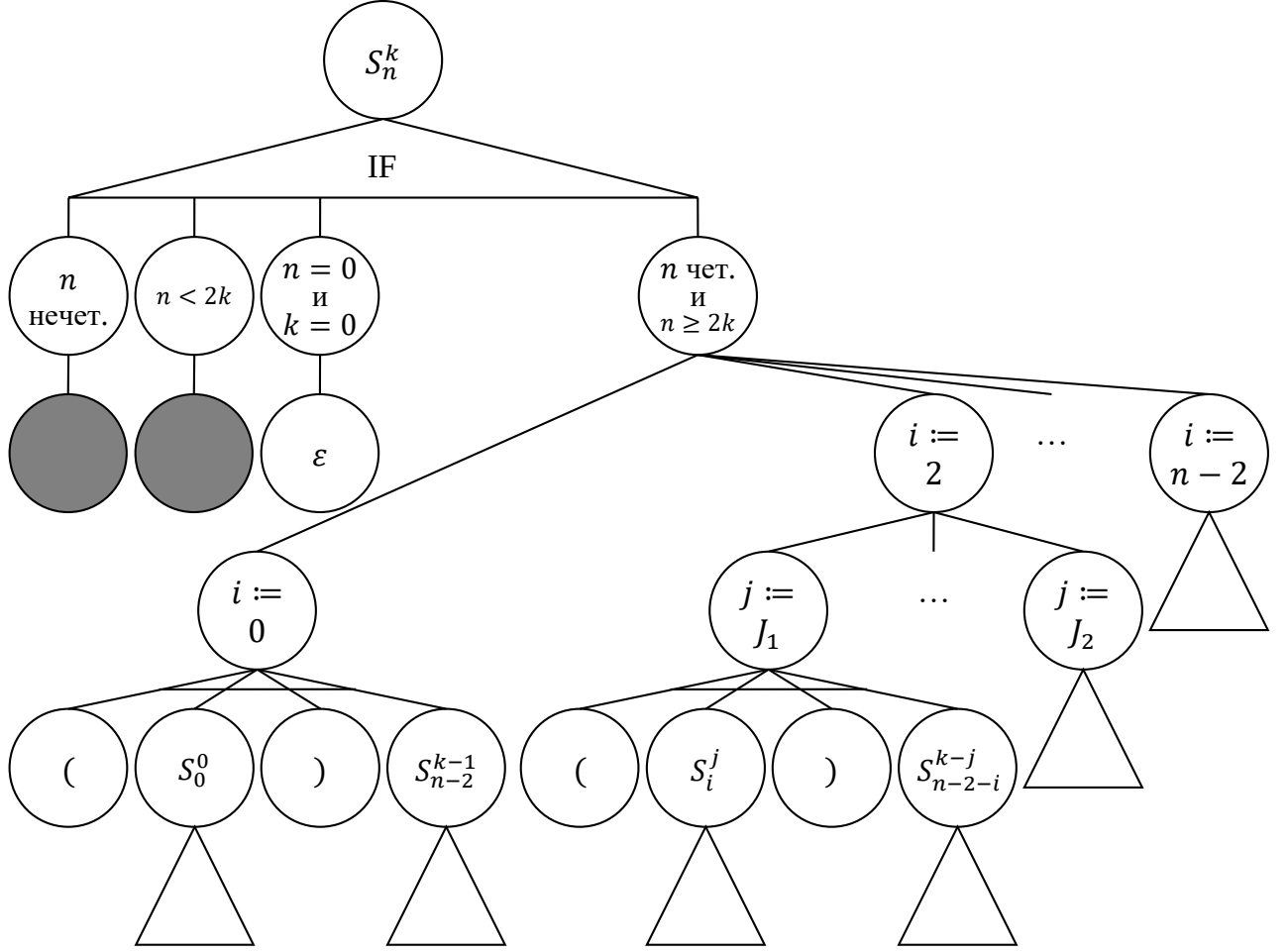


Рисунок 5.3 — Общая структура дерева И/ИЛИ для правил вывода (5.3)

Метка $j := J_1$ соответствует начальному значению $j = \max\left(1, k - \frac{n-2-i}{2}\right)$, а метка $j := J_2$ соответствует конечному значению $j = \min\left(k, \frac{i}{2}\right)$. Поддерево каждого узла, помеченного $j > J_1$, имеет структуру, аналогичную поддереву узла с меткой $j := J_1$. Поддерево каждого узла, помеченного $i > 2$, имеет структуру, аналогичную поддереву узла с меткой $i := 2$.

Используя общие алгоритмы ранжирования и генерации по рангу для структуры дерева И/ИЛИ, представленной на рисунке 5.3, получаем соответствующие алгоритмы для слов ω длины n с k шаблонами вида $()$, строящихся по правилам вывода (5.3).

Алгоритм 5.3: Алгоритм ранжирования слова ω длины n с k шаблонами вида $()$, строящегося по правилам вывода (5.3)

```

1 Rank_w( $\omega$ ,  $n$ ,  $k$ )
2 begin
3   if  $n$  нечетное или  $n < 2k$  then  $r := \text{"ERROR"}$ 
4   else if  $n = 0$  и  $k = 0$  then  $r := 0$ 
5   else
6      $\omega_1 := \text{ParseW}(\omega, 1)$ 
7      $\omega_2 := \text{ParseW}(\omega, 2)$ 
8      $I := |\omega_1|$ 
9      $J := \text{CountK}(\omega_1)$ 
10    if  $I = 0$  then
11       $w_1 := |S_0^0|$ 
12       $l_1 := \text{Rank\_w}(\omega_1, 0, 0)$ 
13       $l_2 := \text{Rank\_w}(\omega_2, n - 2, k - 1)$ 
14       $r := l_1 + w_1 l_2$ 
15    end
16    else
17       $sum := |S_{n-2}^{k-1}|$ 
18       $sum := sum + \sum_{i=2}^{I-1} \sum_{j=\max(1, k-\frac{n-2-i}{2})}^{\min(k, \frac{i}{2})} |S_i^j| \cdot |S_{n-2-i}^{k-j}|$ 
19       $sum := sum + \sum_{j=\max(1, k-\frac{n-2-I}{2})}^{J-1} |S_I^j| \cdot |S_{n-2-I}^{k-j}|$ 
20       $w_1 := |S_I^J|$ 
21       $l_1 := \text{Rank\_w}(\omega_1, I, J)$ 
22       $l_2 := \text{Rank\_w}(\omega_2, n - 2 - I, k - J)$ 
23       $r := sum + l_1 + w_1 l_2$ 
24    end
25  end
26  return  $r$ 
27 end

```

Алгоритм 5.4: Алгоритм генерации по рангу слова ω длины n с k шаблонами вида $(\)$, строящегося по правилам вывода (5.3)

```

1  Unrank_w( $r, n, k$ )
2  begin
3      if  $n$  нечетное или  $n < 2k$  then  $\omega := \text{"ERROR"}$ 
4      else if  $n = 0$  и  $k = 0$  then  $\omega := \varepsilon$ 
5      else
6          if  $|S_{n-2}^{k-1}| > r$  then
7               $\omega_1 := \varepsilon$ 
8               $\omega_2 := \text{Unrank\_w}(r, n - 2, k - 1)$ 
9               $\omega := (\omega_1)\omega_2$ 
10         end
11     else
12          $r := r - |S_{n-2}^{k-1}|$ 
13          $sum := 0$ 
14         for  $i := 2$  to  $n - 2$  step 2 do
15             for  $j := \max(1, k - \frac{n-2-i}{2})$  to  $\min(k, \frac{i}{2})$  do
16                  $s := |S_i^j| \cdot |S_{n-2-i}^{k-j}|$ 
17                 if  $sum + s > r$  then
18                      $r := r - sum$ 
19                      $I := i$ 
20                      $J := j$ 
21                     break
22                 end
23                  $sum := sum + s$ 
24             end
25         end
26          $w_1 := |S_I^J|$ 
27          $l_1 := r \bmod w_1$ 
28          $l_2 := \lfloor \frac{r}{w_1} \rfloor$ 
29          $\omega_1 := \text{Unrank\_w}(l_1, I, J)$ 
30          $\omega_2 := \text{Unrank\_w}(l_2, n - 2 - I, k - J)$ 
31          $\omega := (\omega_1)\omega_2$ 
32     end
33 end
34 return  $\omega$ 
35 end

```

Используемая в алгоритме 5.3 функция $\text{CountK}(\omega)$ реализует подсчет количества шаблонов вида $()$ в слове ω .

Заметим, что для повышения эффективности алгоритмов можно вычислить значения $|S_n^k|$ для правил вывода (5.3) через явную формулу для чисел Нараяны:

$$|S_n^k| = \begin{cases} 0, & n \text{ нечетное или } n < 2k; \\ 1, & n = k = 0; \\ N_{n/2}^k, & n \text{ четное и } n \geq 2k. \end{cases} =$$

$$= \begin{cases} 0, & n \text{ нечетное или } n < 2k; \\ 1, & n = k = 0; \\ \frac{2}{n} \binom{n/2}{k} \binom{n/2}{k-1}, & n \text{ четное и } n \geq 2k. \end{cases}$$

В таблице 5.1 представлен пример ранжирования комбинаторного множества всех слов длины $n = 8$ с $k = 3$ шаблонами вида $()$, строящихся по правилам вывода (5.3). Всего существует $|S_8^3| = 6$ слов, удовлетворяющих заданным требованиям.

Таблица 5.1 — Ранжирование множества слов длины 8 с 3 шаблонами вида $()$, строящихся по правилам вывода (5.3)

Слово	Ранг
$()()(())$	0
$()(())()$	1
$()(())()$	2
$(())()()$	3
$(())()()$	4
$(())()()$	5

Кроме того, представленные примеры разработанных алгоритмов были реализованы в виде программ с помощью системы компьютерной алгебры «Mathima». Ручное тестирование программных реализаций подтвердило корректность работы разработанных алгоритмов, а также показало полиномиальный рост времени выполнения программ в зависимости от длины n генерируемого слова ω при использовании явных формул для вычисления чисел Каталана и чисел Нараяны. При этом использование рекуррентных формул для вычисления $|S_n|$ приводит к экспоненциальной временной сложности в зависимости от n , что подтверждает актуальность получения явных формул.

Пример 3. Одним из основных и важнейших элементов клеточной структуры организма является молекула РНК, которая представляет собой цепочку нуклеотидов с азотистыми основаниями четырех типов. Последовательность нуклеотидов в цепочке молекулы РНК образует первичную структуру РНК. Кроме того, между двумя основаниями в цепочке РНК могут образовываться водородные связи, что приводит к образованию в ней различных петель. Полученная форма цепочки молекулы РНК с водородными связями образует вторичную структуру РНК. С математической точки зрения вторичная структура молекулы РНК представляется в виде графа, где узлами графа являются нуклеотиды, а ребра графа отображают связи между основаниями. Для компактности представления вторичной структуры молекулы РНК широко используется нотация точек и скобок (рисунок 5.4).

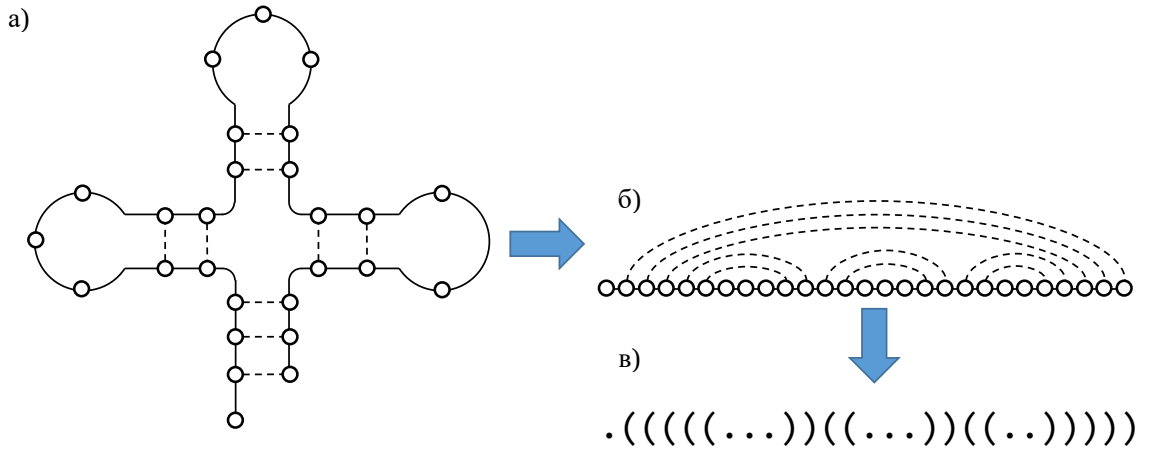


Рисунок 5.4 — Пример представления вторичной структуры РНК без псевдоузлов, состоящей из $n = 27$ нуклеотидов длиной с $m = 9$ водородными связями

Вторичные структуры молекулы РНК в нотации точек и скобок могут быть сгенерированы путем применения следующей контекстно-свободной грамматики:

$$G = (T, N, S, P), \quad T = \{ (,), ., \epsilon \}, \quad N = \{ S, A, B, C \},$$

$$P = \{ S \rightarrow CA, A \rightarrow (B)C, A \rightarrow (B)S, B \rightarrow .C, B \rightarrow S, C \rightarrow .C, C \rightarrow \epsilon \}.$$

При этом количество всех вторичных структур РНК без псевдоузлов длиной n с m водородными связями определяется по следующей формуле [239]:

$$S_n^m = \begin{cases} 0, & n < 0 \text{ или } m \geq n; \\ 1, & n = m = 0; \\ \frac{1}{n-m} \binom{n-m}{m} \binom{n-m}{m+1}, & n \text{ четное.} \end{cases}$$

Применение предложенного метода позволило построить алгоритмы кодирования для вторичных структур РНК без псевдоузлов, состоящих из n нуклеотидов с m водородными связями. Разработанные алгоритмы могут найти применение в задачах хранения датасетов с информацией о таких структурах РНК. Например, для хранения информации о вторичной структуре РНК без псевдоузлов длиной n с m водородными связями можно рассмотреть следующие три способа:

- хранение последовательности точек и скобок в виде строки в текстовом файле, где каждый символ принадлежит множеству $\{ (,), . \}$. Если использовать кодировку ASCII, то для хранения каждого символа потребуется 8 бит. Тогда для хранения последовательности из n символов потребуется $B_1 = 8 \cdot n$ бит;
- хранение последовательности точек и скобок в виде числа в троичной системе счисления, где каждая троичная цифра принадлежит множеству $\{0, 1, 2\}$, что также соответствует элементам множества $\{ (,), . \}$. Тогда для хранения последовательности из n троичных цифр потребуется $B_2 = \lceil \log_2(3^n - 1) \rceil$ бит;
- хранение последовательности точек и скобок в виде ранга соответствующего слова. Тогда в худшем случае для хранения максимально возможного значения ранга потребуется $B_3 = \lceil \log_2(\max_m(S_n^m)) \rceil$ бит.

Отношение B_3 к B_2 показывает, что даже для худшего случая хранение с использованием алгоритма ранжирования сокращает количество требуемых бит более чем в 2 раза по сравнению с использованием для хранения троичной системы счисления.

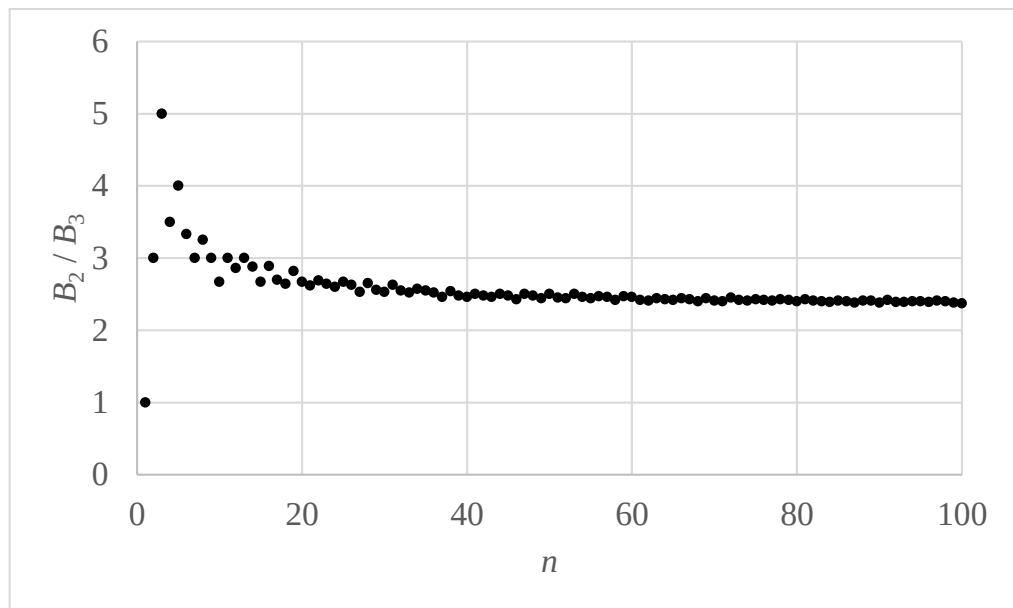


Рисунок 5.5 — Коэффициент сжатия для разных способов хранения вторичной структуры молекулы РНК

5.4 Метод получения явных и рекуррентных формул для подсчета количества слов заданной длины

Для корректной работы алгоритмов кодирования слов контекстно-свободных грамматик, разрабатываемых согласно методу из предыдущего параграфа 5.2, требуется формула для подсчета количества элементов в соответствующем комбинаторном множестве. Если работаем с однозначной контекстно-свободной грамматикой, то на основе ее правил вывода можно получить соответствующие рекуррентные формулы. Однако такие рекуррентные формулы неэффективны с точки зрения вычислительной сложности, так как показывают экспоненциальную временную сложность. Поэтому получение явных формул вместо рекуррентных является важной задачей оптимизации соответствующих алгоритмов кодирования.

Производящие функции являются широко используемым инструментом в области перечислительной комбинаторики [147; 153]. В частности, коэффициенты заданной производящей функции, связанной с комбинаторным множеством, показывают количество элементов определенного размера в данном множестве. В случае однозначных контекстно-свободных грамматик можно записать производящую функцию для последовательности чисел, которые равны количеству выводимых слов заданной длины. В работе [240] авторы рассматривают задачу получения формул для коэффициентов такой производящей функции и показывают, что она имеет класс сложности NC. Однако авторы не приводят никаких конкретных правил получения самих формул. В то же время существуют примеры решения данной задачи с использованием методов теории производящих функций для частных случаев контекстно-свободных грамматик [241–243].

Рассмотрим контекстно-свободную грамматику $G = (T, N, S, P)$ и порождаемые ею множества $L_n(G)$ выводимых слов заданной длины n , то есть $L_n(G) = \{\omega \in T^* | S \Rightarrow^* \omega, |\omega| = n\}$. Если последовательно перебрать все возможные значения параметра n , то получаем следующую числовую последовательность:

$$|L_0(G)|, \quad |L_1(G)|, \quad |L_2(G)|, \quad |L_3(G)|, \quad \dots$$

Такую числовую последовательность можно представить в виде производящей функции

$$|L_0(G)| + |L_1(G)|x + |L_2(G)|x^2 + \dots = \sum_{n \geq 0} |L_n(G)|x^n = \sum_{n \geq 0} s(n)x^n = S(x).$$

Таким образом, коэффициенты $s(n)$ производящей функции $S(x)$ показывают количество слов ω длины n , которые выводятся из начального символа S грамматики $G = (T, N, S, P)$. Согласно теореме перечисления Хомского–Шютценберже [244], можно получить функциональное уравнение для производящей функции последовательности чисел, показывающих количество слов заданной длины, выводимых из начального символа однозначной контекстно-свободной грамматики. Для этого необходимо сгруппировать правила вывода с одинаковой левой частью и представить их в виде $A \rightarrow \beta_1|\beta_2|\dots|\beta_m$, где $S \in N$ и $\beta_i \in (N \cup T)^*$. Далее требуется выполнить следующие действия:

- заменить знак ' $\rightarrow$ ' на знак '=';
- заменить знак '|' на знак '+' (сложение);
- заменить конкатенацию терминальных и нетерминальных символов на знак '*' (умножение);
- заменить нетерминальные символы на соответствующие им обозначения производящих функций (например, начальный символ S меняется на производящую функцию $S(x)$);
- заменить каждый терминальный символ ε на 1;
- заменить все остальные терминальные символы на переменную x .

Таким образом, на основе правил вывода однозначной контекстно-свободной грамматики $G = (T, N, S, P)$ формируется система функциональных уравнений с участием производящих функций, коэффициенты которых показывают количество слов заданной длины, выводимых из соответствующего нетерминального символа. При этом количество функциональных уравнений соответствует количеству нетерминальных символов грамматики.

Функциональное уравнение, полученное для начального символа S однозначной контекстно-свободной грамматики $G = (T, N, S, P)$, описывает производящую функцию

$$S(x) = \sum_{n \geq 0} s(n) x^n = \sum_{n \geq 0} |L_n(G)| x^n.$$

Если найти явную формулу для коэффициентов $s(n)$ производящей функции $S(x)$, то получим эффективный способ вычисления мощности множества слов фиксированной длины n , выводимых из начального символа S грамматики $G = (T, N, S, P)$. Следовательно, требуется решение полученного функционального уравнения относительно производящей функции $S(x)$ с последующим

нахождением формул для вычисления ее коэффициентов $s(n)$. Одним из активно применяемых методов решения функциональных уравнений, связанных с производящими функциями и их коэффициентами, является использование формулы обращения Лагранжа [147]. Согласно этой формуле, если имеется функциональное уравнение

$$F(x) = xG(F(x)), \quad (5.4)$$

где

$$F(x) = \sum_{n>0} f(n) x^n, \quad G(x) = \sum_{n\geq 0} g(n) x^n,$$

тогда

$$n[x^n]F(x)^k = k[x^{n-k}]G(x)^n.$$

Кроме того, если использовать коэффициенты k -й степени производящих функций

$$F(x)^k = \sum_{n>0} f(n,k) x^n, \quad G(x)^k = \sum_{n\geq 0} g(n,k) x^n,$$

тогда получим

$$f(n,k) = \frac{k}{n} g(n-k, n).$$

Таким образом, если заданное функциональное уравнение можно преобразовать к виду (5.4), то можем вычислить коэффициенты производящей функции $F(x)$ и ее k -й степени через соответствующие коэффициенты производящей функции $G(x)$ или наоборот. В то же время для нахождения явных формул для коэффициентов производящей функции может быть использован подход на основе композит производящих функций [245]. Согласно этому подходу, заданная производящая функция должна быть разложена на композицию более простых производящих функций, для вычисления коэффициентов которых известны явные формулы. Более того, композиты производящих функций имеют обобщение на случай производящих функций двух и более переменных [156; 157].

Используя комбинацию вышеперечисленных методов, можно получить явные формулы для коэффициентов производящих функций, связанных с однозначными контекстно-свободными грамматиками. В частности, на основе теоремы перечисления Хомского–Шютценберже можно получить функциональное уравнение для производящей функции $S(x)$, связанной с заданной однозначной контекстно-свободной грамматикой $G = (T, N, S, P)$. Если существует аналитическое решение

полученного функционального уравнения, то можно найти явную формулу для коэффициентов производящей функции $S(x)$, разложив ее на композицию более простых и применив правила вычисления композит. В противном случае можно преобразовать функциональное уравнение к виду уравнения (5.4), чтобы далее воспользоваться формулой обращения Лагранжа. В результате получаем формулу для вычисления коэффициентов $s(n)$ через коэффициенты другой производящей функции, заданной замкнутым выражением.

Далее представлено подробное описание предлагаемого метода получения формул для вычисления коэффициентов производящих функций, связанных с однозначными контекстно-свободными грамматиками [246].

Шаг 1. На основе правил вывода заданной однозначной контекстно-свободной грамматики $G = (T, N, S, P)$ построить систему функциональных уравнений для производящих функций, связанных с каждым нетерминальным символом. Полученная система содержит функциональные уравнения, количество которых равно количеству нетерминальных символов в грамматике G .

Шаг 2. Преобразовать систему функциональных уравнений и получить уравнение, включающее только одну производящую функцию $S(x)$.

Шаг 3. Если $s(0) \neq 0$ (то есть допускается вывод пустого слова $S \Rightarrow^* \epsilon$), то сделать замену $S(x) = \frac{R(x)}{x}$ в функциональном уравнении. В противном случае сделать замену $S(x) = R(x)$. Таким образом, получается следующая производящая функция:

$$R(x) = \sum_{n>0} r(n) x^n.$$

Это требование вызвано ограничениями используемых далее математических методов (формула обращения Лагранжа требует $f(0) = 0$ для $F(x)$ в уравнении (5.4)).

Шаг 4. Получить явную формулу для коэффициентов $r(n)$ производящей функции $R(x)$. Возможны следующие два способа:

Способ 1. Решить функциональное уравнение относительно производящей функции $R(x)$. На основе аналитического решения найти явную формулу для коэффициентов производящей функции $R(x)$ (например, применив подход композит производящих функций).

Способ 2. Преобразовать функциональное уравнение в одну из следующих форм и найти явную формулу для коэффициентов производящей функции $R(x)$:

– случай 1: если функциональное уравнение имеет форму $H(R(x)) = x$, где $R(x) = F(x)$ и

$$F(x) = \sum_{n>0} f(n) x^n,$$

$$H(x) = \sum_{n>0} h(n) x^n, \quad H(x)^k = \sum_{n>0} H^\Delta(n, k) x^n,$$

то

$$f(n) = \frac{1}{n} \sum_{i=0}^{n-1} \binom{2n-1}{n+i} \binom{n-1+i}{i} (-1)^i \frac{H^\Delta(n-1+i, i)}{h(1)^{n+i}} \quad (5.5)$$

или

$$f(n) = \frac{1}{n} g(n-1, n), \quad (5.6)$$

где

$$g(n, k) = [x^n] G(x)^k = [x^n] \left(\frac{x}{H(x)} \right)^k.$$

Следовательно,

$$r(n) = [x^n] R(x) = [x^n] F(x) = f(n); \quad (5.7)$$

– случай 2: если функциональное уравнение имеет форму $H(x, R(x)) = x$, где $R(x) = F(x, x)$ и

$$F(x, y) = \sum_{n \geq 0} \sum_{m > 0} f(n, m) x^n y^m,$$

$$H(x, y) = \sum_{n \geq 0} \sum_{m > 0} h(n, m) x^n y^m, \quad H(x, y)^k = \sum_{n \geq 0} \sum_{m > 0} H^\Delta(n, m, k) x^n y^m,$$

то

$$f(n, m) = \frac{1}{m} \sum_{i=0}^{n+m} \binom{n+2m-1}{m+i} \binom{m-1+i}{i} (-1)^i \frac{H^\Delta(n, m-1+i, i)}{h(0, 1)^{m+i}} \quad (5.8)$$

или

$$f(n, m) = \frac{1}{m} g(n, m-1, m), \quad (5.9)$$

где

$$g(n, m, k) = [x^n y^m] G(x, y)^k = [x^n] \left(\frac{y}{H(x, y)} \right)^k.$$

Следовательно,

$$r(n) = [x^n] R(x) = [x^n] F(x, x) = \sum_{i=0}^{n-1} f(i, n-i). \quad (5.10)$$

Шаг 5. Получить явную формулу для коэффициентов $s(n)$ производящей функции $S(x)$:

– если $s(0) = 0$, то $S(x) = R(x)$ и

$$s(n) = r(n); \quad (5.11)$$

– если $s(0) \neq 0$, то $S(x) = \frac{R(x)}{x}$ и

$$s(n) = r(n+1). \quad (5.12)$$

Заметим, что выбор исследователем способа получения явной формулы (Способ 1 или Способ 2). может быть основан на сложности преобразований, которые необходимо выполнить для заданного функционального уравнения. Для этого необходимо сравнить аналитическое решение для производящей функции $R(x)$ с выражением для производящей функции $H(x)$ или производящей функции $G(x) = \frac{x}{H(x)}$. Затем необходимо выбрать из них более подходящий вариант.

Благодаря комбинации формул, указанных в предложенном методе, формируется цепочка зависимостей между коэффициентами производящих функций $S(x)$, $R(x)$, $F(x)$, $G(x)$ и $H(x)$. Получение явной формулы для коэффициентов любой из указанных производящих функций приводит к получению явной формулы для вычисления $s(n)$. Таким образом, становится возможным получение явных формул для вычисления значений $s(n)$, то есть для вычисления количества слов длины n , которые выводятся из начального символа S заданной однозначной контекстно-свободной грамматики $G = (T, N, S, P)$. В отличие от существующих исследований, которые рассматривают проблему перечисления конкретного формального языка, предлагаемый метод является общим методом и может быть применен к любой однозначной контекстно-свободной грамматике.

Также отметим, что формула (5.6) получена с использованием формулы обращения Лагранжа для функционального уравнения вида (5.4). Формула (5.9) получена с помощью формулы обращения Лагранжа для функционального уравнения вида

$$F(x, y) = yG(x, F(x, y)),$$

то есть для фиксированного значения переменной x . При этом формулы (5.5) и (5.8) получены путем применения подход композит производящих функций к производящим функциям $H(x)$ или $H(x, y)$. Далее представлены детали доказательства данных формул.

Теорема 5.1. Для функционального уравнения

$$H(F(x)) = x,$$

где

$$\begin{aligned} F(x) &= \sum_{n>0} f(n) x^n, & F(x)^k &= \sum_{n>0} F^\Delta(n, k) x^n, \\ H(x) &= \sum_{n>0} h(n) x^n, & H(x)^k &= \sum_{n>0} H^\Delta(n, k) x^n, \end{aligned}$$

верно следующее:

$$F^\Delta(n, k) = \frac{k}{n} \sum_{i=0}^{n-k} \binom{2n-k}{n+i} \binom{n-1+i}{i} (-1)^i \frac{H^\Delta(n-k+i, i)}{h(1)^{n+i}}.$$

Доказательство. Рассмотрим следующее функциональное уравнение:

$$F(x) = xG(F(x)),$$

где

$$G(x) = \sum_{n \geq 0} g(n) x^n, \quad G(x)^k = \sum_{n \geq 0} g(n, k) x^n.$$

Согласно формуле обращения Лагранжа имеем

$$F^\Delta(n, k) = [x^n] F(x)^k = \frac{k}{n} g(n-k, n). \quad (5.13)$$

Для случая, когда

$$H(x) = \frac{x}{G(x)},$$

преобразуем исходное функциональное уравнение к виду

$$H(F(x)) = x.$$

Применяя результат теоремы 5 из [156] для

$$H_x(x)G(x) = 1,$$

где

$$H_x(x) = \frac{H(x)}{x} = \sum_{n \geq 0} h_x(n) x^n, \quad H_x(x)^k = \sum_{n \geq 0} h_x(n, k) x^n,$$

получаем

$$g(n, k) = \sum_{i=0}^n \binom{n+k}{k+i} \binom{k-1+i}{i} (-1)^i \frac{h_x(0(n, i))}{h_x(0)^{k+i}}. \quad (5.14)$$

Применяя

$$H_x(x) = \frac{H(x)}{x} = \sum_{n>0} h(n) x^{n-1} = \sum_{n>0} h(n+1, k) x^n = \sum_{n \geq 0} h_x(n) x^n,$$

$$H_x(x)^k = \left(\frac{H(x)}{x} \right)^k = \sum_{n>0} H^\Delta(n, k) x^{n-k} = \sum_{n>0} H^\Delta(n+k, k) x^n = \sum_{n \geq 0} h_x(n, k) x^n$$

и формулу (5.14) для (5.13), получаем желаемый результат. $\square$

Кроме того, выполняется

$$f(n) = F^\Delta(n, 1) = \frac{1}{n} \sum_{i=0}^{n-1} \binom{2n-1}{n+i} \binom{n-1+i}{i} (-1)^i \frac{H^\Delta(n-1+i, i)}{h(1)^{n+i}}.$$

Теорема 5.2. Для функционального уравнения

$$H(x, F(x, y)) = y,$$

где

$$F(x, y) = \sum_{n \geq 0} \sum_{m > 0} f(n, m) x^n y^m, \quad F(x, y)^k = \sum_{n \geq 0} \sum_{m > 0} F^\Delta(n, m, k) x^n y^m,$$

$$H(x, y) = \sum_{n \geq 0} \sum_{m > 0} h(n, m) x^n y^m, \quad H(x, y)^k = \sum_{n \geq 0} \sum_{m > 0} H^\Delta(n, m, k) x^n y^m,$$

верно следующее:

$$F^\Delta(n, m, k) = \frac{k}{m} \sum_{i=0}^{n+m} \binom{n+2m-k}{m+i} \binom{m-1+i}{i} (-1)^i \frac{H^\Delta(n, m-k+i, i)}{h(0, 1)^{m+i}}.$$

Доказательство. Доказательство аналогично доказательству теоремы 6 в [156], где рассматривается случай функционального уравнения

$$H(F(x, y), y) = x.$$

Однако в данной ситуации необходимо рассмотреть замену переменных x и y друг на друга. $\square$

Кроме того, выполняется

$$f(n, m) = F^\Delta(n, m, 1) = \frac{1}{m} \sum_{i=0}^{n+m} \binom{n+2m-1}{m+i} \binom{m-1+i}{i} (-1)^i \frac{H^\Delta(n, m-1+i, i)}{h(0, 1)^{m+i}}.$$

Обратим внимание, что для $y = x$ имеем

$$H(x, F(x, x)) = x,$$

то есть получаем Случай 2 предлагаемого метода.

Для проверки предлагаемого метода далее приведены несколько примеров нахождения явных формул для подсчета количества слов заданной длины, которые выводятся из начального символа однозначной контекстно-свободной грамматики.

Пример 1. Рассмотрим следующую однозначную контекстно-свободную грамматику:

$$G = (T, N, S, P), \quad T = \{ (,), \varepsilon \}, \quad N = \{ S \}, \quad P = \{ S \rightarrow (S)S \mid \varepsilon \}.$$

В таблице 5.2 представлены примеры для нескольких первых значений $s(n)$ и соответствующее множество слов.

Таблица 5.2 — Пример значений $s(n)$ и соответствующее множество слов

n	$s(n)$	Множество слов
0	1	ε
1	0	
2	1	$()$
3	0	
4	2	$()(), (())$
5	0	
6	5	$()()(), ()(()), (()()), ((())), (((()))$

На основе правил вывода $S \rightarrow (S)S \mid \varepsilon$ получаем следующую рекуррентную формулу для подсчета количества слов заданной длины n , которые выводятся из рассматриваемой грамматики:

$$s(n) = \begin{cases} 0 & n < 0 \text{ или } n \text{ нечетное;} \\ 1 & n = 0; \\ \sum_{i=0}^{n-2} s(i) \cdot s(n-2-i) & \text{иначе.} \end{cases} \quad (5.15)$$

Однако использование данной рекуррентной формулы неэффективно, поскольку она имеет экспоненциальную сложность. Далее применим предложенный метод для получения явной формулы.

Шаг 1: На основе правил вывода $S \rightarrow (S)S \mid \varepsilon$ получаем следующее функциональное уравнение:

$$S(x) = x^2 S(x)^2 + 1. \quad (5.16)$$

Шаг 2: Полученное функциональное уравнение включает в себя только одну производящую функцию $S(x)$. Поэтому никаких дополнительных преобразований не требуется.

Шаг 3: Поскольку $s(0) \neq 0$, то делаем замену $S(x) = \frac{R(x)}{x}$ в функциональном уравнении (5.16) и получаем

$$\frac{R(x)}{x} = x^2 \left(\frac{R(x)}{x} \right)^2 + 1$$

или

$$xR(x)^2 - R(x) + x = 0. \quad (5.17)$$

Шаг 4 (Способ 1): Аналитическое решение для функционального уравнения (5.17) относительно $R(x)$ имеет вид

$$R(x) = \frac{1 - \sqrt{1 - 4x^2}}{2x} = x + x^3 + 2x^5 + 5x^7 + 14x^9 + \dots = \sum_{n>0} r(n) x^n. \quad (5.18)$$

Далее найдем коэффициенты $r(n)$ производящей функции $R(x)$ на основе выражения (5.18). Например, производящая функция $R(x)$ может быть представлена в виде следующей композиции производящих функций:

$$R(x) = xC(A(x)), \quad (5.19)$$

где

$$C(x) = \sum_{n \geq 0} C_n x^n = \frac{1 - \sqrt{1 - 4x}}{2x}, \quad C_n = \frac{1}{n+1} \binom{2n}{n},$$

$$A(x) = \sum_{n \geq 0} a(n) x^n = x^2, \quad a(n) = \delta(n, 2).$$

В указанных производящих функциях используются следующие обозначения: C_n — число Каталана, $\delta(i, j)$ — символ Кронекера.

Композиата производящей функции $A(x)$ равна

$$A^\Delta(n, k) = [x^n] A(x)^k = [x^n] (x^2)^k = [x^n] x^{2k} = \delta(n, 2k).$$

Согласно подходу композит производящих функций, коэффициенты $R(x)$, представленные в виде композиции (5.19), можно вычислить по формуле

$$r(n) = \sum_{k=1}^{n-1} A^{\Delta}(n-1, k) C_k = \sum_{k=1}^{n-1} \delta(n-1, 2k) C_k = \begin{cases} C_{(n-1)/2} & n \text{ нечетное;} \\ 0 & \text{иначе.} \end{cases}$$

Шаг 5: Следовательно, применяя формулу (5.12), получаем

$$s(n) = r(n+1) = \begin{cases} 0 & n \text{ нечетное;} \\ C_{n/2} & \text{иначе.} \end{cases}$$

Далее рассмотрим альтернативный способ получения явной формулы для $s(n)$, описанный в предлагаемом методе.

Шаг 4 (Способ 2): Преобразуем функциональное уравнение (5.17):

$$\frac{R(x)}{1 + R(x)^2} = x.$$

Следовательно, получаем $H(R(x)) = x$ (случай 1), где

$$H(x) = \sum_{n>0} h(n) x^n = \frac{x}{1 + x^2}.$$

Для упрощения вычислений введем производящую функцию

$$G(x) = \frac{x}{H(x)} = 1 + x^2.$$

Используя биномиальную теорему, выводим следующее выражение для ее k -й степени:

$$G(x)^k = \sum_{n \geq 0} g(n, k) x^n = (1 + x^2)^k = \sum_{j \geq 0} \binom{k}{j} x^{2j}.$$

Данное выражение также можно переписать следующим образом:

$$g(n, k) = \begin{cases} 0 & n \text{ нечетное;} \\ \binom{k}{\frac{n}{2}} & \text{иначе.} \end{cases} \quad (5.20)$$

Шаг 5: Объединяя формулу (5.20) с (5.6), (5.7) и (5.12), получаем следующую явную формулу для вычисления значений $s(n)$:

$$s(n) = r(n+1) = \frac{1}{n+1} g(n, n+1) = \begin{cases} 0 & n \text{ нечетное;} \\ \frac{1}{n+1} \binom{n+1}{\frac{n}{2}} & \text{иначе.} \end{cases} \quad (5.21)$$

С целью экспериментального подтверждения эффективности полученных явных формул по сравнению с рекуррентной был проведен вычислительный эксперимент. Для этого были созданы программные реализации соответствующих формул для $s(n)$ на языке системы компьютерной алгебры «Maxima» [212]. Запуск программных реализаций осуществлялся на ноутбуке со следующими характеристиками: Intel i7-9750H, 2,6 ГГц, Windows 10, 64 бит. На рисунке 5.6 показаны результаты измерения зависимости среднего значения времени вычисления значения $s(n)$ от параметра n . Здесь наглядно видно экспоненциальную зависимость для рекуррентной формулы и полиномиальную для явной формулы.

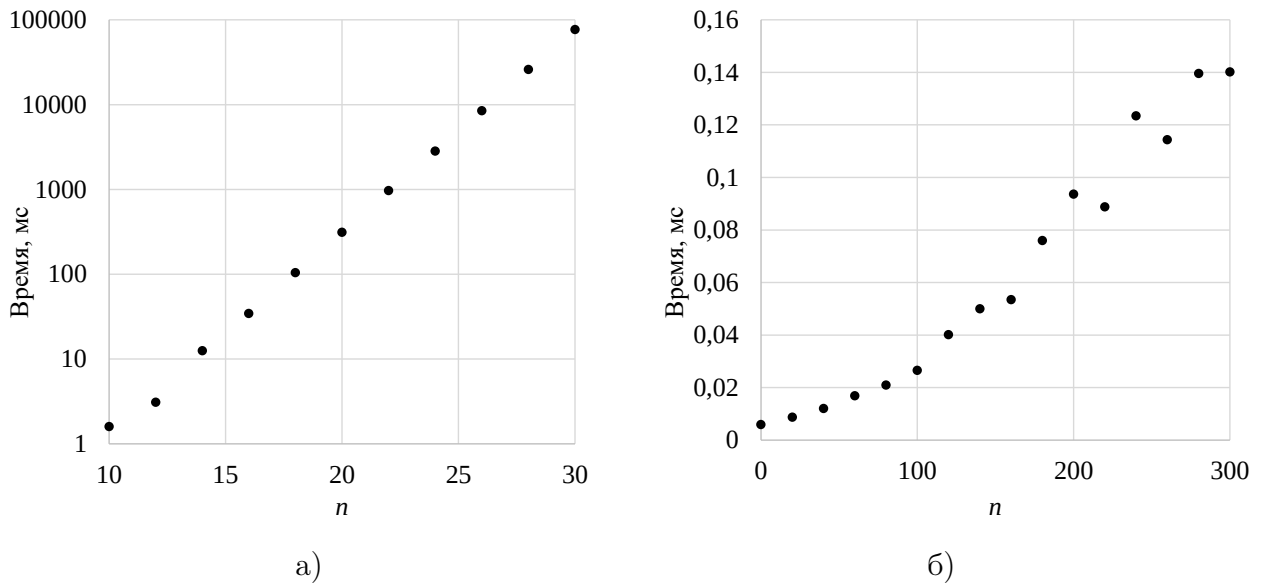


Рисунок 5.6 — Среднее время вычисления значения $s(n)$ от параметра n :
а) на основе формулы (5.15); б) на основе формулы (5.21)

Пример 2. Рассмотрим следующую контекстно-свободную грамматику:

$$G = (T, N, S, P), \quad T = \{ (,), [,], \varepsilon \}, \quad N = \{ S, M \},$$

$$P = \{ S \rightarrow (S)S \mid [M]S \mid \varepsilon, M \rightarrow (M \mid (S)M \mid [M]M \mid \varepsilon \}.$$

В таблице 5.3 представлены примеры для нескольких первых значений $s(n)$ и соответствующее множество слов.

Шаг 1: На основе правил вывода $S \rightarrow (S)S \mid [M]S \mid \varepsilon$ получаем следующее функциональное уравнение:

$$S(x) = x^2 S(x)^2 + x^2 M(x) S(x) + 1.$$

На основе правил вывода $M \rightarrow (M \mid (S)M \mid [M]M \mid \varepsilon$ получаем следующее функциональное уравнение:

$$M(x) = x M(x) + x^2 S(x) M(x) + x^2 M(x)^2 + 1.$$

Таблица 5.3 — Пример значений $s(n)$ и соответствующее множество слов

n	$s(n)$	Множество слов
0	1	ε
1	0	
2	2	$()$, $[]$
3	1	$[()$
4	9	$()()$, $[]()$, $()[]$, $[] []$, $(())$, $([])$, $[()]$, $[[[]]$, $[(()]$

Таким образом, имеем систему двух функциональных уравнений с двумя производящими функциями

$$\begin{cases} S(x) = x^2 S(x)^2 + x^2 M(x) S(x) + 1; \\ M(x) = x M(x) + x^2 S(x) M(x) + x^2 M(x)^2 + 1. \end{cases}$$

Шаг 2: Преобразуем функциональные уравнения в одно уравнение, включающее только одну производящую функцию $S(x)$:

$$x^3 S(x)^3 - (2x^2 + x) S(x)^2 + (x + 1) S(x) - 1 = 0. \quad (5.22)$$

Шаг 3: Поскольку $s(0) \neq 0$, делаем замену $S(x) = \frac{R(x)}{x}$ в функциональном уравнении (5.22) и получаем

$$x^3 \left(\frac{R(x)}{x} \right)^3 - (2x^2 + x) \left(\frac{R(x)}{x} \right)^2 + (x + 1) \left(\frac{R(x)}{x} \right) - 1 = 0$$

или

$$x R(x)^3 - (2x + 1) R(x)^2 + (x + 1) R(x) - x = 0 \quad (5.23)$$

Шаг 4 (Способ 1): Аналитическое решение для функционального уравнения (5.23) относительно $R(x)$ имеет вид

$$\begin{aligned} R(x) = & \left(\frac{\sqrt{-3} - 1}{2} \right) \left(\frac{\sqrt{23x^4 - 6x^3 + 5x^2 + 2x - 1}}{2\sqrt{27}x^2} + \frac{25x^3 - 3x^2 + 3x + 2}{54x^3} \right)^{\frac{1}{3}} - \\ & - \left(\frac{\sqrt{-3} + 1}{2} \right) \left(\frac{\sqrt{23x^4 - 6x^3 + 5x^2 + 2x - 1}}{2\sqrt{27}x^2} + \frac{25x^3 - 3x^2 + 3x + 2}{54x^3} \right)^{-\frac{1}{3}} \times \\ & \times \left(\frac{x^2 + x + 1}{9x^2} \right) + \frac{2x + 1}{3x} = x + x^3 + 2x^5 + 5x^7 + 14x^9 + \dots = \sum_{n>0} r(n) x^n. \end{aligned}$$

Это решение было получено с использованием системы компьютерной алгебры «Matha». Ввиду сложности полученного выражения, получение явной формулы для коэффициентов $r(n)$ производящей функции $R(x)$ является трудной задачей. Поэтому рассмотрим альтернативный способ получения явной формулы для $s(n)$, описанный в предлагаемом методе.

Шаг 4 (Способ 2): Преобразуем функциональное уравнение (5.23):

$$\frac{R(x)^2 - R(x)}{R(x)^3 - 2R(x)^2 + R(x) - 1} = x.$$

Следовательно, получаем $H(R(x)) = x$ (случай 1), где

$$H(x) = \sum_{n>0} h(n) x^n = \frac{x^2 - x}{x^3 - 2x^2 + x - 1}.$$

Для упрощения вычислений введем производящую функцию

$$G(x) = \frac{x}{H(x)} = \frac{x^3 - 2x^2 + x - 1}{x - 1} = x(x - 1) + \frac{1}{1 - x}.$$

Используя биномиальную теорему, выводим следующее выражение для ее k -й степени:

$$G(x)^k = \sum_{n \geq 0} g(n, k) x^n = \left(x(x - 1) + \frac{1}{1 - x} \right)^k = \sum_{j=0}^k \binom{k}{j} A(x)^j B(x)^{k-j},$$

где

$$A(x) = x(x - 1), \quad A^\Delta(n, k) = [x^n] A(x)^k = [x^n] (x(x - 1))^k = (-1)^n \binom{k}{n - k},$$

$$B(x) = \frac{1}{1 - x}, \quad b(n, k) = [x^n] B(x)^k = [x^n] \left(\frac{1}{1 - x} \right)^k = \binom{n + k - 1}{k - 1}.$$

Согласно подходу композит производящих функций, коэффициенты $g(n, k)$ для $G(x) = A(x) + B(x)$ можно вычислить по формуле

$$g(n, k) = \sum_{j=0}^k \binom{k}{j} \sum_{i=j}^n A^\Delta(i, j) b(n - i, k - j). \quad (5.24)$$

Шаг 5: Объединяя формулу (5.24) с (5.6), (5.7) и (5.12), получаем следующую формулу для вычисления значений $s(n)$:

$$s(n) = \frac{1}{n + 1} \sum_{j=0}^n \binom{n + 1}{j} \sum_{i=j}^n (-1)^i \binom{j}{i - j} \binom{2n - i - j}{n - j}.$$

Пример 3. Рассмотрим следующую контекстно-свободную грамматику:

$$G = (T, N, S, P), \quad T = \{+, \times, (,), a\}, \quad N = \{S, M, E\},$$

$$P = \{S \rightarrow S + M \mid M, M \rightarrow M \times E \mid E, E \rightarrow a \mid (S)\}.$$

Шаг 1: На основе правил вывода $S \rightarrow S + M \mid M$ получаем следующее функциональное уравнение:

$$S(x) = xS(x)M(x) + M(x). \quad (5.25)$$

На основе правил вывода $M \rightarrow M \times E \mid T$ получаем следующее функциональное уравнение:

$$M(x) = xM(x)E(x) + E(x). \quad (5.26)$$

На основании правил вывода $E \rightarrow a \mid (S)$ получаем следующее функциональное уравнение:

$$E(x) = x + x^2S(x). \quad (5.27)$$

Таким образом, имеем систему трех функциональных уравнений с тремя производящими функциями

$$\begin{cases} S(x) = xS(x)M(x) + M(x); \\ M(x) = xM(x)E(x) + E(x); \\ E(x) = x + x^2S(x). \end{cases}$$

Шаг 2: Преобразуем функциональные уравнения в одно уравнение, включающее только одну производящую функцию $S(x)$. Для этого выразим $M(x)$ из уравнения (5.26) через $E(x)$ и подставим $E(x)$ из уравнения (5.27) в полученное выражение:

$$M(x) = \frac{E(x)}{1 - xE(x)} = \frac{x + x^2S(x)}{1 - x^2 - x^3S(x)}. \quad (5.28)$$

Применяя формулу (5.28) к уравнению (5.25), после упрощения получаем следующее функциональное уравнение, включающее только одну производящую функцию $S(x)$:

$$2x^3S(x)^2 + (3x^2 - 1)S(x) + x = 0. \quad (5.29)$$

Шаг 3: Поскольку $s(0) = 0$ (невозможно вывести пустое слово), делаем замену $S(x) = R(x)$ в функциональном уравнении (5.29) и получаем

$$2x^3 R(x)^2 + (3x^2 - 1)R(x) + x = 0. \quad (5.30)$$

Шаг 4 (Способ 1): Аналитическое решение для функционального уравнения (5.30) относительно $R(x)$ имеет вид

$$R(x) = \frac{1 - 3x^2 - \sqrt{x^4 - 6x^2 + 1}}{4x^3} = x + 3x^3 + 11x^5 + 45x^7 + \dots = \sum_{n \geq 0} r(n) x^n.$$

Предположим, что декомпозиция производящей функции $R(x)$ на более простые функции является трудной задачей. Тогда рассмотрим альтернативный способ получения явной формулы для $s(n)$, описанный в предлагаемом методе.

Шаг 4 (Способ 2): Преобразуем функциональное уравнение (5.30):

$$(1 - 3x^2)R(x) - 2x^3 R(x)^2 = x.$$

Следовательно, получаем $H(x, R(x)) = x$ (случай 2), где

$$H(x, y) = \sum_{n > 0} \sum_{m > 0} h(n, m) x^n y^m = (1 - 3x^2)y - 2x^3 y^2.$$

Используя биномиальную теорему, выводим следующее выражение для композиты производящей функции $H(x, y)$:

$$\begin{aligned} H(x, y)^k &= \sum_{n > 0} \sum_{m > 0} H^\Delta(n, m, k) x^n y^m = y^k ((1 - 3x^2) - 2x^3 y)^k = \\ &= y^k \sum_{n \geq 0} \binom{k}{n} (1 - 3x^2)^n (-2x^3 y)^{k-n} = y^k \sum_{n \geq 0} \binom{k}{n} \sum_{m \geq 0} \binom{n}{m} (-3x^2)^m (-2x^3 y)^{k-n} = \\ &= \sum_{n \geq 0} \sum_{m \geq 0} \binom{k}{n} \binom{n}{m} (-3)^m (-2)^{k-n} x^{2m+3(k-n)} y^{2k-n}. \end{aligned}$$

Это также можно переписать следующим образом:

$$\begin{aligned} &H^\Delta(n, m, k) = \\ &= \begin{cases} 0 & (n - m + k) \text{ нечетное;} \\ 1 & n = m = k = 0; \\ \binom{k}{m-k} \binom{2k-m}{\frac{-n+m+k}{2}} (-3)^{\frac{n-m+k}{2}} \left(\frac{2}{3}\right)^{m-k} & \text{иначе.} \end{cases} \quad (5.31) \end{aligned}$$

Шаг 5: Объединяя формулу (5.31) с (5.8), (5.10) и (5.11), получаем формулу для вычисления значений $s(n)$.

5.5 Выводы по главе

В данной главе изложены результаты апробации предложенной методологии на примере построения алгоритмов кодирования информационных объектов, представленных словами контекстно-свободных грамматик. Используя биективное отображение деревьев вывода слов контекстно-свободных грамматик на структуры вариантов деревьев И/ИЛИ, предложен метод кодирования слов контекстно-свободных грамматик, который характеризуется следующими преимуществами:

1. Использование структур деревьев И/ИЛИ для построения алгоритмов комбинаторной генерации обеспечивает универсальность применимости предложенного метода к любой контекстно-свободной грамматике, а также позволяет учитывать сразу несколько параметров. Например, в работах [203; 247; 248] представлены исследования по разработке алгоритмов ранжирования и генерации по рангу для языка Дика (аналог разработанных алгоритмов 5.1 и 5.2), но полученные результаты носят частный характер и не могут быть применены к другим примерам контекстно-свободных грамматик. Использование других универсальных методов по разработке алгоритмов комбинаторной генерации (символьный метод или метод на основе префиксов слов) позволяет получать алгоритмы ранжирования и генерации по рангу для языка Дика с полиномиальной вычислительной сложностью [135], но при этом отсутствует возможность учета дополнительных количественных параметров помимо длины генерируемых слов. Данное ограничение отсутствует у предложенного метода, что подтверждается разработанными алгоритмами 5.3 и 5.4 и является его основным преимуществом.

2. Предложенный метод позволяет обрабатывать как однозначные, так и неоднозначные контекстно-свободные грамматики. Если для произвольной контекстно-свободной грамматики построить соответствующую структуру дерева И/ИЛИ, то каждому варианту такого дерева И/ИЛИ будет сопоставлено только одно дерево вывода. В случае однозначной контекстно-свободной грамматики для каждого слова существует только одно дерево вывода, а значит и только один вариант дерева И/ИЛИ. В случае неоднозначной контекстно-свободной грамматики одному слову может соответствовать сразу несколько деревьев вывода, а значит и сразу несколько вариантов дерева И/ИЛИ. Этот факт не нарушает возможность восстановления исходного слова, что является обязательным при реализации кодирования без потерь.

3. Получаемые алгоритмы комбинаторной генерации имеют полиномиальную временную сложность при условии наличия алгоритма вычисления значений $|A_n|$ для всех правил вывода вида $A_n \rightarrow \beta$ с полиномиальной вычислительной сложностью в зависимости от n . При этом увеличение самих значений $|A_n|$ (в том числе их экспоненциальный рост в зависимости от n) влияет только на требование к обработке больших целых чисел размера $\log_2 |A_n|$ бит и не изменяет класс полиномиальной сложности алгоритмов ранжирования и генерации по рангу.

4. Применение алгоритма генерации по рангу для случая однозначной контекстно-свободной грамматики обеспечивает равномерное распределение случайно генерируемых слов. Если преобразовать соответствующую структуру дерева И/ИЛИ путем дублирования некоторых узлов-потомков ИЛИ-узлов и их поддеревьев, то в модифицированной структуре появятся разные варианты, генерирующие одни и те же слова. Таким образом, можно обеспечить неравномерность случайной генерации слов. Применение алгоритмов генерации по рангу для случая неоднозначной контекстно-свободной грамматики реализует неравномерное распределение случайно генерируемых слов. В таком случае, чем больше различных деревьев вывода имеется для вывода конкретного слова, тем больше вероятность его случайной генерации.

Получаемые с помощью предложенного метода алгоритмы кодирования слов контекстно-свободных грамматик могут найти свое применение при решении задач сжатия данных и моделирования объектов, описываемых словами формальных языков.

Кроме того, корректная работа получаемых алгоритмов кодирования слов контекстно-свободных грамматик требует наличия формул для подсчета количества элементов в соответствующем комбинаторном множестве. Для решения данной задачи разработан метод получения явных формул для подсчета количества слов длины n , выводимых из заданного нетерминального символа однозначной контекстно-свободной грамматики. Данный метод дополняет метод кодирования информационных объектов, строящихся по правилам вывода контекстно-свободных грамматик, в части решения задач перечисления комбинаторных множеств. Отличительной особенностью предложенного метода является объединение методов, основанных на формуле обращения Лагранжа и композитах производящих функций, а также их применение в области контекстно-свободных грамматик.

Результаты данной главы опубликованы в [156; 225; 237; 246].

Глава 6. Блочное кодирование цифровых данных

В данной главе представлены результаты разработки метода блочного кодирования цифровых данных, который базируется на получении структурированного представления кодируемого информационного объекта в виде последовательности блоков, каждый из которых формируется кортежем атрибутов. Рассматриваются примеры соответствующих информационных объектов, а именно: текстовые файлы, растровые изображения, журналы событий информационных систем и реляционные базы данных. Решается задача разбиения объекта на блоки и их декомпозиции, а также задача перечисления соответствующих комбинаторных множеств. В качестве апробации рассматривается кодирование файлов с реальными данными, исследуется эффективность сжатия.

6.1 Общая схема блочного кодирования на основе алгоритмов комбинаторной генерации

Хранение информационных объектов в виде цифровых данных требует, чтобы объект строился путем выполнения последовательности конкретных инструкций, составленных с использованием набора допустимых вычислительных операций. В параграфе 1.1 показано, что такие информационные объекты (текстовые данные, растровые изображения, видеозаписи, аудиозаписи, реляционные базы данных) представляют собой дискретные структуры, а фиксирование их параметров формирует комбинаторное множество всех возможных структур. Следовательно, информационный объект может быть закодирован с помощью алгоритмов комбинаторной генерации. Однако цифровые данные характеризуются большим объемом, что при выполнении алгоритма ранжирования потребует работы с большими целыми числами, двоичное представление которых сопоставимо с размером кодируемых данных. Так как информационный объект представляет собой последовательность более простых подструктур, то можно выполнить ранжирование каждой из них. В результате получается закодированный объект в виде последовательности рангов, что соответствует идее блочного кодирования.

Используя методологию построения алгоритмов кодирования на основе рекурсивных композиций структур деревьев И/ИЛИ, предлагается метод блочного кодирования цифровых данных, который заключается в выполнении следующих шагов:

Шаг 1. Разбиение массива цифровых данных на отдельные блоки, размер которых является допустимым для обработки цифровым устройством. Принцип разбиения на блоки может основываться на фиксировании размера блока (например, каждый блок состоит из заданного количества бит) или соответствовать логике построения дискретной структуры, то есть выполняется декомпозиция сложных конструкций на комбинацию более простых.

Шаг 2. Декомпозиция блока на комбинацию элементарных составляющих (атрибуты блока). При этом декомпозиция должна быть реализована единым образом для каждого блока, что позволит в дальнейшем использовать единый алгоритм ранжирования. В результате получаем структурированное представление исходных данных в виде последовательности блоков, то есть

$$\text{Данные} = (\text{Блок}_1, \text{Блок}_2, \dots, \text{Блок}_n),$$

каждый из которых описывается кортежем атрибутов:

$$\text{Блок}_i = \langle \text{Атрибут}_1, \text{Атрибут}_2, \dots, \text{Атрибут}_m \rangle.$$

Шаг 3. Составление справочников уникальных значений для каждого атрибута на основе анализа содержимого всех блоков. В результате получаем, что каждый атрибут принимает значение из фиксированного множества, то есть $\text{Атрибут}_j \in \{\text{Значение}_1, \text{Значение}_2, \dots, \text{Значение}_{k_j}\}$, где значение k_j показывает мощность множества уникальных значений j -го атрибута.

Шаг 4. Определение комбинаторного множества всех возможных блоков для заданного набора справочников уникальных значений атрибутов. Так как блок формируется кортежем m атрибутов, которые принимают значения из фиксированных множеств (справочников уникальных значений атрибутов), то получаем комбинаторное множество $T_{m,k_1,\dots,k_m}$, которое содержит все возможные для построения блоки. Мощность комбинаторного множества $T_{m,k_1,\dots,k_m}$ определяется следующей формулой:

$$|T_{m,k_1,\dots,k_m}| = k_1 \cdot k_2 \cdot \dots \cdot k_m = \prod_{j=1}^m k_j. \quad (6.1)$$

Шаг 5. Построение структуры дерева И/ИЛИ для комбинаторного множества $T_{m,k_1,\dots,k_m}$. Так как функция мощности комбинаторного множества удовлетворяет требуемой алгебре, то получаем соответствующую структуру дерева И/ИЛИ (рисунок 6.1).

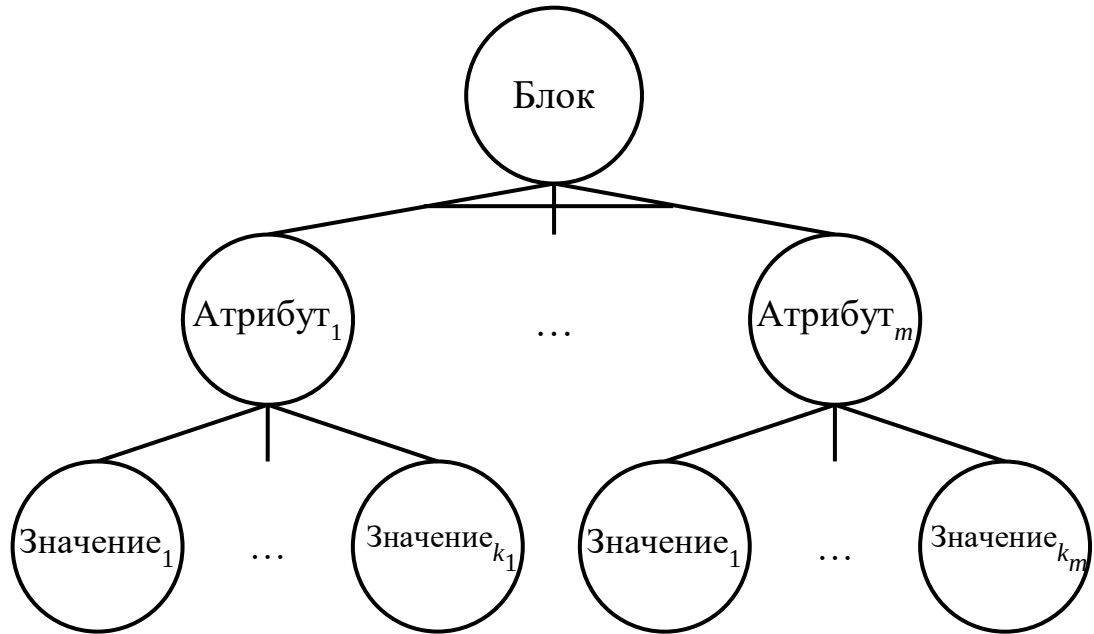


Рисунок 6.1 — Общий вид структуры дерева И/ИЛИ для кодируемого блока

Шаг 6. Вычисление ранга для кортежа идентификаторов атрибутов каждого блока. Для ранжирования варианта дерева И/ИЛИ, представленного на рисунке 6.1, требуется знать порядковый номер выбранного узла-потомка для каждого ИЛИ-узла. Это действие можно реализовать путем получения идентификатора выбранного значения для каждого атрибута, где идентификатором выступает порядковый номер значения атрибута среди упорядоченного множества его уникальных значений. В результате для каждого блока получаем вычисленное значение ранга, а исходные данные кодируются в виде последовательности рангов

$$\text{Данные} = (\text{Блок}_1, \text{Блок}_2, \dots, \text{Блок}_n) \rightarrow (\text{Ранг}_1, \text{Ранг}_2, \dots, \text{Ранг}_n).$$

Предложенный метод кодирования эффективно использовать для информационных объектов, которые характеризуются высокой степенью дублирования содержащейся в них информации. В таком случае получаем значительно меньшее количество уникальных значений атрибутов блока по сравнению с количеством всех кодируемых блоков. В свою очередь, это приводит к сокращению разнообразия множества всех возможных блоков и, следовательно, требует меньших ресурсов компьютерной памяти для хранения рангов блоков.

Алгоритм 6.1 описывает процесс вычисления ранга при кодировании блока данных, сформированного кортежем m атрибутов, на основе структуры дерева И/ИЛИ, представленной на рисунке 6.1. Алгоритм 6.2 описывает обратный процесс декодирования, который восстанавливает исходный блок данных. В данных алгоритмах нумерация значений в упорядоченных справочниках начинается с 1.

Алгоритм 6.1: Алгоритм кодирования блока данных, сформированного кортежем m атрибутов, на основе структуры дерева И/ИЛИ, представленной на рисунке 6.1

```

1 Encode(< Атрибут1, ..., Атрибутm >, {Справочник1, ..., Справочникm})
2 begin
3   Ранг := 0
4   for  $j := m$  to 1 by -1 do
5      $w :=$  Количество уникальных значений в Справочникj
6      $l :=$  Порядковый номер значения Атрибутj в Справочникj
7     Ранг :=  $(l - 1) + w \cdot \text{Ранг}$ 
8   end
9   return Ранг
10 end
```

Алгоритм 6.2: Алгоритм декодирования блока данных, реализующий обратные преобразования к действиям алгоритма 6.1

```

1 Decode(Ранг, {Справочник1, ..., Справочникm})
2 begin
3   for  $j := 1$  to  $m$  do
4      $w :=$  Количество уникальных значений в Справочникj
5      $l := \text{Ранг} \bmod w$ 
6     Атрибутj := Значение с порядковым номером  $(l + 1)$  в Справочникj
7     Ранг :=  $\lfloor \frac{\text{Ранг}}{w} \rfloor$ 
8   end
9   Блок := < Атрибут1, ..., Атрибутm >
10  return Блок
11 end
```

6.2 Структуры деревьев И/ИЛИ для блочного кодирования

Если для всех m атрибутов используется единый справочник, состоящий из k уникальных значений атрибутов, то есть

$$\text{Справочник} = \{\text{Значение}_1, \text{Значение}_2, \dots, \text{Значение}_k\}$$

и $|\text{Справочник}| = k$, тогда получаем частный случай в виде комбинаторного множества $T_{m,k}$, мощность которого определяется следующей формулой:

$$|T_{m,k}| = \prod_{j=1}^m k = k^m. \quad (6.2)$$

На рисунке 6.2 показан результат соответствующих преобразований структуры дерева И/ИЛИ, представленной на рисунке 6.1, при использовании формулы (6.2).

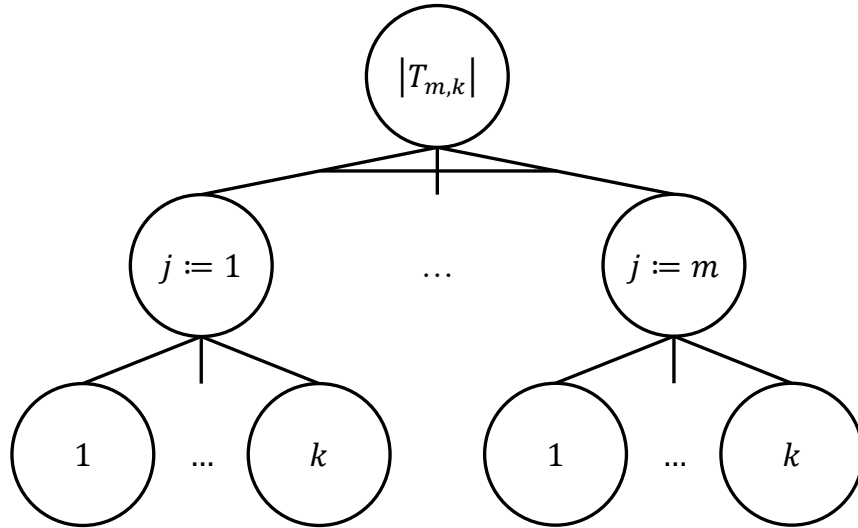


Рисунок 6.2 — Структура дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.2)

С точки зрения построения биективного отображения между множеством блоков данных и множеством вариантов дерева И/ИЛИ для $|T_{m,k}|$, представленного на рисунке 6.2, используются следующие правила:

- поддерево ИЛИ-узла, помеченного j , определяет значение Атрибут $_j$ в кортеже $\langle \text{Атрибут}_1, \dots, \text{Атрибут}_m \rangle$;
- выбор потомка ИЛИ-узла, помеченного j , показывает порядковый номер значения справочника, которое является значением для Атрибут $_j$.

Для построенной структуры дерева И/ИЛИ также разработаны алгоритмы ранжирования и генерации по рангу для множества его вариантов. В данных алгоритмах вариант v дерева И/ИЛИ определяется указателем на его корневой узел, при этом каждый узел содержит список указателей на узлы-потомки и каждый узел может иметь метку.

Алгоритм 6.3: Алгоритм ранжирования варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.2)

```

1 Rank_Variant( $v, m, k$ )
2 begin
3    $(v_1, \dots, v_m) :=$  Получить указатели на узлы-потомки для узла  $v$ 
4    $r := 0$ 
5   for  $j := m$  to 1 by  $-1$  do
6      $l :=$  Получить значение метки узла-потомка для узла  $v_j$ 
7      $r := (l - 1) + k \cdot r$ 
8   end
9   return  $r$ 
10 end
```

Алгоритм 6.4: Алгоритм генерации по рангу варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.2)

```

1 Unrank_Variant( $r, m, k$ )
2 begin
3   Создать И-узел  $v$ 
4   for  $j := 1$  to  $m$  do
5     Создать ИЛИ-узел  $v_j$ 
6     Добавить узел-потомок  $v_j$  для узла  $v$ 
7      $l := r \bmod k$ 
8     Добавить узел-потомок с меткой  $(l + 1)$  для узла  $v_j$ 
9      $r := \lfloor \frac{r}{k} \rfloor$ 
10  end
11  return  $v$ 
12 end
```

Так как $k^m = k \cdot k^{m-1}$, поэтому формулу (6.2) также можно преобразовать в рекуррентную формулу следующего вида:

$$|T_{m,k}| = \begin{cases} k \cdot |T_{m-1,k}|, & m > 0 \text{ и } k > 0; \\ 1, & m = 0 \text{ и } k \geq 0; \\ 0, & \text{иначе.} \end{cases} \quad (6.3)$$

Полученная рекуррентная формула (6.3) позволяет построить рекурсивную композицию деревьев И/ИЛИ для $|T_{m,k}|$, представленную на рисунке 6.3.

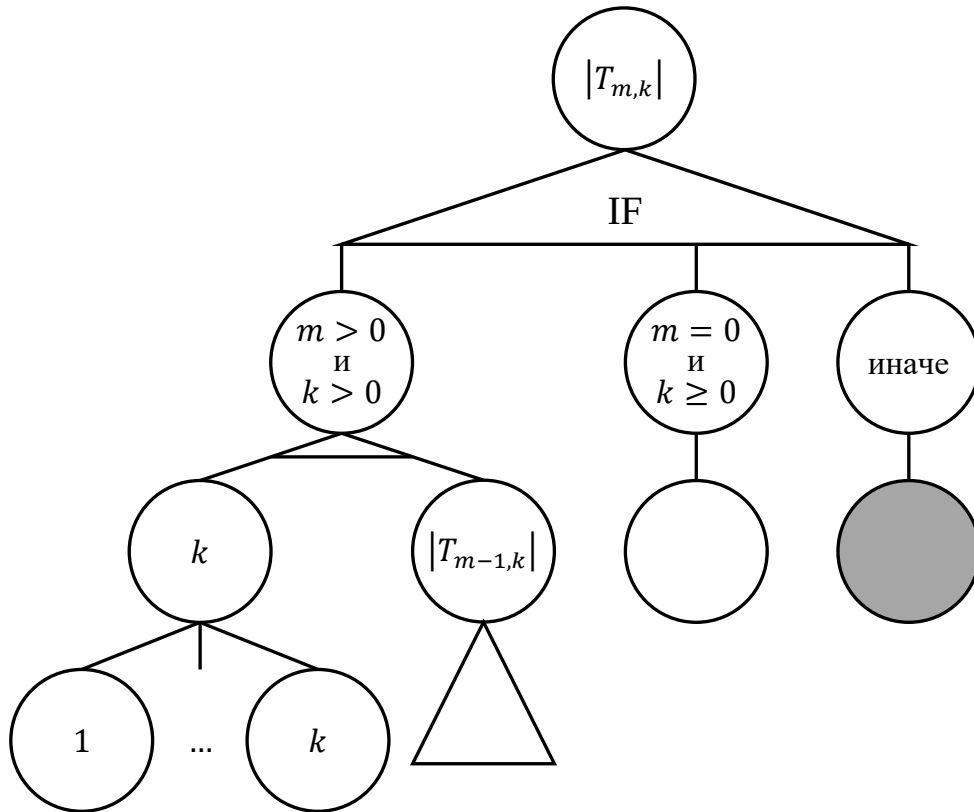


Рисунок 6.3 — Структура дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.3)

С точки зрения построения биективного отображения между множеством блоков данных и множеством вариантов дерева И/ИЛИ для $|T_{m,k}|$, представленного на рисунке 6.3, используются следующие правила:

- выбор потомка ИЛИ-узла, помеченного k , показывает порядковый номер значения справочника, которое является значением для Атрибут _{m} ;
- поддереву узла, помеченного $|T_{m-1,k}|$, определяет значение атрибутов в кортеже $\langle \text{Атрибут}_1, \dots, \text{Атрибут}_{m-1} \rangle$;
- достижение узла, помеченного $|T_{0,k}|$, соответствует пустому кортежу и указывает на завершение рекурсивных вызовов.

Для построенной структуры дерева И/ИЛИ также разработаны алгоритмы ранжирования и генерации по рангу для множества его вариантов. Расчеты в этих алгоритмах идентичны расчетам в алгоритмах 6.3 и 6.4 и различаются только в рекурсивной организации цикла и в обратном порядке просмотра атрибутов.

Алгоритм 6.5: Алгоритм ранжирования варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.3)

```

1 Rank_Variant( $v, m, k$ )
2 begin
3   ( $v_1, v_2$ ) := Получить указатели на узлы-потомки для узла  $v$ 
4    $l_1$  := Получить значение метки узла-потомка для узла  $v_1$ 
5    $l_2$  := Rank_Variant( $v_2, m - 1, k$ )
6    $r := (l_1 - 1) + k \cdot l_2$ 
7   return  $r$ 
8 end
```

Алгоритм 6.6: Алгоритм генерации по рангу варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.3)

```

1 Unrank_Variant( $r, m, k$ )
2 begin
3   Создать И-узел  $v$ 
4   Создать ИЛИ-узел  $v_1$ 
5   Добавить узел-потомок  $v_1$  для узла  $v$ 
6    $l_1 := r \bmod k$ 
7   Добавить узел-потомок с меткой  $(l_1 + 1)$  для узла  $v_1$ 
8    $l_2 := \lfloor \frac{r}{k} \rfloor$ 
9    $v_2 := \text{Unrank\_Variant}(l_2, m - 1, k)$ 
10  Добавить узел-потомок  $v_2$  для узла  $v$ 
11  return  $v$ 
12 end
```

Кроме того, для кодирования элементов комбинаторного множества $T_{m,k}$ можно применить любые другие формулы для вычисления $|T_{m,k}|$, которые удовлетворяют требованиям методологии [249].

Теорема 6.1. *Количество всех блоков, формируемых кортежем значений m атрибутов с использованием единого справочника из k уникальных значений атрибутов, вычисляется с помощью следующей рекуррентной формулы:*

$$|T_{m,k}| = \begin{cases} \sum_{l=0}^m C_m^l \cdot |T_{m-l,k-1}|, & m > 0 \text{ и } k > 0; \\ 1, & m \geq k = 1 \text{ или } k \geq m = 0; \\ 0, & \text{иначе,} \end{cases} \quad (6.4)$$

где C_m^l показывает количество сочетаний из m по l .

Доказательство. Пусть параметр l показывает количество Значение $_k$ в кортеже Блок = < Атрибут $_1, \dots, \text{Атрибут}_m$ >, то есть Атрибут $_j$ = Значение $_k$ для l различных значений индекса j . Минимальное значение l равно 0, когда кортеж не содержит Значение $_k$. Максимальное значение l равно m , когда кортеж состоит только из Значение $_k$, то есть Блок = < Значение $_k, \dots, \text{Значение}_k$ >. Следовательно, требуется просуммировать значения количества всех блоков для каждого возможного значения параметра l .

Для фиксированного значения l необходимо определить l позиций в кортеже Блок = < Атрибут $_1, \dots, \text{Атрибут}_m$ >, для которых Атрибут $_j$ = Значение $_k$. Количество способов сделать это соответствует количеству сочетаний из m по l , которое обозначим через C_m^l и $C_m^l = \binom{m}{l}$. Если из кортежа удалить все элементы, для которых Атрибут $_j$ = Значение $_k$, то в результате получится кортеж $m - l$ атрибутов, который не содержит Значение $_k$. Количество таких кортежей $m - l$ атрибутов при использовании справочника

$$\text{Справочник} \setminus \{\text{Значение}_k\} = \{\text{Значение}_1, \dots, \text{Значение}_{k-1}\},$$

состоящего из $k - 1$ уникальных значений, равно $|T_{m-l,k-1}|$. Так как для каждого из C_m^l сочетаний имеется $|T_{m-l,k-1}|$ разных способов заполнения оставшихся $m - l$ атрибутов, то требуется умножить C_m^l на $|T_{m-l,k-1}|$ для нахождения количества блоков, содержащих l раз Значение $_k$.

Кроме того, рекуррентная формула имеет следующие начальные условия:

– если $k = 1$ и $m \geq 1$, то имеется справочник с единственным уникальным значением (Справочник = {Значение $_1$ }), которое присваивается каждому атрибуту. Следовательно, для $k = 1$ и $m \geq 1$ получаем множество всех блоков $T_{m,k} = \{< \text{Значение}_1, \dots, \text{Значение}_1 >\}$ и $|T_{m,k}| = 1$;

– если $k \geq 0$ и $m = 0$, то формируется пустой кортеж. Следовательно, для $k \geq 0$ и $m = 0$ получаем множество всех блоков $T_{m,k} = \{\emptyset\}$ и $|T_{m,k}| = 1$.

В результате получаем искомую рекуррентную формулу (6.4) для вычисления $|T_{m,k}|$. $\square$

Полученная рекуррентная формула (6.4) позволяет построить рекурсивную композицию деревьев И/ИЛИ для $|T_{m,k}|$, представленную на рисунке 6.4. Кроме того, данная структура дерева И/ИЛИ для $|T_{m,k}|$ содержит в качестве подструктуры дерево И/ИЛИ для C_m^l . При этом допускается использовать любую структуру дерева И/ИЛИ, количество вариантов которой равно значению C_m^l и для которой определено биективное отображение между множеством вариантов дерева И/ИЛИ и комбинаторным множеством сочетаний из m по l .

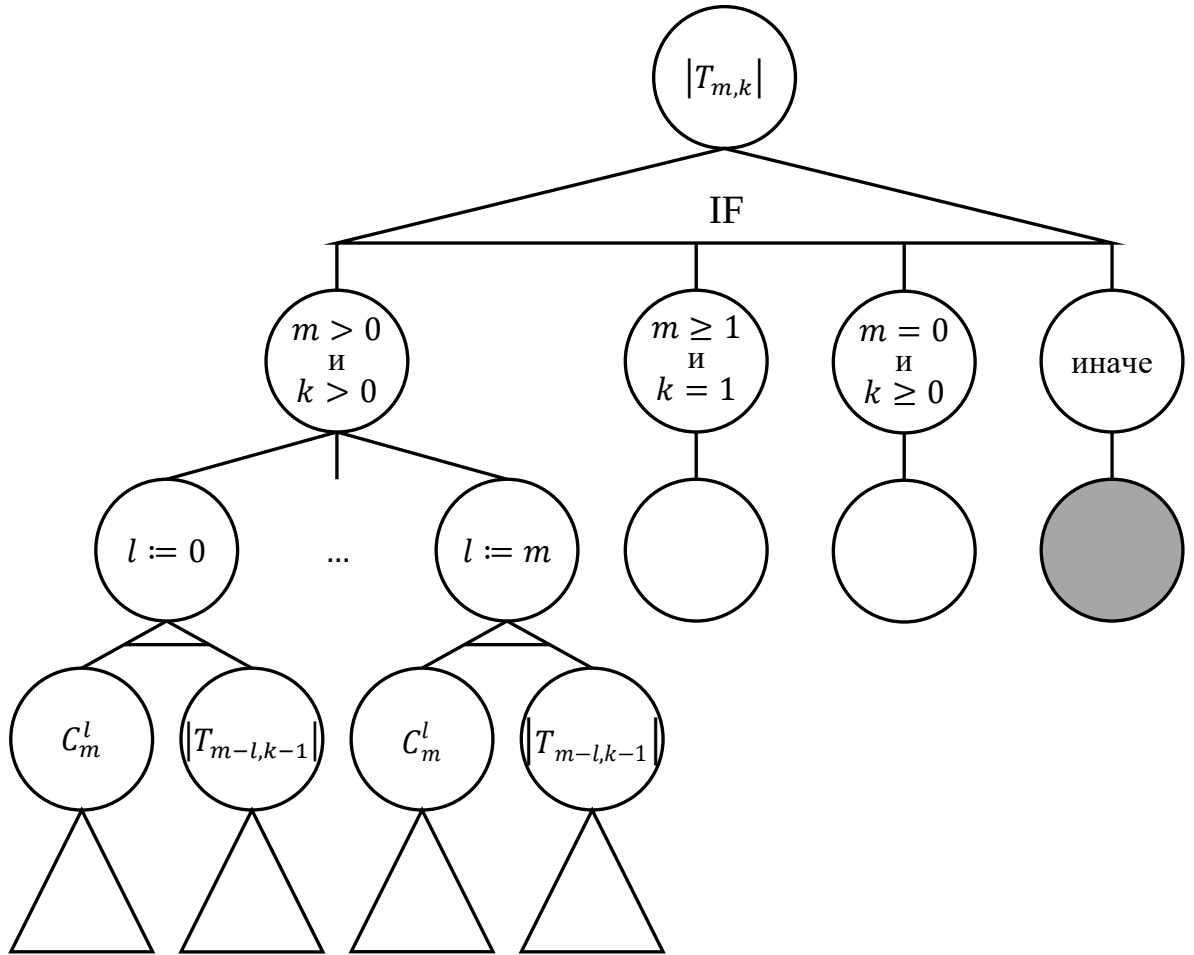


Рисунок 6.4 — Структура дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.4)

С точки зрения построения биективного отображения между множеством блоков данных и множеством вариантов дерева И/ИЛИ для $|T_{m,k}|$, представленного на рисунке 6.4, используются следующие правила:

- выбор И-узла, помеченного l , показывает количество Значение_k в кортеже $\langle \text{Атрибут}_1, \dots, \text{Атрибут}_m \rangle$;
- поддерево узла, помеченного C_m^l , определяет l значений индекса j , для которых $\text{Атрибут}_j = \text{Значение}_k$;
- поддерево узла, помеченного $|T_{m-l, k-1}|$, определяет значение атрибутов в кортеже после удаления всех элементов, для которых $\text{Атрибут}_j = \text{Значение}_k$;
- достижение узла, помеченного $|T_{m,1}|$, соответствует кортежу одинаковых значений m атрибутов, то есть $\langle \text{Значение}_1, \dots, \text{Значение}_1 \rangle$, и указывает на завершение рекурсивных вызовов;
- достижение узла, помеченного $|T_{0,k}|$, соответствует пустому кортежу и указывает на завершение рекурсивных вызовов.

Для построенной структуры дерева И/ИЛИ также разработаны алгоритмы ранжирования и генерации по рангу для множества его вариантов. Данные алгоритмы требуют наличия алгоритмов ранжирования и генерации по рангу вариантов дерева И/ИЛИ для C_m^l .

Алгоритм 6.7: Алгоритм ранжирования варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.4)

```

1 Rank_Variant( $v, m, k$ )
2 begin
3   if  $k = 1$  или  $m = 0$  then  $r := 0$ 
4   else
5      $v_0 :=$  Получить указатель на узел-потомок для узла  $v$ 
6      $L :=$  Получить значение метки узла  $v_0$ 
7      $(v_1, v_2) :=$  Получить указатели на узлы-потомки для узла  $v_0$ 
8      $sum := \sum_{l=0}^{L-1} C_m^l \cdot |T_{m-l, k-1}|$ 
9      $w_1 := C_m^L$ 
10     $l_1 := \text{Rank\_Combination}(v_1, m, L)$ 
11     $l_2 := \text{Rank\_Variant}(v_2, m - L, k - 1)$ 
12     $r := sum + l_1 + w_1 \cdot l_2$ 
13  end
14  return  $r$ 
15 end
```

Алгоритм 6.8: Алгоритм генерации по рангу варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.4)

```

1 Unrank_Variant( $r, m, k$ )
2 begin
3   Создать ИЛИ-узел  $v$ 
4   if  $k = 1$  или  $m = 0$  then Добавить узел-потомок для узла  $v$ 
5   else
6      $sum := 0$ 
7     for  $l := 0$  to  $m$  do
8        $s := C_m^l \cdot |T_{m-l,k-1}|$ 
9       if  $sum + s > r$  then
10         $r := r - sum$ 
11         $L := l$ 
12        break
13      end
14       $sum := sum + s$ 
15    end
16    Создать И-узел  $v_0$  с меткой  $L$ 
17    Добавить узел-потомок  $v_0$  для узла  $v$ 
18     $w_1 := C_m^L$ 
19     $l_1 := r \bmod w_1$ 
20     $l_2 := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
21     $v_1 := \text{Unrank\_Combination}(l_1, m, L)$ 
22    Добавить узел-потомок  $v_1$  для узла  $v_0$ 
23     $v_2 := \text{Unrank\_Variant}(l_2, m - L, k - 1)$ 
24    Добавить узел-потомок  $v_2$  для узла  $v_0$ 
25  end
26  return  $v$ 
27 end

```

Далее рассмотрим способ вычисления $|T_{m,k}|$ с использованием неупорядоченных разбиений множества на непустые подмножества, количество которых определяется значением чисел Стирлинга второго рода [250].

Теорема 6.2. *Количество всех блоков, формируемых кортежем значений m атрибутов с использованием единого справочника из k уникальных значений атрибутов, вычисляется с помощью следующей формулы:*

$$|T_{m,k}| = \begin{cases} \sum_{l=1}^{\min(m,k)} S_m^l \cdot C_k^l \cdot P_l, & m > 0 \text{ и } k > 0; \\ 1, & m = 0 \text{ и } k \geq 0; \\ 0, & \text{иначе.} \end{cases} \quad (6.5)$$

где S_m^l показывает количество разбиений множества m элементов на l подмножеств, C_k^l показывает количество сочетаний из k по l , P_l показывает количество перестановок l элементов.

Доказательство. Пусть параметр l показывает количество разных значений в кортеже Блок = < Атрибут₁, ..., Атрибут _{m} >. Минимальное возможное значение l равно 1, когда кортеж состоит из атрибутов с одинаковым значением, то есть Атрибут₁ = ... = Атрибут _{m} . Максимальное значение l равно $\min(m, k)$:

- если $m < k$ и $l = m$, тогда все атрибуты кортежа имеют разные значения, то есть Атрибут _{j_1} $\neq$ Атрибут _{j_2} для всех $j_1 \neq j_2$;
- если $k < m$ и $l = k$, тогда атрибуты кортежа имеют все возможные значения, то есть {Атрибут₁, ..., Атрибут _{m} } $\supseteq$ {Значение₁, ..., Значение _{k} }.

Следовательно, требуется просуммировать значения количества всех блоков для каждого возможного значения параметра l .

Для фиксированного значения l необходимо определить l подмножеств атрибутов кортежа, где каждое подмножество содержит атрибуты с одинаковым значением. Количество способов сделать это соответствует количеству неупорядоченных разбиений множества m атрибутов на l непустых подмножеств, что определяется числом Стирлинга второго рода S_m^l (числовая последовательность A008277 в OEIS [174]). Затем для каждого из l подмножеств нужно определить соответствующее значение из справочника. Количество способов выбрать l значений из справочника {Значение₁, ..., Значение _{k} } соответствует количеству сочетаний из k по l , обозначаемое через биномиальный коэффициент $C_k^l = \binom{k}{l}$. Распределение выбранных l значений между l подмножествами атрибутов определяется их перестановкой, количество которых обозначим через факториал $P_l = l!$. Таким образом, чтобы найти количество блоков, содержащих l разных значений в кортеже m атрибутов при использовании справочника из k уникальных значений, необходимо перемножить S_m^l , C_k^l и P_l . $\square$

Полученная формула (6.5) позволяет построить композицию деревьев И/ИЛИ для $|T_{m,k}|$, представленную на рисунке 6.5. Кроме того, данная структура дерева И/ИЛИ для $|T_{m,k}|$ содержит в качестве подструктур дерева И/ИЛИ для S_m^l , C_k^l и P_l .

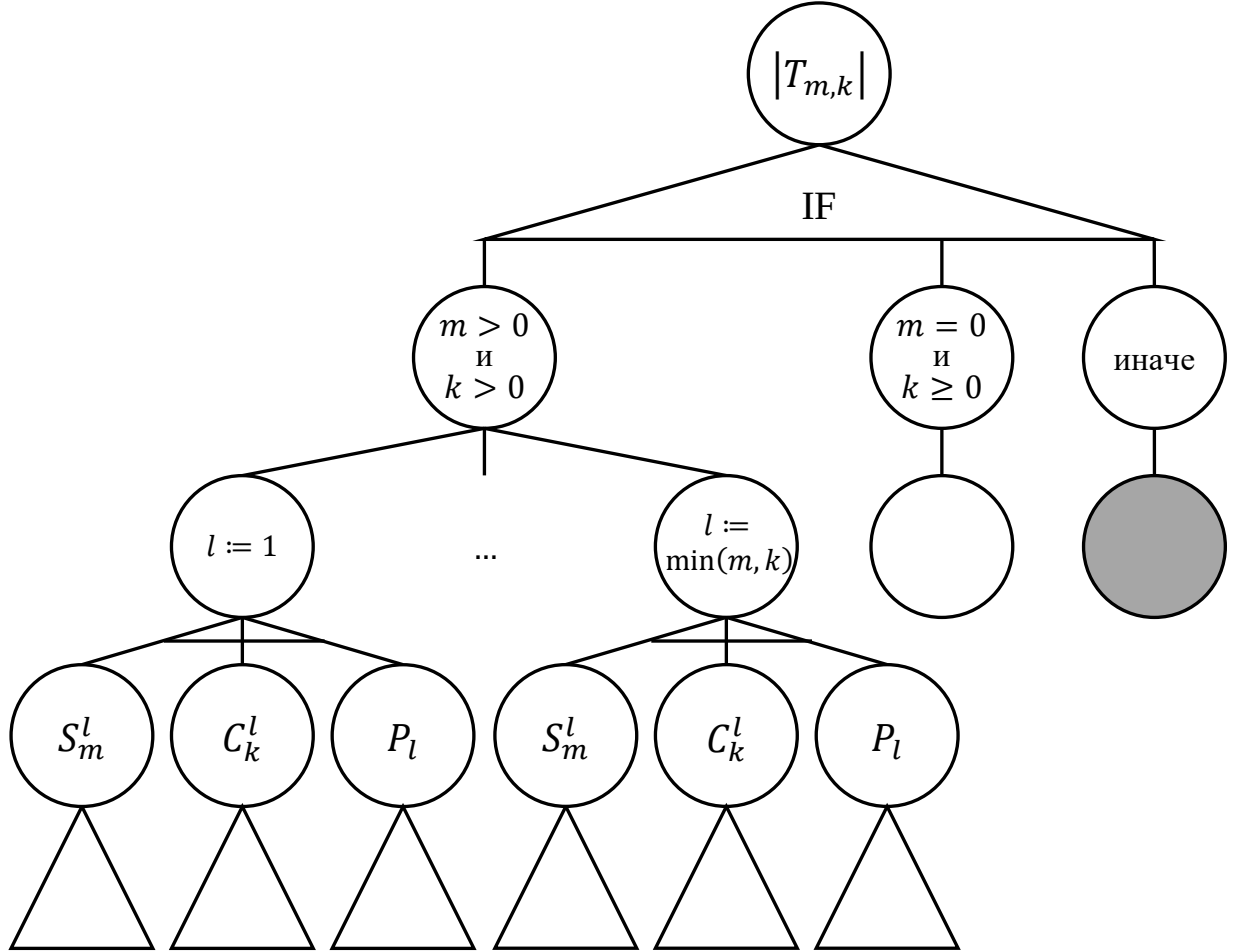


Рисунок 6.5 — Структура дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.5)

С точки зрения построения биективного отображения между множеством блоков данных и множеством вариантов дерева И/ИЛИ для $|T_{m,k}|$, представленного на рисунке 6.5, используются следующие правила:

- выбор И-узла, помеченного l , показывает количество разных значений в кортеже $\langle \text{Атрибут}_1, \dots, \text{Атрибут}_m \rangle$;
- поддерево узла, помеченного S_m^l , определяет способ разбиения множества m атрибутов на l непустых подмножеств с одинаковыми значениями атрибутов;
- поддерево узла, помеченного C_k^l , определяет выбор l значений из справочника $\{\text{Значение}_1, \dots, \text{Значение}_k\}$;

– поддереву узла, помеченного P_l , определяет способ распределения l выбранных значений атрибутов между l подмножествами атрибутов.

Для построенной структуры дерева И/ИЛИ также разработаны алгоритмы ранжирования и генерации по рангу для множества его вариантов. Данные алгоритмы требуют наличия алгоритмов ранжирования и генерации по рангу вариантов дерева И/ИЛИ для S_m^l , C_k^l и P_l .

Таким образом, каждая из представленных структур деревьев И/ИЛИ для комбинаторного множества $|T_{m,k}|$ может быть использована при разработке алгоритмов блочного кодирования цифровых данных. При этом такие алгоритмы будут отличаться последовательностями вычисленных значений рангов блоков, что влияет на вид их статистического распределения и, следовательно, на эффективность их последующего сжатия при помощи статистических методов.

Алгоритм 6.9: Алгоритм ранжирования варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.5)

```

1 Rank_Variant( $v, m, k$ )
2 begin
3   if  $m = 0$  then  $r := 0$ 
4   else
5      $v_0 :=$  Получить указатель на узел-потомок для узла  $v$ 
6      $L :=$  Получить значение метки узла  $v_0$ 
7      $(v_1, v_2, v_3) :=$  Получить указатели на узлы-потомки для узла  $v_0$ 
8      $sum := \sum_{l=1}^{L-1} S_m^l \cdot C_k^l \cdot P_l$ 
9      $w_1 := S_m^L$ 
10     $w_2 := C_k^L$ 
11     $l_1 := \text{Rank\_StirlingII}(v_1, m, L)$ 
12     $l_2 := \text{Rank\_Combination}(v_2, l, L)$ 
13     $l_3 := \text{Rank\_Permutation}(v_3, L)$ 
14     $r := sum + l_1 + w_1 \cdot (l_2 + w_2 \cdot l_3)$ 
15  end
16  return  $r$ 
17 end
```

Алгоритм 6.10: Алгоритм генерации по рангу варианта дерева И/ИЛИ для $|T_{m,k}|$ при использовании формулы (6.5)

```

1  Unrank_Variant( $r, m, k$ )
2  begin
3      Создать ИЛИ-узел  $v$ 
4      if  $m = 0$  then Добавить узел-потомок для узла  $v$ 
5      else
6           $sum := 0$ 
7          for  $l := 1$  to  $m$  do
8               $s := S_m^l \cdot C_k^l \cdot P_l$ 
9              if  $sum + s > r$  then
10                  $r := r - sum$ 
11                  $L := l$ 
12                 break
13             end
14              $sum := sum + s$ 
15         end
16         Создать И-узел  $v_0$  с меткой  $L$ 
17         Добавить узел-потомок  $v_0$  для узла  $v$ 
18          $w_1 := S_m^L$ 
19          $w_2 := C_k^L$ 
20          $l_1 := r \bmod w_1$ 
21          $r := \left\lfloor \frac{r}{w_1} \right\rfloor$ 
22          $l_2 := r \bmod w_2$ 
23          $l_3 := \left\lfloor \frac{r}{w_2} \right\rfloor$ 
24          $v_1 := \text{Unrank\_StirlingII}(l_1, m, L)$ 
25         Добавить узел-потомок  $v_1$  для узла  $v_0$ 
26          $v_2 := \text{Unrank\_Combination}(l_2, k, L)$ 
27         Добавить узел-потомок  $v_2$  для узла  $v_0$ 
28          $v_3 := \text{Unrank\_Permutation}(l_3, L)$ 
29         Добавить узел-потомок  $v_3$  для узла  $v_0$ 
30     end
31     return  $v$ 
32 end

```

6.3 Кодирование информационных объектов, связанных с прикладным программным обеспечением

Рассмотрим примеры информационных объектов, представляющих собой цифровые данные, и построим алгоритмы их кодирования на основе предложенной методологии. В частности, рассмотрим следующие информационные объекты, связанные с прикладным программным обеспечением: текстовые файлы, растровые изображения, журналы событий информационных систем, реляционные базы данных.

6.3.1 Кодирование текстовых файлов

С математической точки зрения простой текстовый файл представляет собой конкатенацию символов, взятых из некоторого конечного множества символов (алфавит). Таким образом, текстом $text$ является строка $t_1 t_2 \dots t_m$, состоящая из m символов t_j ($j \in \{1, \dots, m\}$) из алфавита $symbols = \{s_1, s_2, \dots, s_k\}$, содержащего k различных символов s_l ($l \in \{1, \dots, k\}$). При этом текст может быть предварительно разбит на тексты меньшей длины. В результате получаем, что исходные данные текста делятся на n блоков одинакового размера, каждый из которых представляет собой конкатенацию m символов из алфавита k символов. При необходимости, последний блок может быть заполнен недостающим количеством пустых символов.

Если рассмотреть все возможные блоки текста при фиксированных параметрах m и k , то получим комбинаторное множество $T_{m,k}$. В таком случае символы блока текста t_j ($j \in \{1, \dots, m\}$) соответствуют атрибутам, алфавит $\{s_1, s_2, \dots, s_k\}$ соответствует справочнику уникальных значений атрибутов. Также отметим, что все атрибуты представляют собой одинаковую структуру (символ текста), поэтому допускается хранение единого справочника уникальных значений атрибутов.

Используя формулу (6.2) для вычисления $T_{m,k}$ и соответствующую структуру дерева И/ИЛИ (рисунок 6.2), получаем структуру дерева И/ИЛИ для кодируемого блока текста (рисунок 6.6). Корневой узел данной структуры является И-узлом с меткой $text$ и содержит m узлов-потомков, помеченных t_j ($j \in \{1, \dots, m\}$). При этом каждый такой узел является ИЛИ-узлом и содержит k узлов-потомков, помеченных значениями от 1 до k , что соответствует порядковым номерам символов в упорядоченном алфавите $\{s_1, s_2, \dots, s_k\}$.

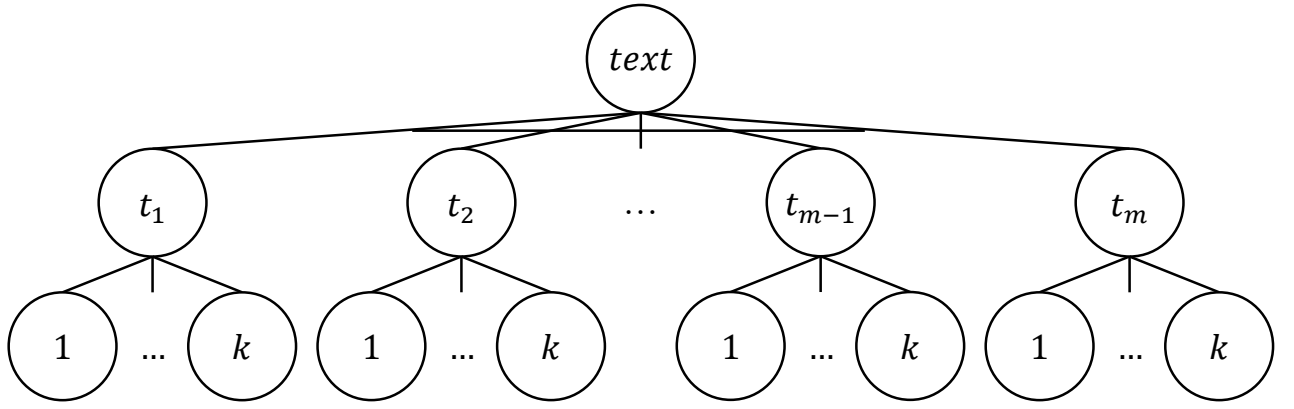


Рисунок 6.6 — Структура дерева И/ИЛИ для кодируемого блока текста

На рисунке 6.7 в общем виде показана структура варианта дерева И/ИЛИ, представленного на рисунке 6.6. В данном случае у каждого ИЛИ-узла, помеченного t_j , оставлен только один узел-потомок, помеченный l_j , что соответствует выбору конкретного символа из упорядоченного алфавита $\{s_1, s_2, \dots, s_k\}$ для j -го символа блока текста, то есть $t_j = s_{l_j}$ в $text = t_1 t_2 \dots t_m$.

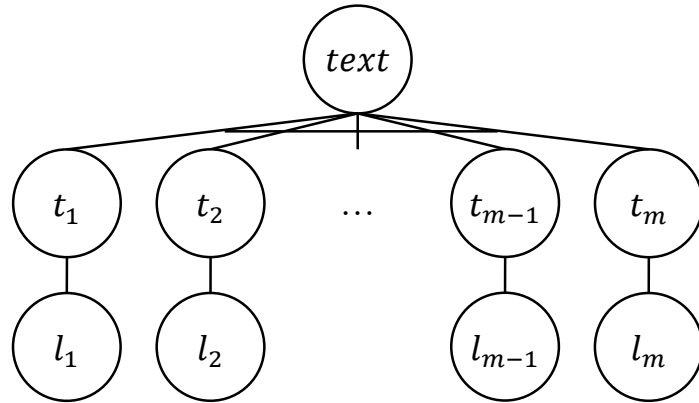


Рисунок 6.7 — Структура варианта дерева И/ИЛИ для кодируемого блока текста

На основе общих алгоритмов кодирования (алгоритмы 6.3 и 6.4) и правил биективного отображения вариантов дерева И/ИЛИ, представленного на рисунке 6.6, на блоки текста, строятся соответствующие алгоритмы кодирования (алгоритмы 6.11 и 6.12). Также заметим, что вычисление ранга в алгоритме 6.11 для блока текста $text = t_1 t_2 \dots t_m$ при использовании единого справочника уникальных значений атрибутов в виде упорядоченного алфавита $symbols = \{s_1, s_2, \dots, s_k\}$ кратко описывается в виде следующей формулы:

$$r := \sum_{j=1}^m l_j \cdot k^{j-1}.$$

Алгоритм 6.11: Алгоритм кодирования блока текста при использовании формулы (6.2)

```

1 Encode_Text( $text = t_1 t_2 \dots t_m$ ,  $symbols = \{s_1, s_2, \dots, s_k\}$ )
2 begin
3    $r := 0$ 
4   for  $j := m$  to 1 by  $-1$  do
5      $l :=$  Порядковый номер значения  $t_j$  в алфавите  $symbols$ 
6      $r := (l - 1) + k \cdot r$ 
7   end
8   return  $r$ 
9 end

```

Алгоритм 6.12: Алгоритм декодирования блока текста при использовании формулы (6.2)

```

1 Decode_Text( $r$ ,  $symbols = \{s_1, s_2, \dots, s_k\}$ )
2 begin
3   for  $j := 1$  to  $m$  do
4      $l := r \bmod k$ 
5      $t_j := s_l$ 
6      $r := \lfloor \frac{r}{k} \rfloor$ 
7   end
8    $text := t_1 t_2 \dots t_m$ 
9   return  $text$ 
10 end

```

В качестве вычислительного эксперимента с помощью программной реализации алгоритмов 6.11 и 6.12 на языке программирования Python было выполнено кодирование текстового файла «alice29.txt» из стандартного тестового набора данных «The Canterbury Corpus» [251]. Данный файл содержит 144 873 символа, каждый из которых занимает 1 байт в исходной кодировке текста, и 3 608 символов переноса строки, которые занимают по 2 байта. В сумме получаем $N = 148\,481$ символов, которые образуют алфавит из $k = 73$ уникальных символов. Размер исходного файла составляет

$$144\,873 \cdot 1 + 3\,608 \cdot 2 = 152\,089 \text{ байт.}$$

При использовании ранжирования с разделением на блоки размером по $m = 8$ символов размер файла, содержащего информацию о рангах каждого блока, составил 116 007 байт. В данном случае $N = 148\,481$ символов образуют следующее количество целых блоков:

$$n = \left\lceil \frac{N}{m} \right\rceil = \left\lceil \frac{148\,481}{8} \right\rceil = 18\,561.$$

Максимально возможное значение ранга равно $k^m = 73^8$, поэтому для хранения ранга каждого блока требуется

$$\lceil \log_2(k^m) \rceil = \lceil \log_2(73^8) \rceil = 50 \text{ бит.}$$

Тогда хранение всех блоков потребует

$$n \cdot \lceil \log_2(k^m) \rceil = 18\,561 \cdot 50 = 928\,050 \text{ бит} = 116\,006,25 \text{ байт.}$$

При этом хранение блока из 8 символов в исходном файле требовало как минимум 64 бит (если в блоке отсутствовал символ переноса строки). Следовательно, коэффициент сжатия равен 1,31, что уступает существующим методам сжатия данных [252]. Кроме того, полученный после ранжирования бинарный файл со значениями рангов в двоичном представлении также может быть подвергнут дополнительному сжатию за счет последующего применения других универсальных методов сжатия (например, алгоритм LZMA [253; 254]). Однако применение статистических методов сжатия данных к бинарному файлу со значениями рангов блоков текста показывает худшие результаты по сравнению с применением этих же методов к исходному файлу. Это объясняется следующими причинами:

- блок символов преобразуется в значение ранга, которое требует для хранения незначительно меньше бит памяти;
- исходный текст характеризуется неравномерностью статистического распределения символов, что положительно влияет на качество результата статистических методов сжатия данных;
- представление в виде рангов меняет статистическое распределение, приближая его к равномерному, что негативно влияет на качество результата статистических методов сжатия данных.

Таким образом, за счет ранжирования получаем незначительный выигрыш при хранении рангов, но при этом сильно снижается эффективность сжатия статистическими методами.

6.3.2 Кодирование растровых изображений

Растровое изображение представляет собой двумерный массив пикселей, каждый из которых имеет свой цвет. В таком случае аналогом алфавита текста для растрового изображения выступает использованная в нем палитра, то есть набор пикселей с уникальным цветом. При этом количество пикселей с уникальным цветом обычно намного меньше общего количества пикселей в изображении, что приводит к дублированию хранимой информации. Кроме того, двумерный массив пикселей может быть представлен в виде единой последовательности, полученной путем последовательной конкатенации всех строк. Тогда для обеспечения возможности восстановления исходного двумерного массива из последовательности пикселей требуется дополнительно указать его размерность (количество строк и количество столбцов).

Таким образом, растровым изображением *image* является последовательность $(p_1, p_2, \dots, p_m)$, состоящая из m пикселей p_i ($j \in \{1, \dots, m\}$) из палитры $colors = \{c_1, c_2, \dots, c_k\}$, содержащей k различных цветов c_l ($l \in \{1, \dots, k\}$). При этом изображение может быть предварительно разбито на изображения меньшего размера (например, последовательность пикселей делится на подпоследовательности или двумерный массив делится на подмассивы). В результате получаем, что исходные данные растрового изображения делятся на n блоков одинакового размера, каждый из которых представляет собой последовательность m пикселей из палитры k цветов [255]. При необходимости, последний блок может быть заполнен недостающим количеством пустых пикселей.

Если рассмотреть все возможные блоки изображения при фиксированных параметрах m и k , то получим комбинаторное множество $T_{m,k}$. В таком случае пиксели блока изображения p_j ($j \in \{1, \dots, m\}$) соответствуют атрибутам, палитра $\{c_1, c_2, \dots, c_k\}$ соответствует справочнику уникальных значений атрибутов. Также отметим, что все атрибуты представляют собой одинаковую структуру (пиксель изображения), поэтому допускается хранение единого справочника уникальных значений атрибутов.

Используя формулу (6.2) для вычисления $T_{m,k}$ и соответствующую структуру дерева И/ИЛИ (рисунок 6.2), получаем структуру дерева И/ИЛИ для кодируемого блока изображения (рисунок 6.8). Данная структура дерева И/ИЛИ идентична структуре дерева И/ИЛИ для кодируемого блока текста (рисунок 6.6), поэтому кодирование блока изображения реализуется аналогично алгоритмам 6.11 и 6.12.

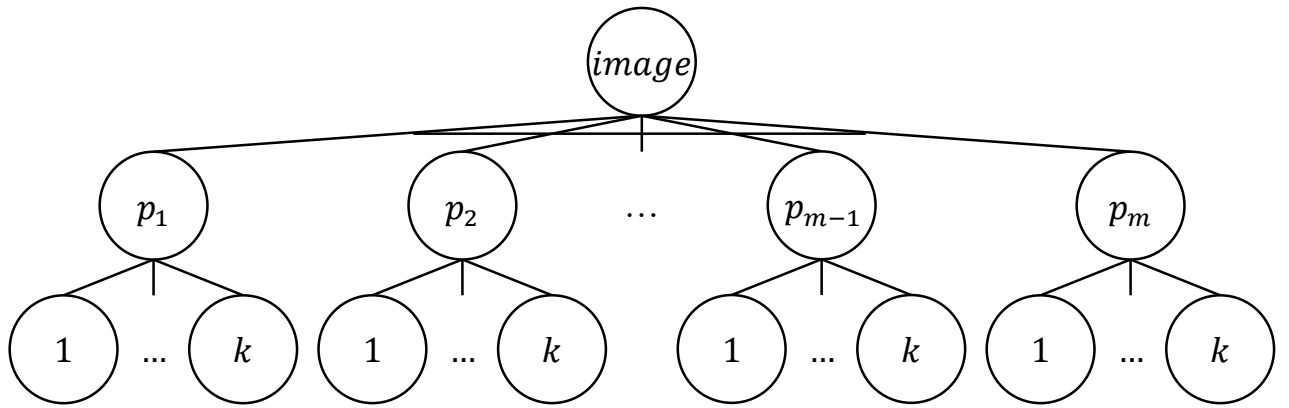


Рисунок 6.8 — Структура дерева И/ИЛИ для кодируемого блока изображения

В качестве вычислительного эксперимента выполнено кодирование растровых изображений из стандартного тестового набора данных «The USC-SIPI Image Database» [256]. Например, файл «4.1.07 Jelly beans» хранит изображение размера 256×256 , содержащее $N = 256 \cdot 256 = 65\,536$ пикселей, цвет которых определяется значением в 3 байта. В то же время палитру изображения составляют $k = 18\,646$ уникальных цветов. Размер исходного файла составляет

$$65\,536 \cdot 3 = 196\,608 \text{ байт.}$$

При использовании ранжирования с разделением на блоки размером по $m = 64$ пикселя (изображение разделено на квадраты размера 8×8) размер файла, содержащего информацию о рангах каждого блока, составил 116 224 байт. В данном случае $N = 65\,536$ пикселей образуют следующее количество целых блоков:

$$n = \left\lceil \frac{N}{m} \right\rceil = \left\lceil \frac{65\,536}{64} \right\rceil = 1\,024.$$

Максимально возможное значение ранга равно $k^m = 18\,646^{64}$, поэтому для хранения ранга каждого блока требуется

$$\lceil \log_2(k^m) \rceil = \lceil \log_2(18\,646^{64}) \rceil = 908 \text{ бит.}$$

Тогда хранение всех блоков потребует

$$n \cdot \lceil \log_2(k^m) \rceil = 1\,024 \cdot 908 = 929\,792 \text{ бит} = 116\,224 \text{ байт.}$$

При этом хранение блока из 64 пикселей в исходном файле требовало $64 \cdot 24 = 1\,536$ бит. Следовательно, коэффициент сжатия равен 1,69, что также уступает существующим методам сжатия данных (формат PNG позволяет сжать изображение до 76 803 байт, то есть коэффициент сжатия равен 2,56).

Таким образом, аналогично кодированию блока текста получаем ситуацию, когда значение ранга блока изображения требует для хранения незначительно меньше бит памяти, чем хранение в виде последовательности пикселей (908 бит против 1 536 бит). Последующее применение других универсальных методов сжатия (например, алгоритм LZMA) также показывает низкую эффективность, так как представление в виде рангов меняет статистическое распределение в сторону равномерности. Для проверки равномерности распределения было проведено исследование статистического распределения значений последовательностей по 8 бит в бинарном файле со значениями рангов блоков изображений. Рисунок 6.9 наглядно демонстрирует близкое к равномерному распределение внутри бинарного файла.

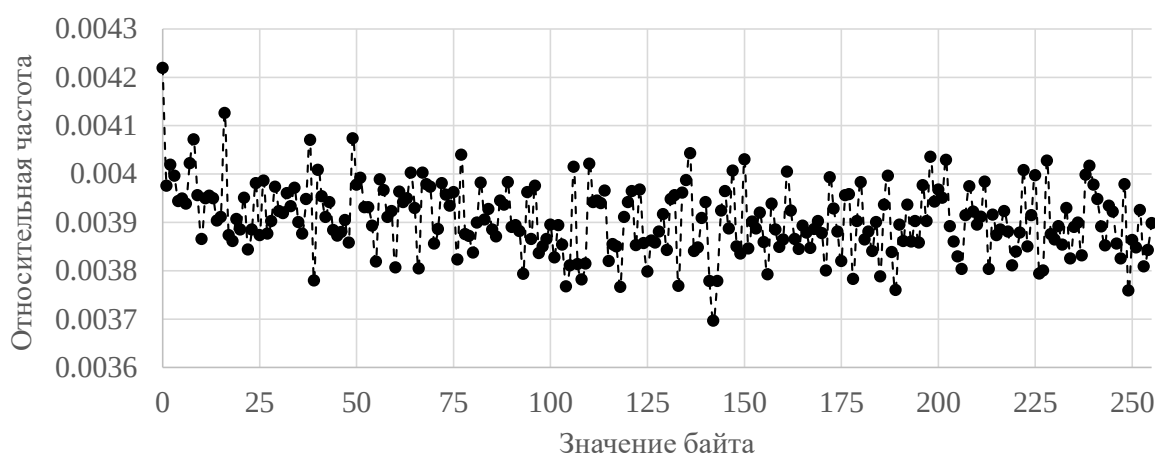


Рисунок 6.9 — Полигон относительных частот последовательностей по 8 бит в бинарном файле со значениями рангов блоков изображений

Равномерность распределения внутри бинарного файла со значениями рангов блоков изображений показывает возможность разработки генератора псевдослучайных чисел на его основе. Актуальность разработки новых методов генерации случайных чисел обусловлена возрастающими требованиями к безопасности информационных систем и ограниченностью традиционных подходов [257]. Классические генераторы псевдослучайных последовательностей, основанные на детерминированных алгоритмах, часто демонстрируют уязвимости, связанные с предсказуемостью и недостаточной энтропией. В свою очередь, это делает их непригодными для применения в задачах криптографии и в высоконадежных системах.

Перспективным направлением является использование цифровых изображений в качестве источника энтропии генератора случайных чисел. Особый интерес представляют изображения, получаемые в реальном времени, которые обладают

высоким уровнем энтропии за счет наличия различных шумов и вариативности освещения. Например, в работе [258] предложен метод, основанный на преобразовании исходного изображения в оттенки серого, при этом в качестве источника случайных чисел используется гистограмма для статистического распределения цветов пикселей. В работе [259] описан метод генерации случайных чисел на основе графических объектов (изображений), а также метода k -средних и алгоритма вихря Мерсенна. Предложенный в работе [260] метод генерации случайных чисел также использует в качестве исходного источника энтропии изображение, при этом данный метод демонстрирует высокие показатели случайности по результатам статистических тестов NIST 800-22 [261].

Получаемые бинарные файлы со значениями рангов блоков изображений, взятых из стандартного тестового набора данных «The USC-SIPI Image Database» [256], также были проверены с помощью статистических тестов NIST 800-22 [261]. Проверке подлежали последовательности размера 10^6 бит с использованием стандартных параметров по умолчанию. Результаты статистических тестов показали, что наилучших значений по критериям случайности числовых последовательностей достигают цветные изображения (таблицы 6.1–6.3). Также на результаты статистических тестов оказывает влияние размер кодируемого блока m , при этом форма блока (последовательность m пикселей или квадрат размера $\sqrt{m} \times \sqrt{m}$) практически не меняет общую картину.

Таблица 6.1 — Доля пройденных статистических тестов пакета NIST 800-22 (форма блока — последовательность пикселей)

Изображение	Размер кодируемого блока				
	1×2	1×4	1×8	1×16	1×32
4.2.02 Lena (gray)	50,93%	81,37%	94,41%	97,52%	99,39%
4.2.02 Lena (color)	79,50%	96,27%	97,52%	98,14%	99,39%
4.2.03 Mandrill (gray)	55,90%	83,23%	95,03%	98,14%	99,39%
4.2.03 Mandrill (color)	85,09%	93,17%	98,14%	98,14%	99,39%
4.2.05 Airplane (gray)	2,48%	19,88%	54,60%	90,68%	96,27%
4.2.05 Airplane (color)	52,76%	91,93%	96,93%	99,39%	99,38%
4.2.06 Sailboat on lake (gray)	7,45%	55,90%	80,75%	96,32%	96,93%
4.2.06 Sailboat on lake (color)	83,85%	95,71%	98,77%	99,38%	99,39%
4.2.07 Peppers (gray)	2,48%	25,47%	72,05%	93,79%	96,32%
4.2.07 Peppers (color)	88,82%	97,52%	99,38%	99,39%	99,39%

Таблица 6.2 — Доля пройденных статистических тестов пакета NIST 800-22 (форма блока — последовательность пикселей)

Изображение	Размер кодируемого блока				
	1×64	1×128	1×256	1×512	1×1024
4.2.02 Lena (gray)	99,38%	99,39%	99,39%	99,38%	99,38%
4.2.02 Lena (color)	99,39%	99,38%	99,39%	99,39%	99,38%
4.2.03 Mandrill (gray)	99,38%	99,39%	99,39%	99,38%	99,38%
4.2.03 Mandrill (color)	99,39%	99,39%	98,77%	99,39%	99,39%
4.2.05 Airplane (gray)	98,16%	99,39%	99,38%	99,38%	99,38%
4.2.05 Airplane (color)	99,39%	99,39%	99,39%	99,39%	99,39%
4.2.06 Sailboat on lake (gray)	99,38%	99,39%	99,38%	99,39%	99,38%
4.2.06 Sailboat on lake (color)	99,39%	99,38%	99,39%	99,38%	99,39%
4.2.07 Peppers (gray)	97,53%	98,77%	98,77%	98,77%	99,39%
4.2.07 Peppers (color)	99,39%	99,38%	98,77%	99,39%	99,38%

Таблица 6.3 — Доля пройденных статистических тестов пакета NIST 800-22 (форма блока — квадрат пикселей)

Изображение	Размер кодируемого блока				
	2×2	4×4	8×8	16×16	32×32
4.2.02 Lena (gray)	81,37%	96,93%	99,39%	99,38%	99,38%
4.2.02 Lena (color)	96,27%	99,39%	99,38%	99,39%	98,77%
4.2.03 Mandrill (gray)	83,23%	98,77%	99,39%	99,38%	99,38%
4.2.03 Mandrill (color)	91,93%	98,76%	99,38%	99,39%	99,39%
4.2.05 Airplane (gray)	21,12%	91,30%	98,16%	99,38%	99,38%
4.2.05 Airplane (color)	92,55%	99,39%	99,39%	99,38%	99,39%
4.2.06 Sailboat on lake (gray)	53,42%	95,09%	99,38%	99,39%	99,39%
4.2.06 Sailboat on lake (color)	95,03%	99,39%	99,39%	98,77%	99,39%
4.2.07 Peppers (gray)	30,43%	91,93%	97,52%	99,39%	99,39%
4.2.07 Peppers (color)	98,76%	99,39%	99,39%	99,39%	99,39%

Тестирование последовательности размера 10^8 бит показало, что результаты сопоставимы с существующими аналогами генераторов случайных чисел на основе изображений. Таким образом, кодирование растровых изображений с использованием методов комбинаторной генерации может стать основой новых технологий по генерации случайных последовательностей чисел.

6.3.3 Кодирование журналов событий информационных систем

Результаты кодирования текстовых файлов и растровых изображений на основе предложенной методологии показывают, что с точки зрения эффекта сжатия данных основополагающим фактором является разница между количеством бит, требуемых для хранения максимального значения ранга, и количеством бит, требуемых для хранения данных в исходном виде. Следовательно, наибольший потенциал метода проявляется для информационных объектов, которые характеризуются дублированием подструктур большого размера. Например, такая ситуация наблюдается в файлах журналов событий информационных систем. Данные журналов событий характеризуются высокой степенью структурированности и избыточности информации, что приводит к неэффективному расходованию ресурсов компьютерной памяти при их хранении.

Журналирование событий является типовым процессом, реализуемым в различного рода информационных системах. Результатом работы такого процесса является создание журналов событий, содержащих краткое описание произошедших в системе событий в хронологическом порядке. При этом обращение к содержимому журналов событий чаще всего совершается лишь изредка, например, при наступлении какого-либо сбоя в системе [262]. Таким образом, данные журналов событий можно считать архивными и их можно хранить в сжатом виде [263; 264]. С точки зрения внутреннего содержимого журналы событий имеют фиксированную структуру и описывают журналируемые события с помощью шаблонных текстовых структур. С точки зрения организации способа хранения журнал событий представляет собой обычный текстовый файл, где каждая строка описывает произошедшее в системе событие. В таком случае файл журнала событий может занимать большой объем памяти для его хранения. Также можно встретить реализацию журнала событий в формате файла базы данных.

Простейшая реализация журнала событий представляет собой текстовый файл, который имеет определенную структуру:

- каждая строка является одной записью журнала событий (если зафиксировать архив журнала событий за некоторый промежуток времени, то он содержит n различных записей);
- каждая запись журнала событий содержит дату и время зарегистрированного в системе события, а также набор значений атрибутов, характеризующих данное событие (событие определяется набором значений m атрибутов).

Если зафиксировать некоторое текущее состояние журнала событий без возможности добавления в него новых записей (например, журнал событий за определенный прошедший период времени), то множество записей журнала событий может быть рассмотрено как комбинаторное множество. Внутреннее содержимое журнала событий имеет фиксированную структуру, следовательно, каждая его запись состоит из значений одних и тех же атрибутов. Кроме того, если рассматривается сжатие некоторого зафиксированного состояния журнала событий без возможности добавления в него новых записей, то множество возможных значений каждого атрибута является конечным. В результате получаем комбинаторное множество $T_{m,k_1,\dots,k_m}$, содержащее все кортежи m атрибутов, которые принимают значения из m справочников уникальных значений атрибутов размером $k_1, \dots, k_m$. Мощность такого комбинаторного множества определяется формулой (6.1), то есть блоком данных выступает запись журнала событий.

Предлагаемый метод кодирования журналов событий заключается в выполнении следующих шагов [265]:

Шаг 1. Определение кодируемых атрибутов журнала событий.

В ходе данного шага требуется сначала определить из каких M атрибутов состоит каждая запись исследуемого журнала событий. Таким образом, каждая запись журнала событий представляется как кортеж значений M атрибутов $\text{Запись} = \langle a_1, \dots, a_M \rangle$. Далее необходимо указать какие именно m атрибутов ($m \in \{1, \dots, M\}$) подлежат кодированию за счет применения методов комбинаторной генерации, при этом оставшиеся атрибуты будут записаны в исходном виде. Тогда кортеж значений M атрибутов $\langle a_1, \dots, a_M \rangle$ может быть представлен следующим образом: $\langle c_1, \dots, c_{M-m}, b_1, \dots, b_m \rangle$, где b_j — значение j -го атрибута ($j \in \{1, \dots, m\}$) среди подлежащих дальнейшему кодированию m атрибутов, c_j — значение j -го атрибута ($j \in \{1, \dots, M - m\}$) среди оставшихся $M - m$ атрибутов. При этом допускается изменение порядка атрибутов в кортеже относительно исходного расположения с сохранением инструкций по восстановлению исходного упорядочивания: ситуация, когда $\langle a_1, \dots, a_M \rangle \neq \langle c_1, \dots, c_{M-m}, b_1, \dots, b_m \rangle$, но при этом кортежи состоят из одних и тех же элементов, то есть $\{a_1, \dots, a_M\} = \{c_1, \dots, c_{M-m}, b_1, \dots, b_m\}$. Рекомендуется оставлять без кодирования атрибуты, не имеющие значительного дублирования в своих значениях, так как в противном случае будет наблюдаться слабый эффект сжатия данных, представленных в виде значений рангов.

Шаг 2. Составление справочников уникальных значений атрибутов журнала событий.

В ходе данного шага требуется для каждого подлежащего дальнейшему кодированию атрибута определить множество его уникальных значений. В результате чего формируются m множеств B_j ($j \in \{1, \dots, m\}$), каждое из которых представляется в виде файла-справочника. Каждая запись (строка) такого файла-справочника представляет собой одно из уникальных значений атрибута, а порядковый номер записи (строки) является ее идентификатором. При этом возможны различные варианты упорядочивания записей внутри файла-справочника: например, если расположить записи в лексикографическом порядке, то это ускорит процесс поиска записей внутри файла-справочника и, следовательно, ускорит работу алгоритмов комбинаторной генерации.

Шаг 3. Построение структуры дерева И/ИЛИ для кодируемых атрибутов журнала событий.

В ходе данного шага требуется построить структуру дерева И/ИЛИ, варианты которого будут описывать кортежи значений, подлежащих дальнейшему кодированию m атрибутов $\langle b_1, \dots, b_m \rangle$. Корнем дерева будет являться И-узел, количество узлов-потомков которого определяется количеством кодируемых атрибутов и каждый из них имеет метку b_j ($j \in \{1, \dots, m\}$). В свою очередь, каждый узел с меткой b_j представляется ИЛИ-узлом, количество узлов-потомков (листьев) которого определяется количеством уникальных значений соответствующего атрибута в множестве B_j , то есть равно $|B_j|$. На рисунке 6.10 представлен итоговый вид полученной структуры дерева И/ИЛИ для кодируемых атрибутов журнала событий.

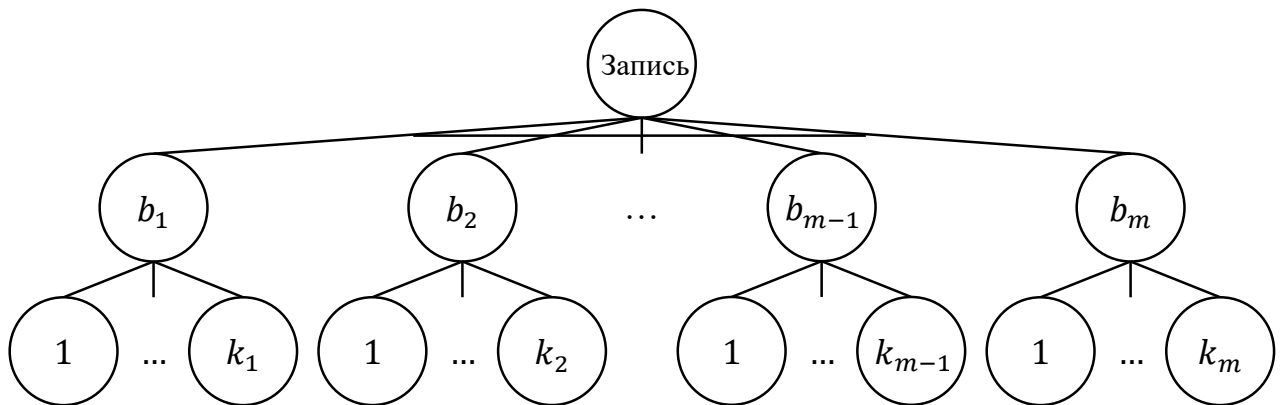


Рисунок 6.10 — Структура дерева И/ИЛИ для кодируемых атрибутов записи журнала событий

Шаг 4. Декомпозиция значений атрибутов журнала событий.

В ходе данного шага требуется рассмотреть возможность усложнения построенной структуры дерева И/ИЛИ для кодируемых атрибутов журнала событий за счет декомпозиции значений каких-либо атрибутов с целью получения выигрыша в количестве хранимой информации. В качестве примера рассмотрим следующие два типовых случая:

Случай 1. Каждое значение b_j для j -го кодируемого атрибута представляет собой конкатенацию значений $d_1 \dots d_l$ некоторых других атрибутов, то есть значению b_j можно сопоставить кортеж $\langle d_1, \dots, d_l \rangle$. Тогда узел с меткой b_j преобразуется из ИЛИ-узла в И-узел. Результат данного преобразования представлен на рисунке 6.11.

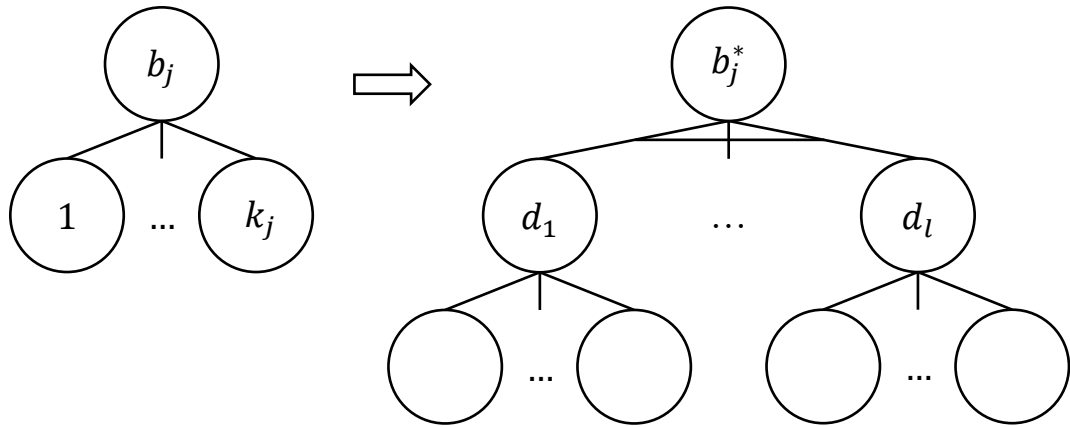


Рисунок 6.11 — Пример декомпозиции значения атрибута журнала событий (случай 1)

Случай 2. Каждое значение b_j для j -го кодируемого атрибута представляет собой один из возможных вариантов конкатенации значений некоторых других атрибутов. То есть наблюдается ситуация, описанная выше в случае 1, но допускается несколько шаблонных вариантов кортежей для сопоставления со значением b_j . Тогда узел с меткой b_j преобразуется в ИЛИ-узел, количество потомков которого определяется количеством шаблонных вариантов кортежей. При этом каждый шаблон представляется И-узлом аналогично представлению узла с меткой b_j в случае 1. Результат данного преобразования представлен на рисунке 6.12.

Так как в результате данных преобразований имевшийся атрибут заменяется на новый составной атрибут, то необходимо заменить файл-справочник уникальных значений данного атрибута на набор файлов-справочников уникальных значений для новых атрибутов. Это приводит к сокращению избыточности информации в имеющихся файлах-справочниках уникальных значений атрибутов, а также к

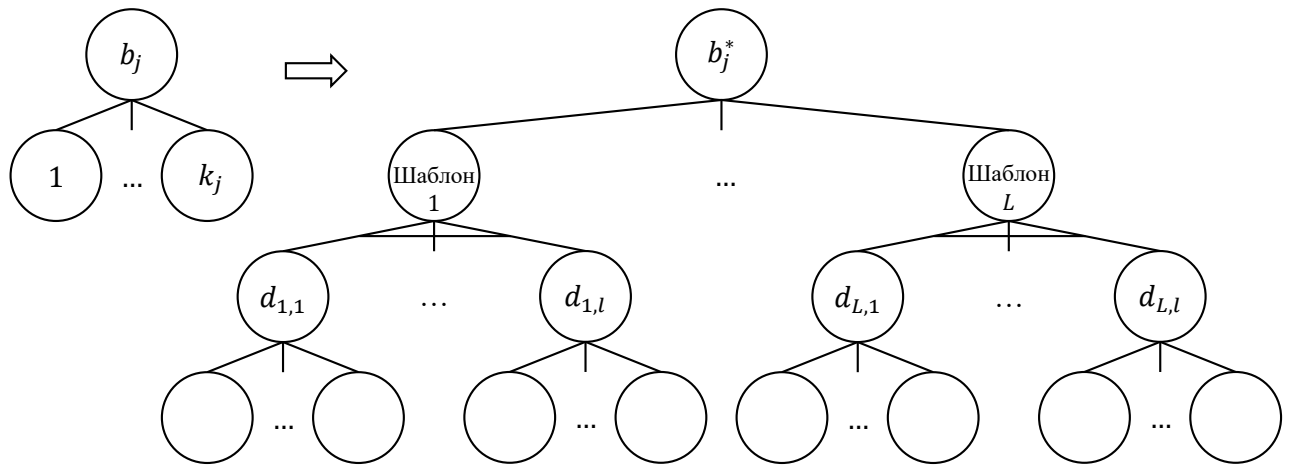


Рисунок 6.12 — Пример декомпозиции значения атрибута журнала событий (случай 2)

уменьшению объема этих файлов. Однако данные преобразования приводят к увеличению количества вариантов дерева И/ИЛИ и, следовательно, увеличению количества бит, требуемой для хранения значения ранга для каждого варианта дерева И/ИЛИ. Целесообразность применения декомпозиции значения атрибута журнала событий проверяется путем выполнения следующего неравенства:

$$\lceil \log_2 (w(\text{Запись}^*) - 1) \rceil \cdot n + \sum_j C_j^* < \lceil \log_2 (w(\text{Запись}) - 1) \rceil \cdot n + \sum_j C_j,$$

где:

- n — количество строк записей в рассматриваемом журнале событий;
- $w(\text{Запись})$ — количество вариантов в исходной версии дерева И/ИЛИ;
- $w(\text{Запись}^*)$ — количество вариантов в преобразованной версии дерева И/ИЛИ;
- C_j — количество бит, требуемой для хранения j -го файла-справочника уникальных значений атрибутов в исходной версии дерева И/ИЛИ;
- C_j^* — количество бит, требуемой для хранения j -го файла-справочника уникальных значений атрибутов в преобразованной версии дерева И/ИЛИ.

Таким образом, если по итогу изменения структуры дерева И/ИЛИ для хранения сжатого журнала событий и всех требуемых файлов-справочников в сумме необходимо меньше памяти, чем при исходной версии дерева И/ИЛИ, то такое преобразование считается целесообразным.

Шаг 5. Нормализация записей журнала событий.

В ходе данного шага требуется представить каждый кортеж подлежащих дальнейшему кодированию m атрибутов $\langle b_1, \dots, b_m \rangle$ в виде кортежа

идентификаторов значений атрибутов, полученных на основе сформированных файлов-справочников. Таким образом, каждому кортежу $\langle b_1, \dots, b_m \rangle$ сопоставляется кортеж вида $\langle Idb_1, \dots, Idb_m \rangle$, где Idb_j — порядковый номер (начиная с 0) выбранного значения b_j среди упорядоченных элементов множества B_j .

Шаг 6. Ранжирование записей журнала событий.

В ходе данного шага требуется для каждого кортежа идентификаторов значений атрибутов $\langle Idb_1, \dots, Idb_m \rangle$ вычислить значение ранга соответствующего варианта дерева И/ИЛИ. Например, если дерево И/ИЛИ имеет структуру, представленную на рисунке 6.10, то ранг варианта дерева И/ИЛИ, которому соответствует кортеж $\langle Idb_1, \dots, Idb_m \rangle$, вычисляется по формуле:

$$Idb_1 + |B_1| \cdot (Idb_2 + |B_2| \cdot (\dots (Idb_{m-1} + |B_{m-1}| \cdot Idb_m) \dots)).$$

Если структура дерева И/ИЛИ была преобразована за счет декомпозиции значений атрибутов, то данные изменения также необходимо учесть в правилах вычисления рангов для вариантов дерева И/ИЛИ.

Таким образом, получаем схему кодирования журнала событий с помощью алгоритмов комбинаторной генерации, представленную на рисунке 6.13.

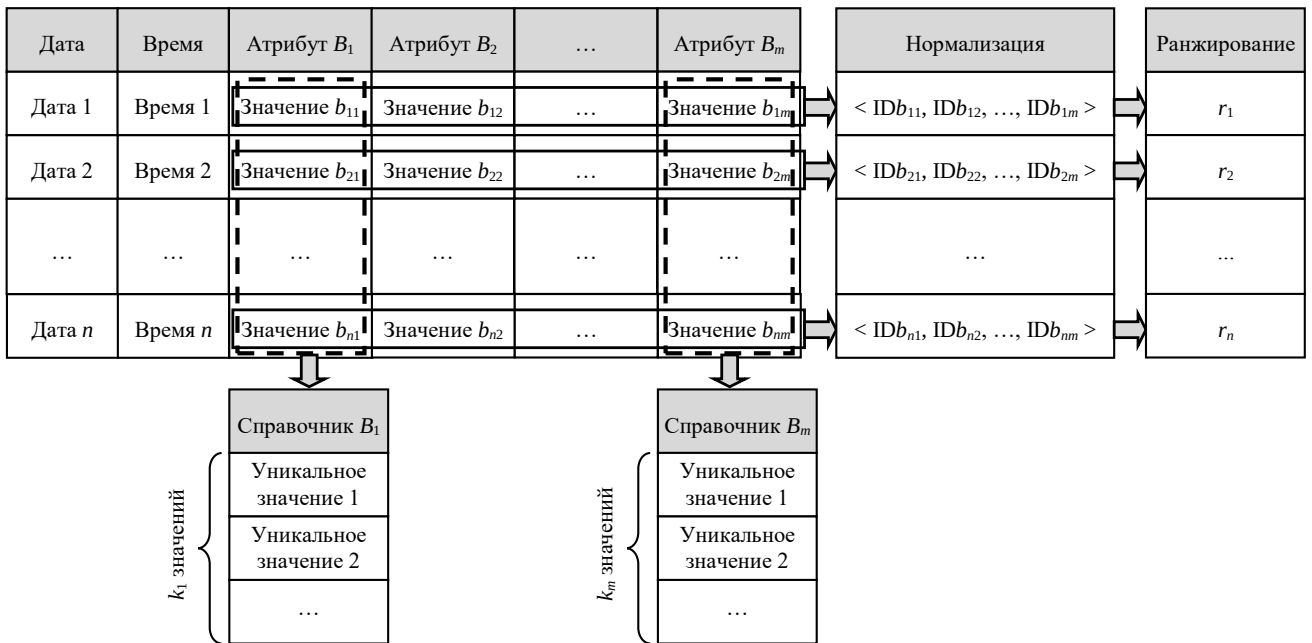


Рисунок 6.13 — Кодирование журнала событий с помощью алгоритмов комбинаторной генерации на основе деревьев И/ИЛИ

Для оценки эффективности предложенного метода рассмотрим пример кодирования данных журналов событий, генерируемых внутри электронных курсов

системы Moodle [266]. Используя данные системы дистанционного обучения Томского государственного университета систем управления и радиоэлектроники [267], был выгружен журнал событий для электронного курса, который:

- содержит информацию о деятельности внутри электронного курса для одного потока студентов в течение одного семестра в рамках обучения по дисциплине «Теория вероятностей и математическая статистика»;
- состоит из строки-заголовка и 116 245 строк записей о журналируемых событиях электронного курса;
- объем файла журнала событий в формате CSV равен 44 324 589 байт или 43 286 КБ.

Далее подробно представлена реализация основных шагов предложенного метода и полученные результаты для указанного журнала событий.

Шаг 1. Первая строка-заголовок рассматриваемого журнала событий содержит названия атрибутов, которые содержит каждая запись журнала событий [268; 269]:

- Атрибут₀: «Время» — дата и время журналируемого события электронного курса;
- Атрибут₁: «Полное имя пользователя» — фамилия, имя и отчество пользователя, сгенерировавшего событие электронного курса;
- Атрибут₂: «Затронутый пользователь» — фамилия, имя и отчество пользователя, над которым совершено действие в электронном курсе;
- Атрибут₃: «Контекст события» — название элемента электронного курса, с помощью которого совершено действие в электронном курсе;
- Атрибут₄: «Компонент» — тип элемента электронного курса, с помощью которого совершено действие в электронном курсе;
- Атрибут₅: «Название события» — название журналируемого события электронного курса;
- Атрибут₆: «Описание» — подробное описание журналируемого события электронного курса;
- Атрибут₇: «Источник» — тип источника журналируемого события электронного курса;
- Атрибут₈: «IP-адрес» — IP-адрес, с которого совершено действие в электронном курсе.

Каждая запись рассматриваемого журнала событий представляет собой одну текстовую строку, полученную конкатенацией текстовых строк с конкретными значениями атрибутов, разделенных символом запятой.

В целях упрощения последующей обработки закодированных данных, Атрибут₀ не будет участвовать в формировании структуры дерева И/ИЛИ, то есть будет храниться в исходном виде без какого-либо кодирования. Такое решение позволит в дальнейшем фильтровать запросы к журналу события по датам и времени без необходимости декодирования неподходящих записей. Объем файла журнала событий в формате CSV, в котором от каждой записи оставлена только подстрока с значением Атрибут₀, равен 2 171 581 байт или 2 121 КБ (4,9% от объема исходного файла журнала событий).

Шаг 2. Исследование содержимого записей журнала событий показало, что атрибуты содержат следующее количество уникальных значений:

- Атрибут₁: 182 уникальных значения;
- Атрибут₂: 208 уникальных значений;
- Атрибут₃: 50 уникальных значений;
- Атрибут₄: 19 уникальных значений;
- Атрибут₅: 87 уникальных значений;
- Атрибут₆: 36 513 уникальных значений;
- Атрибут₇: 4 уникальных значения;
- Атрибут₈: 3 016 уникальных значений.

Так как рассматриваемый журнал событий содержит всего 116 245 строк записей, то из полученной информации о количестве уникальных значений атрибутов очевиден вывод о значительном дублировании хранимых данных. Следовательно, более эффективно с точки зрения объема хранимых данных будет организация хранения записей рассматриваемого журнала событий в виде кортежей идентификаторов уникальных значений атрибутов, а сами уникальные значения атрибутов будут храниться в виде отдельных справочников. При этом каждый такой справочник представляет собой текстовый файл, каждая строка которого является текстовой строкой с одним из уникальных значений атрибута, а порядковый номер строки является ее идентификатором.

Также, учитывая тот факт, что Атрибут₁ и Атрибут₂ охватывают практически одно и то же множество пользователей, было реализовано объединение соответствующих множеств уникальных значений данных атрибутов. В результате было получено одно множество, состоящее из 212 уникальных значений.

Объем файлов сформированных справочников в формате CSV равен:

- Атрибут₁ и Атрибут₂: 10 768 байт;
- Атрибут₃: 2 703 байт;
- Атрибут₄: 516 байт;
- Атрибут₅: 4 574 байт;
- Атрибут₆: 4 553 086 байт;
- Атрибут₇: 26 байт;
- Атрибут₈: 46 456 байт.

Общий объем файлов справочников равен 4 618 129 байт или 4 510 КБ (10,4% от объема исходного файла журнала событий).

Шаг 3. Основываясь на выбранных для кодирования атрибутах, а также информации о количестве уникальных значений данных атрибутов, строится структура дерева И/ИЛИ, представленная на рисунке 6.14.

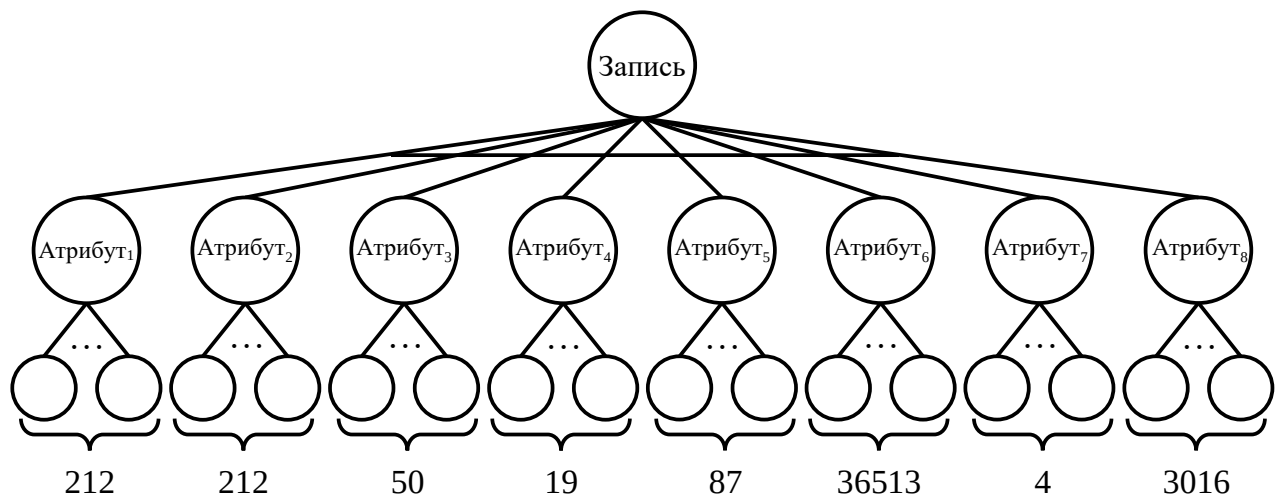


Рисунок 6.14 — Структура дерева И/ИЛИ для кодируемых атрибутов рассматриваемого журнала событий

Построенная структура дерева И/ИЛИ представляет собой корневой узел, помеченный текстом «Запись» и являющийся И-узлом. Данный И-узел содержит 8 узлов-потомков, помеченных названием соответствующего атрибута. Каждый такой узел является ИЛИ-узлом и в качестве узлов-потомков содержит листья, количество которых определяется количеством уникальных значений соответствующего атрибута среди всех записей рассматриваемого журнала событий. Таким образом, каждая запись журнала событий (точнее ее кодируемая часть, состоящая из значений Атрибут₁–Атрибут₈) может быть представлена одним вариантом построенной структуры дерева И/ИЛИ.

Шаг 4. Одним из возможных вариантов декомпозиции значений атрибутов рассматриваемого журнала событий является представление Атрибут₁ в виде комбинации следующих трех атрибутов:

- Атрибут_{1.1}: «Фамилия пользователя»;
- Атрибут_{1.2}: «Имя пользователя»;
- Атрибут_{1.3}: «Отчество пользователя».

Изменение структуры соответствующего ИЛИ-узла дерева И/ИЛИ представлено на рисунке 6.15.

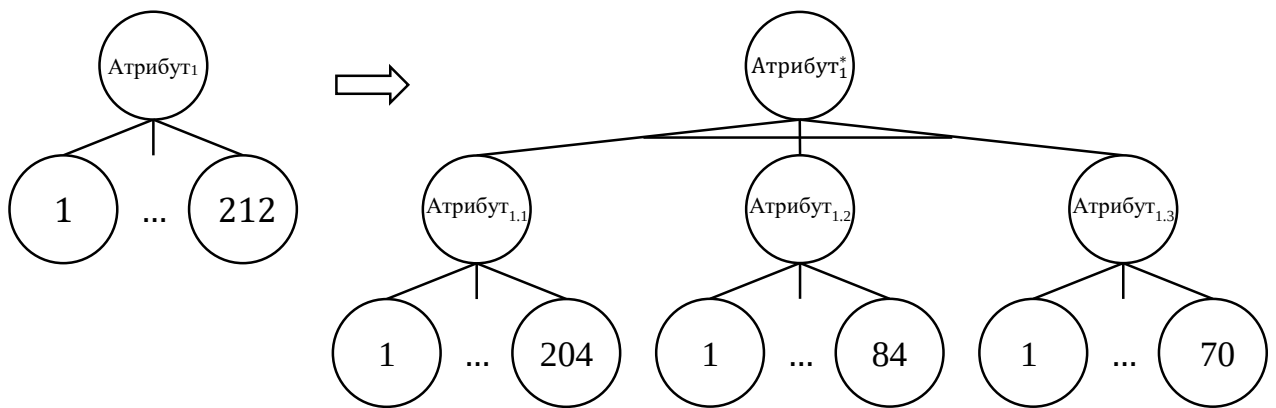


Рисунок 6.15 — Изменение структуры ИЛИ-узла для Атрибут₁

В таком случае будут получены следующие новые справочники уникальных значений атрибутов:

- Атрибут_{1.1}: 204 уникальных значения, объем файла равен 3 414 байт;
- Атрибут_{1.2}: 84 уникальных значений, объем файла равен 1 158 байт;
- Атрибут_{1.3}: 70 уникальных значений, объем файла равен 1 495 байт.

Общий объем полученных файлов справочников равен 6 067 байт, что составляет 56,3% от объема исходного файла справочника для Атрибут₁, то есть меньше на $10\,768 - 6\,067 = 4\,701$ байт. При этом изменение структуры ИЛИ-узла приводит к увеличению количества вариантов в поддереве данного узла:

- поддерево узла, помеченного Атрибут₁, имеет 212 вариантов, то есть максимальное значение ранга варианта равно 211, что требует 8 бит для хранения;
- поддерево узла, помеченного Атрибут₁^{*}, имеет $204 \cdot 84 \cdot 70 = 1\,199\,520$ вариантов, то есть для хранения ранга варианта требуется 21 бит.

Следовательно, на каждую запись журнала событий дополнительно потребуется $21 - 8 = 13$ бит для хранения, то есть для всех имеющихся записей понадобится дополнительно $116\,245 \cdot 13 = 1\,511\,185$ бит или 188 898,125 байт, что в результате приводит к увеличению объема хранимых данных.

Аналогичный отрицательный результат будет получен, если Атрибут₈, значения которого содержат информацию об IP-адресах, представить в виде комбинации четырех значений составляющих его байтов. В таком случае ИЛИ-узел для Атрибут₈ станет И-узлом с четырьмя потомками, каждый из которых содержит 256 вариантов. Это приведет к увеличению общего количества вариантов с 3016 до $256^4 = 4\,294\,967\,296$ и, соответственно, к увеличению объема хранимых данных.

Наибольшим по количеству уникальных значений и по объему файла справочника является Атрибут₆. Анализ значений данного атрибута показал, что в нем содержатся шаблонные предложения с подробным описанием журналируемых событий электронного курса, в которые встроены идентификаторы затронутых элементов электронного курса [270]. Если не учитывать значения идентификаторов, то все 36 513 уникальных значений для Атрибут₆ могут быть описаны с использованием 132 шаблонов. Каждый такой шаблон содержит до пяти идентификаторов затронутых элементов электронного курса, при этом количество уникальных значений самих идентификаторов отличается. Так как шаблоны могут содержать разное количество идентификаторов, то дополнительно введем пустой идентификатор, выбор которого будет означать выбор шаблона с меньшим количеством идентификаторов. Таким образом, можно предложить декомпозицию значений для Атрибут₆ в виде комбинации следующих атрибутов:

- Атрибут_{6.1}: «Шаблон»;
- Атрибут_{6.2}–Атрибут_{6.6}: «Идентификатор».

Измененная структура ИЛИ-узла дерева И/ИЛИ для Атрибут₆ представлена на рисунке 6.16.

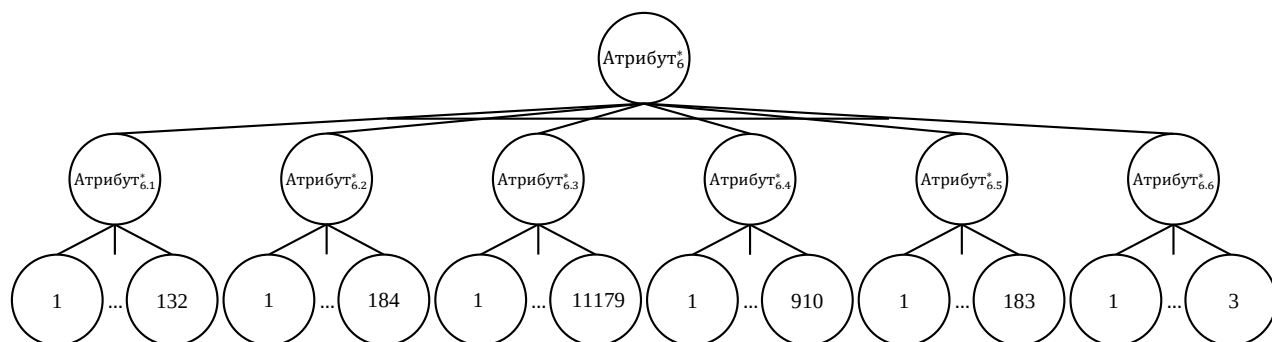


Рисунок 6.16 — Изменение структуры ИЛИ-узла для Атрибут₆

Общий объем полученных файлов справочников для Атрибут_{6.1}–Атрибут_{6.6} равен 122 632 байт, что составляет 2,7% от объема исходного файла справочника для Атрибут₆, то есть меньше на $4\,553\,086 - 122\,632 = 4\,430\,454$ байт. При этом

изменение структуры ИЛИ-узла приводит к увеличению количества вариантов в поддереве данного узла:

- поддерево узла, помеченного Атрибут₆, имеет 36 513 вариантов, то есть максимальное значение ранга варианта равно 36 512, что требует 16 бит для хранения;

- поддерево узла, помеченного Атрибут₆^{*}, имеет $132 \cdot 184 \cdot 11\,179 \cdot 910 \cdot 183 \cdot 3 \approx 1,4 \cdot 10^{14}$ вариантов, то есть для хранения ранга варианта требуется 47 бит.

Следовательно, на каждую запись журнала событий дополнительно потребуется $47 - 16 = 31$ бит для хранения, то есть для всех имеющихся записей понадобится дополнительно $116\,245 \cdot 31 = 3\,603\,595$ бит или 450 449,375 байт, что в результате приводит к уменьшению объема хранимых данных на 3 887 КБ ($4\,430\,454 - 450\,449,375 = 3\,980\,004,625$ байт).

Таким образом, положительный эффект с точки зрения сжатия данных рассматриваемого журнала событий показывает только декомпозиция Атрибут₆. Дополнительное улучшение данного эффекта может быть достигнуто за счет более детальной декомпозиции шаблонов по количеству используемых идентификаторов.

Шаг 5. Следующим этапом необходимо представить каждую запись рассматриваемого журнала событий в виде кортежа идентификаторов значений атрибутов, полученных на основе сформированных после декомпозиции справочников. В таблице 6.4 представлены требования к объему памяти, необходимому для хранения идентификаторов значений атрибутов. Таким образом, для хранения каждой нормализованной записи требуется в сумме 98 бит для кодируемых атрибутов. Тогда для всех имеющихся записей понадобится $116\,245 \cdot 98 = 11\,392\,010$ бит или 1 424 001,25 байт.

Если ограничиться данным этапом, то итоговый суммарный объем сжатого путем нормализации записей журнала событий будет состоять из:

- общего объема файлов справочников, равного 187 675 байт;
- объема текстового файла, равного 2 171 581 байт, с значениями Атрибут₀ для каждой записи исходного журнала событий;
- объема бинарного файла, равного 1 424 001,25 байт, с набором идентификаторов значений Атрибут₁–Атрибут₈ для каждой записи исходного журнала событий.

Суммарно получаем 3 783 257,25 байт или 3 695 КБ, что составляет 8,54% от объема исходного файла журнала событий.

Таблица 6.4 — Требования к объему памяти для хранения идентификаторов

Атрибут	Количество уникальных значений	Множество значений идентификатора	Количество бит для хранения идентификатора	Количество байт для хранения справочника
Атрибут ₁	212	$\{0, \dots, 211\}$	8	10 768
Атрибут ₂	212	$\{0, \dots, 211\}$	8	
Атрибут ₃	50	$\{0, \dots, 49\}$	6	2 703
Атрибут ₄	19	$\{0, \dots, 18\}$	5	516
Атрибут ₅	87	$\{0, \dots, 86\}$	7	4 574
Атрибут _{6.1}	132	$\{0, \dots, 131\}$	8	9 806
Атрибут _{6.2}	184	$\{0, \dots, 183\}$	8	1 263
Атрибут _{6.3}	11 179	$\{0, \dots, 11\,178\}$	14	100 486
Атрибут _{6.4}	910	$\{0, \dots, 909\}$	10	6 997
Атрибут _{6.5}	183	$\{0, \dots, 182\}$	8	4 039
Атрибут _{6.6}	3	$\{0, \dots, 2\}$	2	41
Атрибут ₇	4	$\{0, \dots, 3\}$	2	26
Атрибут ₈	3 016	$\{0, \dots, 3\,015\}$	12	46 456
Сумма:			98	187 675

Шаг 6. Последним этапом необходимо для каждой записи рассматриваемого журнала событий, представленной в виде кортежа идентификаторов значений атрибутов, вычислить значение ранга соответствующего варианта дерева И/ИЛИ. Построенная структура дерева И/ИЛИ для атрибутов рассматриваемого журнала событий, представленная на рисунке 6.14, с учетом внесенных изменений в структуру ИЛИ-узла для Атрибут₆, представленных на рисунке 6.16, содержит в себе следующее количество вариантов:

$$212 \cdot 212 \cdot 50 \cdot 19 \cdot 87 \cdot (132 \cdot 184 \cdot 11\,179 \cdot 910 \cdot 183 \cdot 3) \cdot 4 \cdot 3\,016 \approx 6,1 \cdot 10^{27}.$$

Таким образом, для хранения каждого ранга требуется 93 бит. Тогда для всех имеющихся записей понадобится $116\,245 \cdot 93 = 10\,810\,785$ бит или 1 351 348,125 байт (1 320 КБ), что на 5,1% меньше по сравнению с требуемым объемом памяти для хранения аналогичной информации в формате нормализованных записей. Итоговый суммарный объем сжатого журнала событий будет состоять из:

- общего объема файлов справочников, равного 187 675 байт;
- объема текстового файла, равного 2 171 581 байт, с значениями Атрибут₀ для каждой записи исходного журнала событий;
- объема бинарного файла, равного 1 351 348,125 байт, с рангом для каждой записи исходного журнала событий.

Суммарно получаем 3 710 604,125 байт или 3 624 КБ, что составляет 8,37% от объема исходного файла журнала событий. Следовательно, коэффициент сжатия равен 11,94.

На сегодняшний день существует большое количество готовых программных продуктов, специализирующихся на сжатии файлов без потерь [271; 272]. Например, популярны следующие форматы сжатых файлов: ZIP, 7z, RAR. Если сжать исходный файл журнала событий с помощью соответствующих программ-архиваторов с максимальным уровнем сжатия, то получим сокращение размера файла до 893 КБ в формате ZIP или 7z (алгоритм LZMA) и до 752 КБ в формате RAR. Данные результаты являются лучшими по сравнению с результатами апробации предложенного метода. Однако необходимо учесть, что файлы, которые были получены в ходе применения предложенного метода, также могут быть дополнительно сжаты с помощью соответствующих программ-архиваторов. Совокупный результат показал сжатие до 554 КБ в формате ZIP или 7z (алгоритм LZMA) и до 650 КБ в формате RAR. Следовательно, получаем сжатие без использования предложенного метода в лучшем случае до 752 КБ против сжатия в лучшем случае до 554 КБ с его использованием (увеличение коэффициента сжатия на 35%). При этом комбинация разных архиваторов не демонстрирует какого-либо эффекта дополнительного сжатия данных. Увеличение коэффициента сжатия также было достигнуто на других примерах журналов событий, генерируемых внутри электронных курсов системы Moodle. Таким образом, предварительное кодирование журналов событий с помощью предложенного метода позволяет увеличить коэффициент сжатия существующих методов сжатия данных.

Дополнительно было проведено исследование изменения коэффициента сжатия в динамике по мере заполнения журнала событий новыми записями. В данном случае использовался один журнал событий, на основе которого были сформированы 10 его подмножеств от $1 \cdot 10^4$ первых записей до $10 \cdot 10^4$ первых записей с шагом в 10^4 записей. Далее каждый полученный журнал событий был сжат алгоритмом LZMA (архиватор 7-Zip), а также был закодирован предложенным методом с последующим сжатием алгоритмом LZMA. В таблице 6.5 представлены результаты данного эксперимента, которые подтверждают эффективность предложенного метода кодирования. Комбинация кодирования предложенным методом с последующим сжатием алгоритмом LZMA также выигрывает по сравнению с результатами сжатия архиватором WinRAR, который продемонстрировал лучшие результаты по сжатию исходных некодированных данных (таблица 6.6).

Таблица 6.5 — Результаты сжатия журнала событий в динамике (7-Zip)

Количество записей	Исходный журнал, КБ	Сжатие 7-Zip (LZMA), КБ	Кодирование рангами, КБ	Кодирование рангами и 7-Zip, КБ	Коэффициент сжатия
$1 \cdot 10^4$	3 670	79	323	50	+58%
$2 \cdot 10^4$	7 427	156	644	95	+64%
$3 \cdot 10^4$	11 171	234	939	141	+66%
$4 \cdot 10^4$	14 922	313	1 243	188	+66%
$5 \cdot 10^4$	18 564	395	1 550	254	+56%
$6 \cdot 10^4$	22 297	467	1 841	294	+59%
$7 \cdot 10^4$	26 072	543	2 139	341	+59%
$8 \cdot 10^4$	29 840	621	2 518	371	+67%
$9 \cdot 10^4$	33 615	700	2 829	414	+69%
$10 \cdot 10^4$	37 278	764	3 166	473	+62%

Таблица 6.6 — Результаты сжатия журнала событий в динамике (WinRAR)

Количество записей	Исходный журнал, КБ	Сжатие WinRAR, КБ	Кодирование рангами, КБ	Кодирование рангами и WinRAR, КБ	Коэффициент сжатия
$1 \cdot 10^4$	3 670	71	323	50	+42%
$2 \cdot 10^4$	7 427	136	644	95	+43%
$3 \cdot 10^4$	11 171	200	939	141	+42%
$4 \cdot 10^4$	14 922	265	1 243	188	+41%
$5 \cdot 10^4$	18 564	336	1 550	254	+32%
$6 \cdot 10^4$	22 297	395	1 841	294	+34%
$7 \cdot 10^4$	26 072	457	2 139	341	+34%
$8 \cdot 10^4$	29 840	520	2 518	371	+40%
$9 \cdot 10^4$	33 615	585	2 829	414	+41%
$10 \cdot 10^4$	37 278	644	3 166	473	+36%

Аналогичные результаты были получены при кодировании журнала событий безопасности операционной системы Windows. Журналы событий операционной системы Windows хранятся в бинарном файле в формате EVTХ, который помимо записей о событиях содержит большое количество технической информации [273]. При этом имеется возможность сохранения в текстовый файл формата ТХТ, который содержит только записи журнала событий. Применение предложенного метода к файлу журнала событий в формате ТХТ также демонстрирует повышение коэффициента сжатия. Например, файл журнала событий безопасности, содержащий 29 200 записей размером 37 901 КБ, сжался до 59 КБ с использованием предварительного кодирования рангами против 101 КБ без его использования.

6.3.4 Кодирование реляционных баз данных

Предложенный метод кодирования журналов событий может быть использован при решении задачи кодирования файла реляционной базы данных. Реляционная база данных представляет собой совокупность иерархически связанных таблиц, при этом структура каждой таблицы аналогична рассмотренной выше структуре журнала событий (таблица содержит множество записей, каждая из которых формируется набором значений соответствующих атрибутов). Таким образом, можно проранжировать записи для каждой таблицы по отдельности. При этом для каждой таблицы формируются справочники уникальных значений атрибутов без учета атрибутов с внешними ключами, так как они ссылаются на первичные ключи, которые уже хранятся в соответствующих справочниках.

В качестве вычислительного эксперимента было проведено кодирование основной таблицы реляционной базы данных журнала событий системы управления обучением университета. Предоставленный университетом файл базы данных содержал таблицу с 2 358 056 строк данных, которые занимают 530,2 МБ. При этом файл базы данных занимает 584 МБ, так как в нем содержится дополнительная техническая информация, и сжимается алгоритмом LZMA (архиватор 7-Zip) до 77,2 МБ. Если экспортировать все строки данных в файл в формате CSV, то его размер составит 426,6 МБ, которые сжимаются алгоритмом LZMA до 16,9 МБ. Если предварительно реализовать кодирование записей журнала событий с помощью предложенного метода, то в результате последующего сжатия алгоритмом LZMA достигается значение 11,7 МБ. Таким образом, достигается дополнительный эффект увеличения коэффициента сжатия на 44%.

Кроме того, реляционные базы данных могут использоваться как инструмент хранения вариантов деревьев И/ИЛИ в декодированном состоянии [274].

6.4 Выводы по главе

В данной главе разработан и исследован метод блочного кодирования цифровых данных, отличающийся применением алгоритмов комбинаторной генерации на основе структур деревьев И/ИЛИ. Предложенный метод позволяет представить

цифровые данные в виде последовательности блоков, каждый из которых описывается кортежем атрибутов, что обеспечивает их структурированное представление.

Основные научные и практические результаты, полученные в главе:

1. Разработана общая схема блочного кодирования, включающая этапы разбиения данных на блоки, декомпозиции атрибутов, построения справочников уникальных значений и вычисления рангов на основе структур деревьев И/ИЛИ. Предложены алгоритмы кодирования и декодирования, обеспечивающие биективное отображение между исходными данными и их представлением в виде последовательности значений рангов;

2. Для комбинаторного множества блоков данных $T_{m,k}$ с единым справочником k уникальных значений для m атрибутов получены различные формулы вычисления его мощности. На их основе построены структуры деревьев И/ИЛИ, для которых разработаны алгоритмы ранжирования и генерации по рангу;

3. Проведена апробация метода на примерах кодирования текстовых файлов и растровых изображений. Показано, что прямое применение метода не обеспечивает высокого коэффициента сжатия по сравнению с известными аналогами (LZMA, PNG). Однако преобразование данных в представление в виде последовательности значений рангов приводит к статистически равномерному распределению, что открывает перспективы для использования в генераторах псевдослучайных последовательностей;

4. Разработан метод кодирования журналов событий информационных систем, учитывающий их высокую структурированность и избыточность. Метод включает этапы выделения атрибутов, декомпозиции значений, нормализации и ранжирования. Апробация на реальных данных (журналы событий электронных курсов Moodle и журналы безопасности операционной системы Windows) показала увеличение коэффициента сжатия при комбинации с алгоритмом LZMA;

5. Продемонстрирована возможность применения метода кодирования журналов событий информационных систем для кодирования реляционных баз данных. Эксперименты подтвердили повышение эффективности сжатия по сравнению с использованием только алгоритма LZMA.

Таким образом, предложенный метод блочного кодирования на основе структур деревьев И/ИЛИ является эффективным инструментом для обработки структурированных цифровых данных, особенно в случаях высокой избыточности.

Результаты данной главы опубликованы в [249; 255; 265].

Заключение

В диссертационной работе решена научная проблема кодирования информационных объектов, представляющих собой сложные дискретные структуры с биективным отображением на деревья И/ИЛИ.

Основные результаты диссертационной работы:

1. Проведен аналитический обзор современного состояния исследований в области построения алгоритмов комбинаторной генерации для решения задач кодирования информации. Показана возможность представления множеств информационных объектов, обрабатываемых цифровыми устройствами, в форме комбинаторных множеств. Составлена общая схема кодирования информационных объектов, представляющих собой сложные дискретные структуры, на основе алгоритмов комбинаторной генерации;

2. Изложена методология построения алгоритмов кодирования информационных объектов на основе рекурсивных композиций структур деревьев И/ИЛИ. Предложенная расширенная алгебра функций мощности комбинаторных множеств обеспечивает возможность построения рекурсивных композиций структур деревьев И/ИЛИ и алгоритмов комбинаторной генерации для множеств их вариантов. Кроме того, расширенная алгебра позволяет обрабатывать структуры, которые задаются в общем виде с использованием целочисленных параметров, при этом алгоритмы комбинаторной генерации динамически подстраиваются под частную структуру дерева И/ИЛИ для конкретных значений параметров. Особенностью данной методологии является промежуточное представление кодируемого информационного объекта в виде варианта соответствующей структуры дерева И/ИЛИ, что позволяет выполнить декомпозицию сложной дискретной структуры на более простые и реализовать комбинацию наиболее эффективных алгоритмов комбинаторной генерации. В том числе появляется возможность комбинирования известных алгоритмов, полученных с использованием совершенно другого математического аппарата. В качестве демонстрации возможностей построения алгоритмов комбинаторной генерации для разных рекурсивных композиций структур деревьев И/ИЛИ разработаны три набора алгоритмов ранжирования и генерации по рангу для комбинаторных множеств, связанных со специальными числами Фубини;

3. Исследована связь рекурсивных композиций структур деревьев И/ИЛИ с результатом выполнения операций над производящими функциями для комбинаторных множеств. В частности, декомпозиция производящей функции с использованием операций сложения, умножения, возведения в степень и композиции задает способ комбинирования алгоритмов комбинаторной генерации для простых структур, из которых формируется исследуемая сложная структура. Также рассмотрен класс алгебраических производящих функций, которые задаются в виде функционального уравнения, и доказаны теоремы о рекуррентной формуле для вычисления коэффициентов их степеней. Использование данных рекуррентных формул позволяют строить алгоритмы кодирования информационных объектов, мощность множества которых определяется алгебраической производящей функцией, с полиномиальной оценкой вычислительной сложности. Таким образом, на основе полученных результатов сформирован общий метод кодирования информационных объектов, мощность множества которых определяется коэффициентами алгебраических производящих функций и их степеней;

4. Предложенная методология апробирована на примере построения алгоритмов кодирования информационных объектов, представленных решеточными путями. Рассмотрены частные случаи решеточных путей (северо-восточные решеточные пути, пути Дика, пути Деланнуа, пути Шредера и пути Моцкина), где для каждого из них построена рекурсивная композиция структур деревьев И/ИЛИ и разработаны алгоритмы комбинаторной генерации. Кроме того, решена задача кодирования множеств решеточных путей для общего случая, а именно: разработаны алгоритмы ранжирования и генерации по рангу для множества направленных решеточных путей на плоскости. Также для решения данной задачи доказаны теоремы о рекуррентных формулах для подсчета количества направленных решеточных путей. Таким образом, на основе полученных результатов сформирован общий метод кодирования информационных объектов, представленных решеточными путями на плоскости;

5. Предложенная методология апробирована на примере построения алгоритмов кодирования информационных объектов, представленных словами контекстно-свободных грамматик. Разработан метод кодирования информационных объектов, строящихся по правилам вывода контекстно-свободных грамматик, отличительной особенностью которого является использование биективного отображения деревьев вывода на структуры вариантов деревьев И/ИЛИ. Полученная

связь рекурсивных композиций деревьев И/ИЛИ с контекстно-свободными грамматиками позволяет строить алгоритмы кодирования для информационных объектов, которые определяются более чем одним параметром и для построения которых применимы правила вывода контекстно-свободных грамматик. Разработанный метод кодирования успешно применен к частным случаям контекстно-свободных грамматик, в том числе для грамматики, генерирующей скобочные представления для вторичных структур РНК без псевдоузлов. Также разработан метод получения явных формул для подсчета количества слов заданной длины, выводимых из нетерминального символа однозначной контекстно-свободной грамматики. Данный метод дополняет метод кодирования в части решения задач перечисления комбинаторных множеств;

6. Разработан метод блочного кодирования цифровых данных, который базируется на получении структурированного представления цифровых данных в виде последовательности блоков, каждый из которых формируется кортежем атрибутов. Получен ряд формул для вычисления мощности таких комбинаторных множеств. Рассмотрены примеры соответствующих информационных объектов, а именно: текстовые файлы и растровые изображения. Используя результаты статистических тестов сформулированы выводы о возможности реализации генератора псевдослучайных числовых последовательностей на основе бинарных файлов со значениями рангов для блоков кодируемого изображения в двоичной системе счисления. Разработан метод кодирования журналов событий информационных систем. В качестве апробации разработанный метод успешно применен для кодирования журналов событий, генерируемых внутри электронных курсов системы Moodle, а также для кодирования журналов событий безопасности операционной системы Windows. Кроме того, предложенный метод кодирования журналов событий был использован при решении задачи кодирования файла реляционной базы данных. Показано, что предварительное кодирование журналов событий с помощью предложенного метода позволяет увеличить коэффициент сжатия для алгоритма LZMA;

7. Все разработанные алгоритмы кодирования были реализованы в виде программ с использованием системы компьютерной алгебры «Maxima», а также в виде программ на языке программирования Python. Результаты вычислительных экспериментов, выполненных с помощью созданного комплекса программ, подтверждают корректность работы разработанных алгоритмов кодирования.

Список литературы

1. Федеральный закон от 27.07.2006 № 149-ФЗ (ред. от 23.11.2024) «Об информации, информационных технологиях и о защите информации» (с изм. и доп., вступ. в силу с 01.01.2025).
2. Digital 2024: Global overview report [Электронный ресурс]. — Режим доступа: <https://datareportal.com/reports/digital-2024-global-overview-report>.
3. Taylor, P. Data growth worldwide 2010–2028 | Statista [Электронный ресурс]. — 2024. — Режим доступа: <https://www.statista.com/statistics/871513/worldwide-data-created/>.
4. Марков, А. А. Введение в теорию кодирования / А. А. Марков. — М.: Наука, 1982. — 192 с.
5. Методы сжатия данных. Устройство архиваторов, сжатие изображений и видео / Д. Ватолин, А. Ратушняк, М. Смирнов, В. Юкин. — М.: ДИАЛОГ-МИФИ, 2003. — 384 с.
6. Salomon, D. Handbook of data compression / D. Salomon, G. Motta. — USA: Springer, 2010. — 1360 p.
7. Schneier, B. Applied cryptography: Protocols, algorithms, and source code in C / B. Schneier. — USA: John Wiley & Sons Inc, 1996. — 758 p.
8. Василенко, О. Н. Теоретико-числовые алгоритмы в криптографии / О. Н. Василенко. — М.: МЦНМО, 2003. — 328 с.
9. Блейхут, Р. Э. Теория и практика кодов, контролирующих ошибки / Р. Э. Блейхут. — М.: Мир, 1986. — 576 с.
10. Huffman, W. C. Fundamentals of error-correcting codes / W. C. Huffman, V. Pless. — USA: Cambridge University Press, 2003. — 668 p.
11. Майоров, А. А. Информационные объекты в информационном поле / А. А. Майоров // Образовательные ресурсы и технологии. — 2015. — № 1 (9). — С. 66–73.

12. Knuth, D. E. The art of computer programming. Volume 4A: Combinatorial algorithms, part 1 / D. E. Knuth. — USA: Addison-Wesley, 2011. — 912 p.
13. Knuth, D. E. The art of computer programming. Volume 4B: Combinatorial algorithms, part 2 / D. E. Knuth. — USA: Addison-Wesley, 2022. — 736 p.
14. Kreher, D. L. Combinatorial algorithms: Generation, enumeration, and search / D. L. Kreher, D. R. Stinson. — USA: CRC Press, 1999. — 342 p.
15. Ruskey, F. Combinatorial generation [Электронный ресурс]. — 2003. — Режим доступа: <https://page.math.tu-berlin.de/~felsner/SemWS17-18/Ruskey-Comb-Gen.pdf>.
16. ECO: A methodology for the enumeration of combinatorial objects / E. Barcucci, A. Del Lungo, E. Pergola, R. Pinzani // Journal of Difference Equations and Applications. — 1999. — Vol. 5, no. 4–5. — P. 435–490.
17. Combinatorial generation via permutation languages. I. Fundamentals / E. Hartung, H. P. Hoang, T. Mutze, A. Williams // Transactions of the American Mathematical Society. — 2022. — Vol. 375. — P. 2255–2291.
18. Goldberg, A. V. Compression and ranking / A. V. Goldberg, M. Sipser // SIAM Journal on Computing. — 1991. — Vol. 20, no. 3. — P. 524–536.
19. Данилкин, Ф. А. Комбинаторное кодирование для сжатия данных / Ф. А. Данилкин, М. Л. Гришин // Известия Тульского государственного университета. Технические науки. — 2008. — № 2. — С. 251–258.
20. Гришин, М. Л. Методы построения информационно-измерительных систем глобального геомониторинга подвижных объектов в реальном времени : дис. ... канд. техн. наук : 05.11.16 / Гришин Максим Леонидович. — Тула, 2012. — 168 с.
21. Grammar compression with probabilistic context-free grammar / H. Naganuma, D. Hendrian, R. Yoshinaka et al. // Data Compression Conference. — 2020.
22. Improved algorithms for ranking and unranking (k, m) -ary trees in B -order / Y.-H. Chang, R.-Y. Wu, R.-S. Chang, J.-M. Chang // Journal of Combinatorial Optimization. — 2022. — Vol. 44, no. 3. — P. 1495–1510.

23. Bozinovski, A. Full linearization of ranking and unranking of Catalan cipher vectors using Catalan triangle abstraction / A. Bozinovski, B. Stojcevska // Journal of Information Technology and Applications. — 2024. — Vol. 28, no. 2. — P. 136–141.
24. Bozinovski, A. Enumeration, ranking and generation of binary trees based on level-order traversal using Catalan cipher vectors / A. Bozinovski, B. Stojcevska, V. Pacovski // Journal of Information Technology and Applications. — 2013. — Vol. 6, no. 2. — P. 78–86.
25. Mikawa, K. Efficient linear-time ranking and unranking of derangements / K. Mikawa, K. Tanaka // Information Processing Letters. — 2023. — Vol. 179. — Article 106288.
26. Kagey, P. Ranking and unranking restricted permutations / P. Kagey // Discrete Applied Mathematics. — 2024. — Vol. 355. — P. 247–261.
27. Gabric, D. Ranking and unranking bordered and unbordered words / D. Gabric // Information Processing Letters. — 2024. — Vol. 184. — Article 106452.
28. Radoszewski, J. Faster algorithms for ranking/unranking bordered and unbordered words / J. Radoszewski, W. Rytter, T. Walen // Lecture Notes in Computer Science: International Symposium on String Processing and Information Retrieval. — 2024. — Vol. 14899. — P. 257–271.
29. Adamson, D. Ranking and unranking k -subsequence universal words / D. Adamson // Lecture Notes in Computer Science: International Conference on Combinatorics on Words. — 2023. — Vol. 13899. — P. 47–59.
30. Curiel, A. Lexicographic unranking algorithms for the twelvefold way / A. Curiel, A. Genitrini // International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms. — Vol. 302. — Article 17. — 2024.
31. Стенли, Р. Перечислительная комбинаторика / Р. Стенли. — М.: Мир, 1990. — 440 с.
32. Cameron, B. Pivot Gray codes for the spanning trees of a graph ft. the fan / B. Cameron, A. Grub, J. Sawada // Graphs and Combinatorics. — 2024. — Vol. 40. — Article 78.

33. Generating m -ary Gray codes and related algorithms / S. Bouyuklieva, I. Bouyukliev, V. Bakoev, M. Pashinska-Gadzheva // Algorithms. — 2024. — Vol. 17, no. 7. — Article 311.
34. Dynamic 3D combinatorial generation of hPSC-derived neuromesodermal organoids with diverse regional and cellular identities / D. Whye, D. Wood, K. H. Kim et al. // Current Protocols. — 2022. — Vol. 2, no. 10. — Article e568.
35. Eckroth, J. OneRuleToFindThem: Efficient automated generation of password cracking rules / J. Eckroth, L. Hough, H. ElAarag // Journal of Computing Sciences in Colleges. — 2023. — Vol. 39, no. 3. — P. 226–248.
36. Roussillon, T. Combinatorial generation of planar sets / T. Roussillon // Journal of Mathematical Imaging and Vision. — 2023. — Vol. 65, no. 5. — P. 702–717.
37. Wells, M. B. Generation of permutations by transposition / M. B. Wells // Mathematics of Computation. — 1961. — Vol. 15, no. 74. — P. 192–195.
38. Johnson, S. M. Generation of permutations by adjacent transposition / S. M. Johnson // Mathematics of Computation. — 1963. — Vol. 17, no. 83. — P. 282–285.
39. Heap, B. R. Permutations by interchanges / B. R. Heap // The Computer Journal. — 1963. — Vol. 6, no. 3. — P. 293–298.
40. Sedgewick, R. Permutation generation methods / R. Sedgewick // ACM Computing Surveys. — 1977. — Vol. 9, no. 2. — P. 137–164.
41. Lipski Jr., W. More on permutation generation methods / W. Lipski Jr. // Computing. — 1979. — Vol. 23. — P. 357–365.
42. Zaks, S. A new algorithm for generation of permutations / S. Zaks // BIT Numerical Mathematics. — 1984. — Vol. 24. — P. 196–204.
43. Viktorov, O. V. Permutation generation algorithm / O. V. Viktorov // Asian Journal of Information Technology. — 2007. — Vol. 6, no. 9. — P. 956–957.
44. Kuo, T. A new method for generating permutations in lexicographic order / T. Kuo // Journal of Science and Engineering Technology. — 2009. — Vol. 5, no. 4. — P. 21–29.

45. Ibrahim, H. New algorithm for listing all permutations / H. Ibrahim, Z. Omar, A. Rohni // Modern Applied Science. — 2010. — Vol. 4, no. 2. — P. 89–94.
46. New permutation generation under exchange strategy (PGuES) / S. Karim, H. Ibrahim, Z. Omar, N. Ahmad // Modern Applied Science. — 2013. — Vol. 7, no. 8. — P. 15–21.
47. Lawnik, M. Generating and numbering permutations with the use of chaotic maps / M. Lawnik, A. Banasik // International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications. — 2021. — P. 387–390.
48. Er, M. C. Generation, ranking and unranking of permutations based on the mixed-radix codes / M. C. Er // Journal of Information and Optimization Sciences. — 1985. — Vol. 7, no. 2. — P. 105–111.
49. Critani, F. Ranking and unranking permutations with applications / F. Critani, M. Dall'Aglio, G. Di Biase // Transactions on Engineering Sciences. — 1997. — Vol. 15. — P. 99–106.
50. Myrvold, W. Ranking and unranking permutations in linear time / W. Myrvold, F. Ruskey // Information Processing Letters. — 2001. — Vol. 79, no. 6. — P. 281–284.
51. Mares, M. Linear-time ranking of permutations / M. Mares, M. Straka // Lecture Notes in Computer Science: Algorithms. — 2007. — Vol. 4698. — P. 187–193.
52. Bonet, B. Efficient algorithms to rank and unrank permutations in lexicographic order / B. Bonet // Workshop on Search in Artificial Intelligence and Robotics. — 2008. — P. 18–23.
53. Parks, H. R. Improved linear-time ranking of permutations / H. R. Parks, D. C. Wills // Journal of Applied Mathematics and Computation. — 2021. — Vol. 5, no. 4. — P. 277–282.
54. Effler, S. A CAT algorithm for generating permutations with a fixed number of inversions / S. Effler, F. Ruskey // Information Processing Letters. — 2003. — Vol. 86, no. 2. — P. 107–112.

55. Roelants van Baronaigien, D. Generating permutations with given ups and downs / D. Roelants van Baronaigien, F. Ruskey // Discrete Applied Mathematics. — 1992. — Vol. 36, no. 1. — P. 57–65.
56. Korsh, J. F. Loopless generation of up-down permutations / J. F. Korsh // Discrete Mathematics. — 2001. — Vol. 240, no. 1–3. — P. 97–122.
57. Vajnovszki, V. An efficient Gray code algorithm for generating all permutations with a given major index / V. Vajnovszki // Journal of Discrete Algorithms. — 2014. — Vol. 26. — P. 77–88.
58. Baril, J.-L. Gray code for permutations with a fixed number of cycles / J.-L. Baril // Discrete Mathematics. — 2007. — Vol. 307, no. 13. — P. 1559–1571.
59. Akl, S. G. A new algorithm for generating derangements / S. G. Akl // BIT Numerical Mathematics. — 1980. — Vol. 20. — P. 2–7.
60. Baril, J.-L. Gray code for derangements / J.-L. Baril, V. Vajnovszki // Discrete Applied Mathematics. — 2004. — Vol. 140, no. 1–3. — P. 207–221.
61. Akl, S. G. A comparison of combination generation methods / S. G. Akl // ACM Transactions on Mathematical Software. — 1981. — Vol. 7, no. 1. — P. 42–45.
62. Mifsud, C. J. Algorithm 154: Combination in lexicographical order / C. J. Mifsud // Communications of the ACM. — 1963. — Vol. 6, no. 3. — P. 103.
63. Chan, B. Generating combinations in parallel / B. Chan, S. G. Akl // BIT. — 1986. — Vol. 26, no. 1. — P. 1–6.
64. Chen, G. H. Parallel generation of permutations and combinations / G. H. Chen, M.-S. Chern // BIT. — 1986. — Vol. 26, no. 3. — P. 277–283.
65. Akl, S. G. An optimal parallel algorithm for generating combinations / S. G. Akl, D. Gries, I. Stojmenovic // Information Processing Letters. — 1989. — Vol. 33, no. 3. — P. 135–139.
66. Lin, C.-J. A parallel algorithm for generating combinations / C.-J. Lin // Computers and Mathematics with Applications. — 1989. — Vol. 17, no. 12. — P. 1523–1533.

67. Tsay, J. C. A systolic design for generating combinations in lexicographic order / J. C. Tsay, C. J. Lin // *Parallel Computing*. — 1990. — Vol. 13, no. 1. — P. 119–125.
68. Stojmenovic, I. A simple systolic algorithm for generating combinations in lexicographic order / I. Stojmenovic // *Computers and Mathematics with Applications*. — 1992. — Vol. 24, no. 4. — P. 61–64.
69. Elhage, H. Systolic generation of combinations from arbitrary elements / H Elhage, I Stojmenovic // *Parallel Processing Letters*. — 1992. — Vol. 2, no. 2–3. — P. 241–248.
70. Kapralski, A. New methods for the generation of permutations, combinations, and other combinatorial objects in parallel / A. Kapralski // *Journal of Parallel and Distributed Computing*. — 1993. — Vol. 17, no. 4. — P. 315–326.
71. Xu, C.-W. A new parallel combination generator / C.-W. Xu, X Ma, W.-K. Shiue // *International Conference on Parallel and Distributed Processing Techniques and Applications*. — 1996. — P. 25–28.
72. Kokosinski, Z. On parallel generation of combinations in associative processor architectures / Z. Kokosinski // *IASTED International Conference on Parallel and Distributed Systems*. — 1997. — P. 283–289.
73. Itai, A. Generating permutations and combinations in lexicographical order / A. Itai // *Journal of the Brazilian Computer Society*. — 2001. — Vol. 7, no. 3. — P. 65–68.
74. Eades, P. An algorithm for generating subsets of fixed size with a strong minimal change property / P. Eades, B. McKay // *Information Processing Letters*. — 1984. — Vol. 19, no. 3. — P. 131–133.
75. Xiang, L. On $O(1)$ time algorithms for combinatorial generation / L. Xiang, K. Ushijima // *Computer Journal*. — 2001. — Vol. 44, no. 4. — P. 292–302.
76. Ruskey, F. The coolest way to generate combinations / F. Ruskey, A. Williams // *Discrete Mathematics*. — 2009. — Vol. 309, no. 17. — P. 5305–5320.

77. Torres-Jimenez, J. A low spatial complexity algorithm to generate combinations with the strong minimal change property / J. Torres-Jimenez, I. Izquierdo-Marquez // Discrete Mathematics, Algorithms and Applications. — 2019. — Vol. 11, no. 5. — Article 1950060.
78. Knott, G. D. A numbering system for combinations / G. D. Knott // Communications of the ACM. — 1974. — Vol. 17, no. 1. — P. 45–46.
79. Er, M. C. Lexicographic ordering, ranking and unranking of combinations / M. C. Er // International Journal of Computer Mathematics. — 1985. — Vol. 17, no. 3–4. — P. 277–283.
80. Kokosinski, Z. Algorithms for unranking combinations and their applications / Z. Kokosinski // IASTED/ISMM International Conference on Parallel and Distributed Computing and Systems. — 1995. — P. 216–224.
81. Kokosinski, Z. Unranking combinations in parallel / Z. Kokosinski // International Conference on Parallel and Distributed Processing Techniques and Applications. — 1996. — P. 79–82.
82. Shimizu, T. Unranking of small combinations from large sets / T. Shimizu, T. Fukunaga, H. Nagamochi // Journal of Discrete Algorithms. — 2014. — Vol. 29. — P. 8–20.
83. Parque, V. Towards the succinct representation of m out of n / V. Parque, T. Miyashita // Lecture Notes in Computer Science: Internet and Distributed Computing Systems. — 2018. — Vol. 11226. — P. 16–26.
84. Parque, V. Unranking combinations using gradient-based optimization / V. Parque, T. Miyashita // International Conference on Tools with Artificial Intelligence. — 2018. — P. 579–586.
85. Genitrini, A. Lexicographic unranking of combinations revisited / A. Genitrini, M. Pepin // Algorithms. — 2021. — Vol. 14, no. 3. — Article 97.
86. Unranking small combinations of a large set in co-lexicographic order / V. V. Kruchinin, Y. V. Shablya, D. V. Kruchinin, V. M. Rulevskiy // Algorithms. — 2022. — Vol. 15, no. 2. — Article 36.

87. Stanley, R. P. Catalan numbers / R. P. Stanley. — USA: Cambridge University Press, 2015. — 244 p.
88. Lucas, J. M. On rotations and the generation of binary trees / J. M. Lucas, D. R. Vanbaronaigien, F. Ruskey // Journal of Algorithms. — 1993. — Vol. 15, no. 3. — P. 343–366.
89. Vajnovszki, V. On the loopless generation of binary tree sequences / V. Vajnovszki // Information Processing Letters. — 1998. — Vol. 68, no. 3. — P. 113–117.
90. Pallo, J. M. Enumerating, ranking and unranking binary trees / J. M. Pallo // The Computer Journal. — 1986. — Vol. 29, no. 2. — P. 171–175.
91. A loopless algorithm for generating multiple binary tree sequences simultaneously / R.-Y. Wu, J.-M. Chang, H.-C. Chan, K.-J. Pai // Theoretical Computer Science. — 2014. — Vol. 556. — P. 25–33.
92. Amortized efficiency of generation, ranking and unranking left-child sequences in lexicographic order / K.-J. Pai, J.-M. Chang, R.-Y. Wu, S.-C. Chang // Discrete Applied Mathematics. — 2019. — Vol. 268. — P. 223–236.
93. Ruskey, F. Generating binary trees lexicographically / F. Ruskey, T. C. Hu // SIAM Journal on Computing. — 1977. — Vol. 6, no. 4. — P. 745–758.
94. Proskurowski, A. Binary tree gray codes / A. Proskurowski, F. Ruskey // Journal of Algorithms. — 1985. — Vol. 6, no. 2. — P. 225–238.
95. Ruskey, F. Generating binary trees by transpositions / F. Ruskey, A. Proskurowski // Journal of Algorithms. — 1990. — Vol. 11, no. 1. — P. 68–84.
96. Zaks, S. Lexicographic generation of ordered trees / S. Zaks // Theoretical Computer Science. — 1980. — Vol. 10, no. 1. — P. 63–82.
97. Roelants van Baronaigien, D. A loopless algorithm for generating binary tree sequences / D. Roelants van Baronaigien // Information Processing Letters. — 1991. — Vol. 39, no. 4. — P. 189–194.
98. Ruskey, F. Generating t -ary trees lexicographically / F. Ruskey // SIAM Journal on Computing. — 1978. — Vol. 7, no. 4. — P. 424–439.

99. Akl, S. G. Generating t -ary trees in parallel / S. G. Akl, I. Stojmenovic // Nordic Journal of Computing. — 1996. — Vol. 3, no. 1. — P. 63–71.
100. Ahrabian, H. Generation of t -ary trees with Ballot-sequences / H. Ahrabian, A. Nowzari-Dalini // International Journal of Computer Mathematics. — 2003. — Vol. 80, no. 10. — P. 1243–1249.
101. Ahrabian, H. Parallel generation of t -ary trees with Ballot-sequences / H. Ahrabian, A. Nowzari-Dalini // Journal of Discrete Mathematical Sciences and Cryptography. — 2006. — Vol. 9, no. 2. — P. 225–234.
102. Ranking and unranking of t -ary trees using RD-sequences / R.-Y. Wu, J.-M. Chang, A.-H. Chen, C.-L. Liu // IEICE Transactions on Information and Systems. — 2011. — Vol. E94.D, no. 2. — P. 226–232.
103. Ranking and unranking t -ary trees in a Gray-code order / R.-Y. Wu, J.-M. Chang, A.-H. Chen, C.-L. Liu // The Computer Journal. — 2013. — Vol. 56, no. 11. — P. 1388–1395.
104. Amani, M. Efficient generation, ranking, and unranking of (k, m) -ary trees in B -order / M. Amani, A. Nowzari-Dalini // Bulletin of the Iranian Mathematical Society. — 2019. — Vol. 45. — P. 1145–1158.
105. A loopless algorithm for generating (k, m) -ary trees in Gray code order / Y.-H. Chang, R.-Y. Wu, C.-K. Lin, J.-M. Chang // Optimization Letters. — 2021. — Vol. 15. — P. 1133–1154.
106. Li, L. Ranking and unranking of AVL-trees / L. Li // SIAM Journal on Computing. — 1986. — Vol. 15, no. 4. — P. 1025–1035.
107. Gupta, U. I. Ranking and unranking of B -trees / U. I. Gupta, D. T. Lee, C. K. Wong // Journal of Algorithms. — 2024. — Vol. 4, no. 1. — P. 51–60.
108. Pallo, J. Generating trees with n nodes and m leaves / J. Pallo // International Journal of Computer Mathematics. — 1987. — Vol. 21, no. 2. — P. 133–144.
109. Seyed-Tabari, E. A new algorithm for generation of different types of RNA / E. Seyed-Tabari, H. Ahrabian, A. Nowzari-Dalini // International Journal of Computer Mathematics. — 2010. — Vol. 87, no. 6. — P. 1197–1207.

110. Reingold, E. M. Combinatorial algorithms: Theory and practice / E. M. Reingold, J. Nievergelt, N. Deo. — USA: Prentice-Hall, 1977. — 433 p.
111. Barcucci, E. Random generation of trees and other combinatorial objects / E. Barcucci, A. Del Lungo, E. Pergola // Theoretical Computer Science. — 1999. — Vol. 218, no. 2. — P. 219–232.
112. Exhaustive generation of combinatorial objects by ECO / S. Bacchelli, E. Barcucci, E. Grazzini, E. Pergola // Acta Informatica. — 2004. — Vol. 40, no. 8. — P. 585–602.
113. Mixed succession rules: The commutative case / S. Bacchelli, L. Ferrari, R. Pinzani, R. Sprugnoli // Journal of Combinatorial Theory, Series A. — 2010. — Vol. 117, no. 5. — P. 568–582.
114. Del Lungo, A. ECO method and the exhaustive generation of convex polyominoes / A. Del Lungo, A. Frosini, S. Rinaldi // Lecture Notes in Computer Science: Discrete Mathematics and Theoretical Computer Science. — 2003. — Vol. 2731. — P. 129–140.
115. On the generation and enumeration of some classes of convex polyominoes / A. Del Lungo, E. Duchi, A. Frosini, S. Rinaldi // Electronic Journal of Combinatorics. — 2004. — Vol. 11, no. 1. — Article R60.
116. Baril, J.-L. ECO-generation for p -generalized Fibonacci and Lucas permutations / J.-L. Baril, P.-T. Do // Pure Mathematics and Applications. — 2006. — Vol. 17, no. 1–2. — P. 19–37.
117. Combinatorial Gray codes for classes of pattern avoiding permutations / W. M. B. Dukes, M. F. Flanagan, T. Mansour, V. Vajnovszki // Theoretical Computer Science. — 2008. — Vol. 396, no. 1–3. — P. 35–49.
118. Baril, J.-L. More restrictive Gray codes for some classes of pattern avoiding permutations / J.-L. Baril // Information Processing Letters. — 2009. — Vol. 109, no. 14. — P. 799–804.
119. Vajnovszki, V. Generating involutions, derangements, and relatives by ECO / V. Vajnovszki // Discrete Mathematics and Theoretical Computer Science. — 2010. — Vol. 12, no. 1. — P. 109–122.

120. Baril, J.-L. ECO-generation for some restricted classes of compositions / J.-L. Baril, P.-T. Do // Computer Science Journal of Moldova. — 2013. — Vol. 21, no. 1. — P. 97–119.
121. Do, P. T. Exhaustive generation for permutations avoiding (colored) regular sets of patterns / P. T. Do, T. T. H. Tran, V. Vajnovszki // Discrete Applied Mathematics. — 2019. — Vol. 268. — P. 44–53.
122. Hoang, H. P. Combinatorial generation via permutation languages. II. Lattice congruences / H. P. Hoang, T. Mutze // Israel Journal of Mathematics. — 2021. — Vol. 244, no. 1. — P. 359–417.
123. Merino, A. Combinatorial generation via permutation languages. III. Rectangulations / A. Merino, T. Mutze // Discrete & Computational Geometry. — 2023. — Vol. 70. — P. 51–122.
124. Cardinal, J. Combinatorial generation via permutation languages. IV. Elimination Trees / J. Cardinal, A. Merino, T. Mutze // ACM Transactions on Algorithms. — 2024. — Vol. 21, no. 1. — Article 13.
125. Combinatorial generation via permutation languages. V. Acyclic orientations / J. Cardinal, H. P. Hoang, A. Merino et al. // SIAM Journal on Discrete Mathematics. — 2023. — Vol. 37, no. 3. — P. 1509–1547.
126. Gregor, P. Combinatorial generation via permutation languages. VI. Binary trees / P. Gregor, T. Mutze, Namrata // European Journal of Combinatorics. — 2024. — Vol. 122. — Article 104020.
127. Flajolet, P. A calculus for the random generation of combinatorial structures / P. Flajolet, P. Zimmerman, B. Cutsem // Theoretical Computer Science. — 1994. — Vol. 132, no. 1–2. — P. 1–35.
128. Martinez, C. A generic approach for the unranking of labeled combinatorial classes / C. Martinez, X. Molinero // Random Structures and Algorithms. — 2001. — Vol. 19, no. 3–4. — P. 472–497.
129. Martinez, C. Generic algorithms for the generation of combinatorial objects / C. Martinez, X. Molinero // Lecture Notes in Computer Science: Mathematical Foundations of Computer Science. — 2003. — Vol. 2747. — P. 572–581.

130. Martinez, C. Efficient iteration in admissible combinatorial classes / C. Martinez, X. Molinero // Theoretical Computer Science. — 2005. — Vol. 346, no. 2–3. — P. 388–417.
131. Martinez, C. An experimental study of unranking algorithms / C. Martinez, X. Molinero // Lecture Notes in Computer Science: Experimental and Efficient Algorithms. — 2004. — Vol. 3059. — P. 326–340.
132. Molinero, X. Unranking algorithms for combinatorial structures / X. Molinero, J. Vives // International Journal of Applied Mathematics and Informatics. — 2015. — Vol. 9. — P. 110–115.
133. Ryabko, B. Y. Fast enumeration of combinatorial objects / B. Y. Ryabko // Discrete Mathematics and Applications. — 1998. — Vol. 8, no. 2. — P. 163–182.
134. Медведева, Ю. С. Быстрый алгоритм нумерации слов с заданными ограничениями на длины серий единиц / Ю. С. Медведева, Б. Я. Рябко // Проблемы передачи информации. — 2010. — Т. 46, № 4. — С. 130–139.
135. Медведева, Ю. С. Быстрая нумерация слов, порождаемых грамматиками Дика / Ю. С. Медведева // Математические заметки. — 2014. — Т. 96, № 1. — С. 51–69.
136. Медведева, Ю. С. Быстрая нумерация комбинаторных объектов, находящая применение в системах передачи и хранения информации : дис. ... канд. техн. наук : 05.13.17 / Медведева Юлия Сергеевна. — Новосибирск, 2015. — 113 с.
137. Кручинин, В. В. Методы построения алгоритмов генерации и нумерации комбинаторных объектов на основе деревьев И/ИЛИ / В. В. Кручинин. — Томск: В-Спектр, 2007. — 200 с.
138. Кручинин, В. В. Методы, алгоритмы и программное обеспечение комбинаторной генерации : дис. ... д-ра техн. наук : 05.13.11 / Кручинин Владимир Викторович. — Томск, 2010. — 393 с.
139. Шабля, Ю. В. Модификация метода построения алгоритмов комбинаторной генерации на основе применения теории производящих функций / Ю. В. Шабля, Д. В. Кручинин // Доклады ТУСУРа. — 2019. — Т. 22, № 3. — С. 55–60.

140. Шабля, Ю. В. Алгоритмическое обеспечение комбинаторной генерации на основе применения теории производящих функций : дис. ... канд. техн. наук : 05.13.17 / Шабля Юрий Васильевич. — Томск, 2019. — 123 с.
141. Кручинин, Д. В. Модификация метода построения алгоритмов комбинаторной генерации на основе применения производящих функций многих переменных и приближенных вычислений / Д. В. Кручинин // Доклады ТУСУР. — 2022. — Т. 25, № 1. — С. 55–60.
142. Кручинин, Д. В. Методы, алгоритмы и программное обеспечение на основе производящих функций многих переменных для комплексного исследования информационных объектов : дис. ... д-ра техн. наук : 05.13.17 / Кручинин Дмитрий Владимирович. — Томск, 2022. — 319 с.
143. Риордан, Д. Введение в комбинаторный анализ / Д. Риордан. — М.: Издательство иностранной литературы, 1963. — 288 с.
144. Comtet, L. Advanced combinatorics: The art of finite and infinite expansions / L. Comtet. — Netherlands: D. Reidel Publishing Company, 1974. — 354 p.
145. Flajolet, P. Analytic combinatorics / P. Flajolet, R. Sedgewick. — USA: Cambridge University Press, 2009. — 826 p.
146. Wilf, H. S. Generatingfunctionology / H. S. Wilf. — USA: Academic Press, 1994. — 228 p.
147. Стенли, Р. Перечислительная комбинаторика: Деревья, производящие функции и симметрические функции / Р. Стенли. — М.: Мир, 2009. — 767 с.
148. Егорычев, Г. П. Интегральное представление и вычисление комбинаторных сумм / Г. П. Егорычев. — Новосибирск: Наука, Сибирское отделение, 1977. — 285 с.
149. Кузьмин, О. В. Обобщенные пирамиды Паскаля и их приложения / О. В. Кузьмин. — Новосибирск: Наука, Сиб. издат. фирма РАН, 2000. — 294 с.
150. Ландо, С. А. Лекции о производящих функциях / С. А. Ландо. — М.: МЦНМО, 2007. — 144 с.

151. Boas, R. P. Polynomial expansions of analytic functions / R. P. Boas, R. C. Buck. — Second edition. — Germany: Springer, 1964. — 85 p.
152. Roman, S. The umbral calculus / S. Roman. — USA: Academic Press, 1984. — 193 p.
153. Srivastava, H. M. A treatise on generating functions: Mathematics and its applications / H. M. Srivastava, H. L. Manocha. — UK: Ellis Horwood, 1984. — 572 p.
154. Кручинин, В. В. Степени производящих функций и их применение / В. В. Кручинин, Д. В. Кручинин. — Томск: Издательство ТУСУР, 2013. — 236 с.
155. Кручинин, Д. В. Метод получения явных выражений полиномов на основе степеней производящих функций : дис. ... канд. физ.-мат. наук : 01.01.01 / Кручинин Дмитрий Владимирович. — Томск, 2016. — 97 с.
156. Kruchinin, D. V. Method for obtaining coefficients of powers of bivariate generating functions / D. V. Kruchinin, V. V. Kruchinin, Y. V. Shablya // Mathematics. — 2021. — Vol. 9, no. 4. — Article 428.
157. Kruchinin, D. V. Method for obtaining coefficients of powers of multivariate generating functions / D. V. Kruchinin, V. V. Kruchinin, Y. V. Shablya // Mathematics. — 2023. — Vol. 11, no. 13. — Article 2859.
158. Kruchinin, D. V. Explicit formulas for Meixner polynomials / D. V. Kruchinin, Y. V. Shablya // International Journal of Mathematics and Mathematical Sciences. — 2015. — Vol. 2015. — Article 620569.
159. Realization of a method for calculating Bell polynomials based on compositae of generating functions / V. S. Melman, Y. V. Shablya, D. V. Kruchinin, A. A. Shelupanov // Journal of Informatics and Mathematical Sciences. — 2018. — Vol. 10, no. 4. — P. 659–672.
160. A library for calculating polynomials based on compositae of generating functions / D. V. Kruchinin, V. S. Melman, Y. V. Shablya, A. A. Shelupanov // International Conference of Numerical Analysis and Applied Mathematics (ICNAAM 2018). — AIP Conference Proceedings. — Vol. 2116. — Article 100007. — 2019.

161. Kruchinin, V. V. Some identities for harmonic numbers based on composition of generating functions / V. V. Kruchinin, D. V. Kruchinin, Y. V. Shablya // Montes Taurus Journal of Pure and Applied Mathematics. — 2024. — Vol. 6, no. 3. — P. 363–374.
162. Explicit formulas for the Eulerian numbers of the second kind / D. V. Kruchinin, V. V. Kruchinin, Y. V. Shablya, A. A. Shelupanov // International Conference of Numerical Analysis and Applied Mathematics (ICNAAM 2018). — AIP Conference Proceedings. — Vol. 2116. — Article 100008. — 2019.
163. Shablya, Y. V. New properties of a composition of ordinary generating functions for primes / Y. V. Shablya, D. V. Kruchinin, A. A. Shelupanov // Journal of Discrete Mathematical Sciences and Cryptography. — 2021. — Vol. 24, no. 4. — P. 917–930.
164. Properties of a composition of exponential and ordinary generating functions / D. V. Kruchinin, Y. V. Shablya, V. V. Kruchinin, A. A. Shelupanov // Communications in Mathematics and Applications. — 2018. — Vol. 9, no. 4. — P. 705–711.
165. Integer properties of a composition of exponential generating functions / D. V. Kruchinin, Y. V. Shablya, O. O. Evsutin, A. A. Shelupanov // International Conference of Numerical Analysis and Applied Mathematics (ICNAAM 2016). — AIP Conference Proceedings. — Vol. 1863. — Article 300014. — 2017.
166. Кручинин, Д. В. Программное обеспечение для анализа тестов простоты натурального числа / Д. В. Кручинин, Ю. В. Шабля // Доклады ТУСУРа. — 2014. — № 4 (34). — С. 95–99.
167. Шабля, Ю. В. Генератор критериев простоты натурального числа на основе свойств композиции производящих функций / Ю. В. Шабля, Д. В. Кручинин, А. А. Шелупанов // Доклады ТУСУРа. — 2015. — № 4 (38). — С. 97–101.
168. Bender, E. A. Central and local limit theorems applied to asymptotic enumeration II: Multivariate generating functions / E. A. Bender, L. Bruce Richmond // Journal of Combinatorial Theory, Series A. — 1983. — Vol. 34, no. 3. — P. 255–265.

169. Pemantle, R. Twenty combinatorial examples of asymptotics derived from multivariate generating functions / R. Pemantle, M. C. Wilson // SIAM Review. — 2008. — Vol. 50, no. 2. — P. 199–272.
170. Кручинин, В. В. Алгоритмы и перечислительные свойства деревьев И/ИЛИ / В. В. Кручинин // Вестник Томского государственного университета: Серия «Математика. Кибернетика. Информатика». — 2004. — № 284. — С. 178–181.
171. Кручинин, В. В. Представление множеств деревьями И/ИЛИ / В. В. Кручинин // Доклады ТУСУРа. — 2008. — № 1 (17). — С. 107–112.
172. Кручинин, В. В. Метод кодирования информационных объектов на основе деревьев И/ИЛИ / В. В. Кручинин, Б. А. Люкшин // Доклады ТУСУРа. — 2010. — № 1 (21). — С. 170–172.
173. Sundblad, Y. The Ackermann function. A theoretical, computational, and formula manipulative study / Y. Sundblad // BIT Numerical Mathematics. — 1971. — Vol. 11. — P. 107–119.
174. Sloane, N. J. A. The On-Line Encyclopedia of Integer Sequences [Электронный ресурс]. — Режим доступа: <http://www.oeis.org/>.
175. Pippenger, N. The hypercube of resistors, asymptotic expansions, and preferential arrangements / N. Pippenger // Mathematics Magazine. — 2010. — Vol. 83, no. 5. — P. 331–346.
176. Jacques, M. Greedy universal cycle constructions for weak orders / M. Jacques, D. Wong // Lecture Notes in Computer Science: Algorithms and Discrete Applied Mathematics. — 2020. — Vol. 12016. — P. 363–370.
177. Gross, O. A. Preferential arrangements / O. A. Gross // The American Mathematical Monthly. — 1962. — Vol. 69. — P. 4–8.
178. Good, I. J. The number of orderings of n candidates when ties are permitted / I. J. Good // The Fibonacci Quarterly. — 1975. — Vol. 13. — P. 11–18.
179. Cayley, A. On the analytical forms called trees. — Part II / A. Cayley // The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science. — 1859. — Vol. 18, no. 121. — P. 374–378.

180. Diagana, T. Some new identities and congruences for Fubini numbers / T. Diagana, H. Maiga // Journal of Number Theory. — 2017. — Vol. 173. — P. 547–569.
181. Kilar, N. A new family of Fubini type numbers and polynomials associated with Apostol–Bernoulli numbers and polynomials / N. Kilar, Y. Simsek // Journal of the Korean Mathematical Society. — 2017. — Vol. 54, no. 5. — P. 1605–1621.
182. Gun, D. Combinatorial sums involving Stirling, Fubini, Bernoulli numbers and approximate values of Catalan numbers / D. Gun, Y. Simsek // Advanced Studies in Contemporary Mathematics. — 2020. — Vol. 30, no. 4. — P. 503–513.
183. Sharma, S. K. A parametric kind of the degenerate Fubini numbers and polynomials / S. K. Sharma, W. A. Khan, C. S. Ryoo // Mathematics. — 2020. — Vol. 8, no. 3. — Article 405.
184. Jin, S. Degenerate Fubini-type polynomials and numbers, degenerate Apostol–Bernoulli polynomials and numbers, and degenerate Apostol–Euler polynomials and numbers / S. Jin, M. C. Dagli, F. Qi // Axioms. — 2022. — Vol. 11, no. 9. — Article 477.
185. Velleman, D. J. Permutations and combination locks / D. J. Velleman, G. S. Call // Mathematics Magazine. — 1995. — Vol. 68, no. 4. — P. 243–253.
186. Шабля, Ю. В. Сжатие информационных объектов с помощью методов комбинаторной генерации на основе деревьев И/ИЛИ / Ю. В. Шабля // Доклады ТУСУРа. — 2024. — Т. 27, № 4. — С. 74–79.
187. Shablya, Y. V. Combinatorial generation algorithms for discrete structures associated with the Fubini numbers / Y. V. Shablya, V. A. Polyuga, D. V. Kruchinin // Proceedings of the Jangjeon Mathematical Society. — 2025. — Vol. 28, no. 1. — P. 53–71.
188. Algorithms for ranking and unranking the combinatorial set of closed questionnaire answers / P. P. Shcheglov, G. A. G.A. Filippov, Y. V. Shablya, D. V. Kruchinin // Journal of Physics: Conference Series. — 2020. — Vol. 1611. — Article 012069.
189. Melman, A. S. On the efficiency of combinatorial generation for adaptive image steganography / A. S. Melman, O. O. Evsutin, Y. V. Shablya // International Conference Engineering and Telecommunication (EnT 2021). — IEEE. — 2021.

190. Combinatorial generation algorithms for the set defined by central polygonal numbers / G. A. Filippov, D. V. Kruchinin, Y. V. Shablya, V. M. Rulevskiy // International Conference of Numerical Analysis and Applied Mathematics (ICNAAM 2021). — AIP Conference Proceedings. — Vol. 2849. — Article 060002. — 2023.
191. Ахо, А. В. Разработка и анализ компьютерных алгоритмов / А. В. Ахо, Дж. Э. Хопкрофт, Дж. Д. Ульман. — Киев: Диалектика, 2021. — 544 с.
192. Шабля, Ю. В. Алгоритмы комбинаторной генерации на основе структур деревьев И/ИЛИ для класса алгебраических производящих функций / Ю. В. Шабля // Моделирование и анализ информационных систем. — 2025. — Т. 32, № 4. — С. 360–383.
193. Shablya, Y. V. Relationship between combinatorial sets, generating functions and combinatorial generation algorithms / Y. V. Shablya, M. Y. Perminova // Proceedings book of the 6th Mediterranean International Conference of Pure & Applied Mathematics and Related Areas. — Paris, 2023. — P. 356–359.
194. Shablya, Y. V. Compositions of AND/OR tree structures based on generating functions / Y. V. Shablya, V. V. Kruchinin, D. V. Kruchinin // Abstracts book of the 30th International Conference on Difference Equations and Applications. — Guangzhou, 2025. — P. 60.
195. Humphreys, K. A history and a survey of lattice path enumeration / K. Humphreys // Journal of Statistical Planning and Inference. — 2010. — Vol. 140, no. 8. — P. 2237–2254.
196. Krattenthaler, C. Lattice path enumeration / C. Krattenthaler // Handbook of enumerative combinatorics. — USA: CRC Press, 2015. — P. 598–680.
197. Zaks, S. Generating trees and other combinatorial objects lexicographically / S. Zaks, D. Richards // SIAM Journal on Computing. — 1979. — Vol. 8, no. 1. — P. 73–81.
198. Bent, S. W. Ranking trees generated by rotations / S. W. Bent // Lecture Notes in Computer Science: Scandinavian Workshop on Algorithm Theory. — 1990. — Vol. 447. — P. 132–142.

199. Parque, V. An efficient scheme for the generation of ordered trees in constant amortized time / V. Parque, T. Miyashita // International Conference on Ubiquitous Information Management and Communication. — 2021.
200. Barcucci, E. Exhaustive generation of positive lattice paths / E. Barcucci, A. Bernini, R. Pinzani // CEUR Workshop Proceedings: International Conference on Random and Exhaustive Generation of Combinatorial Structures. — 2018. — Vol. 2113. — P. 79–86.
201. Barcucci, E. Exhaustive generation of some lattice paths and their prefixes / E. Barcucci, A. Bernini, R. Pinzani // Theoretical Computer Science. — 2021. — Vol. 878–879. — P. 47–52.
202. Kuo, T. From enumerating to generating: A linear time algorithm for generating 2D lattice paths with a given number of turns / T. Kuo // Algorithms. — 2015. — Vol. 8, no. 2. — P. 190–208.
203. Liebehenschel, J. Ranking and unranking of a generalized Dyck language and the application to the generation of random trees / J. Liebehenschel // Seminaire Lotharingien de Combinatoire. — 1999. — Vol. 43. — Article B43d.
204. Duchon, P. On the enumeration and generation of generalized Dyck words / P. Duchon // Discrete Mathematics. — 2000. — Vol. 225, no. 1–3. — P. 121–135.
205. Liebehenschel, J. Lexicographical generation of a generalized Dyck language / J. Liebehenschel // SIAM Journal on Computing. — 2003. — Vol. 32, no. 4. — P. 880–903.
206. Shablya, Y. V. Combinatorial generation algorithms for some lattice paths using the method based on AND/OR trees / Y. V. Shablya // Algorithms. — 2023. — Vol. 16, no. 6. — Article 266.
207. Mohanty, G. Lattice path counting and applications / G. Mohanty. — USA: Academic Press, 1979. — 200 p.
208. Mansour, T. Statistics on Dyck paths / T. Mansour // Journal of Integer Sequences. — 2006. — Vol. 9, no. 1. — Article 06.1.5.
209. Banderier, C. Why Delannoy numbers? / C. Banderier, S. Schwer // Journal of Statistical Planning and Inference. — 2005. — Vol. 135, no. 1. — P. 40–54.

210. Shapiro, L. W. Bijections for the Schroder Numbers / L. W. Shapiro, R. A. Sulanke // *Mathematics Magazine*. — 2000. — Vol. 73, no. 5. — P. 369–376.
211. Oste, R. Motzkin paths, Motzkin polynomials and recurrence relations / R. Oste, J. Van der Jeugt // *Electronic Journal of Combinatorics*. — 2019. — Vol. 22, no. 2. — Article P2.8.
212. Maxima, Computer Algebra System [Электронный ресурс]. — Режим доступа: <https://maxima.sourceforge.io/index.html>.
213. Shablya, Y. V. Combinatorial generation algorithms for directed lattice paths / Y. V. Shablya, A. A. Merinov, D. V. Kruchinin // *Mathematics*. — 2024. — Vol. 12, no. 8. — Article 1207.
214. Banderier, C. Basic analytic combinatorics of directed lattice paths / C. Banderier, P. Flajolet // *Theoretical Computer Science*. — 2002. — Vol. 281, no. 1–2. — P. 37–80.
215. Prodinger, H. The kernel method: A collection of examples / H. Prodinger // *Seminaire Lotharingien de Combinatoire*. — 2003. — Vol. 50. — Article B50f.
216. Banderier, C. Analytic combinatorics of lattice paths: Enumeration and asymptotics for the area / C. Banderier, B. Gittenberger // *Discrete Mathematics and Theoretical Computer Science*. — 2006. — Vol. AG. — P. 345–356.
217. Analytic combinatorics of lattice paths with forbidden patterns, the vectorial kernel method, and generating functions for pushdown automata / A. Asinowski, A. Bacher, C. Banderier, B. Gittenberger // *Algorithmica*. — 2020. — Vol. 82. — P. 386–428.
218. Dziemianczuk, M. On directed lattice paths with vertical steps / M. Dziemianczuk // *Discrete Mathematics*. — 2016. — Vol. 339, no. 3. — P. 1116–1139.
219. Explicit formulas for enumeration of lattice paths: Basketball and the kernel method / C. Banderier, C. Krattenthaler, A. Krinik et al. // *Developments in Mathematics*. — 2019. — Vol. 58. — P. 78–118.
220. Bousquet-Melou, M. Linear recurrences with constant coefficients: the multivariate case / M. Bousquet-Melou, M. Petkovsek // *Discrete Mathematics*. — 2000. — Vol. 225, no. 1–3. — P. 51–75.

221. Shablya, Y. V. Euler-Catalan's number triangle and its application / Y. V. Shablya, D. V. Kruchinin // *Symmetry*. — 2020. — Vol. 12, no. 4. — Article 600.
222. Shablya, Y. V. Method for developing combinatorial generation algorithms based on AND/OR trees and its application / Y. V. Shablya, D. V. Kruchinin, V. V. Kruchinin // *Mathematics*. — 2020. — Vol. 8, no. 6. — Article 962.
223. Kruchinin, D. V. On some properties of generalized Narayana numbers / D. V. Kruchinin, V. V. Kruchinin, Y. V. Shablya // *Quaestiones Mathematicae*. — 2022. — Vol. 45, no. 12. — P. 1949–1963.
224. Кручинин, Д. В. Методика использования базы знаний производящих функций двух переменных / Д. В. Кручинин // *Системы анализа и обработки данных*. — 2022. — Т. 85, № 1. — С. 121–139.
225. Shablya, Y. V. Algorithms for ranking and unranking the combinatorial set of RNA secondary structures / Y. V. Shablya, D. V. Kruchinin // *Discrete Mathematics, Algorithms and Applications*. — 2025.
226. Chavan, P. V. Automata theory and formal languages: Concepts and practices / P. V. Chavan, A. Jadhav. — USA: Academic Press, 2023. — 232 p.
227. Huynh, D. T. The complexity of ranking simple languages / D. T. Huynh // *Mathematical systems theory*. — 1990. — Vol. 23. — P. 1–19.
228. Lange, K.-J. Parallel recognition and ranking of context-free languages / K.-J. Lange, P. Rossmanith, W. Rytter // *Lecture Notes in Computer Science: Mathematical Foundations of Computer Science*. — 1992. — Vol. 629. — P. 24–36.
229. Makinen, E. Ranking and unranking left Szilard languages / E. Makinen // *International Journal of Computer Mathematics*. — 1998. — Vol. 68, no. 1–2. — P. 29–38.
230. Formatted encryption beyond regular languages / D. Luchaup, T. Shrimpton, T. Ristenpart, S. Jha // *ACM SIGSAC Conference on Computer and Communications Security*. — 2014. — P. 1292–1303.

231. Hickey, T. Uniform random generation of strings in a context-free language / T. Hickey, J. Cohen // *SIAM Journal on Computing*. — 1983. — Vol. 12, no. 4. — P. 645–655.
232. Mairson, H. G. Generating words in a context-free language uniformly at random / H. G. Mairson // *Information Processing Letters*. — 1994. — Vol. 49, no. 2. — P. 95–99.
233. Nebel, M. E. Random generation of RNA secondary structures according to native distributions / M. E. Nebel, A. Scheid, F. Weinberg // *Algorithms for Molecular Biology*. — 2011. — Vol. 6. — Article 24.
234. Onokpasa, E. RNA secondary structures: from ab initio prediction to better compression, and back / E. Onokpasa, S. Wild, P. W. H. Wong // *Data Compression Conference*. — 2023.
235. Format-preserving encryption / M. Bellare, T. Ristenpart, P. Rogaway, T. Stegers // *Lecture Notes in Computer Science: International Workshop on Selected Areas in Cryptography*. — 2009. — Vol. 5867. — P. 295–312.
236. Miracle, S. Targeted invertible pseudorandom functions and deterministic format-transforming encryption / S. Miracle, S. Yilek // *Lecture Notes in Computer Science: Topics in Cryptology – Cryptographers’ Track at the RSA Conference*. — 2023. — Vol. 13871. — P. 622–642.
237. Шабля, Ю. В. Метод построения алгоритмов комбинаторной генерации для контекстно-свободных языков на основе структур деревьев И/ИЛИ / Ю. В. Шабля // *Информационные технологии*. — 2025. — Т. 31, № 9. — С. 465–476.
238. Earley, J. An efficient context-free parsing algorithm / J. Earley // *Communications of the ACM*. — 1970. — Vol. 13, no. 2. — P. 94–102.
239. Hofacker, I. L. Combinatorics of RNA secondary structures / I. L. Hofacker, P. Schuster, P. F. Stadler // *Discrete Applied Mathematics*. — 1998. — Vol. 88, no. 1–3. — P. 207–237.
240. Bertoni, A. Computing the counting function of context-free languages / A. Bertoni, M. Goldwurm, N. Sabadini // *Lecture Notes in Computer Science*:

- Annual Symposium on Theoretical Aspects of Computer Science. — 1987. — Vol. 247. — P. 169–179.
241. Zhou, R. R. Context-free grammars for several triangular arrays / R. R. Zhou, J. Yeh, F. Ren // *Axioms*. — 2022. — Vol. 11, no. 6. — Article 297.
 242. Chen, W. Y. C. A context-free grammar for the e-positivity of the trivariate second-order Eulerian polynomials / W. Y. C. Chen, A. M. Fu // *Discrete Mathematics*. — 2022. — Vol. 345, no. 1. — Article 112661. — P. 277–283.
 243. Yang, H. R. A context-free grammar associated with Fibonacci and Lucas sequences / H. R. Yang // *Journal of Mathematics*. — 2023. — Vol. 2023. — Article 6497710.
 244. Panholzer, A. Grobner bases and the defining polynomial of a context-free grammar generating function / A. Panholzer // *Journal of Automata, Languages and Combinatorics*. — 2005. — Vol. 10, no. 1. — P. 79–97.
 245. Kruchinin, V. V. Composita and its properties / V. V. Kruchinin, D. V. Kruchinin // *Journal of Analysis and Number Theory*. — 2014. — Vol. 2, no. 2. — P. 37–44.
 246. Shablya, Y. V. Enumeration of words derived from unambiguous context-free grammars based on powers of generating functions / Y. V. Shablya, V. V. Kruchinin, D. V. Kruchinin // *Proceedings of the Jangjeon Mathematical Society*. — 2025. — Vol. 28, no. 3. — P. 299–317.
 247. Ranking and loopless generation of k -ary Dyck words in cool-lex order / S. Durocher, P. C. Li, D. Mondal, A. Williams // *Lecture Notes in Computer Science: International Workshop on Combinatorial Algorithms*. — 2011. — Vol. 7056. — P. 182–194.
 248. Kasa, Z. Generating and ranking of Dyck words / Z. Kasa // *Acta Universitatis Sapientiae, Informatica*. — 2009. — Vol. 1, no. 1. — P. 109–118.
 249. Shablya, Y. V. Method for designing encoding algorithms based on AND/OR trees / Y. V. Shablya, V. A. Polyuga // *Journal of Discrete Mathematical Sciences and Cryptography*. — 2025.

250. Boyadzhiev, K. N. Close encounters with the Stirling numbers of the second kind / K. N. Boyadzhiev // *Mathematics Magazine*. — 2012. — Vol. 85, no. 4. — P. 252–266.
251. The Canterbury Corpus [Электронный ресурс]. — Режим доступа: <https://corpus.canterbury.ac.nz/descriptions/>.
252. Казаков, А. Я. Некоторые модификации алгоритма сжатия ВРЕ / А. Я. Казаков, А. А. Жихарева, П. А. Пасечник // *Информационные технологии и вычислительные системы*. — 2018. — Т. 3. — С. 59–68.
253. Gupta, A. Modern lossless compression techniques: Review, comparison and analysis / A. Gupta, A. Bansal, V. Khanduja // *International Conference on Electrical, Computer and Communication Technologies*. — 2017.
254. Matusiak, D. M. A review of classical lossless data compression algorithms and their contemporary relevance / D. M. Matusiak // *Biuletyn Wojskowej Akademii Technicznej*. — 2025. — Vol. 74, no. 2. — P. 31–40.
255. Меринов, А. А. Сжатие изображений без потерь на основе методов комбинаторной генерации / А. А. Меринов, Ю. В. Шабля // *Перспективы развития фундаментальных наук: сборник научных трудов XXI Международной конференции студентов, аспирантов и молодых ученых. Том 3: Математика*. — Томск: Издательство Томского политехнического университета, 2024. — С. 11–13.
256. SIPI Image Database [Электронный ресурс]. — Режим доступа: <https://sipi.usc.edu/database/>.
257. Lugrin, T. Random number generator / T. Lugrin // *Trends in Data Protection and Encryption Technologies*. — Switzerland: Springer, 2023. — P. 31–34.
258. Dereli, S. True-random number generator based on image histogram / S. Dereli // *Academic Perspective Procedia*. — 2020. — Vol. 3, no. 1. — P. 301–307.
259. Крюков, Д. А. Генерация псевдослучайных чисел на основе преобразований графических объектов / Д. А. Крюков, И. А. Тескер // *Информационно-управляющие системы*. — 2018. — № 2. — С. 2–7.

260. Haider, T. A novel pseudo-random number generator based on multivariable optimization for image-cryptographic applications / T. Haider, S. A. Blanco, U. Hayat // Expert Systems with Applications. — 2024. — Vol. 240. — Article 122446.
261. Rukhin, A. A statistical test suite for random and pseudorandom number generators for cryptographic applications [Электронный ресурс]. — 2010. — Режим доступа: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-22r1a.pdf>.
262. Ko, J. A systematic review of anomaly detection for business process event logs / J. Ko, M. Comuzzi // Business & Information Systems Engineering. — 2023. — Vol. 65. — P. 441–462.
263. Improving state-of-the-art compression techniques for log management tools / K. Yao, M. Sayagh, W. Shang, A. E. Hassan // IEEE Transactions on Software Engineering. — 2022. — Vol. 48, no. 8. — P. 2748–2760.
264. Balakrishnan, R. Lossless compression for large scale cluster logs / R. Balakrishnan, R. K. Sahoo // International Parallel & Distributed Processing Symposium. — 2006.
265. Шабля, Ю. В. Метод сжатия данных журналов событий на основе теории комбинаторной генерации с применением структур деревьев И/ИЛИ / Ю. В. Шабля // Современные информационные технологии и ИТ-образование. — 2023. — Т. 19, № 3. — С. 564–574.
266. Bojiah, J. Effectiveness of Moodle in teaching and learning / J. Bojiah // Journal of Hunan University (Natural Sciences). — 2022. — Vol. 49, no. 12. — P. 320–328.
267. Система управления обучением ТУСУР [Электронный ресурс]. — Режим доступа: <https://sdo.tusur.ru/>.
268. Parise, P. A preliminary look at online learner behavior / P. Parise // Bulletin of Kanagawa Prefectural Institute of Language and Culture Studies. — 2017. — Vol. 6. — P. 15–31.
269. Rotelli, D. Processing and understanding Moodle log data and their temporal dimension / D. Rotelli, A. Monreale // Journal of Learning Analytics. — 2023. — Vol. 10, no. 2. — P. 126–141.

270. Репьюк, Н. С. Программное обеспечение для декомпозиции значений атрибута «Описание» журнала событий Moodle / Н. С. Репьюк, Д. В. Кручинин // Перспективы развития фундаментальных наук: сборник научных трудов XX Международной конференции студентов, аспирантов и молодых ученых. Том 7: IT-технологии и электроника. — Томск: Издательство Томского политехнического университета, 2023. — С. 124–126.
271. Jacob, N. Comparative analysis of lossless text compression techniques / N. Jacob, P. Somvanshi, R. Tornekar // International Journal of Computer Applications. — 2012. — Vol. 56, no. 3. — P. 17–21.
272. Tanjung, A. S. Comparison analysis with Huffman algorithm and Goldbach codes algorithm in file compression text using the method exponential comparison / A. S. Tanjung, S. D. Nasution // International Journal of Informatics and Computer Science. — 2020. — Vol. 4, no. 1. — P. 29–34.
273. Computer operating system logging and security issues: a survey / L. Zeng, Y. Xiao, H. Chen et al. // Security and Communication Networks. — 2016. — Vol. 9. — P. 4804–4821.
274. Шабля, Ю. В. Способы хранения структур деревьев И/ИЛИ и их вариантов в оперативной и постоянной компьютерной памяти / Ю. В. Шабля, А. В. Токарева // Доклады ТУСУРа. — 2024. — Т. 27, № 2. — С. 44–50.

Приложение А

Акт внедрения в учебный процесс

ТУСУР

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
**ТОМСКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ СИСТЕМ УПРАВЛЕНИЯ
И РАДИОЭЛЕКТРОНИКИ**

ОКПО 02069326, ОГРН 1027000867068,
ИНН 7021000043, КПП 701750001

пр. Ленина, д. 40, г. Томск, 634050

тел: (382 2) 510-530
факс: (382 2) 513-262, 526-365
e-mail: office@tusur.ru
http://www.tusur.ru



УТВЕРЖДАЮ

Директор департамента

по учебной работе

М.Ю. Ким

12 2025 г.

АКТ

о внедрении результатов диссертационной работы Шаблия Юрия Васильевича
в учебный процесс

Комиссия в составе:

председателя

- Хабибуллиной Н.Ю., декана факультета вычислительных систем, канд. техн. наук
членов комиссии

- Шурьгина Ю.А., заведующего кафедрой КСУП, д-ра техн. наук,

- Боровского И.Г., заведующего каф. ЭМИС ТУСУР, д-ра физ.-мат. наук
составила настоящий акт о нижеследующем.

Результаты диссертационной работы Шаблия Ю.В. используются в учебном процессе на факультете вычислительных систем Томского государственного университета систем управления и радиоэлектроники при чтении курса лекций и проведении практических/лабораторных занятий по дисциплинам «Дискретная математика», «Структуры данных» и «Теория принятия решений» для подготовки студентов, обучающихся по направлению «09.03.01 — Информатика и вычислительная техника».

Результаты диссертационной работы Шаблия Ю.В. используются:

1. В дисциплине «Дискретная математика» при проведении лабораторных занятий, связанных с разработкой представлений комбинаторных множеств, в том числе в форме структур деревьев И/ИЛИ, а также с разработкой отображений элементов комбинаторных множеств в варианты деревьев И/ИЛИ. Это позволяет студентам освоить процесс реализации биективного отображения между множествами;

2. В дисциплине «Структуры данных» при чтении курса лекций и проведении лабораторных занятий, связанных с изучением древовидных структур данных. Это позволяет студентам ознакомиться со структурой данных деревьев

И/ИЛИ и исследовать возможности их применения в задачах кодирования информации;

3. В дисциплине «Теория принятия решений» при чтении курса лекций и проведении практических занятий, связанных с полным перебором допустимых решений. Это позволяет студентам изучать процесс генерации множества альтернатив и разработки соответствующих алгоритмов комбинаторной генерации.

Освоение студентами предложенного Шаля Ю.В. алгоритмического обеспечения позволяет сформировать навыки работы с дискретными структурами, их перечисления и кодирования.

Декан факультета вычислительных систем,
канд. техн. наук, доцент



Хабидулина Н.Ю.

Заведующий кафедрой КСУП,
д-р. техн. наук, профессор



Шурыгин Ю.А.

Заведующий кафедрой ЭМИС,
д-р физ.-мат. наук, профессор



Боровской И.Г.

Приложение Б

Свидетельства о государственной регистрации программ для ЭВМ

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2023686950

**Программа для сжатия данных журналов событий
электронных курсов системы Moodle**

Правообладатель: **Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Томский государственный университет систем
управления и радиоэлектроники» (RU)**

Автор(ы): **Шабля Юрий Васильевич (RU)**

Заявка № **2023686831**
Дата поступления **07 декабря 2023 г.**
Дата государственной регистрации
в Реестре программ для ЭВМ **11 декабря 2023 г.**

Руководитель Федеральной службы
по интеллектуальной собственности



ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 429b6a0fe3a53164ba96f83b73b4aa7
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.05.2023 по 02.08.2024

Ю.С. Зубов

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2023686963

**Программа для автоматизации процесса построения
структур вариантов деревьев И/ИЛИ на основе их
рекурсивной композиции**

Правообладатель: **Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Томский государственный университет систем
управления и радиозлектроники» (RU)**

Авторы: **Меринов Арсен Арзуманович (RU), Шабля Юрий
Васильевич (RU)**



Заявка № **2023685247**

Дата поступления **24 ноября 2023 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **11 декабря 2023 г.**

*Руководитель Федеральной службы
по интеллектуальной собственности*

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 429b6a0fe3a53164ba96f83b73b4aa7
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.05.2023 по 02.08.2024

Ю.С. Зубов

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2023686543

**Программа для оценки времени активной деятельности
пользователей по данным журнала событий
электронного курса системы Moodle**

Правообладатель: **Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Томский государственный университет систем
управления и радиозлектроники» (RU)**

Авторы: **Шабля Юрий Васильевич (RU), Кручинин Дмитрий
Владимирович (RU)**



Заявка № **2023686223**

Дата поступления **01 декабря 2023 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **07 декабря 2023 г.**

*Руководитель Федеральной службы
по интеллектуальной собственности*

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 429b6a0fe3a53164ba96f83b73b4aa7
Владелец **Зубов Юрий Сергеевич**
Действителен с 10.05.2023 по 02.08.2024

Ю.С. Зубов

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2022683382

Программа для генерации решеточных путей на плоскости

Правообладатель: **Шабля Юрий Васильевич (RU)**

Автор(ы): **Шабля Юрий Васильевич (RU)**



Заявка № **2022683116**

Дата поступления **28 ноября 2022 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **05 декабря 2022 г.**

*Руководитель Федеральной службы
по интеллектуальной собственности*

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 68b80077e14e40f0a394e6bd24145d5c7
Владелец **Зубов Юрий Сергеевич**
Действителен с 20.02.2022 по 26.05.2023

Ю.С. Зубов

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2019660020

**Программа для ранжирования и генерации по рангу
элементов комбинаторных множеств**

Правообладатель: **Федеральное государственное бюджетное
образовательное учреждение высшего образования «Томский
государственный университет систем управления и
радиоэлектроники» (ТУСУР) (RU)**

Авторы: **Шабля Юрий Васильевич (RU),
Кручинин Дмитрий Владимирович (RU)**



Заявка № **2019618795**

Дата поступления **17 июля 2019 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **29 июля 2019 г.**

Руководитель Федеральной службы
по интеллектуальной собственности

Г.П. Ивлиев

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2018611242

**Программа «PCG: Primality Criterion Generator» для
генерации критериев простоты числа**

Правообладатель: *Федеральное государственное бюджетное
образовательное учреждение высшего образования «Томский
государственный университет систем управления и
радиоэлектроники» (ТУСУР) (RU)*

Авторы: *Шабля Юрий Васильевич (RU), Кручинин Дмитрий
Владимирович (RU), Мельман Вадим Сергеевич (RU)*



Заявка № **2017662540**

Дата поступления **04 декабря 2017 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **26 января 2018 г.**

Руководитель Федеральной службы
по интеллектуальной собственности

Г.П. Ивлиев

Г.П. Ивлиев

РОССИЙСКАЯ ФЕДЕРАЦИЯ



СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2018611241

**Программа «РТА: Primality Test Analyser» для анализа
тестов простоты числа**

Правообладатель: *Федеральное государственное бюджетное
образовательное учреждение высшего образования «Томский
государственный университет систем управления и
радиоэлектроники» (ТУСУР) (RU)*

Авторы: *Шабля Юрий Васильевич (RU), Кручинин Дмитрий
Владимирович (RU), Мельман Вадим Сергеевич (RU)*



Заявка № **2017662547**

Дата поступления **04 декабря 2017 г.**

Дата государственной регистрации

в Реестре программ для ЭВМ **26 января 2018 г.**

Руководитель Федеральной службы
по интеллектуальной собственности

Г.П. Ивлиев